CHAPMAN & HALL/CRC TEXTS IN STATISTICAL SCIENCE

Geographic Data Science with Python

Sergio Rey, Dani Arribas-Bel
and Levi John Wolf

A **Chapman & Hall** Book

CRC Press
Taylor & Francis Group

Geographic Data Science with Python

This book provides the tools, the methods, and the theory to meet the challenges of contemporary data science applied to geographic problems and data. In the new world of pervasive, large, frequent, and rapid data, there are new opportunities to understand and analyze the role of geography in everyday life. *Geographic Data Science with Python* introduces a new way of thinking about analysis, by using geographical and computational reasoning, it shows the reader how to unlock new insights hidden within data.

Key Features:

- Showcases the excellent data science environment in Python.
- Provides examples for readers to replicate, adapt, extend, and improve.
- Covers the crucial knowledge needed by geographic data scientists.

It presents concepts in a far more geographic way than competing textbooks, covering spatial data, mapping, and spatial statistics whilst covering concepts, such as clusters and outliers, as geographic concepts.

Intended for data scientists, GIScientists, and geographers, the material provided in this book is of interest due to the manner in which it presents geospatial data, methods, tools, and practices in this new field.

CHAPMAN & HALL/CRC
Texts in Statistical Science Series
Joseph K. Blitzstein, *Harvard University, USA*
Julian J. Faraway, *University of Bath, UK*
Martin Tanner, *Northwestern University, USA*
Jim Zidek, *University of British Columbia, Canada*

Recently Published Titles

Sampling Design and Analysis, Third Edition
Sharon L. Lohr

Theory of Statistical Inference
Anthony Almudevar

Probability, Statistics, and Data
A Fresh Approach Using R
Darrin Speegle and Brain Claire

Bayesian Modeling and Computation in Python
Osvaldo A. Martin, Raviv Kumar and Junpeng Lao

Bayes Rules!
An Introduction to Applied Bayesian Modeling
Alicia Johnson, Miles Ott and Mine Dogucu

Stochastic Processes with R
An Introduction
Olga Korosteleva

Design and Analysis of Experiments and Observational Studies using R
Nathan Taback

Time Series for Data Science: Analysis and Forecasting
Wayne A. Woodward, Bivin Philip Sadler and Stephen Robertson

Statistical Theory
A Concise Introduction, Second Edition
Felix Abramovich and Ya'acov Ritov

Applied Linear Regression for Longitudinal Data
With an Emphasis on Missing Observations
Frans E.S. Tan and Shahab Jolani

Fundamentals of Mathematical Statistics
Steffen Lauritzen

Modelling Survival Data in Medical Research, Fourth Edition
David Collett

Applied Categorical and Count Data Analysis, Second Edition
Wan Tang, Hua He and Xin M. Tu

Geographic Data Science with Python
Sergio J. Rey, Dani Arribas-Bel and Levi John Wolf

For more information about this series, please visit: https://www.routledge.com/Chapman--HallCRC-Texts-in-Statistical-Science/book-series/CHTEXSTASCI

Geographic Data Science with Python

By
Sergio Rey, Dani Arribas-Bel and
Levi John Wolf

CRC Press
Taylor & Francis Group
Boca Raton London New York

CRC Press is an imprint of the
Taylor & Francis Group, an **informa** business

A CHAPMAN & HALL BOOK

Designed cover image: Sergio Rey, Dani Arribas-Bel and Levi John Wolf

First edition published 2023
by CRC Press
6000 Broken Sound Parkway NW, Suite 300, Boca Raton, FL 33487-2742

and by CRC Press
4 Park Square, Milton Park, Abingdon, Oxon, OX14 4RN

CRC Press is an imprint of Taylor & Francis Group, LLC

© 2023 Taylor & Francis Group, LLC

Reasonable efforts have been made to publish reliable data and information, but the author and publisher cannot assume responsibility for the validity of all materials or the consequences of their use. The authors and publishers have attempted to trace the copyright holders of all material reproduced in this publication and apologize to copyright holders if permission to publish in this form has not been obtained. If any copyright material has not been acknowledged please write and let us know so we may rectify in any future reprint.

Except as permitted under U.S. Copyright Law, no part of this book may be reprinted, reproduced, transmitted, or utilized in any form by any electronic, mechanical, or other means, now known or hereafter invented, including photocopying, microfilming, and recording, or in any information storage or retrieval system, without written permission from the publishers.

For permission to photocopy or use material electronically from this work, access www.copyright.com or contact the Copyright Clearance Center, Inc. (CCC), 222 Rosewood Drive, Danvers, MA 01923, 978-750-8400. For works that are not available on CCC please contact mpkbookspermissions@tandf.co.uk

Trademark notice: Product or corporate names may be trademarks or registered trademarks and are used only for identification and explanation without intent to infringe.

Library of Congress Cataloging-in-Publication Data

Names: Rey, Sergio J. (Sergio Joseph), author. | Arribas-Bel, Dani, author. | Wolf, Levi John, author.
Title: Geographic data science with Python / by Sergio Rey, Dani Arribas-Bel and Levi John Wolf.
Description: Boca Raton : CRC Press, 2023. | Series: Chapman & Hall/CRC texts in statistical science | Includes bibliographical references and index.
Identifiers: LCCN 2022056545 (print) | LCCN 2022056546 (ebook) | ISBN 9780367263119 (hardback) | ISBN 9781032445953 (paperback) | ISBN 9780429292507 (ebook)
Subjects: LCSH: Geospatial data--Computer processing. | Python (Computer program language)
Classification: LCC G70.217.G46 R49 2023 (print) | LCC G70.217.G46 (ebook) | DDC 910.285/5133--dc23/eng20230506
LC record available at https://lccn.loc.gov/2022056545
LC ebook record available at https://lccn.loc.gov/2022056546

ISBN: 978-0-367-26311-9 (hbk)
ISBN: 978-1-032-44595-3 (pbk)
ISBN: 978-0-429-29250-7 (ebk)

DOI: 10.1201/9780429292507

Typeset in CMR10
by KnowledgeWorks Global Ltd.

Publisher's note: This book has been prepared from camera-ready copy provided by the authors.

Access the Support Material: geographicdata.science/book

A los recuerdos de María Rey y Sergio Joseph Rey, Sr.

Para Mauri y Naomi.

To my parents, Frank & Debora Wolf.

Collectively we offer this book as a tribute to Luc Anselin, whose visionary research and generous mentoring have been an inspiration to us and a generation of spatial scientists and thinkers.

Taylor & Francis
Taylor & Francis Group
http://taylorandfrancis.com

Contents

Preface **xiii**
Motivation . . . xiv
Why this book? . . . xiv
Who is this book for? . . . xiv
What this book is not . . . xv
Content and Purpose of this Book . . . xv
What we included . . . xv
What we did not include . . . xvi
The book in the future . . . xviii

Acknowledgments **xxi**

Author/editor biographies **xxv**

List of Figures **xxvii**

I Building Blocks **1**

1 Geographic Thinking for Data Scientists **3**
1.1 Introduction to geographic thinking . . . 3
1.2 Conceptual representations: models . . . 4
1.3 Computational representations: data structures . . . 5
1.4 Connecting conceptual to computational . . . 10
1.4.1 This categorization is now breaking up (data is data) . . . 10
1.5 Conclusion . . . 11
1.6 Next steps . . . 12

2 Computational Tools for Geographic Data Science **13**
2.1 Open science . . . 13
2.1.1 Computational notebooks . . . 16
2.1.2 Open source packages . . . 17
2.1.3 Reproducible platforms . . . 17
2.2 The (computational) building blocks of this book . . . 18
2.2.1 Jupyter notebooks and JupyterLab . . . 18
2.2.1.1 Notebooks and cells . . . 19
2.2.1.2 Rich content . . . 20

2.2.1.3 JupyterLab . . . 22
2.2.2 Python and open source packages . . . 24
2.2.2.1 Open source packages . . . 24
2.2.2.2 Contextual help . . . 26
2.2.3 Containerized platform . . . 28
2.2.4 Running the book in a container . . . 29
2.3 Conclusion . . . 32
2.4 Next steps . . . 33

3 Spatial Data **35**
3.1 Fundamentals of geographic data structures . . . 36
3.1.1 Geographic tables . . . 36
3.1.2 Surfaces . . . 44
3.1.3 Spatial graphs . . . 48
3.2 Hybrids . . . 51
3.2.1 Surfaces as tables . . . 51
3.2.1.1 One pixel at a time . . . 52
3.2.1.2 Pixels to polygons . . . 55
3.2.2 Tables as surfaces . . . 61
3.2.3 Networks as graphs *and* tables . . . 62
3.3 Conclusion . . . 64
3.4 Questions . . . 64

4 Spatial Weights **67**
4.1 Introduction . . . 67
4.2 Contiguity weights . . . 68
4.2.1 Spatial weights from real-world geographic tables . . . 75
4.2.2 Spatial weights from surfaces . . . 78
4.3 Distance based weights . . . 79
4.3.1 K-nearest neighbor weights . . . 79
4.3.2 Kernel weights . . . 80
4.3.3 Distance bands and hybrid weights . . . 84
4.3.4 Great circle distances . . . 85
4.4 Block weights . . . 86
4.5 Set operations on weights . . . 87
4.5.1 Editing/connecting disconnected observations . . . 88
4.5.2 Using the `union` of matrices . . . 89
4.6 Visualizing weight set operations . . . 89
4.7 Use case: boundary detection . . . 91
4.8 Conclusion . . . 99
4.9 Questions . . . 99
4.10 Next steps . . . 102

II Spatial Data Analysis 103

5 Choropleth Mapping 105
5.1 Principles . . . 106
5.2 Quantitative data classification . . . 106
5.2.1 Equal intervals . . . 109
5.2.2 Quantiles . . . 110
5.2.3 Mean-standard deviation . . . 112
5.2.4 Maximum breaks . . . 112
5.2.5 Boxplot . . . 113
5.2.6 Head-tail breaks . . . 114
5.2.7 Jenks-Caspall breaks . . . 115
5.2.8 Fisher-Jenks breaks . . . 115
5.2.9 Max-p . . . 116
5.2.10 Comparing classification schemes . . . 116
5.3 Color . . . 120
5.3.1 Sequential palettes . . . 122
5.3.2 Diverging palettes . . . 122
5.3.3 Qualitative palettes . . . 124
5.4 Advanced topics . . . 126
5.4.1 User-defined choropleths . . . 126
5.4.2 Pooled classifications . . . 128
5.5 Conclusion . . . 130
5.6 Questions . . . 131
5.7 Next steps . . . 131

6 Global Spatial Autocorrelation 133
6.1 Understanding spatial autocorrelation . . . 133
6.2 An empirical illustration: the EU Referendum . . . 135
6.3 Global spatial autocorrelation . . . 138
6.3.1 Spatial lag . . . 140
6.3.2 Binary case: join counts . . . 143
6.3.3 Continuous case: Moran Plot and Moran's I . . . 147
6.3.4 Other global indices . . . 151
6.3.4.1 Geary's C . . . 151
6.3.4.2 Getis and Ord's G . . . 152
6.4 Questions . . . 153
6.5 Next steps . . . 155

7 Local Spatial Autocorrelation 157
7.1 An empirical illustration: the EU Referendum . . . 158
7.2 Motivating local spatial autocorrelation . . . 160
7.3 Local Moran's I_i . . . 163
7.4 Getis and Ord's local statistics . . . 173
7.5 Bonus: local statistics on surfaces . . . 176

7.6 Conclusion . . . 182
7.7 Questions . . . 182
7.8 Next Steps . . . 184

8 Point Pattern Analysis 185
8.1 Introduction . . . 186
8.2 Patterns in Tokyo photographs . . . 186
8.3 Visualizing point patterns . . . 188
8.3.1 Showing patterns as dots on a map . . . 188
8.3.2 Showing density with hexbinning . . . 189
8.3.3 Another kind of density: kernel density estimation . . . 192
8.4 Centrography . . . 193
8.4.1 Tendency . . . 194
8.4.2 Dispersion . . . 196
8.4.3 Extent . . . 198
8.5 Randomness and clustering . . . 203
8.5.1 Quadrat statistics . . . 206
8.5.2 Ripley's alphabet of functions . . . 210
8.6 Identifying clusters . . . 213
8.7 Conclusion . . . 217
8.8 Questions . . . 219
8.9 Next steps . . . 219

III Advanced Topics 221

9 Spatial Inequality Dynamics 223
9.1 Introduction . . . 223
9.2 Data: U.S. state per capita income 1969-2017 . . . 224
9.3 Global inequality . . . 226
9.3.1 20:20 ratio . . . 228
9.3.2 Gini index . . . 230
9.3.3 Theil's index . . . 235
9.4 Personal vs. regional income . . . 236
9.5 Spatial inequality . . . 238
9.5.1 Spatial autocorrelation . . . 238
9.5.2 Regional decomposition of inequality . . . 240
9.5.3 Spatializing classic measures . . . 245
9.6 Conclusion . . . 248
9.7 Questions . . . 250
9.8 Next steps . . . 250

10 Clustering and Regionalization 251
10.1 Introduction . . . 251
10.2 Data . . . 253
10.3 Geodemographic clusters in san diego census tracts . . . 259

10.3.1 K-means . . . 260
10.3.2 Spatial distribution of clusters . . . 261
10.3.3 Statistical analysis of the cluster map . . . 262
10.4 Hierarchical Clustering . . . 268
10.5 Regionalization: spatially constrained hierarchical clustering . . . 272
10.5.1 Contiguity constraint . . . 272
10.5.2 Changing the spatial constraint . . . 274
10.5.3 Geographical coherence . . . 276
10.5.4 Feature coherence (goodness of fit) . . . 278
10.5.5 Solution similarity . . . 279
10.6 Conclusion . . . 280
10.7 Questions . . . 280
10.8 Next steps . . . 281

11 Spatial Regression **283**
11.1 *What* is spatial regression and *why* should I care? . . . 283
11.2 Data: San Diego Airbnb . . . 285
11.3 Non-spatial regression, a (very) quick refresh . . . 285
11.3.1 Hidden structures . . . 288
11.4 Bringing space into the regression framework . . . 294
11.4.1 Spatial feature engineering: proximity variables . . . 294
11.4.2 Spatial heterogeneity . . . 298
11.4.2.1 Spatial fixed effects . . . 298
11.4.2.2 Spatial regimes . . . 304
11.4.3 Spatial dependence . . . 308
11.4.3.1 Exogenous effects: The SLX model . . . 308
11.4.3.2 Spatial error . . . 315
11.4.3.3 Spatial lag . . . 316
11.4.3.4 Other ways of bringing space into regression . . . 318
11.5 Questions . . . 318
11.5.1 Challenge questions . . . 319
11.5.1.1 The random coast . . . 319
11.5.1.2 The K-neighbor correlogram . . . 321
11.6 Next steps . . . 324

12 Spatial Feature Engineering **325**
12.1 What is spatial feature engineering? . . . 326
12.2 Feature engineering using map matching . . . 327
12.2.1 Counting *nearby* features . . . 328
12.2.2 Assigning point values from surfaces: elevation of Airbnbs . . . 335
12.2.3 Point interpolation using scikit-learn . . . 338
12.2.4 Polygon to point . . . 345
12.2.5 Area to area interpolation . . . 347
12.3 Feature engineering using map synthesis . . . 351
12.3.1 Spatial summary features in map synthesis . . . 352

12.3.1.1 Counting neighbors 352
12.3.1.2 Distance buffers within a single table 353
12.3.1.3 “Ring” buffer features 357
12.3.2 Clustering as feature engineering 359
12.4 Conclusion 363
12.5 Questions 364
12.6 Next steps 364

References **367**

Index **375**

Preface

This book provides the tools, methods, and theory to meet the challenges of contemporary data science applied to geographic problems and data. Social media, emerging forms of data, and computational techniques are revolutionizing social science. In the new world of pervasive, large, frequent, and rapid data, we have new opportunities to understand and analyze the role of geography in everyday life. This book provides the first comprehensive curriculum in geographic data science.

Geographic data is ubiquitous. On the whole, social processes, physical contexts, and individual behaviors show striking regularity in their geographic patterns, structures, and spacing. As data relating to these systems grows in scope, intensity, and depth, it becomes more important to extract meaningful insights from common geographical properties like location, but also how to leverage geographical *relations* between data that are less commonly seen in standard data science.

This book introduces a new way of thinking about analysis. Using geographical and computational reasoning, it shows the reader how to unlock new insights hidden within data. The book is structured around the excellent data science environment available in Python, providing examples and worked analyses for the reader to replicate, adapt, extend, and improve.

Motivation

Why this book?

Writing a book like this is a major undertaking, and this suggests the authors must have some intrinsic motivations for taking on such a task. We do. Each of the authors is an active participant in both open source development of spatial analytical tools and academic geographic science. Through our research and teaching, we have come to recognize a need for a book to fill the niche that sits at the intersection of GIS/Geography and the world of Data Science. We have seen the explosion of interest in all things Data Science on the one hand and, on the other, the longer standing and continued evolution of GIScience. This book represents our attempt at helping to emerge the intersection between these two fields. It is at that common ground where we believe the intellectual and methodological magic occurs.

Who is this book for?

In writing the book, we envisaged two communities of readers whom we want to bring together. The first are GIScientists and geographers who may be wondering what all the fuss is about Data Science, and questioning whether they should engage with the methods, tools, and practices of this new field. Our response to such a reader is an emphatic "Yes!". We see so much to be gained and contributed by geographers who enter these new waters. The second community we have held in mind in writing this material is data scientists who are beginning to turn their attention to working with geographical data. Here we have encountered members of the data science community who are wondering what is so special about geographical data and problems. Data science currently has an impressive array of models and methods; surely these are all that geographers need? Our response is "No! There is a need for new forms of data science when working with geospatial data." Moreover, we see the *collaboration* between these two communities as *critical* to the development of these new advances.

We also recognize that neither of these two communities is a monolithic whole, but they are in fact composed of individuals from different sectors, academic science, industry, public sector, and independent researchers, as well as at different career stages. We hope this book provides material that will be of interest to all of these readers.

What this book is not

Having described our motivation and intended audience for the book, we find it useful to also point out what the book is not. First, we do not intend the work to be viewed as a GIS starter for data scientists. A number of excellent titles are available that serve that role, such as the *Introduction to Python for Geographic Data Analysis* book by Tenkanen, Heikinheimo, and Whipp in this very series (pythongis.org). Second, in a similar sense the book is not an introduction to Python programming for GIScientists like that offered by Ningchuan Xiao's fantastic *GIS Algorithms*, among other offerings. Finally, we have consciously chosen breadth over depth in the selection of our topics. Each of the topics we cover is an active area of research of which our treatment should be viewed as providing an entry point to more advanced study. As the admonition goes:

"A couple of months in the laboratory can frequently save a couple of hours in the library." (Frank Westheimer[1])

Speaking to our intended audiences, geographers new to data science and data scientists new to geography, we hope our book serves as a metaphorical library.

Content and Purpose of this Book

Every book reflects a combination of the authors' perspectives and the social and technological context in which the authors write. Thus, we see this book as a core component of the project of codifying *what a geographic data science does* and, in turn, what kinds of knowledge are important for aspiring geographic data scientists. We also see the *medium* and *method* of writing this book as important for its purpose. Hence, let's discuss first the content, then the purpose of our medium and message.

What we included

This book delves throughly into a few core topics. From our background as academic geographers, we seek to present concepts in a more *geographic* way than a standard textbook on *data science*. This means that we cover spatial data, mapping, and spatial statistics *right* off the bat, and talk at length about some concepts (such as clusters or outliers) as geographic concepts. We know that this can, in some cases, be confusing for readers who are familiar with these terms as they're used in data science. But, as we hope is shown throughout the book, the difference in language and framing is superficial, while the concepts are foundational to both perspectives.

With that in mind, we discuss the central data structures and representations in geographic data science, and then move immediately to visualization and analysis of

[1] Crampon, Jean E. 1988. Murphy, Parkinson, and Peter: Laws for librarians. Library Journal 113. no. 17 (October 15), p. 41.

geographic data. We use descriptive *spatial statistics* that summarize the structure of maps in order to build the intuition of how spatial thinking can be embedded in data science problems. For the analysis sections, we opt for a presentation of a classic subject in spatial analysis -inequality-, and then pivot to discussing important methods across geographic analysis, such as those that help understand when points are clustered in space, when geographic regions are latent within data, and when geographical spillovers are present in standard supervised learning approaches. The book closes with a discussion of how to use spatial principles to improve your typical analytical workflows.

What we did not include

Despite the "breadth over depth" approach we take in this book, there are many topics that we omit in our treatment. Every book must exhibit some kind of editorial discipline, and we use three principles to inform our own.

First, we sought to avoid topics that get "too complicated too quickly"; instead, we sought to maximize the benefit to a casual reader by focusing on simple but meaningful methods of analysis. This precludes many of the interesting but more complex topics and methods, like Bayesian inference or generative models (like cellular automata or agent-based models). We felt that GeoAI developments at the cutting edge of quantitative geographic analysis are also a bit too complex for this treatment. Further, geographical problems of *scale* and *uncertainty* fall in this category, since these questions generally pose issues that demand theoretical, not empirical, solutions that are specific to the analytical task at hand. We expect this list to change as methods in geographical analysis and data science get simpler and better.

Second, we tried to pick topics that did not have contemporary treatment in computational teaching. This includes spatial optimization problems (such as location allocation or coverage problems) as well as the generative and geostatistical models mentioned above. With this book, we are trying to cover areas where we see a clear opportunity in (re)framing them in new ways for the benefit of the two communities we mention above. Where there is already a wheel, we have not reinvented it.

Third, we admit that we chose topics that are intellectually adjacent to our own experiences and training in quantitative geography. The world of spatial statistics is vast, and very deep, but any one person only gains so much perspective on it. Thus, our backgrounds strongly informed our decisions of what to cover in the second and third sections, where we generally avoid more complex methods like Gaussian Process (geostatistical) models or geospatial knowledge graph methods. We would like to emphasize that this omission is not based on the approaches' merits, which we recognise, but on our own ability to present them clearly, honestly, and effectively.

Altogether, these three editorial principles help keep this book focused precisely on the set of techniques we think give readers the most benefit in the shortest space. It covers both methods to summarize and describe geographical pattern, correct analysis for the

artifacts induced by geographical structure, and leverage geographical relationships to do data analysis better.

Why we wrote an ``open'' book

In addition to the content included (or omitted) from the book, we strongly feel that writing this book as we have—online in public using computational notebooks—provides a novel and distinctive utility for our readers. Thus, the book is open in the sense that it is hosted freely online and replicable, since we try to show the reader all of the code and analytical steps required to generate the outputs we discuss. And, although chapters often start with pre-cleaned datasets, we also include the cleaning code in online supplemental material so that interested readers can see. In addition, nearly every graphic has its code included in the book, and is developed directly within the narrative of the book itself. This approach helps illustrate a few things.

First, this approach facilitates learning and teaching. Geographic Data Science is a new field, but it has many academic influences and precursors. Currently, textbooks in this space either include *no* code, or they separate the discussion of the content from the code. When code is not included in a book, students looking to apply new methods (or teachers of these methods) have to cobble together bits of unrelated code in documentation or StackOverflow and also write new code to combine various packages in the Python data science ecosystem. In contrast, this book provides all the code necessary to repeat the analytical stories we tell. We hope that this both improves the usefulness of this book for readers who can follow along with each step of the computation, and also for people more interested in code examples than reading. Second, by including the code directly within the text, we also make the connection between code and idea more clear. This provides learners with the narrative scaffolding *around* code that learners need to see in order to integrate their own code with analytical writing. Third, this method of presentation shows how tightly coupled the ideas we present are to their actual implementation in code. Analytical techniques are only useful when they help someone do something they could not otherwise. Code is now the *main* way that analytical techniques are made real, and thus useful, to people. And now, new analytical techniques are often informed by the programming environments in which they are implemented. So, we made sure to keep these very tightly integrated to reinforce their reciprocal relationship.

The fact that code and analysis are so tightly coupled also presents some unique challenges for learners. For example, there are many computational hurdles over which students must jump before ever starting the first example of many introductory textbooks. These might include installing a software environment, configuring the environment in a similar fashion to the book's, and executing the code correctly. Because solutions to these issues change frequently and can be different from person to person, books often do not address these issues. This leaves students stranded at the starting block. Our book provides a full view of Geographic Data Science, from setting up and organizing computational environments to preparing data, through to developing novel spatial

insight and presenting it cogently to others. We hope this makes it possible for students to start *from scratch*, rather than having to have extensive experience in setting up computational environments. However, coding experience will certainly help get the most out of the book as a whole.

Finally, we recognize that preparing a book in a fundamentally *interactive* medium does change both the nature and the tenor of the content being presented. Books are (usually) not *mutable* in the same sense as a computational notebook: they can't be changed and re-compiled by the reader. But, this also changes how *we* present content, in that we explicitly provide code cells for the reader to change and rerun. This kind of exploration-driven presentation could be mimicked by more static presentation methods, but we think that this approach provides a much clearer "on-ramp" for developing independent use, thought, and reasoning about these techniques.

The book in the future

In this section, we consider some of the main trends that have shaped the conception of the book. As mentioned, every project like this is in part a reflection of the time in which it is conceived and created. In our case, this "era effect" has had both very tangible ramifications, as well as other ones that, though perhaps less visible at first, signal major shifts of the ground on which geographic data science stands. Some of them are unequivocally positive, others more of a price to pay to be able to develop a project like this one.

Starting with the obvious (but powerful): writing the book in the way we have done *is* possible. This is a statement we would have not been able to make a mere ten years ago. What you are holding in your hands (or displaying on your laptop) is an academic textbook released under an open license, entirely based on open technology, and using a platform that treats both narrative *and* code as first-class citizens. It is as much a book as a software artifact, and its form embodies many of the principles that inspire its content.

Though possible, the process has not been straightforward. Many of the technologies we rely on heavily were *just* available when we started writing back in 2018. Computational notebooks were stable by then, but ways of combining them and using them as the building block of long-form writing were not. In particular, this book owes much of its current form to two projects, `jupyterbook` and `jupytext`, which make it possible to build complex documents starting from Jupyter notebooks and to mirror their content to other formats such as markdown, respectively. Both projects were in their early days when we adopted them and, using them in production at the same time they were being developed into a stable shape has not been without its challenges. But this has also reminded us the very best of the open-source ethos: their teams have been a phenomenal example of how a an open, fast-paced project can bring together a community around it. Although many of the changes broke things constantly, clear documentation, signposting, and responsiveness to our questions made it all possible.

In effect, not only infrastructure-wise, the wider landscape of Python for geographic data science evolves very fast. Our scientific stack has changed significantly over the period of writing. New packages appear, existing ones change, and some also lose support and maintenance to a point that they are unusable. Writing a book that tries to set up the main tool set in this context is challenging. In some ways, by the time this book is in print, some of its parts will be outdated or even obsolete. We think this is a problem, albeit a small and good one. It is small because the core value proposition of the book is not as a technical guide teaching a set of specific computational tools. It is rather a companion to help you think geographically when you work with modern data, and get the most of state-of-the-art data technologies when you work with geographic problems. It is also a *good* problem to have, because it is a sign that the ecosystem is constantly getting better. New packages only become significant if they do more, better, or both than the existing ones. At any rate, this constantly and rapidly changing context made us think more thoroughly about the computational infrastructure and, over time, we learned to take it more as a feature rather than a bug (it also inspired us to write *Chapter 2*!).

Besides technical challenges, creating a textbook based on notebooks has also unearthed more conceptual aspects we had not anticipated. Writing computational notebooks is qualitatively different from writing a traditional textbook. As discussed before, the writing process changes when you weave code and narrative, and that takes additional effort and explicit design choices. Furthermore, we wanted this book's content to be available online as a free website, so we were effectively catering to both print *and* the web in the same document. This often meant tricky tradeoff's and, sometimes, settling for a (smaller) common shared subset of options and functionality. This book has taught us in very practical ways that the medium often frames the message, and that we were exploring a lesser-known medium that has its own rules.

Finally, we believe the book was written at an inflection point where the computational landscape for data science and GISc has left its previous steady state, but it is not quite clear yet what the new one fully looks like. Perhaps, as the famous William Gibson's quote goes, the "future is already here - it's just not evenly distributed". Scientific computing is open by default and, more and more, very interoperable. Tooling to work with such stack, from low-level components to the end-user, has improved enormously and continues to do so. At the same time, we think some of these changes bring about more substantial shifts that we have not fully accommodated yet. As we mention above, we have only scratched the surface of what new media like computational notebooks allow, and much of the social infrastructure around science (e.g., publishing) has been largely detached from these changes. With this book, we hope to demonstrate what is already possible in this new world, but also "nudge the way" for the uneven bits of the future that are still not here. We hope you enjoy it, and it inspires you to further nudge away!

Taylor & Francis
Taylor & Francis Group
http://taylorandfrancis.com

Acknowledgments

Many people have contributed to this book by enhancing our understanding of geographic data science, educating us in the practices of open science and open source development, and through constructive feedback as we wrote the book in the open.

We are fortunate to be members of the PySAL family of developers. The core team, both current and emeriti, have made PySAL what it is today. For their contributions we thank: Carson Farmer, Charles Schmidt, David Folch, Eli Knaap, Erin Olson, Germano Barcelos, Hu Shao, James Gaboardi, Jay Laura, Jeff Sauer, Luc Anselin, Martin Fleischmann, Marynia Kolak, Matthew Conway, Mragank Shekhar, Myunghwa Hwang, Pedro Amaral, Phil Stephens, Qunshan Zhao, Ran Wei, Renan Cortes, Sizhe Wang, Stefanie, Lumnitz, Taylor Oshan, Wei Kang, Xin Feng, and Ziqi Li.

We also are grateful to the many community contributors to PySAL, the users, and participants at the many workshops we have given in road-testing the content.

A special thanks is due to Julia Koschinsky for alighting our worlds with an infectious spirit for all things spatial.

This project was conceived in the open from Day 0. This means that our creation process was fully exposed to the world. Writing in the open meant, among other things, we started receiving feedback by readers from very early on. We are forever grateful to everyone who used the book, skimmed its pages, or simply gave us a shout in social media. Your words were re-assurance in the hardest writing times. We are particularly indebted to readers who reported bugs, typos, or other comments through the project's Github page. In no particular order (Github handle only when name not available): Nikos Patias (`@patnik`), Raul Nanclares (`@rnanclares`), Jo Wilkin (`jo-wilkin`), Patricio Reyes (`@pareyesv`), Cillian Berragan (`@cjber`), Thomas

Statham (@tastatham), Wenfei Xu (@iamwfx), David C. Folch (@dfolch), Germano Barcelos (@gegen07), @dujiaxin, and @thakur18.

A book like this would simply not exist were it not for the revolutionary scientific and technical stack that the Python scientific community has built. We are indebted to the developers and contributors of simply too many packages in this ecosystem to list here.

Numerous people at Taylor & Francis were instrumental in ensuring this book was finished in a timely fashion despite the massive disruption of the COVID-19 global pandemic. We thank John Kimmel and Lara Spieker, Editors, for their support of this book. Robin Lloyd-Starkes, Production Editor, patiently reminded us of production deadlines and kept us focused on transitioning from an interactive text to a traditional published book.

Serge is grateful for having the chance to collaborate with Dani and Levi on this book, and is thankful that their friendship has been strengthened by all the challenges a book like this presents. He would also like to express his gratitude to colleagues at the Center for Geospatial Sciences at UC Riverside for their support during the early part of this book's writing. His new (and old) colleagues at San Diego State University make Serge feel like he never left - and for that he is grateful. Eli Knaap has been with Serge at both institutions and has made this period as productive and fun as one could imagine. Serge would like to thank Luc Anselin for his long-time friendship, support, and collaboration. Finally, Serge thanks his immediate family, Janet, Savannah, Connor, and *little miss magic* Laney for their love.

This was Dani's first (but hopefully not last!) book. Embarquing on a project like this helpfully looks much easier and quicker at the beginning than it always ends up being. Dani is grateful for his convenient ability to ignore the real costs and sober assessments of time availability until it is too late. More than anything, Dani is eternally grateful for his two travel companions, Levi and Serge. It has been a lot more work than originally planned, intense at times, and definitely more fun than everyone says writing a book is. Dani would do it all over again any time. For continuous support and (at least pretended) understanding with his impunctuality, Dani would like to thank his ``extended'' families at the Geographic Data Science Lab in Liverpool and Urban Analytics at The Alan Turing Institute in London. This book, like most of what Dani has created over his career, would most likely not exist was it not for Fernando Sanz-Gracia who, decades ago, sparked Dani's academic interest and continues to nurture it with regular Spotify URLs of obscure (and less so) pieces of classical music. For their continuous friendship over the years and across continents, Dani would like to thank the Rare Kids in Zaragoza, for whom this book will probably do little in clarifying what it is that Dani does for a living. After recently becoming a parent himself, Dani has only renewed appreciation and love for his parents, Nati y Antonio, and his brother, Miguel, who not only sustained the infant, child and adolescent who Dani once was, but also made him the adult he is today. To Ellen and Naomi, for everything.

Levi really enjoyed working on this project. Writing a book with Serge and Dani was an awesome, creative, and enjoyable endeavor, and he is excited for what the future

holds. It would have been impossible to write this with any other set of coauthors. The experience was truly transformative for Levi's understanding of academic collaboration, teaching, writing, and coding. Levi is also grateful to Taylor Oshan and Qunshan Zhao, both of whom indelibly shaped his thinking about writing this book, as well as his colleagues at the University of Bristol, at CARTO in Brooklyn, and at the Alan Turing Institute in London. He would like to thank Maev for her love, patience, and support through all of the long days and nights spent on this project. And, he would also like to thank the Moran family for their profound hospitality, care, and support over these years. Finally, Levi would like to thank his family, Frank, Debora, Zach, and Atticus, for their love and inspiration.

Sergio J. Rey $32.77°, -117.07°$
Dani Arribas-Bel $53.40°, 2.96°$
Levi J. Wolf $51.46°, 2.60°$
Spring 2023

Taylor & Francis
Taylor & Francis Group
http://taylorandfrancis.com

Author/editor biographies

Sergio Rey Sergio Rey is Professor of Geography and Founding Director of the Center for Open Geographical Science at San Diego State University. Rey is the creator and lead developer of the Open-Source package STARS: Space-Time Analysis of Regional Systems as well as co-founder and lead developer of PySAL: A Python Library for Spatial Analysis. He is an elected fellow of the Regional Science Association International, a fellow of the Spatial Econometrics Association, and has served as the Editor of the International Regional Science Review from 1999-2014, editor of Geographical Analysis 2014-2017, and the president of the Western Regional Science Association.

Dani Arribas-Bel is a Professor in Geographic Data Science at the Department of Geography and Planning of the University of Liverpool (UK), and Deputy Programme Director for Urban Analytics at the Alan Turing Institute, where he is also ESRC Fellow. At Liverpool, he is a member of the Geographic Data Science Lab, and directs the MSc in Geographic Data Science.

Levi John Wolf is a Senior Lecturer/Assistant Professor in Quantitative Human Geography at the University of Bristol's Quantitative Spatial Science Lab, Fellow at the University of Chicago Center for Spatial Data Science, an Affiliate Faculty at the University of California, Riverside's Center for Geospatial Sciences, and Fellow at the Alan Turing Institute. He works in spatial data science, building new methods and software to learn new things about social and natural processes.

Taylor & Francis
Taylor & Francis Group
http://taylorandfrancis.com

List of Figures

1.1 A geographic table as a GeoDataFrame. 7
1.2 Surface data structures for field data models. 8
1.3 Spatial graph representation of adjacency relations. 9

2.1 Galileo's drawings of Jupyter and the Medician stars, showing the power of diagrams inside of scientific texts. 15
2.2 Embedding rich media in a notebook. 20
2.3 This book's logo, built from Stamen Toner map tiles and from code. . 22
2.4 An annotated view of the JupyterLab interface 23
2.5 The authentification screen for Jupyter notebooks 32

3.1 Map of the world made using the GeoDataFrame.plot() method. . . . 38
3.2 Plotting centroids and boundaries of polygon geometries. 40
3.3 Plotting Bolivia based on a query. 41
3.4 Plotting Indonesia via a query. 42
3.5 Population surface of Sao Paulo, Brazil 47
3.6 Population surface of Sao Paulo, Brazil omitting NAN values. 48
3.7 OSMNX graph for a street network. 50
3.8 Combining points with Contextily. 54
3.9 Digital Elevation Model as a raster. 56
3.10 San Diego California census tracts. 57
3.11 DEM clipped to San Diego 58
3.12 Digital elevation model estimates by census tract, San Diego. 60
3.13 Point locations of Tokyo Photographs. 61
3.14 Point locations of Tokyo Photographs, and Point Density as a Surface. 62

4.1 A three-by-three grid of squares. 69

4.2 Grid cells connected by a red line are 'neighbors' under a 'Rook' contiguity rule. Code generated for this figure is available on the web version of the book. . . . 70
4.3 Grid cells connected by a red line are considered 'neighbors' under 'Queen' contiguity. Code generated for this figure is available on the web version of the book. . . . 73
4.4 Histogram of cardinalities (i.e., the number of neighbors each cell has) in the Queen grid. . . . 75
4.5 The Queen contiguity graph for San Diego tracts. Tracts connected with a red line are neighbors. Code generated for this figure is available on the web version of the book. . . . 76
4.6 Cardinalities for the Queen contiguity graph among San Diego tracts. . 77
4.7 Cardinalities for the Rook contiguity graph among San Diego tracts. . 78
4.8 Centroids of some tracts in San Diego are (nearly) evenly spaced. . . . 82
4.9 A Gaussian kernel centered on two different tracts. . . . 83
4.10 The three graphs discussed above are shown side-by-side. Code generated for this figure is available on the web version of the book. . . . 90
4.11 Median household incomes in San Diego. . . . 92
4.12 Diferences between median incomes among neighboring (and non-neighboring) tracts in San Diego. . . . 95
4.13 Differences between neighboring incomes for the observed map (orange) and maps arising from randomly reshuffled maps (black) of tract median incomes. Code generated for this figure is available on the web version of the book. . . . 97
4.14 The two starkest differences in median household income among San Diego tracts. Code generated for this figure is available on the web version of the book. . . . 98

5.1 Distribution of per capita GDP across 1940s Mexican states . . . 107
5.2 Absolute deviation around class medians. Alternative classification schemes, Mexican state per capita GDP in 1940. . . . 118
5.3 Assignment differences between alternative classification schemes, Mexican state per capita GDP in 1940. . . . 119
5.4 Quantile choropleth, Mexican state per capita GDP in 1940. . . . 121
5.5 Quantile choropleth with black borderlines, Mexican state per capita GDP in 1940. . . . 123
5.6 Divergent palette, Mexican state per capita income rank change. . . . 124
5.7 (Incorrect) sequential palette, Mexican regions. . . . 125
5.8 Qualitative palette, Mexican regions. . . . 126
5.9 Choropleth map colored to focus on areas of southern Mexico eligible for a target policy, showcasing user-defined map classifications. . . . 127
5.10 User-defined palette, `pandas` approach. . . . 128
5.11 Pooled quantile classification of per capita GDP for 1940, 1960, 1980, and 2000, Mexican states. . . . 130

6.1 Percentage of voters wanting to leave the EU in the 2016 UK Referendum known as the 'Brexit' vote. . . . 139
6.2 Vote to leave the EU and its spatial lag. . . . 142
6.3 Places with a majority voting leave in the Brexit vote . . . 144
6.4 Brexit vote, % leave Moran Scatterplot. . . . 148
6.5 Brexit vote, Moran's I replicate distribution and Scatterplot. . . . 151

7.1 Percentage of voters wanting to leave the EU in the 2016 UK Referendum known as the 'Brexit' vote. . . . 161
7.2 Brexit % leave Moran scatterplot. . . . 162
7.3 Brexit % leave Moran scatterplot with labelled quadrants. . . . 164
7.4 Brexit % Leave vote, observed distribution LISA statistics for all sites. 165
7.5 Brexit % Leave vote, Pct_Leave. LISA (top-left), Quadrant (top-right), Signficance (bottom-left), Cluster Map (bottom-right). . . . 169
7.6 Brexit Leave vote, Pct_Leave, Getis-Ord G (left) and G* (right) statistics. 176
7.7 Colormap for Local Moran's I Maps, starting with non-significant local scores in grey,and proceeding through high-high local statistics, low-high, low-low, then high-low. . . . 180
7.8 LISA map for Sao Paulo population surface. . . . 181

8.1 Tokyo photographs jointplot showing the longitude and latitude where photographs were taken. . . . 189
8.2 Tokyo jointplot showing longitude and latitude of photographs with a basemap via contextily. . . . 190
8.3 Tokyo photographs two-dimensional histogram built with hexbinning. 191
8.4 Tokyo photographs kernel density map. . . . 193
8.5 Tokyo photographs mean and median centers. . . . 195
8.6 Tokyo photographs standard deviational ellipse. . . . 197
8.7 Concave hull and (green) and convex hull (blue) for a subset of Tokyo photographs, with the bounding circles for the concave hull (red). . . . 200
8.8 Alpha shape/concave hull, convex hull, minimum rotated rectangle, minimum bounding rectangle, and minimum bounding circle for the Tokyo photographs. . . . 204
8.9 Observed locations for Tokyo Photographs and random locations around Tokyo. . . . 206
8.10 Tokyo points, random and observed patterns within the alpha shape. . 207
8.11 Quadrat counts for the Tokyo photographs. . . . 208
8.12 Quadrat counts for the Tokyo photographs. . . . 209
8.13 Quadrat statistics for the random points across constrained to alpha shape of the Tokyo photographs. . . . 210
8.14 Tokyo points and nearest neighbor graph. Code generated for this figure is available on the web version of the book. . . . 211
8.15 Tokyo points, Ripley's *G* Function. Code generated for this figure is available on the web version of the book. . . . 212

8.16 Tokyo points, Cluster vs. non-cluster points. Code generated for this figure is available on the web version of the book. 213
8.17 Tokyo points, DBSCAN clusters. 216
8.18 Tokyo points, clusters with DBSCAN and minp=0.01. 218

9.1 Distribution of U.S. per capita income at county level in 1969. 226
9.2 Quintiles of per capita income by county, 1969. 228
9.3 The 20-20 ratio for US county incomes. 229
9.4 The Lorenz curve for county per capita income 1969. 232
9.5 Lorenz curves for county per capita incomes since 1969. 233
9.6 Gini coefficients for per capita income since 1969. 235
9.7 Theil index for county per capita income distributions since 1969. . . 236
9.8 Relationship between Gini and Theil indices for county per capita income distributions since 1969. 236
9.9 Moran's I, a measure of spatial autocorrelation, for per capita incomes since 1969 together with pseudo p-values. 239
9.10 Map of census regions in the United States. 241
9.11 Average county per capita incomes among census regions since 1969. . 242
9.12 Inequality indices (Gini, Theil), shown alongside Moran's I, with the Theil decomposition into between-region and within-region components at bottom. 244
9.13 Relationship between the 'near differences' term of the spatial Gini coefficient and Moran's I. The top, as a measure of spatial dissimilarity, should move in an opposite direction to the bottom, which measures spatial similarity (albeit in a different fashion). 249

10.1 The complex, multi-dimensional human geography of San Diego. . . . 255
10.2 A scatter matrix demonstrating the various pair-wise dependencies between each of the variables considered in this section. Each 'facet', or little scatterplot, shows the relationship between the vairable in that column (as its horizontal axis) and that row (as its vertical axis). Since the diagonal represents the situation where the row and column have the same variable, it instead shows the univariate distribution of that variable. 257
10.3 Clusters in the socio-demographic data, found using K-means with k=5. Note that the large eastern part of San Diego actually contains few observations, since those tracts are larger. 262
10.4 Measuring cluster size by the number of tracts per cluster and land area per cluster. 264
10.5 Distributions of each variable for the different clusters. 267
10.6 Distributions of each variable in clusters obtained from Ward's hierarchical clustering. 271
10.7 Two clustering solutions, one for the K-means solution, and the other for Ward's hierarchical clustering. Note that colorings cannot be directly compared between the two maps. 272

10.8 Spatially constrained clusters, or 'regions', of San Diego using Ward's hierarchical clustering. 275
10.9 Regions from a spatially constrained socio-demographic clustering, using a different connectivity constraint. Code generated for this figure is available on the web version of the book. 276

11.1 Distributions of prediction errors (residuals) for the basic linear model. Residuals for coastal Airbnbs are generally positive, meaning that the model under-predicts their prices. 289
11.2 Boxplot of prediction errors by neighborhood in San Diego, showing that the basic model systematically over- (or under-) predicts the nightly price of some neighborhoods' Airbnbs. 291
11.3 The relationship between prediction error for an Airbnb and the nearest Airbnb's prediction error. This suggests that if an Airbnb's nightly price is over-predicted, its nearby Airbnbs will also be over-predicted. . . . 292
11.4 Map of clusters in regression errors, according to the Local Moran's I_i. 293
11.5 A map showing the 'Distance to Balboa Park' variable. 295
11.6 The relationship between prediction error and the nearest Airbnb's prediction error for the model including the 'Distance to Balboa Park' variable. Note the much stronger relationship here than before. 297
11.7 Neighborhood effects on Airbnb nightly prices. Neighborhoods shown in grey are 'not statistically significant' in their effect on Airbnb prices. 304
11.8 Distributions showing the differences between coastal and non-coastal prediction errors. Some 'random' simulations are shown in black in each plot, where observations are randomly assigned to either 'Coastal' or 'Not Coastal' groups. 321
11.9 Correlogram showing the change in correlation between prediction error at an Airbnb and its surroundings as the number of nearest neighbors increases. The null hypothesis, where residuals are shuffled around the map, shows no significant correlation at any distance. 323

12.1 Convex hull of the Airbnbs in San Diego. 329
12.2 Points of interest (POIs) and Airbnbs in San Diego. 331
12.3 Number of POIs within 500 meters of each Airbnb. 335
12.4 Digital elevation model of the San Diego area. 336
12.5 Elevation above sea level at each Airbnb. 338
12.6 Example grid showing the coordiantes used for interpolation. 340
12.7 Grid underlaid Airbnb locations used for interpolation. 341
12.8 Predicted Airbnb price using ten nearest neighbor interpolation. 342
12.9 Predicted nightly price using a varying number of nearest neighbors. Note the plot smooths considerably as more neighbors are added. . . . 343
12.10 Focus on downtown San Diego predictions for nearest neighbor interpolation. 345

12.11 Interpolation of areal information to a different geometry. The Uber H3 hexagon grid is shown in the middle, and the interpolated values for population are shown on the right. 350
12.12 Interpolation of population density from Census Tracts to Uber H3 Hexagons. 351
12.13 Number of Airbnbs within 500 meters of each listing. 353
12.14 Relationship between the number of bedrooms at an Airbnb and the typical number of bedrooms among nearby Airbnbs. 354
12.15 Relationship between the size of Airbnbs between successive distance buffers around an Airbnb. 360
12.16 Clusters in the locations of Airbnbs within San Diego. 362
12.17 Boxplot of price by detected 'competition cluster.' The clusters vary significantly in prices and could be used to train a model. 363

Part I
Building Blocks

Geography is a longstanding academic discipline, so why do we need fancy new concepts, methods, and algorithms? Indeed, why is this book about geographic *data science* and not some other kind of quantitative study in geography, such as *geocomputation* or *Geographic Information Science*? This section addresses these questions by outlining the conceptual and practical fundamentals of *geographic data science*, as well as a few of the innovations and important new frames of reference that make geographic data science distinct from its precursors.

First, in Chapter 1, we discuss the fundamental differences in how data science is *done*. Reproducible, literate and interactive programming environments have seriously changed the game for how analysis is done. Second, we outline the fundamentals of geographic theory for data scientists. The main distinctions between geographic *models* and the *data structures* that represent them are explained in Chapter 2. The linkages between these models and structures are also discussed. Then, starting in Chapter 3, we show how these notions translate into patterns to read/write/represent geographic data formats. Finally, we close this part by discussing how to represent and store geographical relationships in an efficient data structure. Together, this provides a comprehensive overview of the main models of geographical processes, as well as the nuts and bolts of how to interact with geographical data.

Taylor & Francis
Taylor & Francis Group
http://taylorandfrancis.com

1 Geographic Thinking for Data Scientists

Data scientists have long worked with geographical data. Maps, particularly, are a favorite kind of "infographic" now that we are in the age of responsive web cartography. While we discuss a bit about computational cartography in the chapter on choropleth mapping, it's useful to think a bit more deeply about how geography works in the context of data science. So, this chapter delves a bit into "geographic thinking," which represents the collected knowledge geographers have about why geographical information deserves special care and attention, especially when using geographic data in computations.

1.1 Introduction to geographic thinking

Geographical data has two very useful traits. First, geographic data is ubiquitous. Everything has a *location* in space-time, and this location can be used directly to make better predictions or inferences. But, akin to how "time" is more than a clock position, geography is more than an Earth position: location allows you to understand the *relations* between observations. It is often the *relations* that are useful in data science because they let us *contextualize* our data, building links within our existing data and beyond to other relevant data. As argued by the geographer Waldo Tobler, near things are likely to be more related than distant things, both in space and in time. Therefore, if we learn from this contextual information appropriately, we may be able to build better models.

Speaking of models, it is important to discuss how "location" and "relation" are represented. As the classic saying about statistical models goes,

> All models are wrong, but some are useful [Box76]

In this, the author (statistician George Box) suggests that models are simplified representations of reality. Despite the fact that these representations are *not* exactly correct in some sense, they *are* useful in understanding what is important about a statistical process. Reality is so complex that we simply cannot capture all of the interactions and feedback loops that exist in our model. And, indeed, even if we could, the model would not be *useful*, since it would be so complex that it would be unlikely that any individual could understand it in totality or that a computer could estimate it. Thus, simplification is necessary.

In a similar fashion, we paraphrase geographer Keith Ord in suggesting:

> All maps are wrong, but some are useful [Ord10] (p. 167)

Like a statistical model, a *map* is a *representation* of the underlying geographical process, but is not the process itself. Despite the fact that these representations are *not* exactly correct in some sense, they *are* useful in understanding what is important about a geographical process. In this text, we will use the term "data model" to refer to how we conceptually represent a geographical process. We'll use "data structure" in later sections to refer to how geographic data is represented in a computer. Below, we discuss a few common geographic data models and then present their links to typical geographic data structures.

1.2 Conceptual representations: models

It is often challenging to develop a useful conceptual representation for geographic things. For example, maps of population density generally require that we count the number of people who live within some specified "enumeration area," and then we divide by the total area. This represents the density of the area as a constant value over the entire enumeration unit. But, in fact, people are discrete: we each exist only at one specific point in space and time. So, at a sufficiently fine-grained scale of measurement (in both time and space), the population density is zero in most places and times! And, in a typical person's day, they may move from work to home, possibly moving through a few points in space-time in the process. For instance, most shopping malls have few (if any) residents, but their population density is very high at specific points in time, and they draw this population from elsewhere.

This example of population density helps illustrate the classic data models in geographic information science. Geographic processes are represented using *objects*, *fields*, and *networks*.

- *Objects* are discrete entities that occupy a specific position in space and time.
- *Fields* are continuous surfaces that could, in theory, be measured at any location in space and time.
- *Networks* reflect a set of *connections* between objects or between positions in a field.

In our population density example, an "enumeration unit" is an object, as is a person. The field representation would conceptualize density as a smooth, continuous surface containing the total number of persons at all locations. The network representation would model the inter-related system of places whose densities arise from people moving around.[1]

The differences between these three representations are important to understand because they affect what kinds of *relations* are appropriate. For instance, the relationships among geographical processes with objects can be modelled using simple distances. Near objects might then be "strongly related," and distant objects "weakly related." Alternatively, we could consider (or construct) a network that relates the objects based on their interactions. Geographical processes with networks must account for this *topology*, or structure of the connections between the "nodes" (i.e., the origins or destinations). We cannot assume that every node is connected, and these connections also cannot be directly determined from the nodes alone. For example, two subway stations may be very far apart but could be connected by a frequent direct express train; given their connectivity, the raw distances (treating stops as geographic objects) may not be a good indication of their true geographic relationship. Finally, in a field, measurements can occur *anywhere*, so models need to account for the hypothetical realizations that could happen in the unobserved space between points of measurement.

These kinds of structures, in turn, arise directly from how processes are conceptualized and what questions the analyst seeks to answer. And, since the measurement of a process is often beyond the analyst's control, it is useful to recognize that *how a geographical process actually operates* (that is, its "causal" or "generative" structure) can be different from *how we are actually able to measure it*. In the subsequent sections, we discuss the common frames of measurement you may encounter, and the traditional linkages between data model and data structure that are found in classical geographic information systems.

1.3 Computational representations: data structures

Above, we have discussed how data models are abstractions of reality that allow us to focus on the aspects of a process that we are interested in and measure them in a way that helps us answer the questions we care about. In this context, models are one piece of a broader process of simplification and operationalization that turns reality into representations suitable for computers and statistics. This is necessary for us to tap into computers' analytical capabilities. Data models clarify our thinking about which parts of the real world are relevant and which we can discard. Sometimes, they even guide how we record or measure those aspects.

[1] A useful reference on the topic of common models in geographic information science is Goodchild et al. 2007[GYC07], who focus on establishing a very general framework with which geographic processes can be described, and which inspires our present framework.

However, most of data and geographical science is *empirical*, in that it consists of the analysis and interpretation of measurements. To get to analysis, we require one further step of operationalization and simplification. The object, network, and field data models discussed in the previous section are usually still too abstract to help us in this context. So, we pair these data models with other constructs that tell us how quantities can be stored in computers and made amenable for analysis. This is the role that "data structures" play.

Data structures are digital representations that connect data models to computer implementations. They form the middle layer that connects conceptual models to technology. At best, they represent the data model's principles as well as what is technologically possible. In doing so, data structures enable data models to guide the computation. At worst, a bad match between data structure and model can make it challenging to design or execute analyses and make it necessary to transfer to a different data structure.

While we generally think of the fidelity a data structure has for a given data model, this relationship can also run in the opposite direction: once established, a technology or data structure can exert an important influence about how we see and model the world. This is not necessarily a bad thing. Embedding complex ideas in software helps widen the reach of a discipline. For example, desktop GIS software in the 1990s and 2000s made geographic information useful to a much wider audience. It made it so that geographic data *users* did not necessarily require specific training to work with geographic data or conduct spatial analysis. It did this largely by standardizing the data structure central to geographic analysis, the "geographic matrix" [Ber64] which we now call a "geotable." This made it easy to represent most geographical processes in a format that people could easily understand.

However, making conceptual decisions based on technological implementations can be limiting. In the 1990s, Mark Gahegan proposed the concept of "disabling technology" to express how the technological systems we use may affect the structure of our thinking [Gah99]. As a metaphor, we can think of technology as a pair of eyeglasses and data models as the "instructions" to build lenses: if all we use to look at the world is the one pair we already have, we miss all the other ways of looking at the world that arise if we built lenses differently. Of course, one may believe they can operate *without* lenses at all, but even healthy eyes with perfect vision contain optical imperfections in their lenses!

So, "*what main geographic data structures should the data scientist care about?*", we hear you ask. Of course, as with everything in technological, this evolves. In fact, as we will see below in this chapter, much is changing rapidly, redefining how we translate conceptual models into computational constructs to hold data. However, there are a few standard data structures that have been around for a long time because they are so useful. In particular, we will cover three of them: geographic tables, surfaces (and cubes), and spatial graphs. We have selected these because each serves as a technological mirror for the concepts discussed in the previous section.

	pop_est	continent	name	iso_a3	gdp_md_est	geometry
0	920938	Oceania	Fiji	FJI	8374.0	MULTIPOLYGON (((180.00000 -16.06713, 180.00000...
1	53950935	Africa	Tanzania	TZA	150600.0	POLYGON ((33.90371 -0.95000, 34.07262 -1.05982...
2	603253	Africa	W. Sahara	ESH	906.5	POLYGON ((-8.66559 27.65643, -8.66512 27.58948...
3	35623680	North America	Canada	CAN	1674000.0	MULTIPOLYGON (((-122.84000 49.00000, -122.9742...
4	326625791	North America	United States of America	USA	18560000.0	MULTIPOLYGON (((-122.84000 49.00000, -120.0000...
5	18556698	Asia	Kazakhstan	KAZ	460700.0	POLYGON ((87.35997 49.21498, 86.59878 48.54918...
6	29748859	Asia	Uzbekistan	UZB	202300.0	POLYGON ((55.96819 41.30864, 55.92892 44.99586...
7	6909701	Oceania	Papua New Guinea	PNG	28020.0	MULTIPOLYGON (((141.00021 -2.60015, 142.73525 ...
8	260580739	Asia	Indonesia	IDN	3028000.0	MULTIPOLYGON (((141.00021 -2.60015, 141.01706 ...
9	44293293	South America	Argentina	ARG	879400.0	MULTIPOLYGON (((-68.63401 -52.63637, -68.25000...

Fig. 1.1: A geographic table as a GeoDataFrame.

Geographic tables store information about discrete *objects*. Tables are two-dimensional structures made up of rows and columns. Each row represents an independent object, while each column stores an attribute of those objects. Geographic tables are like typical data tables where one column stores geographic information. The tabular structure fits well with the object model because it clearly partitions space into discrete entities, and it assigns a geometry to each entity according to their spatial nature. More importantly, geographic tables can seamlessly combine geographic and non-geographic information. In this data structure, geography becomes simply "one more attribute", when it comes to storage and computer representation.

This is powerful because there is wide support in the world of databases for tabular formats. Geographic tables integrate spatial data into this typically non-spatial domain and allow it to leverage much of its power. Technically speaking, geographic tables are widely supported in a variety of platforms. Popular examples include: PostGIS tables (as a geographic extension of the PostgreSQL database), R's `sf` data frames or, more relevant for this book, Python's `GeoDataFrame` objects, provided by `geopandas` (shown in Figure 1.1). Although each of them has its own particularities, they all represent implementations of an object model.

Surface data structures are used to record empirical measurements for *field* data models. For a field (in theory), there is an infinite set of locations for which a field may be measured. In practice, fields are measured at a finite set of locations. This aim to represent continuity in space (and potentially time) is important because it feeds directly into how surface data are structured. In practice, surfaces are recorded and stored in uniform grids, or arrays whose dimension is closely linked to the geographic extent of the area they represent. In geography, we generally deal with arrays with two or more dimensions. Unlike geographic tables, the arrays used in a surface data structure use both rows and columns to signify location, and they use cell values to store information about that location. For example, a surface for air pollution will be represented as an array where each row will be linked to the measured pollutant level across a specific latitude, and each column for a specific longitude. If we want to represent more than one phenomenon (e.g., air pollution *and* elevation), or the same phenomenon at different

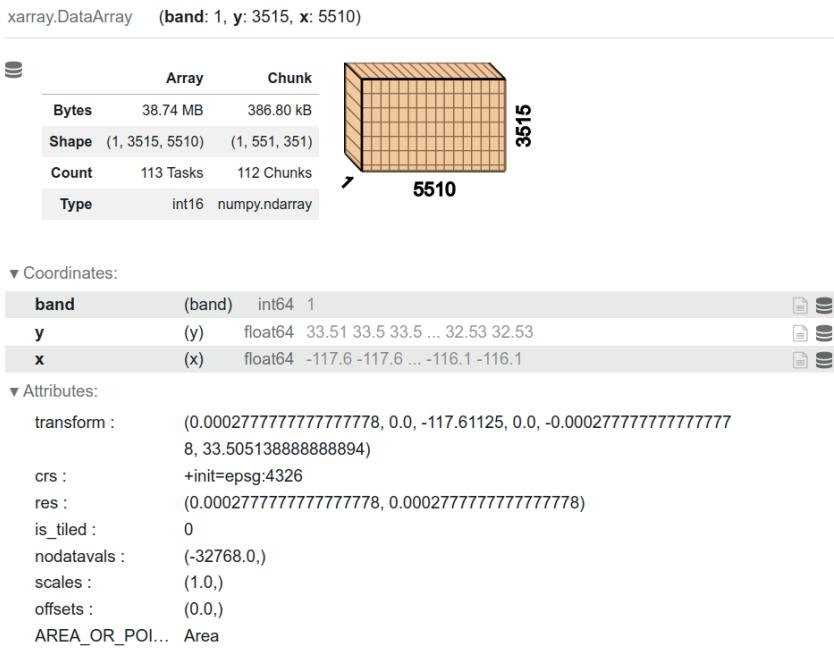


Fig. 1.2: Surface data structures for field data models.

points in time, we will need different arrays that are possibly connected. These multi-dimensional arrays are sometimes called *data cubes* or *volumes*. An emerging standard in Python to represent surfaces and cubes is that provided by the `xarray` library, shown in Figure 1.2.

Spatial graphs capture relationships between objects that are mediated through space. In a sense, they can be considered geographic *networks*, a data structure to store topologies. There are several ways to define spatial relationships between features, and we explore many of them in Chapter 4. The important thing to note for now is that, whichever rules we follow, spatial graphs provide a way to encode them into a data structure that can support analytics. As we will see throughout the book, the range of techniques that rely on these topologies is pretty large, spanning from exploratory statistics of spatial autocorrelation (Chs. 6 and 7), to regionalization (Ch. 10) to spatial econometrics (Ch. 11). Ironically, each of these fields and others in computer science and mathematics have come up with their own terminology to describe similar structures. Hence, when we talk or read about spatial weights matrices, adjacency matrices, geo-graphs, or spatial networks, we are thinking of very similar fundamental structures deployed in different contexts.

Spatial graphs record information about how a given observation is spatially connected to others in the dataset (Figure 1.3). For this reason, they are an obvious complement to geographic tables, which store information about individual observations in isolation.

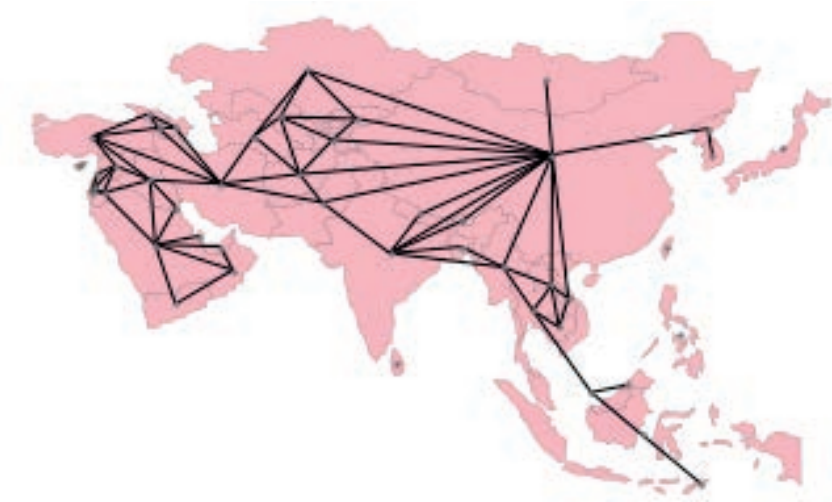


Fig. 1.3: Spatial graph representation of adjacency relations.

Spatial graphs can also be derived from surfaces but here the situation is slightly different because, although surfaces record discrete measurements, they usually relate to a continuous phenomenon. In theory, one could take these measurements at any point in space, so spatial graph of a surface should have an infinite number of observations. In practice however spatial graphs *are* now sometimes used with grids because, as we will discuss in the following section, the connections and distinctions between data models and structures are changing very quickly. Since many fields have theoretical constructs that resemble spatial graphs, there exist several slightly different data structures that store them both in memory and on disk. In this book, we will focus on graph objects provided by the `networkX` library as well as the spatial weights objects in `pysal` which rely to a great extent on sparse adjacency matrix data structures from `scipy`.

The term *spatial graph* is sometimes interchangeably used with that of *spatial network*. This is of course a matter of naming conventions and, to the extent it is only that, it is not very important. However, the confusion can sometimes reflect a more profound misconception of what is being represented. Take the example of the streets in a city or, of the interconnected system of rivers in a catchment area. Both are usually referred to as networks (e.g., city network or river network), although in many cases what is being recorded is actually a collection of objects stored in a geographic table. To make the distinction clear, we need to think about what aspect of the street layout or the river system we want to record. If it is the exact shape, length and location of each segment or stream, this resembles much more a collection of independent lines or polygons that happen to "touch each other" at their ends. If what we are interested in is to understand how each segment or river *is related* to each other, who is connected to whom and how the individual connections comprise a broader interconnected system, then a spatial graph is a more helpful structure to use. This dichotomy of the object versus the graph

is only one example of a larger point: the *right* link between a data model and data structure does not solely depend on the phenomenon we are trying to capture, but also our analytical goal.

1.4 Connecting conceptual to computational

Now that the distinction between the *conceptual* data model and *computational* data structure is clear, we should explore the ways in which these are traditionally aligned. In presenting this traditional relationship between data model and data structure, we also seek to highlight the recent developments where this traditional mapping is breaking down.

First, the main conceptual mapping of data model to data structure is inherited from advances made in computer graphics. This traditional view represents fields as rasters and objects as vector-based tables. In this mode of analysis, there is generally no space for *networks* as a first-class geographic data structure. They are instead computed on the fly from a given set of objects or fields.

The separation of raster/image from vector/table and general omission of networks both stem from implementation decisions made in the one of the first commercially-successful geographic information systems, the Environmental System Research Institute (ESRI)'s ARC/INFO package. This was a command-line precursor to modern graphical information processing systems, such as the free and open source *QGIS* or the ESRI's *ArcGIS* packages. This association between field-and-surface, object-and-table is sometimes called the "desktop view" of geographic information due to the dominance of these graphical GIS packages, although some argue that this is fundamental to geographic representation [GYC07] and cannot be transcended.

1.4.1 This categorization is now breaking up (data is data)

Current trends in geographic data science suggest that this may not necessarily be the case, though. Indeed, contemporary geographic data science is moving beyond these mappings in two very specific ways.

First, statistical learning methods are getting very good at efficiently translating data between different representations. If nothing else, the rise of machine learning has generated extremely efficient "black box" prediction algorithms across a wide class of problems. If you can handle the fact that these algorithms generally are not *explainable* in their internal operations, then these methods can generally provide significant improvements in prediction problems. Change of support problems, which arise when attempting to move data from one geographical data structure to another, are wholly focused on *accuracy*; the interpretation of a change of support algorithm is generally never of substantive interest. Therefore, machine learning has made it much easier to

move between data structures, which reduces the importance of picking the "right" representation from the outset. The "costs" of moving between data structures have been lowered.

Second, this means that there are an increasingly large number of attempts to find a "fundamental" underlying scale or representation that can be used for interchange between geographic data structures. This has largely grown out of corporate data science environments, where transferring all geographic data to a single underlying representation has significant benefits for data storage, computation, and visualization. Projects such as Uber's "Hierarchical Hexagonal Geospatial Index," or "h3" for short, provide something like this, and many other internal proprietary systems (such as the S2 "Earth cube" by Google) serve the same purpose. In addition, projects like WorldPop [Tat17] are also shifting the traditional associations between "types" of data and the representations in which they are typically made available. Population data is generally presented in "object-based" representations, with the census enumeration units as "objects" and the population counts as features of that object. WorldPop (and others like it), though, have shifted to presenting population data in worldwide rasters at varying resolutions, conceptualizing population *distribution* as a continuous field over which population *counts* (both modeled and known) can be presented.

These are only two examples of the drive towards field-based representations in contemporary geographic data science, and they will no doubt change as rapidly as the field itself. However, the incentives to create new representations will likely only intensify, as standard, shared architectures increasingly dominate the free and open source scientific software ecosystem. While scientific results stress the issues of a one-size-fits-all scale of analysis and indicate that results often do change when the geographical scale of analysis changes, this may not matter in most practical applications where explanation is less important than prediction.

1.5 Conclusion

This chapter has discussed the main conceptual *data models* for geographical processes and their typical implementations in computational *data structures*. Objects, which are generally used to represent distinct "bounded" agents, are typically represented by "vector" data structures through a combination of points, lines, polygons. Fields, representations of the continuous smooth surfaces, are typically represented by "raster" data structures that look like images, with *pixels* recording values at each possible site and *bands* recording the kind of data recorded. Networks, which reflect relationships between objects, have typically been core to the data models of many geographical processes, but have not historically been represented easily in many of the data *structures* common in geographic information systems. This is changing, however, as networks become (or return to being [UvanMeeteren21]) more central to our computational and conceptual understanding of geographical systems.

By recognizing that the conceptual data model can be distinct from the computational data structure, we can move more easily between different data structures. Further, recent developments in computation and data storage are breaking down the traditional connections between data models and data structures. These have involved focusing much more intently on *change of support problems* that arise when converting between geographic data structures and attempts at finding a single "canonical" geographic data structure. Despite these trends, the main choice about the *data model* can be made independently of the *data structure*, so it is important to be very clear about the "entities" involved in your problem and how they are related. Be attentive to what the entities you're interested in analyzing or predicting *are*, and be aware of when that may be at odds with *how they are measured*. While it is increasingly common to use hexagonal or square grids to represent data, be attentive to how this obscures the *actual* process you seek to measure or behavior you aim to model. Be prepared to answer: yes, we can grid this, but should we? The answer will depend on your goal.

Overall, this chapter provides a grounding for the concepts and language used in the rest of the book to describe geographic processes, the datasets that describe them, and the models that analyze them. In some subsequent chapters, we will integrate information from many different data structures to study some processes; in other chapters, this will not be necessary. Recognizing these conceptual differences will be important throughout the book.

1.6 Next steps

For further information on the fundamental thinking and concepts behind geographic information, it helps to consider the influential paper by Goodchild, Yuan, and Cova about the fundamental concepts in geographic representation:

Goodchild, Michael, May Yuan, and Thomas Cova. 2007. "Towards a general theory of geographic representation in GIS." *International Journal of Geographical Information Science* 21(3): 329-260.

In addition, a very comprehensive conceptual overview is provided by Worboys and Duckham. Those dissuaded by its publication date will be surprised by the freshness of the conceptual and theoretical perspectives provided therein:

Worboys, Michael and Matt Duckham. 2004. *GIS, A Computing Perspective*. CRC Press: Boca Raton.

Finally, for more information on where the geographic data science perspective *came from* both historically and conceptually, consult Gahegan's mannifesto:

Gahegan, Mark. 1999. "Guest Editorial: What is Geocomputation?" *Transactions in GIS* 3: 203-206.

2 Computational Tools for Geographic Data Science

This chapter provides an overview of the scientific and computational context in which the book is framed. Many of the ideas discussed here apply beyond geographic data science but, since they have been a fundamental pillar in shaping the character of the book, they need to be addressed. First, we will explore debates around "Open Science," its origins, and how the computational community is responding to contemporary pressures to make science more open and accessible to all. In particular, we will discuss three innovations in open science: computational notebooks, open-source packages, and reproducible science platforms. Having covered the conceptual background, we will turn to a practical introduction of the key infrastructure this book relies on: Jupyter Notebooks and JupyterLab, Python packages, and a containerized platform to run the Python code in this book.

2.1 Open science

The term Open Science has grown in popularity in recent years. Although it is used in a variety of contexts with slightly different meanings [CruwellvDE+19], one statement of the intution behind Open Science is that the scientific process, at its core, is meant to be transparent and accessible to anyone. In this context, openness is not to be seen as an "add-on" that only makes cosmetic changes to the general scientific approach, but as a key component of what makes our scientific practices *Science*. Indeed the scientific process, understood as one where we "stand on the shoulders of giants" and progress through dialectics, can only work properly if the community can access and study both results *and* the process that created them. Thus, transparency, accessibility, and inclusiveness are critical for good science.

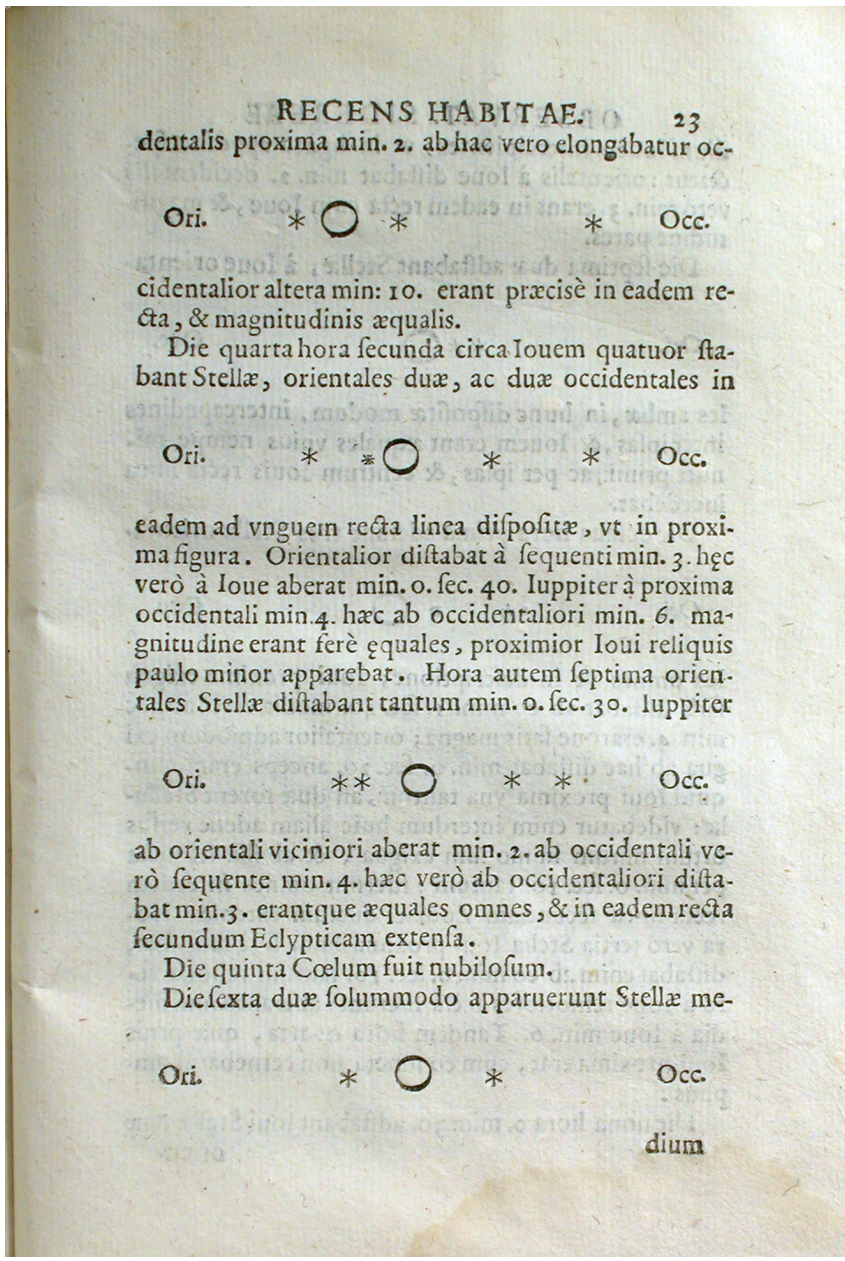
To better understand the argument behind contemporary Open Science, it is useful to understand its history. The idea of openness was a core committment of early scientists. In fact, that was one of the key differences with their contemporary "alchemists" who, in many respects, were working on similar topics albeit in a much more opaque way [Nie20]. Scientists would record the field or lab experiments on paper notebooks or diaries, providing enough detail to, first, remember what they had done and how they had arrived at their results; but also to ensure other members of the scientific community could study, understand, and replicate their findings. One of the most famous of these annotations is Galileo's drawings of Jupiter (source) and the Medicean stars, shown in Figure 2.1.

There is a growing perception that much of this original scientific ethos—operating transparently and arguing with accessible materials—has been lost. A series of recent high profile scandals have even prompted some to call a state of crisis [Ioa07]. This "crisis" arises because the analyses that scientists conduct are difficult to repeat. Sometimes, it is even impossible to clearly understand the steps that were taken to arrive at results.

Why is there a sense that Science is no longer open and transparent in the way Galileo's notebooks were? Although certainly not the only or even the most important factor, technology plays a role. The process and workflow of original scientists relied on a set of analog technologies for which a parallel set of tools was developed to keep track of procedures and document knowledge. Hence, the paper notebooks where biologists drew species, or chemists painstakingly detailed each step they took in the lab. In the case of social sciences, this was probably easier: quantitative data was scarce and much of the analysis relied either on math to minimize the amount of computation required [EH16] or small datasets which could be directly documented in the publication itself.

However, science has evolved a great deal since then, and much of the experimental workflow is dominated by a variety of machinery, most prominently by computers. Most of the science done today, at some point in the process, takes the form of operations mediated through software programs. In this context, the traditional approach of writing down every step in a paper notebook separates from the medium in which these steps actually take place.

The current state of science, in terms of transparency and openness, is prompting for action [Rey09]. On the back of these debates, the term "reproducibility" is also gaining traction. Again, this is a rather general term but in one popular definition [BCK+15], it suggests that scientific results need to be accompanied by enough information and detail so they could be repeated by a third party. Since much of modern science is mediated through computers, reproducibility thus poses important challenges for the computational tools and practices the scientific community builds and relies on. Although there are a variety of approaches, in this book we focus on what we see as an emerging consensus. This framework enables scientists to record and express entire workflows in a way that is both transparent and that fosters efficiency and collaboration.



RECENS HABITAE. 23

dentalis proxima min. 2. ab hac vero elongabatur oc-

Ori. * ○ * * Occ.

cidentalior altera min: 10. erant præcisè in eadem recta, & magnitudinis æqualis.

Die quarta hora ſecunda circa Iouem quatuor ſtabant Stellæ, orientales duæ, ac duæ occidentales in

Ori. * * ○ * * Occ.

eadem ad vnguem recta linea diſpoſitæ, vt in proxima figura. Orientalior diſtabat à ſequenti min. 3. hęc verò à Ioue aberat min. 0. ſec. 40. Iuppiter à proxima occidentali min. 4. hæc ab occidentaliori min. 6. magnitudine erant ferè ęquales, proximior Ioui reliquis paulo minor apparebat. Hora autem ſeptima orientales Stellæ diſtabant tantum min. 0. ſec. 30. Iuppiter

Ori. ** ○ * * Occ.

ab orientali viciniori aberat min. 2. ab occidentali verò ſequente min. 4. hæc verò ab occidentaliori diſtabat min. 3. erantque æquales omnes, & in eadem recta ſecundum Eclypticam extenſa.

Die quinta Cœlum fuit nubiloſum.

Die ſexta duæ ſolummodo apparuerunt Stellæ me-

Ori. * ○ * Occ.

dium

Fig. 2.1: Galileo's drawings of Jupyter and the Medician stars, showing the power of diagrams inside of scientific texts.

We structure our approach to reproducibility in three main layers that build on each other. At the top of this "stack" are *computational notebooks*; supporting the code written in notebooks are *open source packages*; and making possible to transfer computations across different hardware devices and/or architectures are what we term *reproducible platforms*. Let us delve into each of them with a bit more detail before we show practically how this book is built on this infrastructure (and how you too can reproduce it yourself).

2.1.1 Computational notebooks

Computational notebooks are the twenty-first century sibling of Galileo's notebooks. Like their predecessors, they allow researchers, (data) scientists, and computational practitioners to record their practices and steps taken as they are going about their work; unlike the pen and paper approach, computational notebooks are fully integrated in the technological paradigm in which research takes place today. For these reasons, they are rapidly becoming the modern-day version of the traditional academic paper, the main vehicle on which (computational) knowledge is created, shared, and consumed. Computational notebooks (or notebooks, from now on) are spreading their reach into industry practices, being used, for example, in reports. Although, they were designed within a broader scope of application, computational notebooks have several advantages for geographic data science work [BAB20].

All implementations of notebooks share a series of core features. First, a notebook comprises a single file that stores narrative text, computer code, and the output produced by code. Storing both narrative and computational work in a *single file* means that the entire workflow can be recorded and documented in the same place, without having to resort to ancillary devices (like a paper notebook). A second feature of notebooks is that they allow *interactive work*. Modern computational work benefits from the ability to try, fail, tinker, and iterate quickly until a working solution is found. Notebooks embody this quality and enable the user to work interactively. Whether the computation takes place on a laptop or on a data center, notebooks provide the same interface for interactive computing, lowering the cognitive load required to scale up procedures for larger data or more scientists. Third, notebooks have *interoperability* built in. The notebook format is designed for recording and sharing computational work, but not necessarily for other stages of the research cycle. To widen the range of possibilities and applications, notebooks are designed to be easily convertible into other formats. For example, while a specific application is required to open and edit most notebook file formats, no additional software is required to convert them into PDF files that can be read, printed, and annotated without the need of technical software.

Notebooks represent the top layer on the reproducibility stack. They can capture the detail of reproducible work specific to a given project: what data is used, how it is read, cleaned, and transformed; what algorithms are used, how they are combined; how each figure in the project is generated, etc. Stronger guidance on how to write notebooks in

efficient ways is also emerging (e.g., [RBZ+19]), so this too represents an evolving set of practices which we document incompletely in the following sections.

2.1.2 Open source packages

To make notebooks an efficient medium to communicate computational work, it is important that they are concise and streamlined. One way to achieve this goal is to only include the parts of the work that are unique to the application being recorded in the notebook, and to avoid duplication. From this it follows that a piece of code used several times across the notebook, or even across several notebooks, should probably be taken out of the notebook and into a centralized place where it can be accessed whenever needed. In other words, such functionality should be turned into a package.

Packages are modular, flexible, and reusable compilations of code. Unlike notebooks, they do not capture specific applications but abstractions of functionality that can be used in a variety of contexts. Their function is to avoid duplication "downstream" by encapsulating functionality in a way that can be accessed and used in a variety of contexts without having to re-write code every time it is needed. In doing so, packages (or libraries, an interchangeable term in this context) embody the famous hacker motto of DRY: "don't repeat yourself".

Open source packages is software that provides its own code for the user to inspect. In many cases, the user can then modify this code to make their own new software, or redistribute the package so that others may use it. These open source packages fulfill the same functions as any package in terms of modularizing code, but they also enable transparency: any user can access the exposed functionality *and* the underlying code that generates it. For this reason, for code packages to serve Open Science and reproducibility, they need to be open source.

2.1.3 Reproducible platforms

For computational work to be fully reproducible and open, it needs to be possible to replicate it in a different (computational) environment than where it was originally created. This means that it is not sufficient to use notebooks to specify the code that creates the final outputs, or even to rely on open source packages for shared functionality; the computational environment in which the notebook and packages get executed needs to be reproducible too. This statement, which might seem obvious and straightforward, is not always so due to the scale and complexity of modern computational workflows and infrastructures. It is no longer enough for an analysis to just work on the author's laptop, it needs to be able to work on "any laptop" (or computer).

Reproducible platforms encompass the more general aspects that enable open source packages and notebooks to be reproducible. A reproducible platform thus specifies the infrastructure required to ensure a notebook that uses certain open source packages can

be successfully executed. Infrastructure, in this context, relates to lower-level aspects of the software stack, such as the operating system, and even some hardware requirements, such as the use of specific chips such as graphics processing units (GPU). Additionally, a reproducible platform will also specify the versions of packages that are required to recreate the results presented in a notebook, since changes to packages can change the results of computations or break analytical workflows entirely.

Unlike open source packages, the notion of reproducible platforms is not as widespread and generally agreed upon. Its necessity has only become apparent more recently, and work on providing them in standardized ways is less developed than in the case of notebook technology or code packaging and distribution. Nevertheless, some inroads are being made. One area which has experienced significant progress in recent years and holds great promise in this context is container technology. Containers are a lightweight version of a virtual machine, which is a program that enables an entire operating system to run compartmentalized on top of another operating system. Containers allow to encapsulate an entire environment (or platform) in a format that is easy to transfer and reproduce in a variety of computational contexts. The most popular technology for containers nowadays is Docker, and the opportunities that it provides to build transparent and transferrable infrastructure for data science are starting to be explored [Coo17].

2.2 The (computational) building blocks of this book

The main format of this book is the "notebook" and is the first medium in which our content is created and is intended to be presented. Each chapter is written as a separate notebook and can be run interactively. At the same time, we collect all chapters and convert them into different formats for "static consumption" (i.e., read only), either in HTML format for the web, or PDF to be printed in a physical copy. This section will present the specific format of notebooks we use, and illustrate its building blocks in a way that allows you to then follow the rest of the book interactively.

2.2.1 Jupyter notebooks and JupyterLab

Our choice of notebook system is Jupyter [KRKPerez+16]. A Jupyter notebook is a plain text file with the `.ipynb` extension, which means that it is an easy file to move around, sync, and track over time. Internally, it is structured as a plain-text document containing JavaScript Object Notation that records the state of the notebook, so they also integrate well with a host of modern web technologies. Other notebook systems, such as Apache Foundation's "Zeppelin", the Alan Turing Institute's "Wrattler", or ObservableHQ, implement different systems and structures for creating a computational notebook, but all are generally web-based. They often separate the system that executes analytical code from the system that displays the notebook document. In addition, many

notebook systems (including Jupyter), allow computational steps to be (re)run repeatedly out of the order in which they appear. This allows users to experiment with the code directly, but it can be confusing when code is executed out of order. Finally, because of their web-focused nature, many notebook systems allow for the direct integration of visualizations and tables within the document. Overall, notebooks provide a very good environment in which to experiment, present, and explain scientific results.

2.2.1.1 Notebooks and cells

The atomic element that makes up a notebook is called a *cell*. Cells are "chunks" of content that usually contain either text or code. In fact, a notebook overall can be thought of as a collection of code and text cells that, when executed in the correct order, describe and conduct an analysis.

Text cells contain text written in the Markdown text formatting language. Markdown is a popular set of rules to create rich content (e.g., headers, lists, links) from plain text. It is designed so that the plain text version looks similar to the outputted document. This makes it less complex than other typesetting approaches, but this means it also supports fewer features. After writing a text cell, the notebook engine will process the markdown into HTML, which you then see as rich text with images, styling, headers, and so forth. For more demanding or specific tasks, text cells can also integrate LaTeX notation. This means we can write most forms of narrative relying on Markdown, which is more straightforward, and use LaTeX for more sophisticated parts, such as equations. Covering all of the Markdown syntax and structure in detail is beyond the scope of this chapter, but the interested reader can inspect the official GitHub specification of the so-called GitHub-Flavored Markdown [Git] that is adopted by Jupyter notebook.

Code cells are boxes that contain snippets of computer code. In this book, all code will be Python, but Jupyter notebooks are flexible enough to work with other languages.[1] In Jupyter, code cells look like this:

```
# This is a code cell
```

A code cell can be "run" to execute the code it contains. If the code produces an output (e.g., table or figure), this will be produced below the code cell as output. Every time a cell is run, its counter on the notebook interface will go up by one[2]. This counter indicates the order in which a cell is run, in that lower-number cells have been run before higher-number cells. As mentioned before, however, notebooks generally allow the user to "go back" and execute cells again, so there may be lower-number cells that come *after* higher-number cells.

[1] A full list of supported kernels for Jupyter is available at https://github.com/jupyter/jupyter/wiki/Jupyter-kernels

[2] This counter may not always be visible in all of the formats a notebook may be viewed. In this book, cell numbers will not be visible on paper, but they will be visible in the online versions of the notebook.

Fig. 2.2: Embedding rich media in a notebook.

2.2.1.2 Rich content

Code cells in a notebook also enable the embedding of rich (web) content. The `IPython` package provides methods to access as series of media and bring them directly to the notebook environment. Let us see how this can be done practically. To be able to demonstrate it, we will need to *import* the `display` module[3]:

```
import IPython.display as display
```

This makes available additional functionality that allows us to embed rich content. For example, we can include a YouTube clip (Figure 2.2) by passing the video ID[4]:

```
display.YouTubeVideo("iinQDhsdE9s")
```

[3] Skip to the next section if you want to learn more about importing packages.

[4] We regret to inform the reader that the printed page is not the best for displaying YouTube videos, or indeed any web content. So if you are reading this book on paper, the video will not render. We recommend viewing the notebooks online at `geographicdata.science/book` to see the full power of the notebook.

Or we can pass standard HTML code:

```
display.HTML(
    """<table>
<tr>
<th>Header 1</th>
<th>Header 2</th>
</tr>
<tr>
<td>row 1, cell 1</td>
<td>row 1, cell 2</td>
</tr>
<tr>
<td>row 2, cell 1</td>
<td>row 2, cell 2</td>
</tr>
</table>"""
)
```

```
<IPython.core.display.HTML object>
```

Note that this opens the door for including a large number of elements from the web, since an `iframe` of any other website can also be included. Of more relevance for this book, for example, this is one way we can embed interactive maps with an `iframe`:

```
osm = """
<iframe width="425"
        height="350"
        frameborder="0"
        scrolling="no"
        marginheight="0"
        marginwidth="0"
        src="http://www.openstreetmap.org/export/embed.html?
↪bbox=-2.9662737250328064%2C53.400500637844594%2C-2.
↪964626848697662%2C53.402550738394034&amp;layer=mapnik"
        style="border: 1px solid black">
</iframe>
<br/>
<small>
    <a href="http://www.openstreetmap.org/#map=19/53.40153/-2.
↪96545">View Larger Map</a>
</small>
"""
display.HTML(osm)
```

```
<IPython.core.display.HTML object>
```

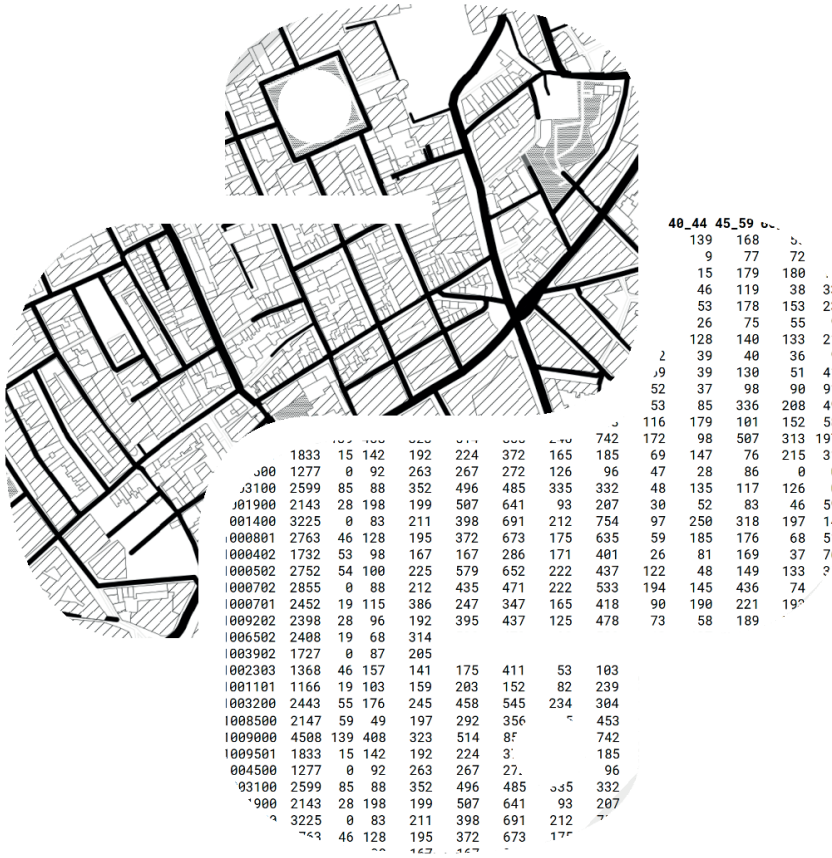


Fig. 2.3: This book's logo, built from Stamen Toner map tiles and from code.

Finally, using a similar approach, we can also load and display local images (Figure 2.3), which we will do throughout the book. For that, we use the `Image` method:

```
path = "../infrastructure/logo/logo_transparent-bg.png"
display.Image(path, width=250)
```

2.2.1.3 JupyterLab

Our recommended way to interact with Jupyter notebooks is through JupyterLab. JupyterLab is an interface to the Jupyter ecosystem that brings together several tools for

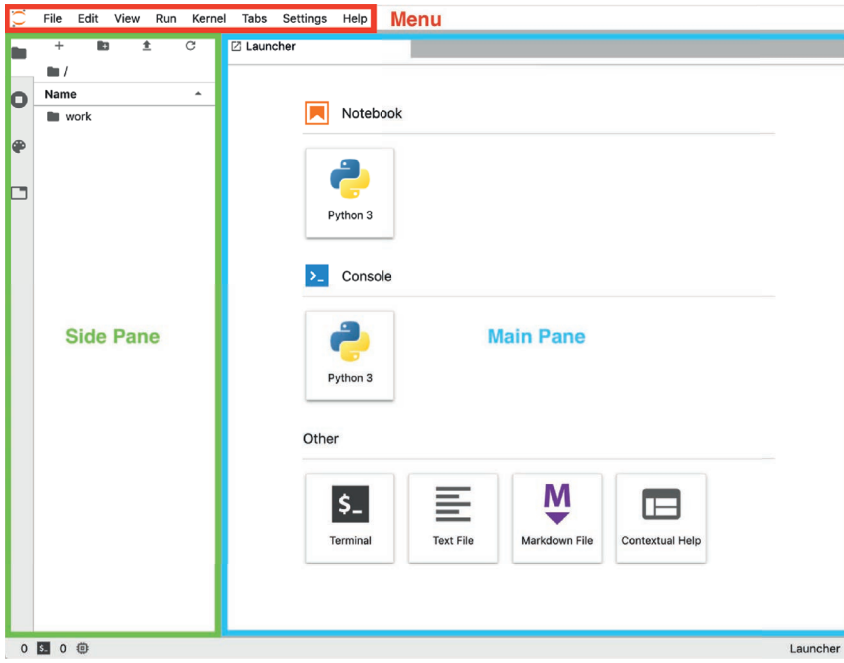


Fig. 2.4: An annotated view of the JupyterLab interface

data science into a consistent interface that enables the user to accomplish most of her workflows. It is built as a web app following a client-server architecture. This means the computation is decoupled from the interface. This decoupling allows each to be hosted in the most convenient and efficient solution. For example, you might be following this book interactively on your laptop. In this case, it is likely both the server that runs all the Python computations you specify in code cells (what we call the *kernel*) is running on your laptop, and you are interacting with it through your browser of preference. But the same technology could power a situation where your kernel is running in a cloud data center, and you interact with JupyterLab from a tablet.

JupyterLab's interface has three main areas. These are shown in Figure 2.4.[5]

At the top, we find a menu bar (red box in the figure) that allows us to open, create and interact with files, as well as to modify the appearance and behavior of JupyterLab. The largest real estate is occupied by the main pane (blue box). By default, there is an option to create a new notebook, open a console, a terminal session, a (Markdown) text file, and a window for contextual help. JupyterLab provides a flexible workspace in that the user can open as many windows as needed and rearrange them as desired by dragging and dropping. Finally, on the left of the main pane we find the side pane (green box), which has several tabs that toggle on and off different auxiliary information. By default, we find a file browser based on the folder from where the session has been launched.

[5] Depending on the version of JupyterLab you are using, the layout and appearance may slightly change.

But we can also switch to a pane that lists all the currently open kernels and terminal sessions, a list of all the commands in the menu (the command *palette*), and a list of all the open windows inside the lab.

2.2.2 Python and open source packages

The main component of this book relies on the Python programming language. Python is a high-level programming language used widely in (data) science. From satellites controlled by NASA[6] to courses in economics by Nobel Prize-winning professors[7], Python is a fundamental component of "consensus" data science [Don17]. This book uses Python because it is a good language for beginners and high-performance science alike. For this reason, it has emerged as one of the main options for Data Science [Eco]. Python is widely used for data processing and analysis both in academia and in industry. There is a vibrant and growing scientific community (through the Scientific Python library `scipy` and the PyData organization), working in both universities and companies, to support and enhance Python's capabilities. New methods and usability improvements of existing packages (also known as libraries) are continuously being released. Within the geographic domain, Python is also very widely adopted: it is the language used for scripting in both the main proprietary enterprise geographic information system, ArcGIS, and the leading open geographic information system, QGIS. All of this means that, whether you are thinking of higher education or industry, Python will be an important asset, valuable to employers and scientists alike.

Python code is "dynamically interpreted", which means it is run on-the-fly without needing to be compiled. This is in contrast to other kinds of programming languages, which require an additional non-interactive step where a program is converted into a binary file, which can then be run. With Python, one does not need to worry about this non-interactive compilation step. Instead, we can simply write code, run it, fix any issues directly, and rerun the code in a rapid iteration cycle. This makes Python a very productive tool for science, since you can prototype code quickly.

2.2.2.1 Open source packages

The standard Python language includes some data structures (such as lists and dictionaries) and allows many basic mathematical operations (e.g., sums, differences, products). For example, right out of the box, and without any further action needed, you can use Python as a calculator. For instance, three plus five is eight:

```
3 + 5
```

```
8
```

[6] For more details, see https://www.python.org/about/success/usa/
[7] For more details, see https://lectures.quantecon.org

Two divided by three is $.6\bar{6}$:

```
2 / 3
```

```
0.6666666666666666
```

And $(3 + 5) * \frac{2}{3}$ is $5\frac{1}{3}$:

```
(3 + 5) * 2 / 3
```

```
5.333333333333333
```

However, the strength of Python as a data analysis tool comes from additional packages, software that adds functionality to the language itself. In this book, we will introduce and use many of the core libraries of the Python ecosystem for (geographic) data science, a set of widely-used libraries that make Python fully featured. We will discuss each package as we use them throughout the chapters. Here, we will show how an installed package can be loaded into a session so that its functionality can be accessed. Package loading in Python is called *importing*. We will use the library `geopandas` as an example. The simplest way to import a library is by typing the following:

```
import geopandas
```

We now have access to the entire library of methods and classes within the session, which we can call by prepending "`geopandas.`" to the name of the function we want. Sometimes, however, we will want to shorten the name to save keystrokes. This approach, called *aliasing*, can be done as follows:

```
import geopandas as gpd
```

Now, every time we want to access a function from `geopandas`, we need to type "`gpd.`" before the function's name. However, sometimes we want to only import parts of a library. For example, we might only want to use one function. In this case, it might be cleaner and more efficient to bring the function itself only:

```
from geopandas import read_file
```

which allows us to use `read_file` directly in the current session, without writing "`gpd.read_file`."

Our approach in this book is to *not* alias to make our code more readable (`geopandas` instead of `gpd`). Also, we will introduce each new library within the text of the chapter where it is first required, so that our code is threaded pedagogically with narrative. Once a package has been introduced, if subsequent chapters rely on it, we will import it at the beginning of the chapter with all the other, already introduced libraries.

2.2.2.2 Contextual help

A very handy feature of Python is the ability to access on-the-spot help for functions. This means that you can check what a function is supposed to do, or how to use it from directly inside your Python session. Of course, this also works handsomely inside a notebook, too. There are a couple of ways to access the help. Take the `read_file` function we have imported above. One way to check its help dialog from within the notebook is to add a question mark after it:

```
?read_file
```

```
Signature: read_file(filename, bbox=None, mask=None,␣
↪rows=None, **kwargs)
Docstring:
Returns a GeoDataFrame from a file or URL.

.. versionadded:: 0.7.0 mask, rows

Parameters
----------
filename : str, path object or file-like object
    Either the absolute or relative path to the file or URL to
    be opened, or any object with a read() method (such as an␣
↪open file
    or StringIO)
bbox : tuple | GeoDataFrame or GeoSeries | shapely Geometry,␣
↪default None
    Filter features by given bounding box, GeoSeries,␣
↪GeoDataFrame or a
    shapely geometry. CRS mis-matches are resolved if given a␣
↪GeoSeries
    or GeoDataFrame. Cannot be used with mask.
mask : dict | GeoDataFrame or GeoSeries | shapely Geometry,␣
↪default None
    Filter for features that intersect with the given dict-
↪like geojson
    geometry, GeoSeries, GeoDataFrame or shapely geometry.
    CRS mis-matches are resolved if given a GeoSeries or␣
↪GeoDataFrame.
    Cannot be used with bbox.
rows : int or slice, default None
    Load in specific rows by passing an integer (first `n`␣
↪rows) or a
    slice() object.
**kwargs :
    Keyword args to be passed to the `open` or␣
↪`BytesCollection` method
```

(continued on next page)

(continued from previous page)

```
    in the fiona library when opening the file. For more␣
↪information on
    possible keywords, type:
    ``import fiona; help(fiona.open)``

Examples
--------
>>> df = geopandas.read_file("nybb.shp")  # doctest: +SKIP

Specifying layer of GPKG:

>>> df = geopandas.read_file("file.gpkg", layer='cities')  #␣
↪doctest: +SKIP

Reading only first 10 rows:

>>> df = geopandas.read_file("nybb.shp", rows=10)  # doctest:␣
↪+SKIP

Reading only geometries intersecting ``mask``:

>>> df = geopandas.read_file("nybb.shp", mask=polygon)  #␣
↪doctest: +SKIP

Reading only geometries intersecting ``bbox``:

>>> df = geopandas.read_file("nybb.shp", bbox=(0, 10, 0, 20))␣
↪ # doctest: +SKIP

Returns
-------
:obj:`geopandas.GeoDataFrame` or :obj:`pandas.DataFrame` :
    If `ignore_geometry=True` a :obj:`pandas.DataFrame` will␣
↪be returned.

Notes
-----
The format drivers will attempt to detect the encoding of␣
↪your data, but
may fail. In this case, the proper encoding can be specified␣
↪explicitly
by using the encoding keyword parameter, e.g. ``encoding='utf-
↪8'``.
File:      /opt/conda/lib/python3.8/site-packages/geopandas/
↪io/file.py
Type:      function
```

In the notebook, this brings up a sub-window in the browser with all the information you need.[8] Additionally, JupyterLab offers the "Contextual Help" box in the initial launcher. If you open it, the help of every function where your cursor lands will be dynamically displayed in the contextual help.

If, for whatever reason, you needed to print that information into the notebook itself, you can use the following `help` function instead:

```
help(geopandas.read_file)
```

2.2.3 Containerized platform

As mentioned earlier in this chapter, reproducible platforms encompass technology and practices that help reproduce a set of analyses or computational work in a different environment than that in which it was produced. There are several approaches to implement this concept in a practical setting. For this book, we use a piece of software called Docker. Docker is based on the idea of a "container," which allows users to create computational environments and run processes within them in a way that is isolated from the host operating system. We decided to use Docker for three main reasons: first, it is widely adopted as an industry standard (e.g., many websites run on Docker containers), which means it is well supported and is likely to be maintained for a while; second, it has also become a standard in the world of data science, which means foundational projects such as Jupyter create official containers for these packages; and third, because of the two previous reasons, developing the book on top of Docker allows us to easily integrate it with cloud services or local (clusters of) servers, widening the set of delivery channels through which we can make the book available to broader audiences.

With Docker, we can create a "container" that includes all the tools required to access the content of the book interactively. But, what exactly is a container? There are several ways to describe it, from very technical to more intuitive ones. In this context, we will focus on a general understanding rather than on the technical details behind its implementation.

At a high level, we can think of a container as a "box" that includes *everything* that is required to run a certain piece of software. This box can be moved around, from machine to machine, and the computations it executes will remain exactly the same. In fact, the content inside the box remains exactly the same, bit by bit. When we download a container into a computer, be it a laptop or a data center, we are not performing an install of the software it contains from the usual channels, for the platform on which we are going to run it. Instead, we are downloading the software in the form that was installed when the container was originally built and packaged, and for the operating system that was also packaged originally. This is the real advantage: build once, run everywhere. For the experienced reader, this might sound very much like their older

[8] In the book, this will instead show the full help text provided by the function.

sibling: virtual machines. Although there are similarities between both technologies, containers are more lightweight, meaning that they can be run much more swiftly and with less computational power and memory than virtual machines. The isolated content inside a container interacts with the rest of the computer through several links that connect the two. For this book, since JupyterLab is a client-server application, the server runs inside the container and we access it through two main "doors": one, through the browser, we will access the main Lab interface; and two, we will "mount" a folder inside the container so we can use software inside the container to edit files that are stored outside in the host machine.

"*Containers sound great but, how can I install and run one?*", you might be asking yourself at this point. First, you will need to install Docker on your computer. This assumes you have administrative rights (i.e., you can install software). If that is the case, you can go to the Docker website[9] and install the version that suits your operating system. Note that, although container technology is Linux-based, Docker provides tools to run it smoothly in macOS and Windows. An install guide for Docker is beyond the scope of this chapter, but there is much documentation available on the web to this end. We personally recommend the official documentation,[10] but you might find other resources that suit your needs better.

2.2.4 Running the book in a container

Once you have Docker up and running on your computer, you can download the container we rely on for the book. We have written the book using the `gds_env` project[11], and that is what we recommend. The `gds_env` provides a containerised platform for geographic data science. It relies on the official Jupyter Docker release and builds on top of it a large set of libraries and add-on's that make the life of the geographic data scientist easier. A new version of the container including the most recent versions of libraries is released twice a year. We have released the book using version `9.0` (we wrote much of it with using versions `6.1-7.0`), but it is likely later versions will work as well.

Downloading the container is akin to installing the software you need to interact with the book, so you will only need to run it once. However, keep in mind that the Docker "image," the file that stores the container, is relatively large (around 4GB), so you will need the space on your machine as well as a good internet connection. If you check those two boxes, you are ready to go. Here are the steps to take:

1. Open a terminal or shell. How to do this will depend on your operating system:
    - `Windows`: we recommend PowerShell or the Terminal app. Type

[9] https://www.docker.com/
[10] https://docs.docker.com/
[11] https://darribas.org/gds_env

"PowerShell" or "Terminal" on the startup menu and, when it comes up, hit enter. This will open a terminal for you.

- `macOS`: use the `Terminal.app`. You can find it on the Applications folder, within the Utilities subfolder.
- `Linux`: if you are running Linux, you probably already have a terminal application of preference. Almost any Linux distribution comes with a terminal app built in.

2. Download, or "pull", the GDS container. For this run on the terminal the following command:

```
docker pull darribas/gds_py:9.0
```

That's it! Once the command above completes, you have all the software you need to interact with this book.

You can now run the container with the following command:

```
docker run \
        --rm \
        -ti \
        -p 8888:8888 \
        -v ${PWD}:/home/jovyan/work \
        darribas/gds_py:9.0
```

Let's unpack the command so that we understand everything that is going on here to get further insight into how the container works:

- `docker run`: Docker does a lot of things; to communicate that we want to *run* a new container, we need specify it.
- `--rm`: this flag will ensure the container is removed when you close it. This in turn makes sure every time you run it again, you start afresh with the exact same set up.
- `-ti`: this flag further ensures that the container is not run in the background but in an interactive mode.
- `-p 8888:8888`: with this, we ensure we forward the *port* from inside the container out to the host machine (e.g., your laptop). This flag is important because it allows the Jupyter server running inside the container to communicate with the browser so we can render the app and send commands. It is one of the two doors we discuss above connecting the container with the computer it runs on.
- `-v ${PWD}:/home/jovyan/work`: this is the second door. This flag "mounts" the folder from where the command is being run in the terminal (`${PWD}` is the `W`orking `D`irectory) into the container so it is visible and editable from inside the container. Such folder will be available at the container's `work` folder.

- `darribas/gds_py:9.0`: this specifies which container we want to run. In this example, we run the `9.0` version of the `gds_py` container. Depending on when you read this, there might be a more recent version that you can try.

The command above will generate output that will look, more or less, like the following:

```
Executing the command: jupyter notebook
[I 14:45:34.681 NotebookApp] Writing notebook server cookie␣
↪secret to /home/jovyan/.local/share/jupyter/runtime/
↪notebook_cookie_secret
[I 14:45:36.504 NotebookApp] Loading IPython parallel␣
↪extension
[I 14:45:36.730 NotebookApp] JupyterLab extension loaded from␣
↪/opt/conda/lib/python3.7/site-packages/jupyterlab
[I 14:45:36.731 NotebookApp] JupyterLab application directory␣
↪is /opt/conda/share/jupyter/lab
[I 14:45:36.738 NotebookApp] [Jupytext Server Extension]␣
↪NotebookApp.contents_manager_class is (a subclass of)␣
↪jupytext.TextFileContentsManager already - OK
[I 14:45:37.718 NotebookApp] Serving notebooks from local␣
↪directory: /home/jovyan
[I 14:45:37.718 NotebookApp] The Jupyter Notebook is running␣
↪at:
[I 14:45:37.719 NotebookApp] http://0fb71d146102:8888/?
↪token=ae7e8017f3e97658a218ec2c2d1fbcc894f09d80f6b5f79c
[I 14:45:37.719 NotebookApp]  or http://127.0.0.1:8888/?
↪token=ae7e8017f3e97658a218ec2c2d1fbcc894f09d80f6b5f79c
[I 14:45:37.719 NotebookApp] Use Control-C to stop this␣
↪server and shut down all kernels (twice to skip␣
↪confirmation).
[C 14:45:37.725 NotebookApp]

    To access the notebook, open this file in a browser:
        file:///home/jovyan/.local/share/jupyter/runtime/
↪nbserver-6-open.html
    Or copy and paste one of these URLs:
        http://0fb71d146102:8888/?
↪token=ae7e8017f3e97658a218ec2c2d1fbcc894f09d80f6b5f79c
     or http://127.0.0.1:8888/?
↪token=ae7e8017f3e97658a218ec2c2d1fbcc894f09d80f6b5f79c
```

With this, you can then open your browser of preference (ideally Mozilla Firefox or Google Chrome) and enter `http://localhost:8888` as if it were a website. This should then load a landing page that looks approximately like the one shown in Figure 2.5.

To access the lab, copy the token from the terminal (in the example above, that would be `ae7e8017f3e97658a218ec2c2d1fbcc894f09d80f6b5f79c`), enter it

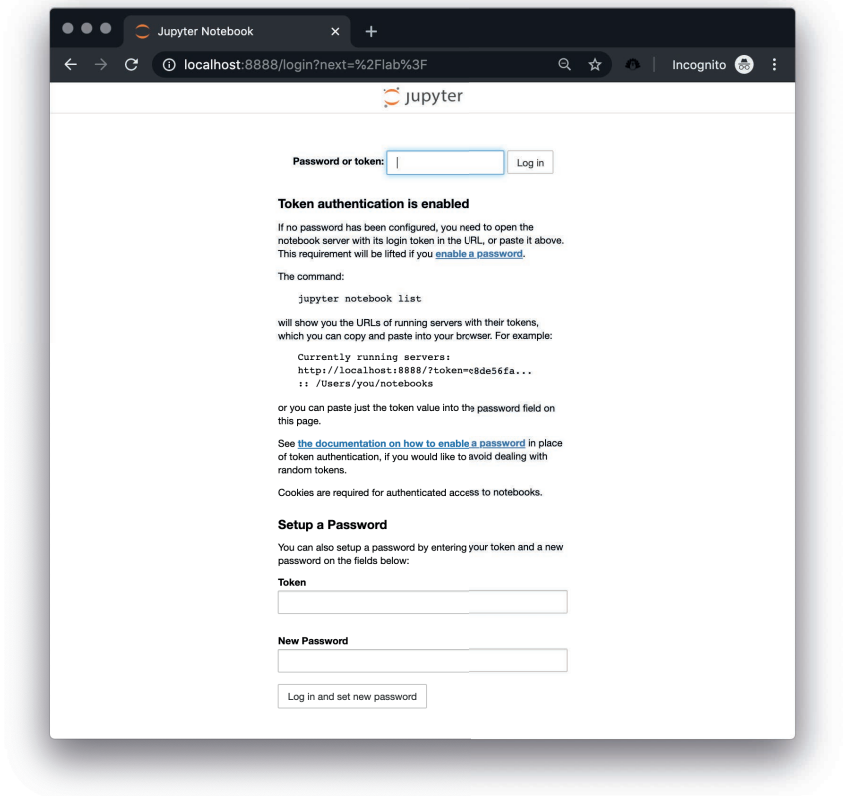


Fig. 2.5: The authentification screen for Jupyter notebooks

on the box and click on “Log in”. Now you are in, and you can view and execute the chapters in this book.

2.3 Conclusion

Reproducibility requires us to take our tools seriously. Since the onset of computational science, we have sought methods to keep the “code” that implements science linked tightly to the writing that explains it. Notebooks offer us a way to do this. Further, keeping in the spirit of open science from long ago, we will do better science if we keep our software public and open to the world. To guarantee that our code can, in fact, be executed, we must also document the environment in which the code is executed using tools like Docker. In the following chapter, we use these tools in concert with geographical thinking to learn about *geographic data science.*

2.4 Next steps

For those interested in further reading on the topic, we suggest the following articles.

For more information on how to write "good" computational notebooks, consider:

Rule, Adam, Amanda Birmingham, Cristal Zuniga, Ilka Altintas, Shih-Cheng Huang, Rob Knight, Niema Moshiri, Mai H. Nguyen, Sara Brin Rosenthal, Fernando Pérez et al. 2019. "Ten Simple Rules for writing and sharing computational analyses in Jupyter Notebooks." *PLOS Computational Biology* 15(7).

For additional perspective on how open code and reproducibility matters in geographic applications, consult the following publications:

Rey, Sergio. 2009. "Show me the code: spatial analysis and open source." *Journal of Geographical Systems* 11(2): 191-207.

Brunsdon, Chris. 2015. "Quantitative Methods I: Reproducible Research and Quantitative Geography." *Progress in Human Geography* 40(5): 687-696.

Taylor & Francis
Taylor & Francis Group
http://taylorandfrancis.com

3

Spatial Data

This chapter grounds the ideas discussed in the previous two chapters into a practical context. We consider how data structures, and the data models they represent, are implemented in Python. We also cover how to interact with these data structures. This will happen alongside the code used to manipulate the data in a single computational laboratory notebook. This, then, unites the two concepts of open science and geographical thinking.

Further, we will spend most of the chapter discussing how Python represents data *once read* from a file or database, rather than focusing on specific *file* formats used to store data. This is because the libraries we use will read any format into one of a few canonical data structures that we discuss in Chapter 1. We take this approach because these data structures are what we interact with during our data analysis: they are our interface with the data. File formats, while useful, are secondary to this purpose. Indeed, part of the benefit of Python (and other computing languages) is *abstraction*: the complexities, particularities and quirks associated with each file format are removed as Python represents all data in a few standard ways, regardless of provenance. We take full advantage of this feature here.

We divide the chapter in two main parts. The first part looks at each of the three main data structures reviewed in Chapter 1 (*Geographic Thinking*): geographic tables, surfaces and spatial graphs. Second, we explore combinations of different data structures that depart from the traditional data model/structure matchings discussed in Chapter 2. We cover how one data in one structure can be effectively transferred to another, but also we discuss why that might (or might not) be a good idea in some cases. A final note before we delve into the content of this book is in order: this is not a comprehensive account of *everything* that is possible with each of the data structures we present. Rather, you can think of it as a preview that we will build on throughout the book to showcase much of what is possible with Python.

```
import pandas
import osmnx
import geopandas
import rioxarray
import xarray
import datashader
import contextily as cx
from shapely import geometry
import matplotlib.pyplot as plt
```

3.1 Fundamentals of geographic data structures

As outlined in Chapter 1, there are a few main data structures that are used in geographic data science: geographic tables (which are generally matched to an object data model), rasters or surfaces (which are generally matched to a field data model), and spatial networks (which are generally matched to a graph data model). We discuss these in turn throughout this section.

3.1.1 Geographic tables

Geographic objects are usually matched to what we called the *geographic table*. Geographic tables can be thought of as a tab in a spreadsheet where one of the columns records geometric information. This data structure represents a single geographic object as a row of a table; each column in the table records information about the object, its attributes or features, as we will see below. Typically, there is a special column in this table that records the *geometry* of the object. Computer systems that use this data structure are intended to add geography into a *relational database*, such as PostgreSQL (through its PostGIS extension) or sqlite (through its spatialite extension). Beyond this, however, many data science languages (such as R, Julia, and Python), have packages that adopt this data structure as well (such as `sf`, `GeoTables.jl`, and `geopandas`), and it is rapidly becoming the main data structure for object-based geographic data.

Before proceeding, though, it helps to mention a quick clarification on terminology. Throughout this book, regardless of the data structure used, we will refer to a measurement about an observation as a *feature*. This is consistent with other work in data science and machine learning. Then, one set of measurements is a *sample*. For tables, this means a feature is a column, and a sample is a row. Historically, though, geographic information scientists have used the word "feature" to mean an individual observation, since a "feature" in cartography is an entity on a map, and "attribute" to describe characteristics of that observation. Elsewhere, a feature may be called a "variable," and a sample is referred to as a "record." So, consistent terminology is important: for this

book, a *feature* is one measured trait pertaining to an observation (column), and a *sample* is one set of measurements (row).

To understand the structure of geographic tables, it will help to read in the `countries_clean.gpkg` dataset included in this book that describes countries in the world. To read in this data, we can use the `read_file()` method in `geopandas`:[1]

```
gt_polygons = geopandas.read_file(
    "../data/countries/countries_clean.gpkg"
)
```

And we can examine the top of the table with the `.head()` method:

```
gt_polygons.head()
```

```
       ADMIN                                                 ␣
↪geometry
0  Indonesia  MULTIPOLYGON (((13102705.696 463877.598, 13102..
↪.
1   Malaysia  MULTIPOLYGON (((13102705.696 463877.598, 13101..
↪.
2      Chile  MULTIPOLYGON (((-7737827.685 -1979875.500, -77..
↪.
3    Bolivia  POLYGON ((-7737827.685 -1979875.500, -7737828...
↪.
4       Peru  MULTIPOLYGON (((-7737827.685 -1979875.500, -77..
↪.
```

Each row of this table is a single country. Each country only has two features: the administrative name of the country and the geometry of the country's boundary. The name of the country is encoded in the `ADMIN` column using the Python `str` type, which is used to store text-based data. The geometry of the country's boundary is stored in the `geometry` column, and it is encoded using a special class in Python that is used to represent geometric objects. As with other table-based data structures in Python, every row and column have an index that identifies them uniquely and is rendered in bold on the left-hand side of the table. This geographic table is an instance of the `geopandas.GeoDataFrame` object, used throughout Python's ecosystem to represent geographic data.

Geographic tables store geographic information as an additional column. But, how is this information encoded? To see, we can check the type of the object in the first row:

```
type(gt_polygons.geometry[0])
```

[1] We will generally use two curved brackets (such as `method_name()`) to denote a *function*, and will omit them (such as `package`) when referring to an object or package.]

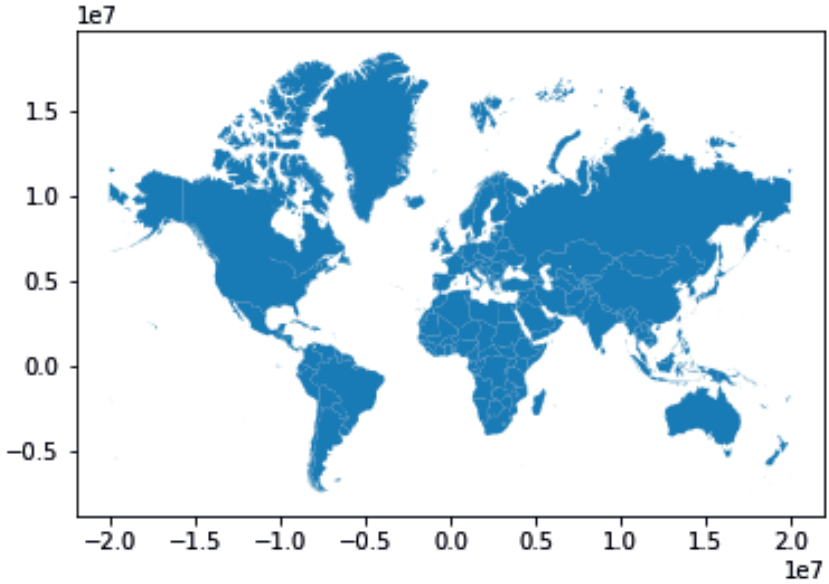


Fig. 3.1: Map of the world made using the GeoDataFrame.plot() method.

```
shapely.geometry.multipolygon.MultiPolygon
```

In `geopandas` (as well as other packages representing geographic data), the `geometry` column has special traits which a "normal" column, such as `ADMIN`, does not. For example, when we plot the dataframe, the `geometry` column is used as the main shape to use in the plot, as shown in Figure 3.1.

```
gt_polygons.plot();
```

Changing the geometric representation of a sample must be done carefully: since the `geometry` column is special, there are special functions to adjust the geometry. For example, if we wanted to represent each country using its *centroid*, a point in the middle of the shape, then we must take care to make sure that a new geometry column was set properly using the `set_geometry()` method. This can be useful when you want to work with two different geometric representations of the same underlying sample.

Let us make a map of both the boundary and the centroid of a country. First, to compute the centroid, we can use the `gt_polygons.geometry.centroid` property. This gives us the point that minimizes the average distance from all other points on the boundary of the shape. Storing that back to a column, called `centroid`:

```
gt_polygons["centroid"] = gt_polygons.geometry.centroid
```

We now have an additional feature:

```
gt_polygons.head()
```

```
       ADMIN                                                 ␣
↪geometry  \
0  Indonesia  MULTIPOLYGON (((13102705.696 463877.598, 13102..
↪.
1   Malaysia  MULTIPOLYGON (((13102705.696 463877.598, 13101..
↪.
2      Chile  MULTIPOLYGON (((-7737827.685 -1979875.500, -77..
↪.
3    Bolivia  POLYGON ((-7737827.685 -1979875.500, -7737828...
↪.
4       Peru  MULTIPOLYGON (((-7737827.685 -1979875.500, -77..
↪.

                            centroid
0   POINT (13055431.810 -248921.141)
1    POINT (12211696.493 422897.505)
2  POINT (-7959811.948 -4915458.802)
3  POINT (-7200010.945 -1894653.148)
4  POINT (-8277554.831 -1032942.536)
```

Despite the fact that `centroid` is a geometry (you can tell because each cell starts with `POINT`), it is not currently set as the active geometry for our table. We can switch to the `centroid` column using the `set_geometry()` method. Finally, we can plot the centroid and the boundary of each country after switching the geometry column with `set_geometry()`, with the results displayed in Figure 3.2.

```
# Plot centroids
ax = gt_polygons.set_geometry("centroid").plot("ADMIN",␣
↪markersize=5)
# Plot polygons without color filling
gt_polygons.plot(
    "ADMIN", ax=ax, facecolor="none", edgecolor="k",␣
↪linewidth=0.2
);
```

Note again how we can create a map by calling `.plot()` on a `GeoDataFrame`. We can thematically color each feature based on a column by passing the name of that column to the plot method (as we do on with `ADMIN` in this case), and that the current geometry is used.

Thus, as should now be clear, nearly any kind of geographic object can be represented in one (or more) geometry column(s). Thinking about the number of different kinds of shapes or geometries one could use quickly boggles the mind. Fortunately the Open Geospatial Consortium (OGC) has defined a set of "abstract" types that can be used to

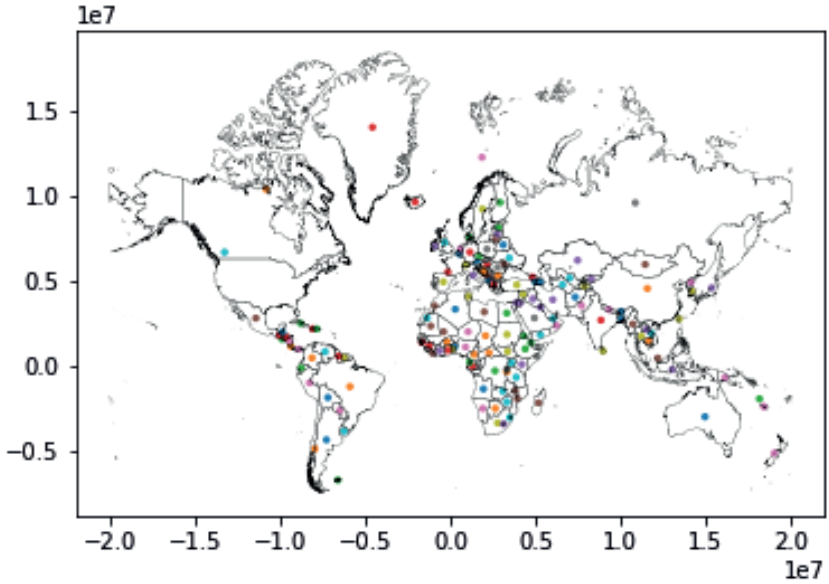


Fig. 3.2: Plotting centroids and boundaries of polygon geometries.

define any kind of geometry. This specification, codified in ISO 19125-1—the "simple features" specification—defines the formal relationships between these types: a `Point` is a zero-dimensional location with an x and y coordinate, a `LineString` is a path composed of a set of more than one `Point`, and a `Polygon` is a surface that has at least one `LineString` that starts and stops with the same coordinate. All of these types *also* have `Multi` variants that indicate a collection of multiple geometries of the same type. So, for instance, Bolivia is represented as a single polygon in Figure 3.3:

```
gt_polygons.query('ADMIN == "Bolivia"')
```

```
    ADMIN                                                geometry␣
↪ \
3  Bolivia  POLYGON ((-7737827.685 -1979875.500, -7737828....

                              centroid
3  POINT (-7200010.945 -1894653.148)
```

```
gt_polygons.query('ADMIN == "Bolivia"').plot();
```

while Indonesia is a `MultiPolygon` containing many `Polygons` for each individual island in the country in Figure 3.4:

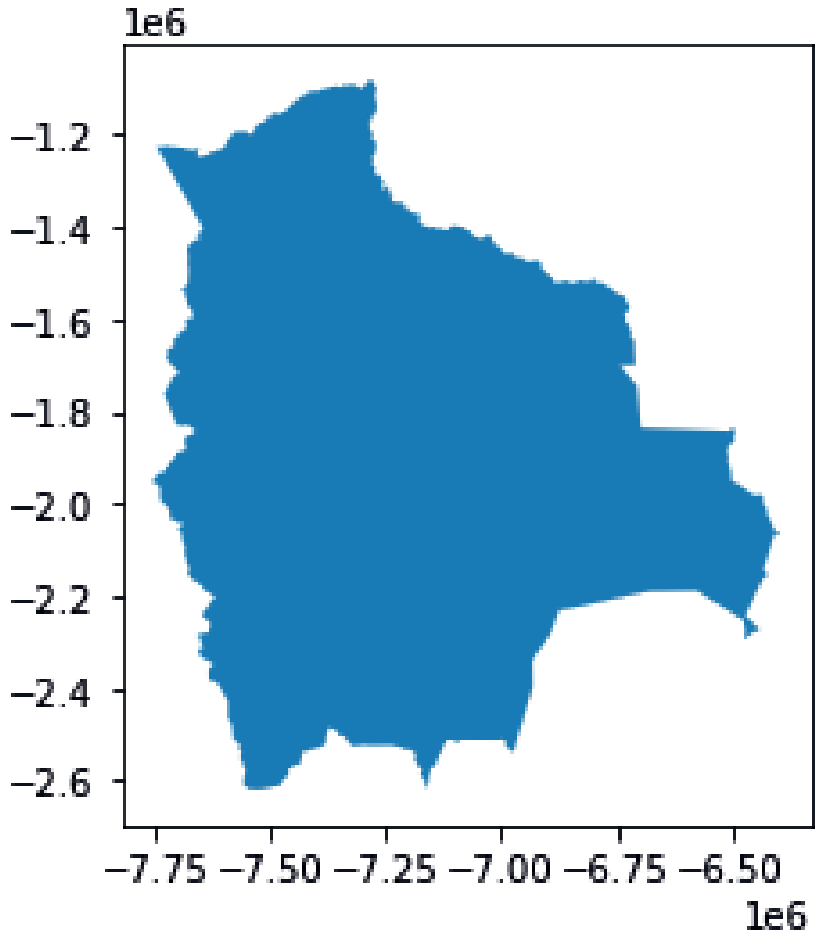


Fig. 3.3: Plotting Bolivia based on a query.

```
gt_polygons.query('ADMIN == "Indonesia"')
```

```
       ADMIN                                                    ␣
↪geometry  \
0  Indonesia  MULTIPOLYGON (((13102705.696 463877.598, 13102..
↪.

                          centroid
0  POINT (13055431.810 -248921.141)
```

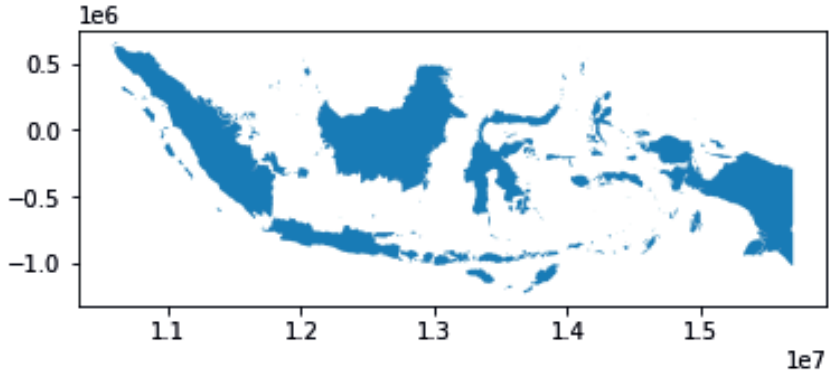


Fig. 3.4: Plotting Indonesia via a query.

```
gt_polygons.query('ADMIN == "Indonesia"').plot();
```

In many cases, geographic tables will have geometries of a single type; records will *all* be `Point` or `LineString`, for instance. However, there is no formal requirement that a *geographic table* has geometries that all have the same type.

Throughout this book, we will use geographic tables extensively, storing polygons, but also points and lines. We will explore lines a bit more in the second part of this chapter but, for now, let us stop on points for a second. As mentioned above, these are the simplest type of feature in that they do not have any dimension, only a pair of coordinates attached to them. This means that points can sometimes be stored in a non-geographic table, simply using one column for each coordinate. We find an example of this on the Tokyo dataset we will use more later. The data is stored as a comma-separated value table, or `.csv`:

```
gt_points = pandas.read_csv("../data/tokyo/tokyo_clean.csv")
```

Since we have read it with `pandas`, the table is loaded as a `DataFrame`, with no explicit spatial dimension:

```
type(gt_points)
```

```
pandas.core.frame.DataFrame
```

If we inspect the table, we find there is not a `geometry` column:

```
gt_points.head()
```

```
           user_id   longitude   latitude             date_
↪taken  \
0     10727420@N00  139.700499  35.674000  2010-04-09␣
↪17:26:25.0
1      8819274@N04  139.766521  35.709095  2007-02-10␣
↪16:08:40.0
2     62068690@N00  139.765632  35.694482  2008-12-21␣
↪15:45:31.0
3  49503094041@N01  139.784391  35.548589  2011-11-11␣
↪05:48:54.0
4     40443199@N00  139.768753  35.671521  2006-04-06␣
↪16:42:49.0

                                photo/video_page_url         ␣
↪   x  \
0  http://www.flickr.com/photos/10727420@N00/4545...  1.
↪555139e+07
1  http://www.flickr.com/photos/8819274@N04/26503...  1.
↪555874e+07
2  http://www.flickr.com/photos/62068690@N00/3125...  1.
↪555864e+07
3  http://www.flickr.com/photos/49503094041@N01/6...  1.
↪556073e+07
4  http://www.flickr.com/photos/40443199@N00/2482...  1.
↪555899e+07

              y
0  4.255856e+06
1  4.260667e+06
2  4.258664e+06
3  4.238684e+06
4  4.255517e+06
```

Many point datasets are provided in this format. To make the most of them, it is convenient to convert them into `GeoDataFrame` tables. There are two steps involved in this process. First, we turn the raw coordinates into geometries:

```
pt_geoms = geopandas.points_from_xy(
    x=gt_points["longitude"],
    y=gt_points["latitude"],
    # x,y are Earth longitude & latitude
    crs="EPSG:4326",
)
```

Second, we create a `GeoDataFrame` object using these geometries:

```
gt_points = geopandas.GeoDataFrame(gt_points, geometry=pt_
↪geoms)
```

And now `gt_points` looks and feels exactly like the one of countries we have seen before, with the difference the `geometry` column stores `POINT` geometries:

```
gt_points.head()
```

```
           user_id   longitude   latitude             date_
↪taken  \
0     10727420@N00  139.700499  35.674000  2010-04-09␣
↪17:26:25.0
1      8819274@N04  139.766521  35.709095  2007-02-10␣
↪16:08:40.0
2     62068690@N00  139.765632  35.694482  2008-12-21␣
↪15:45:31.0
3  49503094041@N01  139.784391  35.548589  2011-11-11␣
↪05:48:54.0
4     40443199@N00  139.768753  35.671521  2006-04-06␣
↪16:42:49.0

                                photo/video_page_url         ␣
↪    x  \
0  http://www.flickr.com/photos/10727420@N00/4545...  1.
↪555139e+07
1  http://www.flickr.com/photos/8819274@N04/26503...  1.
↪555874e+07
2  http://www.flickr.com/photos/62068690@N00/3125...  1.
↪555864e+07
3  http://www.flickr.com/photos/49503094041@N01/6...  1.
↪556073e+07
4  http://www.flickr.com/photos/40443199@N00/2482...  1.
↪555899e+07

              y                     geometry
0  4.255856e+06  POINT (139.70050 35.67400)
1  4.260667e+06  POINT (139.76652 35.70909)
2  4.258664e+06  POINT (139.76563 35.69448)
3  4.238684e+06  POINT (139.78439 35.54859)
4  4.255517e+06  POINT (139.76875 35.67152)
```

3.1.2 Surfaces

Surfaces are used to record data from a field data model. In theory, a field is a continuous surface and thus has an infinite number of locations at which it could be measured. In reality, however, fields are measured at a finite sample of locations that, to provide a sense of continuity and better conform with the field model, are uniformly structured across space. Surfaces thus are represented as grids where each cell contains a sample. A grid can also be thought of as a table with rows and columns but, as we discussed in

the previous chapter, both of them are directly tied to a geographic location. This is in sharp contrast with geographic tables, where geography is confined to a single column.

To explore how Python represents surfaces, we will use an extract for the Brazilian city of Sao Paulo of a *global population dataset*. This dataset records population counts in cells of the same dimensions uniformly covering the surface of the Earth. Our extract is available as a GeoTIF file, a variation of the TIF image format that includes geographic information. We can use the `open_rasterio()` method from the `xarray` package to read in the GeoTIF:

```
pop = xarray.open_rasterio("../data/ghsl/ghsl_sao_paulo.tif")
```

This reads the data into a `DataArray` object:

```
type(pop)
```

```
xarray.core.dataarray.DataArray
```

`xarray` is a package to work with multi-dimensional labeled arrays. Let's unpack this: we can use arrays of not only two dimensions as in a table with rows and columns, but also with an arbitrary number of them; each of these dimensions is "tracked" by an index that makes it easy and efficient to manipulate. In `xarray`, these indices are called coordinates, and they can be retrieved from our `DataArray` through the `coords` attribute:

```
pop.coords
```

```
Coordinates:
  * band     (band) int64 1
  * y        (y) float64 -2.822e+06 -2.822e+06 ... -2.926e+06␣
↪-2.926e+06
  * x        (x) float64 -4.482e+06 -4.482e+06 ... -4.365e+06␣
↪-4.365e+06
```

Interestingly, our surface has *three* dimensions: `x`, `y`, and `band`. The former two track the latitude and longitude that each cell in our population grid covers. The third one has a single value (1) and, in this context, it is not very useful. But it is easy to imagine contexts where a third dimension would be useful. For example, an optical color image may have three bands: red, blue, and green. More powerful sensors may pick up additional bands, such as near infrared (NIR) or even radio bands. Or, a surface measured over time, like the geocubes that we discussed in Chapter 2, will have bands for each point in time at which the field is measured. A geographic surface will thus have two dimensions recording the location of cells (`x` and `y`), and at least one `band` that records other dimensions pertaining to our data.

An `xarray.DataArray` object contains additional information about the values stored under the `attrs` attribute:

```
pop.attrs
```

```
{'transform': (250.0, 0.0, -4482000.0, 0.0, -250.0, -2822000.
↪0),
 'crs': '+proj=moll +lon_0=0 +x_0=0 +y_0=0 +datum=WGS84␣
↪+units=m +no_defs=True',
 'res': (250.0, 250.0),
 'is_tiled': 0,
 'nodatavals': (-200.0,),
 'scales': (1.0,),
 'offsets': (0.0,),
 'AREA_OR_POINT': 'Area',
 'grid_mapping': 'spatial_ref'}
```

In this case, we can see this includes information required to convert pixels in the array into locations on the Earth surface (e.g., `transform`, and `crs`), the spatial resolution (250 meters by 250 meters), and other metadata that allows us to better understand where the data comes from and how it is stored.

Thus, our `DataArray` has three dimensions:

```
pop.shape
```

```
(1, 416, 468)
```

A common operation will be to reduce this to only the two geographic ones. We can do this with the `sel` operator, which allows us to select data by the value of their coordinates:

```
pop.sel(band=1)
```

```
<xarray.DataArray (y: 416, x: 468)>
[194688 values with dtype=float32]
Coordinates:
    band     int64 1
  * y        (y) float64 -2.822e+06 -2.822e+06 ... -2.926e+06␣
↪-2.926e+06
  * x        (x) float64 -4.482e+06 -4.482e+06 ... -4.365e+06␣
↪-4.365e+06
Attributes:
    transform:      (250.0, 0.0, -4482000.0, 0.0, -250.0, -
↪2822000.0)
    crs:            +proj=moll +lon_0=0 +x_0=0 +y_0=0␣
↪+datum=WGS84 +units=m +...
    res:            (250.0, 250.0)
    is_tiled:       0
```

(continued on next page)

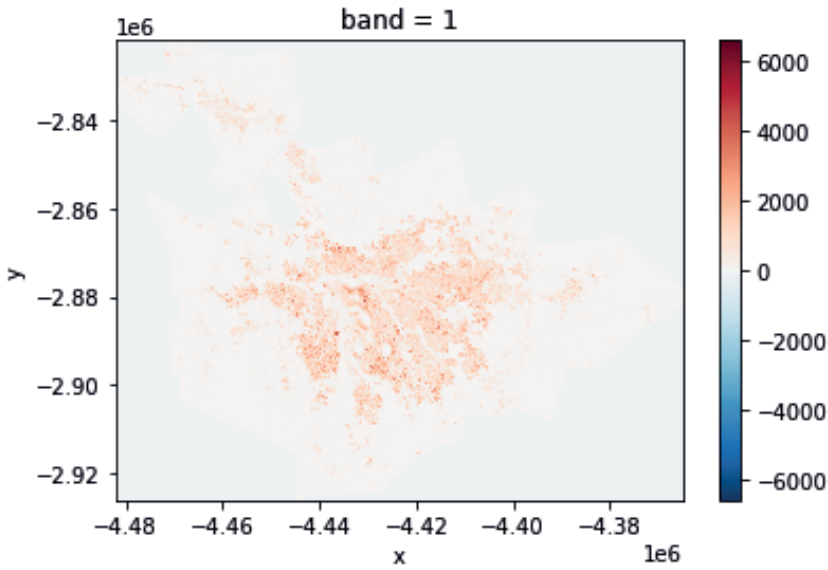


Fig. 3.5: Population surface of Sao Paulo, Brazil

(continued from previous page)

```
    nodatavals:      (-200.0,)
    scales:          (1.0,)
    offsets:         (0.0,)
    AREA_OR_POINT:   Area
    grid_mapping:    spatial_ref
```

The resulting object is thus a two-dimensional array. Similar to geographic tables, we can quickly plot the values in our dataset as shown in Figure 3.5:

```
pop.sel(band=1).plot();
```

This gives us a first overview of the distribution of population in the Sao Paulo region. However, if we inspect the map further, we can see that the map includes negative counts! How could this be? As it turns out, missing data is traditionally stored in surfaces not as a class of its own (e.g., `NaN`) but with an impossible value. If we return to the `attrs` printout above, we can see how the `nodatavals` attribute specifies missing data recorded with -200. With that in mind, we can use the `where()` method to select only values that are *not* -200:

```
pop.where(pop != -200).sel(band=1).plot(cmap="RdPu");
```

The colorbar in Figure 3.6 now looks more sensible, and indicates *real* counts, rather than including the missing data placeholder values.

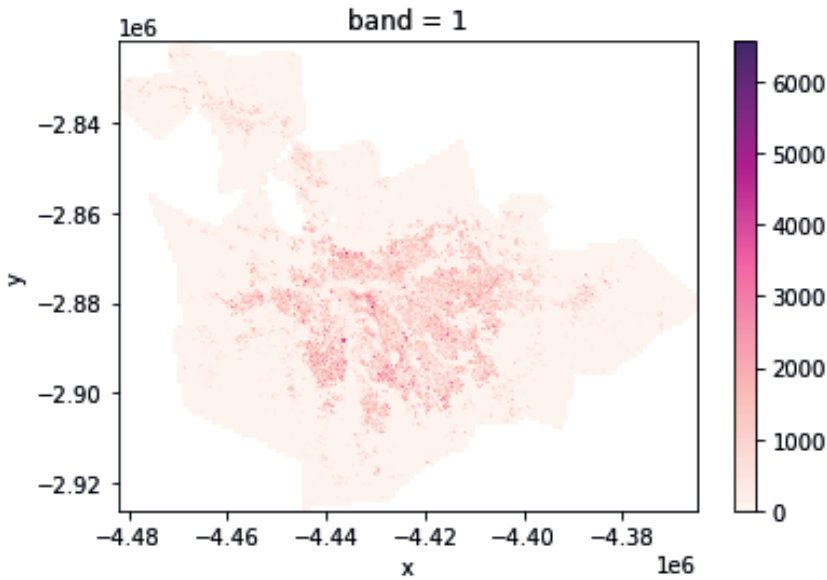


Fig. 3.6: Population surface of Sao Paulo, Brazil omitting NAN values.

3.1.3 Spatial graphs

Spatial graphs store connections between objects through space. These connections may derive from geographical topology (e.g., contiguity), distance, or more sophisticated dimensions such as interaction flows (e.g., commuting, trade, communication). Compared to geographic tables and surfaces, spatial graphs are rather different. First, in most cases they do not record measurements about given phenomena, but instead focus on *connections*, on storing relationships between objects as they are facilitated (or impeded in their absence) by space. Second, because of this relational nature, the data are organized in a more unstructured fashion: while one sample may be connected to only one other sample, another one can display several links. This is in stark contrast to geographic tables and surfaces, both of which have a clearly defined structure, shape and dimensionality in which data are organized.

These particularities translate into a different set of Python data structures. Unlike the previous data structures we have seen, there are quite a few data structures to represent spatial graphs, each optimized for different contexts. One such case is the use of spatial connections in statistical methods such as exploratory data analysis or regression. For this, the most common data structure are spatial weights matrices, to which we devote the next chapter.

In this chapter, we briefly review a different way of representing spatial graphs that is much closer to the mathematical concept of a graph. A graph is composed of *nodes* that

are linked together by *edges*. In a spatial network, *nodes* may represent geographical places, and thus have a specific location; likewise, *edges* may represent geographical paths between these places. Networks require both *nodes* and *edges* to analyze their structure.

For illustration, we will rely on the `osmnx` library, which can query data from OpenStreetMap. For example, we extract the street-based graph of Yoyogi Park, near our earlier data from Tokyo:

```
graph = osmnx.graph_from_place("Yoyogi Park, Shibuya, Tokyo,␣
↪Japan")
```

The code snippet above sends the query to the OpenStreetMap server to fetch the data. Note that the cell above *requires* internet connectivity to work. If you are working on the book *without* connectivity, a cached version of the graph is available on the data folder and can be read as:

```
graph = osmnx.load_graphml("../data/cache/yoyogi_park_graph.
↪graphml")
```

Once the data is returned to `osmnx`, it gets processed into the `graph` Python representation:

```
type(graph)
```

```
networkx.classes.multidigraph.MultiDiGraph
```

We can have a quick inspection of the structure of the graph with the `plot_graph` method, shown in Figure 3.7:

```
osmnx.plot_graph(graph);
```

The resultant `graph` object is actually a `MultiDiGraph` from `networkx`, a graph library written in Python. The graph here is stored as a collection of 106 nodes (street intersections):

```
len(graph.nodes)
```

```
106
```

and 287 edges (streets) that connect them:

```
len(graph.edges)
```

```
287
```

Fig. 3.7: OSMNX graph for a street network.

Each of these elements can be queried to obtain more information such as the location and ID of a node:

```
graph.nodes[1520546819]
```

```
{'y': 35.6711267, 'x': 139.6925951, 'street_count': 4}
```

The characteristics of an edge:

```
graph.edges[(1520546819, 3010293622, 0)]
```

```
{'osmid': 138670840,
 'highway': 'footway',
 'oneway': False,
 'length': 59.113,
 'geometry': <shapely.geometry.linestring.LineString at␣
↪0x7f3123456190>}
```

Or how the different components of the graph relate to each other. For example, what other nodes are directly connected to node `1520546819`?

```
list(graph.adj[1520546819].keys())
```

```
[3010293622, 5764960322, 1913626649, 1520546959]
```

Thus, networks are easy to represent in Python, and are one of the three main data structures in geographic data science.

3.2 Hybrids

We have just seen how geographic tables, surfaces, and networks map onto `GeoDataFrame`, `DataArray` and `Graph` objects in Python, respectively. These represent the conventional pairings that align data models to data structures with Python representations. However, while the conventional pairings are well-used, there are others in active use and many more to yet be developed. Interestingly, many new pairings are driven by new developments in technology, enabling approaches that were not possible in the past or creating situations (e.g., large datasets) that make the conventional approach limiting. Therefore, in this second section of the chapter, we step a bit "out of the box" to explore cases in which it may make sense to represent a dataset with a data structure that might not be the most obvious initial choice.

3.2.1 Surfaces as tables

The first case we explore is treating surfaces as (geo-)tables. In this context, we shift from an approach where each dimension has a clear mapping to a spatial or temporal aspect of the dataset, to one where each sample, cell of the surface/cube is represented as a row in a table. This approach runs contrary to the general consensus that fields are best represented as surfaces or rasters because that allows us to index space and time "by default" based on the location of values within the data structure. Shifting to a tabular structure implies either losing that space-time reference, or having to build it manually with auxiliary objects (e.g., a spatial graph). In almost any case, operating on this format is less efficient than it *could* be if we had bespoke algorithms built around surface structures. Finally, from a more conceptual point of view, treating pixels as independent realizations of a process that we *know* is continuous can be computationally inefficient and statistically flawed.

This perspective, however, also involves important benefits. First, sometimes we *don't* need location for our particular application. Maybe we are interested in calculating overall descriptive statistics; or maybe we need to run an analysis that is entirely atomic in the sense that it operates on each sample in isolation from all the other ones. Second, by "going tabular" we recast our specialized, spatial data into the most common data structure available, for which a large amount of commodity technology is built. This means many new tools can be used for analysis. So-called "big data" technologies, such as distributed systems, are much more common, robust, and tested for tabular data than for spatial surfaces. *If* we can translate our spatial challenge into a tabular challenge, we can immediately plug in technology that is more optimized and, in some cases, reliable. Further, some analytic toolboxes common in (geographic) data science are entirely built around tabular structures. Machine learning packages such as `scikit-learn`, or some spatial analytics (such as most methods in the Pysal family of packages) are designed around this data structure. Converting our surfaces into tables thus allows us to plug into a much wider suite of (potentially) efficient tools and techniques.

We will see two ways of going from surfaces to tables: one converts every pixel into a table row, and another aggregates pixels into pre-determined polygons.

3.2.1.1 One pixel at a time

Technically, going from surface to table involves traversing from `xarray` to `pandas` objects. This is actually a well-established bridge. To illustrate it with an example, let's revisit the population counts in *Sao Paulo* used earlier. We can read the surface into a `DataArray` object with the `open_rasterio()` method:

```
surface = xarray.open_rasterio("../data/ghsl/ghsl_sao_paulo.
↪tif")
```

Transferring to a table is as simple as calling the `DataArray`'s `to_series()` method:

```
t_surface = surface.to_series()
```

The resulting object is a `pandas.Series` object indexed on each of the dimensions of the original `DataArray`:

```
t_surface.head()
```

```
band  y           x
1     -2822125.0  -4481875.0   -200.0
                  -4481625.0   -200.0
                  -4481375.0   -200.0
                  -4481125.0   -200.0
                  -4480875.0   -200.0
dtype: float32
```

At this point, everything we know about `pandas` and tabular data applies! For example, it might be more convenient to express it as a `DataFrame`:

```
t_surface = t_surface.reset_index().rename(columns={0: "Value
↪"})
```

With the power of a tabular library, some queries and filter operations become much easier. For example, finding cells with more than 1,000 people can be done with the usual `query()` method.[2]

```
t_surface.query("Value > 1000").info()
```

[2] Although, if all you want to do is this type of query, `xarray` is well equipped for this kind of task too.

```
<class 'pandas.core.frame.DataFrame'>
Int64Index: 7734 entries, 3785 to 181296
Data columns (total 4 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   band    7734 non-null   int64
 1   y       7734 non-null   float64
 2   x       7734 non-null   float64
 3   Value   7734 non-null   float32
dtypes: float32(1), float64(2), int64(1)
memory usage: 271.9 KB
```

The table we have built has no geometries associated with it, only rows representing pixels. It takes a bit more effort, but it is possible to convert it, or a subset of it, into a full-fledged geographic table, where each pixel includes the grid geometry it represents. For this task, we develop a function that takes a row from our table and the resolution of the surface, and returns its geometry:

```
def row2cell(row, res_xy):
    res_x, res_y = res_xy  # Extract resolution for each␣
↪dimension
    # XY Coordinates are centered on the pixel
    minX = row["x"] - (res_x / 2)
    maxX = row["x"] + (res_x / 2)
    minY = row["y"] + (res_y / 2)
    maxY = row["y"] - (res_y / 2)
    poly = geometry.box(
        minX, minY, maxX, maxY
    )  # Build squared polygon
    return poly
```

For example:

```
row2cell(t_surface.loc[0, :], surface.attrs["res"])
```

```
<shapely.geometry.polygon.Polygon at 0x7f3122262b80>
```

One of the benefits of this approach is that we do not require entirely filled surfaces and can only record pixels where we have data. For the example above or cells with more than 1,000 people, we could create the associated geo-table as follows:

```
max_polys = (
    t_surface.query(
        "Value > 1000"
    )  # Keep only cells with more than 1k people
    .apply(  # Build polygons for selected cells
        row2cell, res_xy=surface.attrs["res"], axis=1
```

(continued on next page)

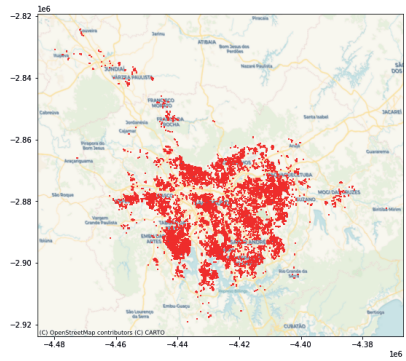


Fig. 3.8: Combining points with Contextily.

(continued from previous page)

```
    )
    .pipe(  # Pipe result from apply to convert into a␣
↪GeoSeries
        geopandas.GeoSeries, crs=surface.attrs["crs"]
    )
)
```

And generate a map with the same tooling that we use for any standard geo-table (Figure 3.8):

```
# Plot polygons
ax = max_polys.plot(edgecolor="red", figsize=(9, 9))
# Add basemap
cx.add_basemap(
    ax, crs=surface.attrs["crs"], source=cx.providers.CartoDB.
↪Voyager
);
```

Finally, once we have operated on the data as a table, we may want to return to a surface-like data structure. This involves taking the same journey in the opposite direction as how we started. The sister method of `to_series` in `xarray` is `from_series`:

```
new_da = xarray.DataArray.from_series(
    t_surface.set_index(["band", "y", "x"])["Value"]
)
new_da
```

```
<xarray.DataArray 'Value' (band: 1, y: 416, x: 468)>
array([[[-200., -200., -200., ..., -200., -200., -200.],
        [-200., -200., -200., ..., -200., -200., -200.],
```

(continued on next page)

(continued from previous page)

```
        [-200., -200., -200., ..., -200., -200., -200.],
        ...,
        [-200., -200., -200., ..., -200., -200., -200.],
        [-200., -200., -200., ..., -200., -200., -200.],
        [-200., -200., -200., ..., -200., -200., -200.]]],␣
↪dtype=float32)
Coordinates:
  * band     (band) int64 1
  * y        (y) float64 -2.926e+06 -2.926e+06 ... -2.822e+06␣
↪-2.822e+06
  * x        (x) float64 -4.482e+06 -4.482e+06 ... -4.365e+06␣
↪-4.365e+06
```

3.2.1.2 Pixels to polygons

A second use case involves moving surfaces directly into geographic tables by aggregating pixels into pre-specified geometries. For this illustration, we will use the digital elevation model *(DEM)* surface containing elevation for the San Diego (US) region, and the set of *census tracts*. For an example, we will investigate the average altitude of each neighborhood.

Let's start by reading the data. First, the elevation model (Figure 3.9):

```
dem = xarray.open_rasterio("../data/nasadem/nasadem_sd.tif").
↪sel(
    band=1
)
dem.where(dem > 0).plot.imshow();
```

And the neighborhood areas (tracts) from the census (Figure 3.10):

```
sd_tracts = geopandas.read_file(
    "../data/sandiego/sandiego_tracts.gpkg"
)
sd_tracts.plot();
```

There are several approaches to compute the average altitude of each neighborhood. We will use `rioxarray`to clip parts of the surface *within* a given set of geometries. By this, we mean that we will cut out the part of the raster that falls within each geometry, and then we can summarize the values in that sub-raster. This is sometimes called computing a "zonal statistic" from a raster, where the "zone" is the geometry.

Since this is somewhat complicated, we will start with a single polygon. For the illustration, we will use the largest one, located on the eastern side of San Diego. We can find the ID of the polygon with:

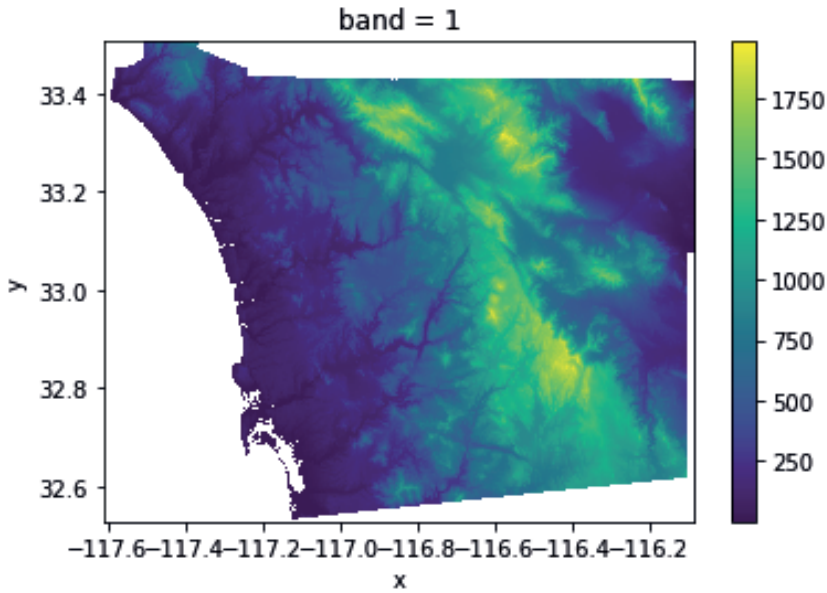


Fig. 3.9: Digital Elevation Model as a raster.

```
largest_tract_id = sd_tracts.query(
    f"area_sqm == {sd_tracts['area_sqm'].max()}"
).index[0]

largest_tract_id
```

```
627
```

And then pull out the polygon itself for the illustration:

```
largest_tract = sd_tracts.loc[largest_tract_id, "geometry"]
```

Clipping the section of the surface that is within the polygon in the DEM can be achieved with the `rioxarray` extension to clip surfaces based on geometries (Figure 3.11):

```
# Clip elevation for largest tract
dem_clip = dem.rio.clip(
    [largest_tract.__geo_interface__], crs=sd_tracts.crs
)

# Set up figure to display against polygon shape
```

(continued on next page)

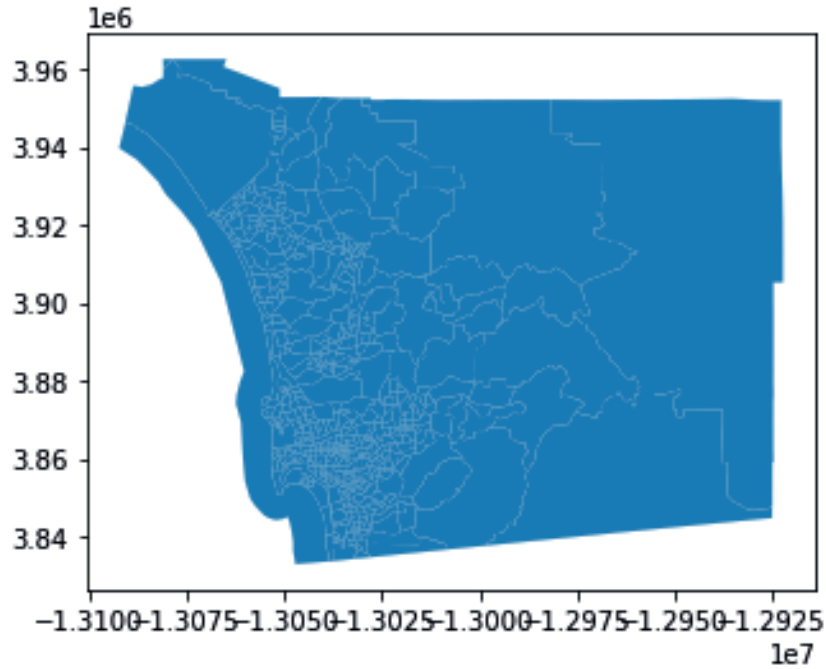


Fig. 3.10: San Diego California census tracts.

(continued from previous page)

```
f, axs = plt.subplots(1, 2, figsize=(6, 3))
# Display elevation of largest tract
dem_clip.where(dem_clip > 0).plot(ax=axs[0], add_
↪colorbar=True)

# Display largest tract polygon
sd_tracts.loc[[largest_tract_id]].plot(
    ax=axs[1], edgecolor="red", facecolor="none"
)
axs[1].set_axis_off()
# Add basemap
cx.add_basemap(
    axs[1], crs=sd_tracts.crs, source=cx.providers.Stamen.
↪Terrain
);
```

Once we have elevation measurements for all the pixels within the tract, the average one can be calculated with `mean()`:

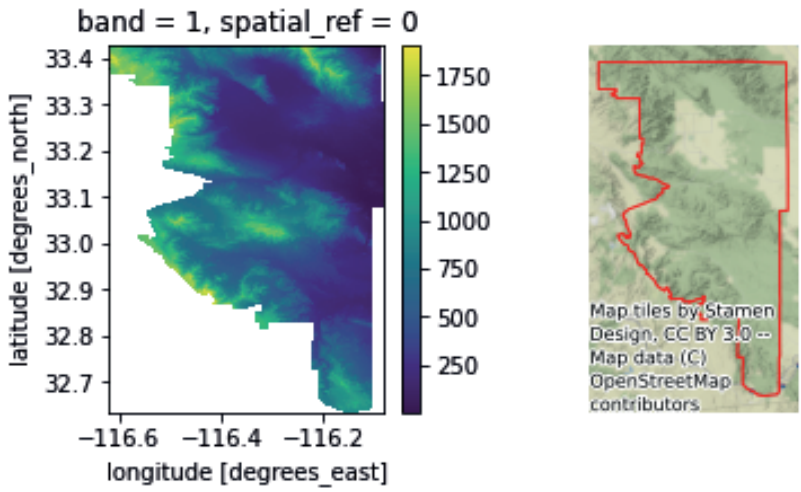


Fig. 3.11: DEM clipped to San Diego

```
dem_clip.where(dem_clip > 0).mean()
```

```
<xarray.DataArray ()>
array(585.11375946)
Coordinates:
    band         int64 1
    spatial_ref  int64 0
```

Now, to scale this to the entire geo-table, there are several approaches. Each has its benefits and disadvantages. We opt for applying the method above to each row of the table. We define an auxiliary function that takes a row containing one of our tracts and returns its elevation:

```
def get_mean_elevation(row, dem):
    # Extract geometry object
    geom = row["geometry"].__geo_interface__
    # Clip the surface to extract pixels within `geom`
    section = dem.rio.clip([geom], crs=sd_tracts.crs)
    # Calculate mean elevation
    elevation = float(section.where(section > 0).mean())
    return elevation
```

Applied to the same tract, it returns the same average elevation:

```
get_mean_elevation(sd_tracts.loc[largest_tract_id, :], dem)
```

```
585.1137594576915
```

This method can then be run on each polygon in our series using the `apply()` method:

```
elevations = sd_tracts.head().apply(
    get_mean_elevation, dem=dem, axis=1
)
elevations
```

```
0      7.144268
1     35.648492
2     53.711389
3     91.358777
4    187.311972
dtype: float64
```

This simple approach illustrates the main idea well: find the cells that pertain to a given geometry and summarize their values in some manner. This can be done with any kind of geometry. Further, this simple method plays well with `xarray` surface structures and is scalable in that it is not too involved to run in parallel and distributed form using libraries like `dask`. Further, it can be extended using arbitrary Python functions, so it is simple to extend.

However, this approach can be quite slow in big data. A more efficient alternative for our example uses the `rasterstats` library. This is a purpose-built library to construct so-called "zonal statistics" from surfaces. Here, the "zones" are the polygons and the "surface" is our `DataArray`. Generally, this library will be faster than the simpler approach used above, but it may be more difficult to extend or adapt:

```
from rasterstats import zonal_stats

elevations2 = zonal_stats(
    sd_tracts.to_crs(dem.rio.crs),  # Geotable with zones
    "../data/nasadem/nasadem_sd.tif",  # Path to surface file
)
elevations2 = pandas.DataFrame(elevations2)
```

```
elevations2.head()
```

```
    min    max        mean   count
0 -12.0   18.0    3.538397    3594
1  -2.0   94.0   35.616395    5709
2  -5.0  121.0   48.742630   10922
3  31.0  149.0   91.358777    4415
4 -32.0  965.0  184.284941  701973
```

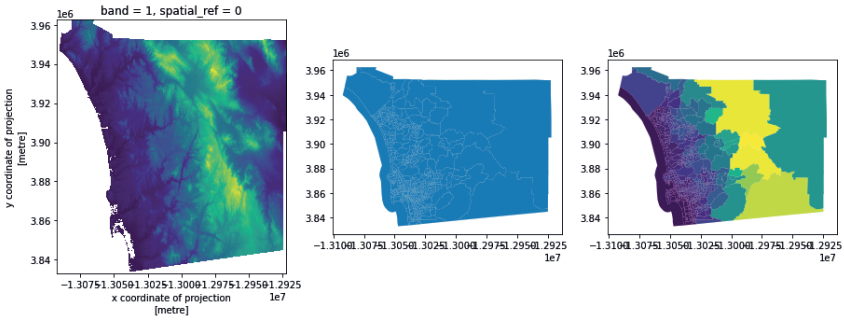


Fig. 3.12: Digital elevation model estimates by census tract, San Diego.

To visualize these results, we can make an elevation map (Figure 3.12):

```
# Set up figure
f, axs = plt.subplots(1, 3, figsize=(15, 5))

# Plot elevation surface
dem.where(  # Keep only pixels above sea level
    dem
    > 0
    # Reproject to CRS of tracts
).rio.reproject(
    sd_tracts.crs
    # Render surface
).plot.imshow(
    ax=axs[0], add_colorbar=False
)

# Plot tract geography
sd_tracts.plot(ax=axs[1])

# Plot elevation on tract geography
sd_tracts.assign(  # Append elevation values to tracts
    elevation=elevations2["mean"]
).plot(  # Plot elevation choropleth
    "elevation", ax=axs[2]
);
```

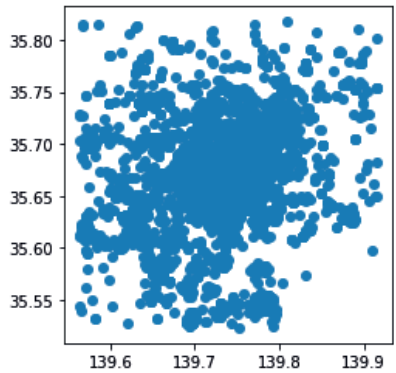


Fig. 3.13: Point locations of Tokyo Photographs.

3.2.2 Tables as surfaces

The case for converting tables into surfaces is perhaps less controversial than that for turning surfaces into tables. This is an approach we can take in cases where we are interested in the *overall* distribution of objects (usually points) and we have so many that it is not only technically more efficient to represent them as a surface, but conceptually it is also easier to think about the points as uneven measurements from a continuous field. To illustrate this approach, we will use the dataset of *Tokyo photographs* we loaded above into `gt_points`.

From a purely technical perspective, for datasets with too many points, representing every point in the data on a screen can be seriously overcrowded:

```
gt_points.plot();
```

In Figure 3.13, it is hard to tell anything about the density of points in the center of the image due to *overplotting*: while points *theoretically* have no width, they *must* have some dimension in order for us to see them! Therefore, point *markers* often plot on top of one another, obscuring the true pattern and density in dense areas. Converting the dataset from a geo-table into a surface involves laying out a grid and counting how many points fall within each cell. In one sense, this is the reverse operation to what we saw when computing zonal statistics in the previous section: instead of aggregating cells into objects, we aggregate objects into cells. Both operations, however, involve aggregation that reduces the amount of information present in order to make the (new) data more manageable.

In Python, we can rely on the `datashader` library, which does all the computation in a very efficient way. This process involves two main steps. First, we set up the grid (or canvas, `cvs`) into which we want to aggregate points:

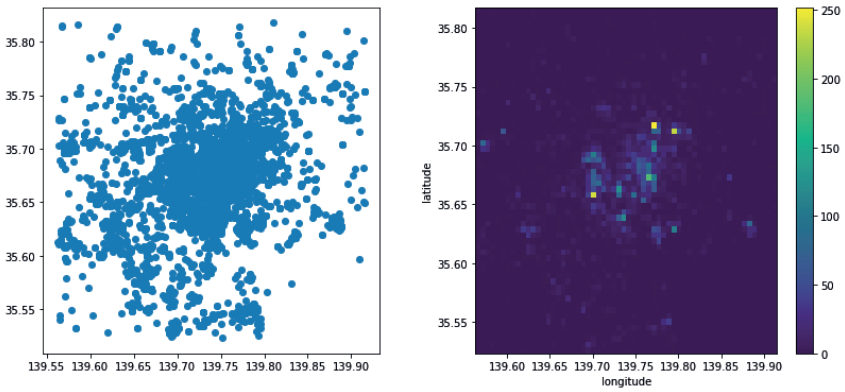


Fig. 3.14: Point locations of Tokyo Photographs, and Point Density as a Surface.

```
cvs = datashader.Canvas(plot_width=60, plot_height=60)
```

Then we "transfer" the points into the grid:

```
grid = cvs.points(gt_points, x="longitude", y="latitude")
```

The resulting `grid` is a standard `DataArray` object that we can then manipulate as we have seen before. When plotted below, the amount of detail that the resampled data allows for is much greater than when the points were visualized alone. This is shown in Figure 3.14.

```
f, axs = plt.subplots(1, 2, figsize=(14, 6))
gt_points.plot(ax=axs[0])
grid.plot(ax=axs[1]);
```

3.2.3 Networks as graphs *and* tables

In the previous chapter, we saw networks as data structures that store *connections* between objects. We also discussed how this broad definition includes many interpretations that focus on different aspects of the networks. While spatial analytics may use graphs to record the topology of a table of objects such as polygons, transport applications may treat the network representation of the street layout as a set of objects itself, in this case lines. In this final section we show how one can flip back and forth between one representation and another, to take advantage of different aspects.

We start with the `graph` object from the *previous section*. Remember this captures the street layout around Yoyogi park in Tokyo. We have seen how, stored under this data

structure, it is easy to query which node is connected to which, and which ones are at the end of a given edge.

However, in some cases, we may want to convert the graph into a structure that allows us to operate on each component of the network independently. For example, we may want to map streets, calculate segment lengths, or draw buffers around each intersection. These are all operations that do not require topological information, that are standard for geo-tables, and that are irrelevant to the graph structure. In this context, it makes sense to convert our `graph` to two geo-tables, one for intersections (graph nodes) and one for street segments (graph edges). In `osmnx`, we can do that with the built-in converter:

```
gt_intersections, gt_lines = osmnx.graph_to_gdfs(graph)
```

Now each of the resulting geo-tables is a collection of geographic objects:

```
gt_intersections.head()
```

```
                   y           x  street_count highway  \
osmid
886196069  35.670087  139.694333             3     NaN
886196073  35.669725  139.699508             3     NaN
886196100  35.669442  139.699708             3     NaN
886196106  35.670422  139.698564             4     NaN
886196117  35.671256  139.697470             3     NaN

                            geometry
osmid
886196069  POINT (139.69433 35.67009)
886196073  POINT (139.69951 35.66972)
886196100  POINT (139.69971 35.66944)
886196106  POINT (139.69856 35.67042)
886196117  POINT (139.69747 35.67126)
```

```
gt_lines.info()
```

```
<class 'geopandas.geodataframe.GeoDataFrame'>
MultiIndex: 287 entries, (886196069, 1520546857, 0) to␣
↪(7684088896, 3010293702, 0)
Data columns (total 8 columns):
 #   Column    Non-Null Count  Dtype
---  ------    --------------  -----
 0   osmid     287 non-null    object
 1   highway   287 non-null    object
 2   oneway    287 non-null    bool
 3   length    287 non-null    float64
 4   geometry  287 non-null    geometry
 5   bridge    8 non-null      object
```

(continued on next page)

(continued from previous page)

```
 6   name       9 non-null      object
 7   access     2 non-null      object
dtypes: bool(1), float64(1), geometry(1), object(5)
memory usage: 26.9+ KB
```

If we were in the opposite situation, where we had a set of street segments and their intersections in geo-table form, we can generate the graph representation with the `graph_from_gdfs` sister method:

```
new_graph = osmnx.graph_from_gdfs(gt_intersections, gt_lines)
```

The resulting object will behave in the same way as our original `graph`.

3.3 Conclusion

In conclusion, this chapter provides an overview of the mappings between data models, presented in Chapter 2, and data structures that are common in Python. Beyond the data structures discussed here, the Python ecosystem is vast, deep, and ever-changing. Part of this is the ease with which you can create your own representations to express different aspects of a problem at hand. However, by focusing on our shared representations and the interfaces between these representations, you can generally conduct any analysis you need. By creating unique, bespoke representations, your analysis might be more efficient, but you can also inadvertently isolate it from other developers and render useful tools inoperable. Therefore, a solid understanding of the basic data structures (the `GeoDataFrame`, `DataArray`, and `Graph`) will be sufficient to support nearly any analysis you need to conduct.

3.4 Questions

1. One way to convert from `Multi`-type geometries into many individual geometries is using the `explode()` method of a GeoDataFrame. Using the `explode()` method, can you find out how many islands are in Indonesia?
2. Using `osmnx`, are you able to extract the street graph for your hometown?
3. As you have seen with the `osmnx.graph_to_gdfs()` method, it is possible to convert a graph into the constituent nodes and edges. Graphs have many other kinds of non-geographical representations. Many of these are provided in `networkx` methods that start with `to_`. How many representations of graphs are currently supported?
4. Using `networkx.to_edgelist()`, what "extra" information does `osmnx` include when building the dataframe for edges?

5. Instead of computing the average elevation for each neighborhood in San Diego, can you answer the following queries?

   - What neighborhoods (or neighborhood) have *the highest average elevation*?
   - What neighborhoods (or neighborhood) have *the highest point single point*?
   - Can you find the neighborhood (or neighborhoods) with *the largest elevation change*?

Taylor & Francis
Taylor & Francis Group
http://taylorandfrancis.com

4 Spatial Weights

"Spatial weights" are one way to represent graphs in geographic data science and spatial statistics. They are widely used constructs that represent geographic relationships between the observational units in a spatially referenced dataset. Implicitly, spatial weights connect objects in a geographic table to one another using the spatial relationships between them. By expressing the notion of geographical proximity or connectedness, spatial weights are the main mechanism through which the spatial relationships in geographical data is brought to bear in the subsequent analysis.

4.1 Introduction

Spatial weights often express our knowledge about spatial relationships. For example, proximity and adjacency are common spatial questions: *What neighborhoods are you surrounded by? How many gas stations are within 5 miles of my stalled car?* These are spatial questions that target specific information about the spatial configuration of a specific target ("a neighborhood," "my stalled car") and geographically connected relevant sites ("adjacent neighborhoods", "nearby gas stations"). For us to use this information in statistical analysis, it's often necessary to compute these relationships between all pairs of observations. This means that, for many applications in geographic data science, we are building a *topology*—a mathematical structure that expresses the connectivity between observations—that we can use to examine the data. Spatial weights matrices express this topology, letting us embed all of our observations in space together, rather than asking and answering single questions about features nearby a unit.

```
import contextily
import geopandas
import rioxarray
```

(continued on next page)

(continued from previous page)

```
import seaborn
import pandas
import numpy
import matplotlib.pyplot as plt
from shapely.geometry import Polygon
from pysal.lib import cg as geometry
```

Since they provide a way to represent these spatial relationships, spatial weights are widely used throughout spatial and geographic data science. In this chapter, we first consider different approaches to construct spatial weights, distinguishing between those based on contiguity/adjacency relations from weights obtained from distance based relationships. We then discuss the case of hybrid weights which combine one or more spatial operations in deriving the neighbor relationships between observations. We illustrate all of these concepts through the spatial weights class in `pysal`, which provides a rich set of methods and characteristics for spatial weights and it is stored under the `weights` sub-module:

```
from pysal.lib import weights
```

We also demonstrate its set-theoretic functionality, which permits the derivation of weights through the application of set operations. Throughout the chapter, we discuss common file formats used to store spatial weights of different types, and we include visual discussion of spatial weights, making these sometimes abstract constructs more intuitive.

4.2 Contiguity weights

A contiguous pair of spatial objects are those that share a common border. At first glance, this seems straightforward. However, in practice this turns out to be more complicated. The first complication is that there are different ways that objects can "share a common border". Let's start with the example of a three-by-three grid (see Figure 4.1). We can create it as a geo-table from scratch:

```
# Get points in a grid
l = numpy.arange(3)
xs, ys = numpy.meshgrid(l, l)
# Set up store
polys = []
# Generate polygons
for x, y in zip(xs.flatten(), ys.flatten()):
    poly = Polygon([(x, y), (x + 1, y), (x + 1, y + 1), (x, y␣
↪+ 1)])
    polys.append(poly)
# Convert to GeoSeries
```

(continued on next page)

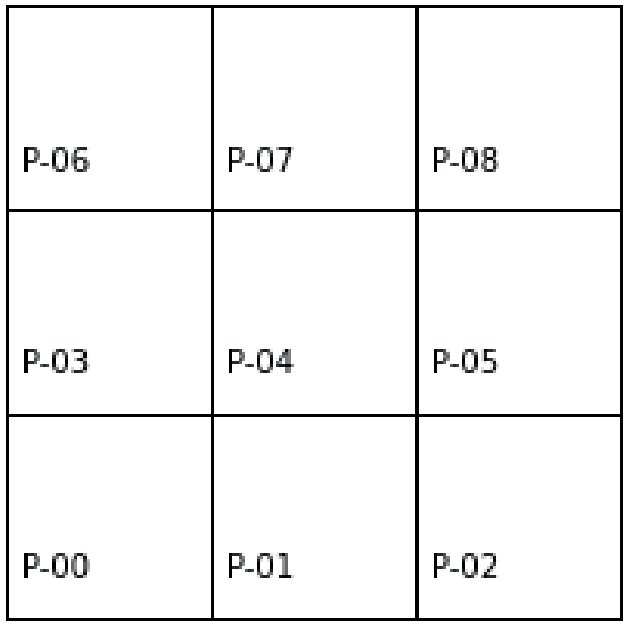


Fig. 4.1: A three-by-three grid of squares.

(continued from previous page)

```
polys = geopandas.GeoSeries(polys)
gdf = geopandas.GeoDataFrame(
    {
        "geometry": polys,
        "id": ["P-%s" % str(i).zfill(2) for i in␣
↪range(len(polys))],
    }
)
```

A common way to express contiguity/adjacency relationships arises from an analogy to the legal moves that different chess pieces can make. *Rook* contiguity requires that the pair of polygons in question share an *edge*. According to this definition, polygon 0 would be a Rook neighbor of 1 and 3, while 1 would be a Rook neighbor with 0, 2, and 4. Applying this rule to all nine polygons we can model our neighbor relations as:

```
# Build a rook contiguity matrix from a regular 3x3
# lattice stored in a geo-table
wr = weights.contiguity.Rook.from_dataframe(gdf)
```

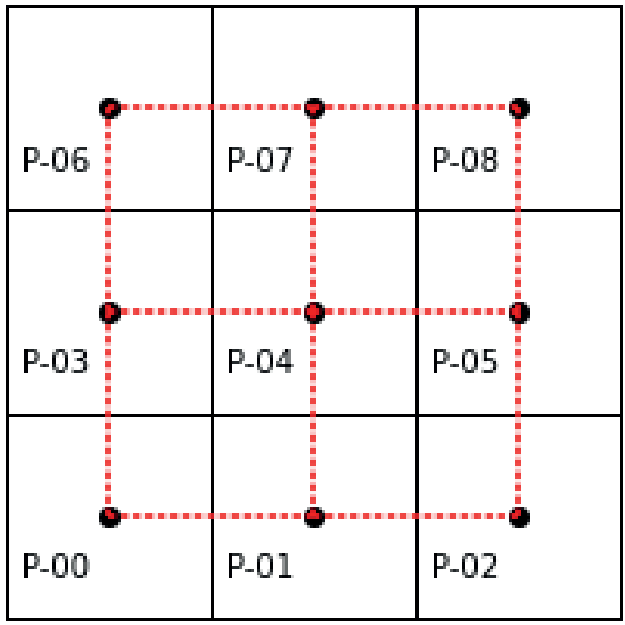


Fig. 4.2: Grid cells connected by a red line are 'neighbors' under a 'Rook' contiguity rule. Code generated for this figure is available on the web version of the book.

Note the pattern we use to build the `w` object, which is similar across the library: we specify the criterium we want for the weights (`weights.contiguity.Rook`) and then the "constructor" we will use (`from_dataframe`). We can visualize the result plotted on top of the same grid of labeled polygons, using red dotted lines to represent the edges between a pair of nodes (polygon centroids in this case). We can see this in Figure 4.2.

The `neighbors` attribute of our `pysal` W object encodes the neighbor relationships by expressing the *focal* observation on the left (in the `key` of the dictionary), and expressing the *neighbors* to the *focal* in the list on the right (in the `value` of the dictionary). This representation has computational advantages, as it exploits the sparse nature of contiguity weights matrices by recording only non-zero weights:

```
wr.neighbors
```

```
{0: [1, 3],
 1: [0, 2, 4],
```

(continued on next page)

(continued from previous page)

```
 2: [1, 5],
 3: [0, 4, 6],
 4: [1, 3, 5, 7],
 5: [8, 2, 4],
 6: [3, 7],
 7: [8, 4, 6],
 8: [5, 7]}
```

More specifically, knowing that the neighbors of polygon 0 are 3 and 1 implies that polygons $2, 4, 5, 6, 7, 8$ are not Rook neighbors of 0. As such, there is no reason to store the "non-neighbor" information and this results in significant reductions in memory requirements. However, it is possible to create the fully dense, matrix representation if needed:

```
pandas.DataFrame(*wr.full()).astype(int)
```

	0	1	2	3	4	5	6	7	8
0	0	1	0	1	0	0	0	0	0
1	1	0	1	0	1	0	0	0	0
2	0	1	0	0	0	1	0	0	0
3	1	0	0	0	1	0	1	0	0
4	0	1	0	1	0	1	0	1	0
5	0	0	1	0	1	0	0	0	1
6	0	0	0	1	0	0	0	1	0
7	0	0	0	0	1	0	1	0	1
8	0	0	0	0	0	1	0	1	0

As you can see from the matrix above, most entries are zero. In fact out of all of the possible $9^2 = 81$ linkages that there could be in this matrix, there are only 24 non-zero entries:

```
wr.nonzero
```

```
24
```

Thus, we can save a significant amount of memory and lose no information using these sparse representations, which only record the non-zero values.

More generally, the spatial weights for our 3-by-3 grid can be represented as a matrix that has 9 rows and 9 columns, matching the number of polygons $(n = 9)$. An important thing to note is that geography has more than one dimension. When compared to common representations of relationships *in time* used in data science, using information about spatial relationships can be more complex: spatial relationships are bi-directional, while temporal relationships are unidirectional. Further complicating things, the ordering of the observations in the weights matrix is arbitrary. The first row is not first for a specific mathematical reason; it just happens to be the first entry in the input. Here we

use the alphanumeric ordering of the unit identifiers to match a polygon with a row or column of the matrix, but any arbitrary rule could be followed and the weights matrix would look different. The graph, however, would be isomorphic and retain the mapping of relationships.

Spatial weights matrices may look familiar to those acquainted with social networks and graph theory in which **adjacency** matrices play a central role in expressing connectivity between nodes. Indeed, spatial weights matrices can be understood as a graph adjacency matrix where each observation is a node and the spatial weight assigned between a pair represents the weight of the edge on a graph connecting the arcs. Sometimes, this is called the **dual graph** of the input geographic data. This is advantageous, as geographic data science can borrow from the rich graph theory literature. At the same time, spatial data has numerous distinguishing characteristics that necessitate the development of specialized procedures and concepts in the handling of spatial weights. This chapter will cover many of these features.

But for now, let's get back to the Rook contiguity graph. A close inspection reveals that this criterion actually places a restriction on the spatial relation. More specifically, polygons 0 and 5 are not Rook neighbors, but they do in fact share a common border. However, in this instance the sharing is due to a common *vertex* rather than a shared *edge*. If we wanted them to be considered as neighbours, we can switch to the more inclusive notion of *Queen* contiguity, which requires the pair of polygons to only share one or more *vertices*. We can create the neighbor relations for this same configuration as follows:

```
# Build a queen contiguity matrix from a regular 3x3
# lattice stored in a geo-table
wq = weights.contiguity.Queen.from_dataframe(gdf)
wq.neighbors
```

```
{0: [1, 3, 4],
 1: [0, 2, 3, 4, 5],
 2: [1, 4, 5],
 3: [0, 1, 4, 6, 7],
 4: [0, 1, 2, 3, 5, 6, 7, 8],
 5: [1, 2, 4, 7, 8],
 6: [3, 4, 7],
 7: [3, 4, 5, 6, 8],
 8: [4, 5, 7]}
```

In addition to this neighbors representation, we can also express the graph visually, as done before. This is shown in Figure 4.3.

By using `Contiguity.Queen` rather than `Contiguity.Rook`, we consider observations that share a vertex to be neighbors. The result is that the neighbors of 0 now include 4 along with 3 and 1.

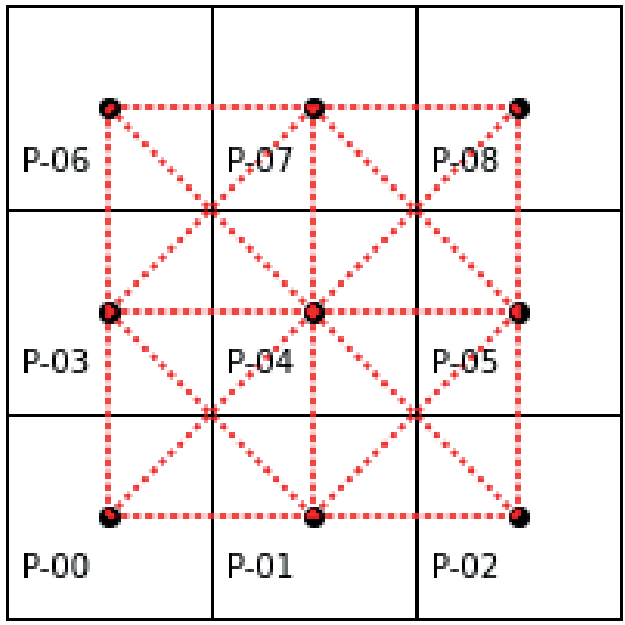


Fig. 4.3: Grid cells connected by a red line are considered 'neighbors' under 'Queen' contiguity. Code generated for this figure is available on the web version of the book.

Akin to how the `neighbors` dictionary encodes the contiguity relations, the `weights` dictionary encodes the strength of the link connecting the focal to each neighbor. For contiguity weights, observations are usually either considered "linked" or "not linked," so the resulting weights matrix is binary. As in any `pysal W` object, the actual weight values are contained in the `weights` attribute:

```
wq.weights
```

```
{0: [1.0, 1.0, 1.0],
 1: [1.0, 1.0, 1.0, 1.0, 1.0],
 2: [1.0, 1.0, 1.0],
 3: [1.0, 1.0, 1.0, 1.0, 1.0],
 4: [1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0],
 5: [1.0, 1.0, 1.0, 1.0, 1.0],
 6: [1.0, 1.0, 1.0],
 7: [1.0, 1.0, 1.0, 1.0, 1.0],
 8: [1.0, 1.0, 1.0]}
```

Similar to the `neighbors` attribute, the `weights` object is a Python dictionary

that only stores the non-zero weights. Although the weights for a given observations neighbors are all the same value for contiguity weights, it is important to note that the `weights` and `neighbors` are aligned with one another; for each observation, its first neighbor in `neighbors` has the first weight in its `weights` entry. This will be important when we examine distance based weights further on, when observations will have different weights.

In addition to the `neighbor` and `weights` attributes, the `w` object has a large number of other attributes and methods that can be useful. The `cardinalities` attribute reports the number of neighbors for each observation:

```
wq.cardinalities
```

```
{0: 3, 1: 5, 2: 3, 3: 5, 4: 8, 5: 5, 6: 3, 7: 5, 8: 3}
```

The related `histogram` attribute provides an overview of the distribution of these cardinalities:

```
wq.histogram
```

```
[(3, 4), (4, 0), (5, 4), (6, 0), (7, 0), (8, 1)]
```

We can obtain a quick visual representation by converting the cardinalities into a `pandas.Series` and creating a histogram shown in Figure 4.4:

```
pandas.Series(wq.cardinalities).plot.hist(color="k");
```

The `cardinalities` and `histogram` attributes help quickly spot asymmetries in the number of neighbors. This, as we will see later in the book, is relevant when using spatial weights in other analytical techniques (e.g., spatial autocorrelation analysis or spatial regression). Here we see that there are four corner observations with three neighbors, four edge observations with five neighbors, and the one central observation has eight neighbors. There are also no observations with four, six, or seven neighbors.

By convention, an ordered pair of contiguous observations constitutes a *join* represented by a non-zero weight in a W. The attribute `s0` records the number of joins.

```
wq.s0
```

```
40.0
```

Thus, the Queen weights here have just under twice the number of joins in this case. The `pct_nonzero` attribute provides a measure of the density (compliment of sparsity) of the spatial weights matrix (if we had it stored explicitly, which we don't):

```
wq.pct_nonzero
```

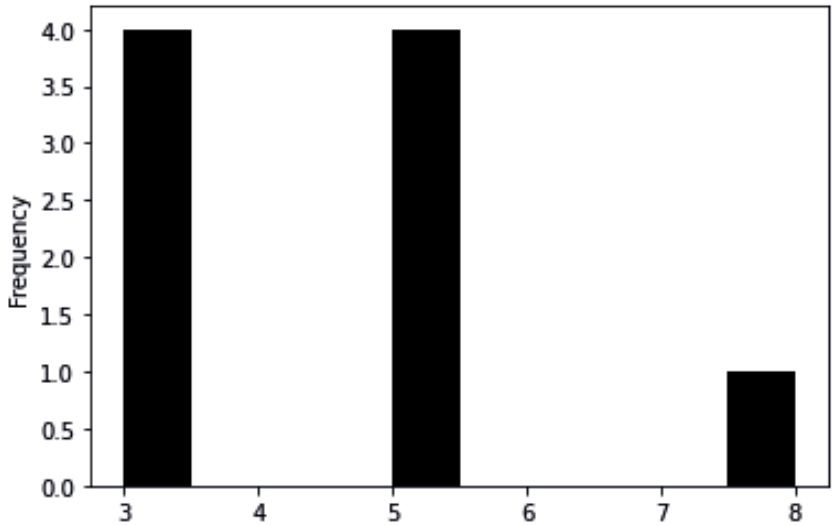


Fig. 4.4: Histogram of cardinalities (i.e., the number of neighbors each cell has) in the Queen grid.

```
49.382716049382715
```

which is equal to $100 \times (\texttt{w.s0}/\texttt{w.n}^2)$.

4.2.1 Spatial weights from real-world geographic tables

The regular lattice map encountered above helps us to understand the logic and properties of `pysal`'s spatial weights class. However, the artificial nature of that geography is of limited relevance to real world research problems. `pysal` supports the construction of spatial weights objects from a number of commonly used spatial data formats. Here we demonstrate this functionality for the case of census tracts in San Diego, California. Most spatial data formats, such as shapefiles, are non-topological in that they encode the polygons as a collection of vertices defining the edges of the geometry's boundary. No information about the neighbor relations is explicitly encoded, so we must construct it ourselves. Under the hood, `pysal` uses efficient spatial indexing structures to extract these.

```
san_diego_tracts = geopandas.read_file(
    "../data/sandiego/sandiego_tracts.gpkg"
)
w_queen = weights.contiguity.Queen.from_dataframe(san_diego_
↪tracts)
```

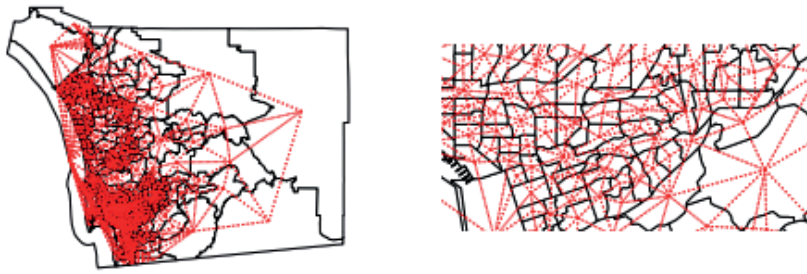


Fig. 4.5: The Queen contiguity graph for San Diego tracts. Tracts connected with a red line are neighbors. Code generated for this figure is available on the web version of the book.

Like before, we can visualize the adjacency relationships (Figure 4.5), but they are much more difficult to see without showing a closer detail. This higher level of detail is shown in the right pane of the plot.

The weights object for San Diego tracts have the same attributes and methods as we encountered with our artificial layout above:

```
print(w_queen.n)
print(w_queen.pct_nonzero)
```

```
628
1.018296888311899
```

First, we have a larger number of spatial units. The spatial weights are also much sparser for the tracts than what we saw for our smaller toy grid. Moreover, the cardinalities have a radically different distribution (Figure 4.6):

```
s = pandas.Series(w_queen.cardinalities)
s.plot.hist(bins=s.unique().shape[0]);
```

As the minimum number of neighbors is 1, while there is one polygon with 29 Queen neighbors. The most common number of neighbors is 6. For comparison, we can also plot the equivalent for Rook weights of the same dataframe:

```
w_rook = weights.contiguity.Rook.from_dataframe(san_diego_
↪tracts)
print(w_rook.pct_nonzero)
```

(continued on next page)

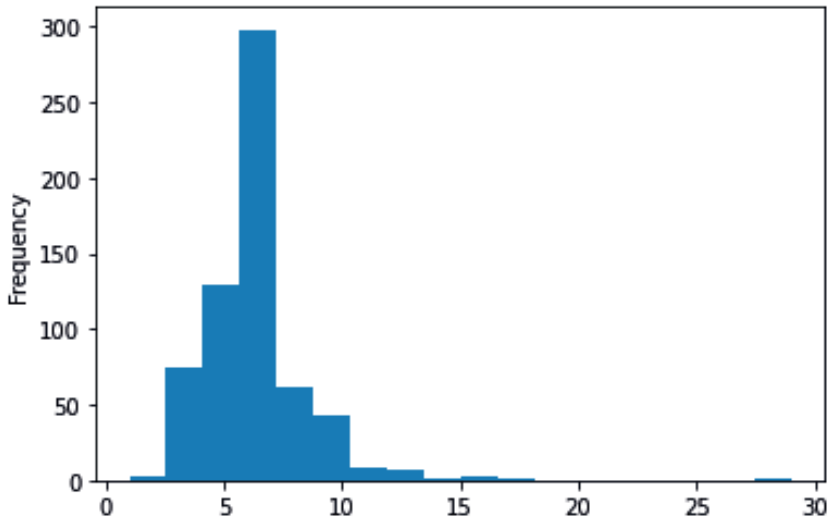


Fig. 4.6: Cardinalities for the Queen contiguity graph among San Diego tracts.

(continued from previous page)

```
s = pandas.Series(w_rook.cardinalities)
s.plot.hist(bins=s.unique().shape[0]);
```

```
0.8722463385938578
```

The cardinality histogram (Figure 4.7) shifts downward due to the increasing sparsity of the weights for the rook case relative to the Queen criterion. Conceptually, this makes sense: all Rook neighbors are also Queen neighbors, since Queen includes neighbors that share an edge; but, not all Queen neighbors are Rook neighbors, since some Queen neighbors only share a point on their boundaries in common.

The example above shows how the notion of contiguity, although more straightforward in the case of a grid, can be naturally extended beyond the particular case of a regular lattice. The principle to keep in mind is that we consider contiguous (and hence call neighbors) observations which share part of their border coordinates. In the Queen case, a single point is enough to make the join. For Rook neighbors, we require a join to consist of one or more shared edges. This distinction is less relevant in the real world than it appears in the grid example above. In any case, there are some cases where this distinction can matter and it is useful to be familiar with the differences between the two approaches.

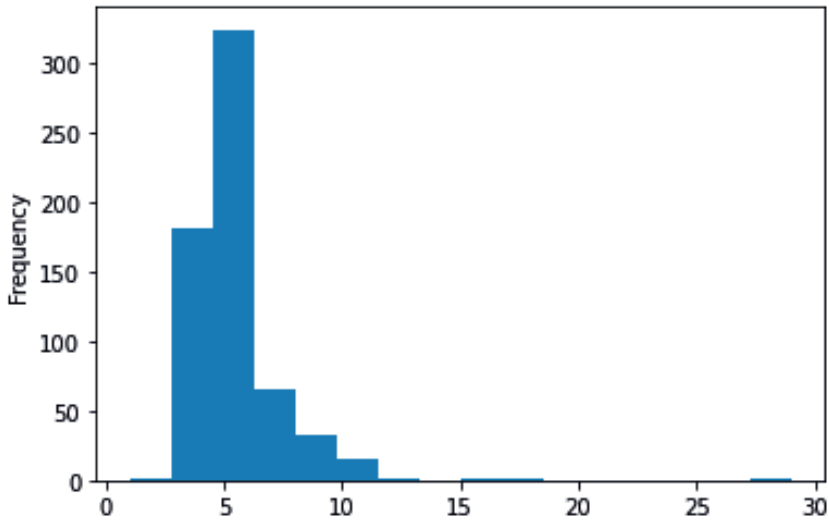


Fig. 4.7: Cardinalities for the Rook contiguity graph among San Diego tracts.

4.2.2 Spatial weights from surfaces

Most often, we will use spatial weights as a way to connect features stored in rows of a geographic table. A more niche application is spatial weights derived from surfaces. Recalling from *Chapter 1*, the boundary between which phenomena get stored as tables and which ones as surfaces is blurring. This means that analytics that were traditionally developed for tables are increasingly being used on surfaces. Here, we illustrate how one can build spatial weights from data stored in surfaces. As we will see later in the book, this widens the range of analytics that we can apply to surface data.

For the illustration, we will use a surface that contains population counts for the Sao Paulo region in Brazil:

```
sao_paulo = rioxarray.open_rasterio("../data/ghsl/ghsl_sao_
↪paulo.tif")
```

From version 2.4 onwards, `pysal` added support to build spatial weights from `xarray.DataArray` objects.

```
w_sao_paulo = weights.contiguity.Queen.from_xarray(sao_paulo)
```

Although the internals differ quite a bit, once built, the objects are a sparse version of the same object that is constructed from a geographic table.

```
w_sao_paulo
```

```
<libpysal.weights.weights.WSP at 0x7f3f8e1002b0>
```

4.3 Distance based weights

In addition to contiguity, we can also define neighbor relations as a function of the distance separating spatial observations. Usually, this means that a matrix expressing the distances between all pairs of observations are required. These are then provided to a **kernel** function which uses the proximity information to model proximity as a smooth function of distance. `pysal` implements a family of distance functions. Here we illustrate a selection beginning with the notion of *nearest neighbor* weights.

4.3.1 K-nearest neighbor weights

The first type of distance based weights defines the neighbor set of a particular observation as containing its nearest k observations, where the user specifies the value of k. To illustrate this for the San Diego tracts, we take $k = 4$. This still leaves the issue of how to measure the distance between these polygon objects, however. To do so we develop a representative point for each of the polygons using the centroid.

```
wk4 = weights.distance.KNN.from_dataframe(san_diego_tracts,␣
↪k=4)
```

The centroids are calculated from the spatial information stored in the `GeoDataFrame` as we have seen before. Since we are dealing with polygons in this case, `pysal` uses inter-centroid distances to determine the k nearest observations to each polygon.

The k-nearest neighbor weights displays no island problem, that is *everyone* has at least one neighbor:

```
wk4.islands
```

```
[]
```

This is the same for the contiguity case above but, in the case of k-nearest neighbor weights, this is by construction. Examination of the cardinality histogram for the k-nearest neighbor weights shows another built-in feature:

```
wk4.histogram
```

```
[(4, 628)]
```

Everyone has the same number of neighbors. In some cases, this is not an issue but a desired feature. In other contexts, however, this characteristic of k-nearest neighbor weights can be undesirable. In such situations, we can turn to other types of distance-based weights.

4.3.2 Kernel weights

The k-nearest neighbor rule assigns binary values to the weights for neighboring observations. `pysal` also supports continuously valued weights to reflect Tobler's first law [Tob70] in a more direct way: observations that are close to a unit have larger weights than more distant observations.

Kernel weights are one of the most commonly-used kinds of distance weights. They reflect the case where similarity/spatial proximity is assumed or expected to decay with distance. The essence of kernel weights is that the weight between observations i and j is based on their distance, but it is further modulated by a kernel function with certain properties. `pysal` implements several kernels. All of them share the properties of distance decay (thus encoding Tobler's First Law), but may decay at different rates with respect to distance.

As a computational note, it is worth mentioning that many of these distance-based decay functions require more resources than the contiguity weights or k-nearest neighbor weights discussed above. This is because the contiguity and k-nearest neighbor structures embed simple assumptions about how shapes relate in space, while kernel functions relax several of those assumptions. Thus, they provide more flexibility at the expense of computation.

The simplest way to compute Kernel weights in `pysal` involves a single function call:

```
w_kernel = weights.distance.Kernel.from_dataframe(gdf)
```

Like k-nearest neighbor weights, the Kernel weights are based on distances between observations. By default, if the input data is an areal unit, we use a central representative point (like the centroid) for that polygon. The value of the weights will be a function of two main options for kernel weights: choice of kernel function and bandwidth. The former controls how distance between i and j is "modulated" to produce the weight that goes in w_{ij}. In this respect, `pysal` offers a large number of functions that determine the shape of the distance decay function. The bandwidth specifies the distance from each focal unit over which the kernel function is applied. For observations separated by distances larger than the bandwidth, the weights are set to zero.

The default values for kernels are to use a triangular kernel with a bandwidth distance equal to the maximum knn=2 distance for all observations. The latter implies a so-called fixed bandwidth where all observations use the same distance for the cut-off. We can inspect this from the generated `W` object:

```
w_kernel.function
```

```
'triangular'
```

for the kernel function, and:

```
# Show the first five values of bandwidths
w_kernel.bandwidth[0:5]
```

```
array([[1.0000001],
       [1.0000001],
       [1.0000001],
       [1.0000001],
       [1.0000001]])
```

For the bandwidth applied to each observation.

Although simple, a fixed bandwidth is not always the best choice. For example, in cases where the density of the observations varies over the study region, using the same threshold anywhere will result in regions with a high density of neighbors while others with observations very sparsely connected. In these situations, an *adaptive* bandwidth -one which varies by observation and its characteristics- can be preferred.

Adaptive bandwidths are picked again using a K-nearest neighbor rule. A bandwidth for each observation is chosen such that, once the k-nearest observation is considered, all the remaining observations have zero weight. To illustrate it, we will use a subset of tracts in our San Diego dataset. First, visualizing the centroids, we can see that they are not exactly regularly-spaced, although others do nearly fall into a regular spacing (Figure 4.8):

```
# Create subset of tracts
sub_30 = san_diego_tracts.query("sub_30 == True")
# Plot polygons
ax = sub_30.plot(facecolor="w", edgecolor="k")
# Create and plot centroids
sub_30.head(30).centroid.plot(color="r", ax=ax)
# Remove axis
ax.set_axis_off();
```

If we now build a weights object with adaptive bandwidth (`fixed=False`), the values for bandwidth differ:

```
# Build weights with adaptive bandwidth
w_adaptive = weights.distance.Kernel.from_dataframe(
    sub_30, fixed=False, k=15
)
```

(continued on next page)

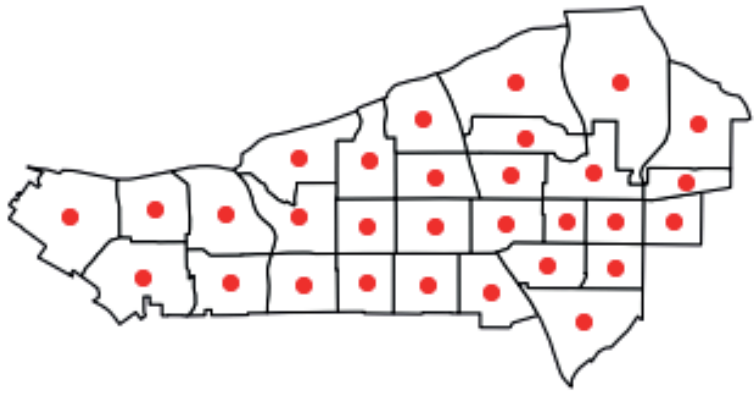


Fig. 4.8: Centroids of some tracts in San Diego are (nearly) evenly spaced.

(continued from previous page)

```
# Print first five bandwidth values
w_adaptive.bandwidth[:5]
```

```
array([[7065.74020822],
       [3577.22591841],
       [2989.74807871],
       [2891.46196945],
       [3965.08354232]])
```

And, we can visualize what these kernels look like on the map, too, by focusing on an individual unit and showing how the distance decay attenuates the weight by grabbing the corresponding row of the full kernel matrix (Figure 4.9):

```
# Create full matrix version of weights
full_matrix, ids = w_adaptive.full()
# Set up figure with two subplots in a row
f, ax = plt.subplots(
    1, 2, figsize=(12, 6), subplot_kw=dict(aspect="equal")
)
# Append weights for first polygon and plot on first subplot
sub_30.assign(weight_0=full_matrix[0]).plot(
    "weight_0", cmap="plasma", ax=ax[0]
)
# Append weights for 18th polygon and plot on first subplot
sub_30.assign(weight_18=full_matrix[17]).plot(
    "weight_18", cmap="plasma", ax=ax[1]
)
```

(continued on next page)

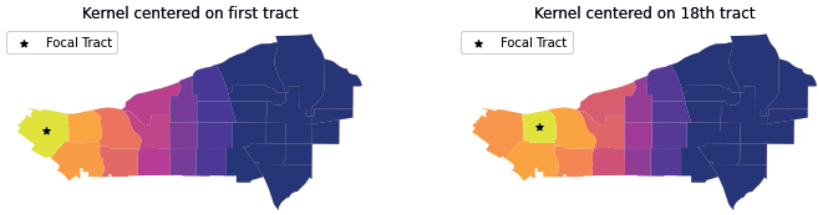


Fig. 4.9: A Gaussian kernel centered on two different tracts.

(continued from previous page)

```
# Add centroid of focal tracts
sub_30.iloc[[0], :].centroid.plot(
    ax=ax[0], marker="*", color="k", label="Focal Tract"
)
sub_30.iloc[[17], :].centroid.plot(
    ax=ax[1], marker="*", color="k", label="Focal Tract"
)
# Add titles
ax[0].set_title("Kernel centered on first tract")
ax[1].set_title("Kernel centered on 18th tract")
# Remove axis
[ax_.set_axis_off() for ax_ in ax]
# Add legend
[ax_.legend(loc="upper left") for ax_ in ax];
```

What the kernel looks like can be strongly affected by the structure of spatial proximity, so any part of the map can look quite different from any other part of the map. By imposing a clear distance decay over several of the neighbors of each observation, kernel weights incorporate Tobler's law explicitly. Often, this comes at the cost of increased memory requirements, as every single pair of observations within the bandwidth distance is considered:

```
w_kernel.pct_nonzero
```

```
40.74074074074074
```

In many instances, this may be at odds with the nature of the spatial interactions at hand, which operate over a more limited range of distance. In these cases, expanding the neighborhood set beyond might lead us to consider interactions which do not take place, or are inconsequential. Thus, for both substantive and computational reasons, it might make sense to further limit the range, keeping impacts hyper-local.

4.3.3 Distance bands and hybrid weights

In some contexts, it makes sense to draw a circle around each observation and consider as neighbors every other observation that falls within the circle. In the GIS terminology, this is akin to drawing a buffer around each point and performing a point-in-polygon operation that determines whether the other observations are within the buffer. If they are, they are assigned a weight of one in the spatial weights matrix; if not they receive a zero.

```
w_bdb = weights.distance.DistanceBand.from_dataframe(
    gdf, 1.5, binary=True
)
```

This creates a binary distance weights where every other observation within a distance of 1.5 is considered neighbor.

Distance band weights can also be continuously weighted. These could be seen as a kind of "censored" kernel, where the kernel function is applied only within a pre-specified distance. For example, let us calculate the DistanceBand weights that use inverse distance weights up to a certain threshold and then truncate the weights to zero for everyone else. For this example we will return to the small lattice example covered in the beginning:

```
w_hy = weights.distance.DistanceBand.from_dataframe(
    gdf, 1.5, binary=False
)
```

We apply a threshold of 1.5 for this illustration. `pysal` truncates continuous weights at this distance. It is important to keep in mind that the threshold distance must use the same units of distance as the units used to define the matrix.

To see the difference, consider polygon 4, in the middle of the grid. The Queen set of weights includes eight neighbors with a uniform weight of one:

```
wq.weights[4]
```

```
[1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0]
```

while the hybrid weights object modulates, giving less relevance to further observations (i.e., in this case those that only share a point):

```
w_hy.weights[4]
```

```
[0.7071067811865475,
 1.0,
 0.7071067811865475,
```

(continued on next page)

(continued from previous page)

```
 1.0,
 1.0,
 0.7071067811865475,
 1.0,
 0.7071067811865475]
```

4.3.4 Great circle distances

We must make one final curve before leaving the distance based weights. It is important that the calculation of distances between objects takes the curvature of the Earth's surface into account. This can be done before computing the spatial weights object, by transforming the coordinates of data points into a projected reference system. If this is not possible or convenient, an approximation that considers the curvature implicit in non-projected reference systems (e.g., longitude/latitude) can be a sufficient workaround. `pysal` provides such approximation as part of its functionality.

To illustrate the relevance of ignoring this aspect altogether, we will examine distance based weights for the case of counties in the state of Texas. First, let us compute a KNN-4 object that ignores the curvature of the Earth's surface (note how we use in this case the `from_shapefile` constructor to build the weights directly from a shapefile full of polygons):

```
# ignore curvature of the earth
knn4_bad = weights.distance.KNN.from_shapefile(
    "../data/texas/texas.shp", k=4
)
```

Next, let us take curvature into account. To do this, we require the radius of the Earth expressed in a given metric. `pysal` provides this number in both miles and kilometers. For the sake of the example, we will use miles:

```
radius = geometry.sphere.RADIUS_EARTH_MILES
radius
```

```
3958.755865744055
```

With this measure at hand, we can pass it to the weights constructor (either straight from a shapefile or from a `GeoDataFrame`), and distances will be expressed in the units we have used for the radius, that is in miles in our case:

```
knn4 = weights.distance.KNN.from_shapefile(
    "../data/texas/texas.shp", k=4, radius=radius
)
```

Comparing the resulting neighbor sets, we see that ignoring the curvature of the Earth's surface can create erroneous neighbor pairs. For example, the four *correct* nearest neighbors to observation 0 when accounting for the Earth's curvature are 6, 4, 5, and 3. However, observation 13 is *ever so slightly* closer when computing the straight line distance instead of the distance that accounts for curvature.

```
knn4[0]
```

```
{6: 1.0, 4: 1.0, 5: 1.0, 3: 1.0}
```

```
knn4_bad[0]
```

```
{6: 1.0, 4: 1.0, 5: 1.0, 13: 1.0}
```

4.4 Block weights

A final type of spatial weight we examine here is block weights. In this case, it is membership in a geographic group that defines the neighbor relationships. Block weights connect every observation in a dataset that belongs to the same category in a provided list.[1] In essence, a block weight structure groups individual observations and considers all members of the group as "near" one another. This means that they then have a value of one for every pair of observations in the same group. Contrariwise, all members *not* in that group are considered disconnected from any observation within the group, and given a value of zero. This is done for every group, so the resulting matrix looks like "blocks" of 1s stacked on the diagonal (assuming that observations in the same group are near one another in the input data table), hence the "block" weights.

To demonstrate this class of spatial weights, we will use the tract dataset for San Diego and focus on their county membership:

```
san_diego_tracts[["GEOID", "state", "county", "tract"]].head()
```

```
         GEOID state county   tract
0  06073018300    06    073  018300
1  06073018601    06    073  018601
2  06073017601    06    073  017601
3  06073019301    06    073  019301
4  06073018700    06    073  018700
```

Every tract has a unique ID (`GEOID`) and a county ID, shared by all tracts in the same

[1] Usually, this list will have some relation to the spatial configuration of the data, but, technically speaking, all one needs to create block weights is a list of memberships.

county. Since the entire region of San Diego is in California, the state ID is the same across the dataset.

To build a block weights object, we do not even need spatial data beyond the list of memberships. In this case, we will use the county membership:

```
# NOTE: since this is a large dataset, it might take a while␣
↪to process
w_bl = weights.util.block_weights(
    san_diego_tracts["county"].values,
    ids=san_diego_tracts["GEOID"].values,
)
```

As a check, let's consider the first two rows in the table above. If the block weights command has worked out correctly, both should be neighbors:

```
"06073000201" in w_bl["06073000100"]
```

```
True
```

We can use block weights as an intermediate step in more involved analyses of "hybrid" spatial relationships. Suppose, for example, the researcher wanted to allow for Queen neighbors within counties but not for tracts across different counties. In this case, tracts from different counties would not be considered neighbors. To create such as spatial weights matrix would require a combination of the Queen and the block criteria, and `pysal` can implement that blending through one of the set operations shown in the next section.

4.5 Set operations on weights

So far, we have seen different principles that guide how to build spatial weights matrices. In this section, we explore how we can create new matrices by *combining* different existing ones. This is useful in contexts where a single neighborhood rule is inapt or when guiding principles point to combinations of criteria.

We will explore these ideas in the section by returning to the San Diego tracts. A number of ways exist to expand the basic criteria we have reviewed above and create hybrid weights. In this example, we will generate a combination of the original contiguity weights and the nearest neighbor weights. We will examine two different approaches that provide similar solutions, thus illustrating the value of set operations in `pysal`.

4.5.1 Editing/connecting disconnected observations

Imagine one of our tracts was an island and did not have any neighbors in the contiguity case. This can create issues in the spatial analytics that build on spatial weights, so it is good practice to amend the matrix before using it. The first approach we adopt is to find the nearest neighbor for the island observation and then add this pair of neighbors to extend the neighbor pairs from the original contiguity weight to obtain a fully connected set of weights.

We will assume, for the sake of the example, that the disconnected observation was number 103. For us to reattach this tract, we can assign it to be "connected" to its nearest neighbor. Let's first extract our "problem" geometry:

```
disconnected_tract = san_diego_tracts.iloc[[103]]
```

As we have seen above, this tract *does* have neighbors:

```
w_queen[103]
```

```
{160: 1.0, 480: 1.0, 98: 1.0, 324: 1.0, 102: 1.0, 107: 1.0,␣
↪173: 1.0}
```

But, for this example, we will assume it does not and thus we find ourselves in the position of having to create additional neighboring units. This approach does not only apply in the context of islands. Sometimes, the process we are interested in may require that we manually edit the weights to better reflect connections we want to capture.

We will connect the observation to its nearest neighbor. To do this, we can construct the K-NN graph as we did above, but set `k=1`, so observations are only assigned to their nearest neighbor:

```
wk1 = weights.distance.KNN.from_dataframe(san_diego_tracts,␣
↪k=1)
```

In this graph, all our observations are connected to one other observation by construction:

```
wk1.histogram
```

```
[(1, 628)]
```

So is, of course, our tract of interest:

```
wk1.neighbors[103]
```

```
[102]
```

To connect it in our initial matrix, we need to create a copy of the `neighbors` dictionary and update the entry for `103`, including `102` as a neighbor. We copy the neighbors:

```
neighbors = w_rook.neighbors.copy()
```

and then we change the entry for the island observation to include its nearest neighbor (`102`) as well as update `102` to have `103` as a neighbor:

```
neighbors[103].append(102)
neighbors[102].append(103)
w_new = weights.W(neighbors)
w_new[103]
```

```
{480: 1.0, 160: 1.0, 324: 1.0, 102: 1.0, 107: 1.0, 173: 1.0}
```

4.5.2 Using the `union` of matrices

A more elegant approach to the island problem makes use of `pysal`'s support for *set theoretic operations* on `pysal` weights. For example, we can construct the *union* of two weighting schemes, connecting any pair of observations if they are connected in *either* the Rook or if they are nearest neighbors:

```
w_fixed_sets = weights.set_operations.w_union(w_rook, wk1)
```

It is important to mention that this approach is not exactly the same, at least in principle, as the one above. It could be that the nearest observation was not originally a Rook neighbor and, in this case, the resulting matrices would differ. This is a rare but theoretically possible situation.

4.6 Visualizing weight set operations

To further build the intuition behind different criteria, in this section we illustrate these concepts using the 32 states of Mexico. We compare the neighbor graphs that results from some of the criteria introduced to define neighbor relations. We first read in the data for Mexico:

```
mx = geopandas.read_file("../data/mexico/mexicojoin.shp")
```

We will contrast the look of the connectivity graphs built following several criteria; so, to streamline things, let's build the weights objects first:

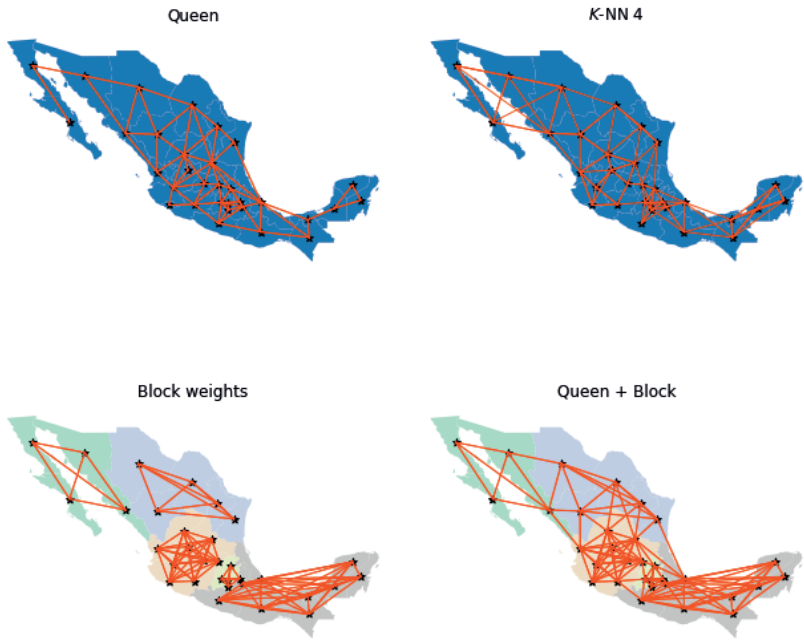


Fig. 4.10: The three graphs discussed above are shown side-by-side. Code generated for this figure is available on the web version of the book.

- Queen contiguity

```
mx_queen = weights.contiguity.Queen.from_dataframe(mx)
```

- K-NN with four nearest neighbors

```
mx_knn4 = weights.KNN.from_dataframe(mx, k=4)
```

- Block weights at the federal region level

```
mx_bw = weights.util.block_weights(mx["INEGI2"].values)
```

- A combination of block and Queen that connects contiguous neighbors *across* regions

```
mx_union = weights.set_operations.w_union(mx_bw, mx_queen)
```

With these at hand, we will build a figure that shows the connectivity graph of each weights object. For cases where the federal regions are used to define blocks, we will color states based on the region they belong to (Figure 4.10).

Queen and K-NN graphs are relatively similar but, as one would expect, the K-NN is sparser than Queen in areas with high density of irregular polygons (Queen will connect each to more than four), and denser in sparser areas with less but larger polygons (e.g., northwest). Focusing on the Queen and Block graphs, there are clear distinctions between the connectivity structures. The Block graph is visually denser in particular areas relative to the Queen graph, and this is captured in their sparsity measures:

```
mx_bw.pct_nonzero
```

```
19.140625
```

```
mx_queen.pct_nonzero
```

```
13.4765625
```

The other distinguishing characteristic can be seen in the number of connected components in the different graphs. The Queen graph has a single connected component, which in graph theory terms, means for all pairs of states there is at least one path of edges that connects the two states. The Block graph has five connected components, one for each of the five regions. Moreover, each of these connected components is fully-connected, meaning there is an edge that directly connects each pair of member states. However, there are no edges between states belonging to different blocks (or components).

As we will see in later chapters, certain spatial analytical techniques require a fully connected weights graph. In these cases, we could adopt the Queen definition since this satisfies the single connected component requirement. However, we may wish to use the Union weights graph, as that provides a single connected component, but offers a blend of different types of connectivity intensities, with the intra-regional (block) linkages being very dense, while the inter-regional linkages are thinner but provide for the single connected component.

4.7 Use case: boundary detection

We close the chapter with an illustration of how weights can be useful by themselves in geographic data science. Note that the application displayed below involves some concepts and code that are a bit more advanced than in the rest of the chapter. If you are up for the challenge, we think the insights it enables are worth the effort!

Spatial weights are ubiquitous in the analysis of spatial patterns in data, since they provide a direct method to represent spatial structure. However, spatial weights are also useful in their own right, such as when examining latent structures directly in the graphs themselves or when using them to conduct descriptive analysis. One clear use case that arises in the analysis of social data is to characterize latent *data discontinuities*. By *data*

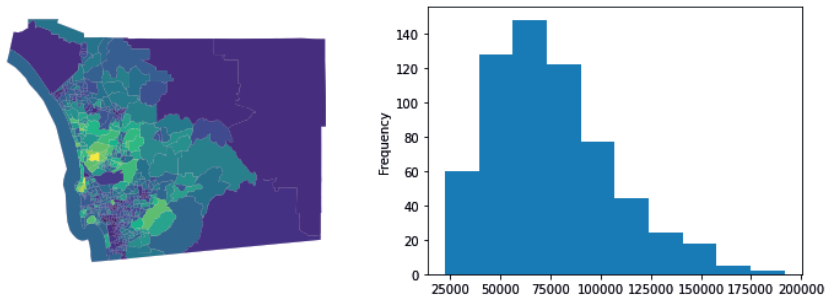


Fig. 4.11: Median household incomes in San Diego.

discontinuity, we mean a single border (or collection of borders) where data for a variable (or many variables) of interest change abruptly. These can be used in models of inequality [LC05][FPP+10][DDPP19] or used to adapt classic empirical outlier detection methods. Below, we'll show one model-free way to identify empirical boundaries in your data.

First, let's consider the median household income for our census tracts in San Diego, shown in Figure 4.11.

```
f, ax = plt.subplots(1, 2, figsize=(12, 4))
san_diego_tracts.plot("median_hh_income", ax=ax[0])
ax[0].set_axis_off()
san_diego_tracts["median_hh_income"].plot.hist(ax=ax[1])
plt.show()
```

Now, we see some cases where there are very stark differences between neighboring areas, and some cases where there appear to be no difference between adjacent areas. Digging into this, we can examine the *distribution of differences* in neighboring areas using the adjacency list, a different representation of a spatial graph:

```
adjlist = w_rook.to_adjlist()
adjlist.head()
```

	focal	neighbor	weight
0	0	1	1.0
1	0	385	1.0
2	0	4	1.0
3	0	548	1.0
4	0	27	1.0

This provides us with a table featuring three columns. `Focal` is the column containing the "origin" of the link, `neighbor` is the column containing the "destination" of

the link, or neighbor of the focal polygon, and `weight` contains how strong the link from `focal` to `neighbor` is. Since our weights are *symmetrical*, this table contains two entries per pair of neighbors, one for `(focal,neighbor)` and the other for `(neighbor,focal)`.

Now we want to connect this table representing spatial structure with information on median household income. Using `pandas`, we can merge up the focal units' and neighboring units' median household incomes:

```
adjlist_income = adjlist.merge(
    san_diego_tracts[["median_hh_income"]],
    how="left",
    left_on="focal",
    right_index=True,
).merge(
    san_diego_tracts[["median_hh_income"]],
    how="left",
    left_on="neighbor",
    right_index=True,
    suffixes=("_focal", "_neighbor"),
)
adjlist_income.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3440 entries, 0 to 3439
Data columns (total 5 columns):
 #   Column                     Non-Null Count  Dtype
---  ------                     --------------  -----
 0   focal                      3440 non-null   int64
 1   neighbor                   3440 non-null   int64
 2   weight                     3440 non-null   float64
 3   median_hh_income_focal     3440 non-null   float64
 4   median_hh_income_neighbor  3440 non-null   float64
dtypes: float64(3), int64(2)
memory usage: 134.5 KB
```

This operation brings together the income at both the focal observation and the neighbor observation. The difference between these two yields income differences between *adjacent* tracts:

```
adjlist_income["diff"] = (
    adjlist_income["median_hh_income_focal"]
    - adjlist_income["median_hh_income_neighbor"]
)
```

With this information on difference we can now do a few things. First, we can compare whether or not this *distribution* is distinct from the distribution of non-neighboring tracts' differences in wealth. This will give us a hint at the extent to which income

follows a spatial pattern. This is also discussed more in depth in the spatial inequality chapter, specifically in reference to the Spatial Gini.

To do this, we can first compute the all-pairs differences in income using the `numpy.subtract` function. Some functions in `numpy` have special functionality; these `ufuncs` (short for "universal functions") often support special applications to your data. Here, we will use `numpy.subtract.outer` to take the difference over the "outer Cartesian product" of two vectors.

```
all_pairs = numpy.subtract.outer(
    san_diego_tracts["median_hh_income"].values,
    san_diego_tracts["median_hh_income"].values,
)
```

In practice, this results in an $N \times N$ array that stores the subtraction of all combinations of the input vectors.

Then, we need to filter out those cells of `all_pairs` that are neighbors. Fortunately, our weights matrix is *binary*. So, subtracting it from an $N \times N$ matrix of 1s will result in the *complement* of our original weights matrix:

```
complement_wr = 1 - w_rook.sparse.toarray()
```

Note `complement_wr` inserts a 0 where `w_rook` includes a 1, and vice versa. Using this complement, we can filter the `all_pairs` matrix to only consider the differences in median household income for tracts that are not neighboring:

```
non_neighboring_diffs = (complement_wr * all_pairs).flatten()
```

Now, we can compare the two distributions of the difference in wealth:

```
f = plt.figure(figsize=(12, 3))
plt.hist(
    non_neighboring_diffs,
    color="lightgrey",
    edgecolor="k",
    density=True,
    bins=50,
    label="Nonneighbors",
)
plt.hist(
    adjlist_income["diff"],
    color="salmon",
    edgecolor="orangered",
    linewidth=3,
    density=True,
    histtype="step",
    bins=50,
```

(continued on next page)

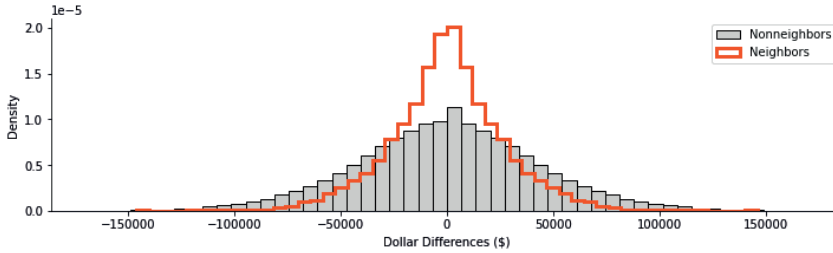


Fig. 4.12: Diferences between median incomes among neighboring (and non-neighboring) tracts in San Diego.

(continued from previous page)

```
    label="Neighbors",
)
seaborn.despine()
plt.ylabel("Density")
plt.xlabel("Dollar Differences ($)")
plt.legend();
```

From Figure 4.12, we can see that the two distributions are distinct, with the distribution of difference in *non-neighboring* tracts being slightly more dispersed than that for *neighboring* tracts. Thus, on the whole, this means that neighboring tracts have more *smaller differences in wealth* than non-neighboring tracts. This is consistent with the behavior we will talk about in later chapters concerning *spatial autocorrelation*, the tendency for observations to be statistically more similar to nearby observations than they are to distant observations.

The adjacency table we have built can also help us find our *most extreme* observed differences in income, hinting at possible hard boundaries between the areas. Since our links are symmetric, we can then focus only on focal observations with *the most extreme* difference in wealth from their immediate neighbors, considering only those where the *focal* is higher, since they each have an equivalent *negative* back-link.

```
extremes = adjlist_income.sort_values("diff",␣
↪ascending=False).head()
extremes
```

```
      focal  neighbor  weight  median_hh_income_focal  \
2605    473       163     1.0                183929.0
2609    473       157     1.0                183929.0
1886    343       510     1.0                151797.0
2610    473       238     1.0                183929.0
```

(continued on next page)

(continued from previous page)

```
54         8         89     1.0                  169821.0

      median_hh_income_neighbor      diff
2605                    37863.0  146066.0
2609                    64688.0  119241.0
1886                    38125.0  113672.0
2610                    74485.0  109444.0
54                      66563.0  103258.0
```

Thus, we see that observation 473 appears often on the `focal` side, suggesting it's quite distinct from its nearby polygons.

To verify whether these differences are truly significant, we can use a map randomization strategy. In this case, we shuffle values across the map and compute *new* `diff` columns. This time, `diff` represents the difference between random incomes, rather than the neighboring incomes we actually observed using our Rook contiguity matrix. Using many `diff` vectors, we can find the observed differences which tend to be much larger than those encountered in randomly-drawn maps of household income.

To start, we can construct many random `diff` vectors:

```
## NOTE: this cell runs a simulation and may take a bit longer
## If you want it to run faster, decrease the number of␣
↪shuffles
## by setting a lower value in `n_simulations`

# Set number or random shuffles
n_simulations = 1000
# Create an empty array to store results
simulated_diffs = numpy.empty((len(adjlist), n_simulations))
# Loop over each random draw
for i in range(n_simulations):
    # Extract income values
    median_hh_focal = adjlist_income["median_hh_income_focal
↪"].values
    # Shuffle income values across locations
    random_income = (
        san_diego_tracts[["median_hh_income"]]
        .sample(frac=1, replace=False)
        .reset_index()
    )
    # Join income to adjacency
    adjlist_random_income = adjlist.merge(
        random_income, left_on="focal", right_index=True
    ).merge(
        random_income,
        left_on="neighbor",
```

(continued on next page)

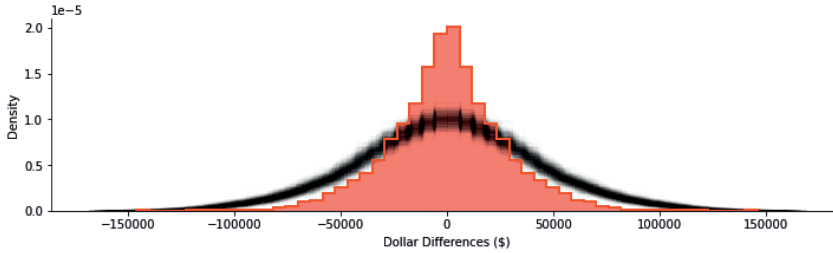


Fig. 4.13: Differences between neighboring incomes for the observed map (orange) and maps arising from randomly reshuffled maps (black) of tract median incomes. Code generated for this figure is available on the web version of the book.

(continued from previous page)

```
        right_index=True,
        suffixes=("_focal", "_neighbor"),
    )
    # Store reslults from random draw
    simulated_diffs[:, i] = (
        adjlist_random_income["median_hh_income_focal"]
        - adjlist_random_income["median_hh_income_neighbor"]
    )
```

After running our simulation, we get many distributions of pairwise differences in household income. Below, we plot the shroud of all the simulated differences, shown in black, and our observed differences, shown in red (Figure 4.13):

Again, our random distribution is much more dispersed than our observed distribution of the differences between nearby tracts. Empirically, we can pool our simulations and construct and use their quantiles to summarize how unlikely any of our *observed* differences are if neighbors' household incomes were randomly assigned:

```
simulated_diffs.flatten().shape
```

```
(3440000,)
```

```
# Convert all simulated differences into a single vector
pooled_diffs = simulated_diffs.flatten()
# Calculate the 0.5th, 50th and 99.5th percentiles
lower, median, upper = numpy.percentile(
    pooled_diffs, q=(0.5, 50, 99.5)
)
# Create a swith that is True if the value is "extreme"
```

(continued on next page)

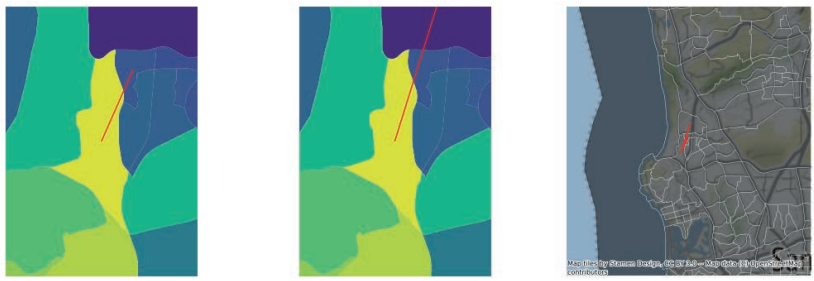


Fig. 4.14: The two starkest differences in median household income among San Diego tracts. Code generated for this figure is available on the web version of the book.

(continued from previous page)

```
# (in the 0.5th percentile or`|` in the 00.5th), False␣
↪otherwise
outside = (adjlist_income["diff"] < lower) | (
    adjlist_income["diff"] > upper
)
```

Despite the fact that our observed differences are less dispersed on average, we can identify two boundaries in the data that are in the top 1% most extreme differences in neighboring household incomes across the map. These boundaries are shown in the table below:

```
adjlist_income[outside]
```

```
      focal  neighbor  weight  median_hh_income_focal  \
885     157       473     1.0                 64688.0
915     163       473     1.0                 37863.0
2605    473       163     1.0                183929.0
2609    473       157     1.0                183929.0

      median_hh_income_neighbor      diff
885                    183929.0 -119241.0
915                    183929.0 -146066.0
2605                    37863.0  146066.0
2609                    64688.0  119241.0
```

Note that one of these, observation 473, appears in both boundaries. This means that the observation is likely to be *outlying*, extremely unlike *all* of its neighbors. These kinds of generalized neighborhood comparisons are discussed in the subsequent chapter on local spatial autocorrelation. For now we can visualize this on a map, focusing on the two boundaries around observation 473, shown also in the larger context of San Diego incomes (Figure 4.14).

These are the starkest contrasts in the map, and result in the most distinctive divisions between adjacent tracts' household incomes.

4.8 Conclusion

Spatial weights are central to how we *represent* spatial relationships in mathematical and computational environments. At their core, they are a "geo-graph," or a network defined by the geographical relationships between observations. They form kind of a "spatial index," in that they record which observations have a specific geographical relationship. Since spatial weights are fundamental to how spatial relationships are represented in geographic data science, we will use them again and again throughout the book.

4.9 Questions

1. Rook contiguity and Queen contiguity are two of three kinds of contiguity that are defined in terms of chess analogies. The third kind, *Bishop contiguity*, applies when two observations are considered connected when they share single vertices, but are considered *disconnected* if they share an edge. This means that observations that exhibit Queen contiguity are those that exhibit either Rook or Bishop contiguity. Using the Rook and Queen contiguity matrices we built for San Diego and the `Wsets.w_difference` function, are there any Bishop-contiguous observations in San Diego?
2. Different kinds of spatial weights objects can result in very different kinds of graph structures. Considering the `cardinalities` of the Queen, Block, and the union of Queen and Block,
   (a) Which graph type has the highest average cardinality?
   (b) Which graph has more non-zero entries?
   (c) Why might this be the case?
3. Graphs are considered "connected" when you can construct a path from any observation to every other observation. A "disconnected" graph has at least one node where there is no path from it to every other node. And, a "connected component" is a part of the graph that is connected internally, but it is disconnected from another part of the graph. This is reported for every spatial weights object in its `w.n_components`.
   (a) How many components does the Queen Contiguity weights for San Diego have?
   (b) Using a K-nearest Neighbor Graph for San Diego tracts where $k = 1$, how many connected components are there in this graph?

(c) Increase k by one until the `n_components` is 1. Make a plot of the relationship between k and `n_components`.

(d) What value of k does `n_components` become 1?

(e) How many non-zero links does this network have?

4. Comparing their average cardinality and percentage of non-zero links, which graph in this chapter has the *most sparse* structure? That is, which graph is the most sparsely connected?

5. In this chapter, we worked with regular *square* lattices using the `lat2W` function. In the same manner, the `hexLat2W` function can generate *hexagonal regular lattices*. For lattices of size (3,3), (6,6), and (9,9) for Rook and Queen `lat2W`, as well as for `hexLat2W`:

   (a) Examine the average cardinality. Does `lat2W` or `hexLat2W` have higher average cardinality?

   (b) Further, make a histogram of the cardinalities. Which type of lattice has higher variation in its number of neighbors?

   (c) Why is there no `rook=True` option in `hexLat2W`, as there is in `lat2W`?

6. The *Voronoi diagram* is a common method to construct polygons from a point dataset. A Voronoi diagram is built up from *Voronoi cells*, each of which contains the area that is closer to its source point than any other source point in the diagram. Further, the Queen contiguity graph for a *Voronoi diagram* obeys a number of useful properties, since it is the *Delaunay Triangulation* of a set of points.

   (a) Using the following code, build and plot the Voronoi diagram for the *centroids* of Mexican states, with the states and their centroids overlayed:

```
from pysal.lib.weights.distance import get_points_
↪array
from pysal.lib.cg import voronoi_frames

centroid_coordinates = get_points_array(mx.centroid)
cells, centers = voronoi_frames(centroid_coordinates)

ax = cells.plot(facecolor='none', edgecolor='k')
mx.plot(ax=ax, edgecolor='red', facecolor='whitesmoke
↪', alpha=.5)
mx.centroid.plot(ax=ax,color='red', alpha=.5,␣
↪markersize=10)
ax.axis(mx.total_bounds[[0,2,1,3]])
plt.show()
```

(a) Using the `weights.Voronoi` function, build the Voronoi weights for the Mexico states data.

(b) Compare the connections in the Voronoi and Queen weights for the Mexico states data. Which form is more connected?

(c) Make a plot of the Queen contiguity and Voronoi contiguity graphs to compare them visually, like we did with the block weights and Queen weights. How do the two graphs compare in terms of the length of their links and how they connect the Mexican states?

(d) Using `weights.set_operations`, find any links that are in the Voronoi contiguity graph, but not in the Queen contiguity graph. Alternatively, find any links that are in the Queen contiguity graph, but not the Voronoi contiguity graph.

7. Interoperability is important for the Python scientific stack. Thanks to standardization around the `numpy` array and the `scipy.sparse` array data structures, it is simple and computationally-easy to convert objects from one representation to another:

(a) Using `w.to_networkx()`, convert the Mexico Regions Queen+Block weights matrix to a `networkx` graph. Compute the Eigenvector Centrality of that new object using `networkx.eigenvector_centrality`

(b) Using `w.sparse`, compute the number of connected components in the Mexico Regions Block weights matrix using the `connected_components` function in `scipy.sparse.csgraph`.

(c) Using `w.sparse`, compute the all-pairs shortest path matrix in the Mexico Queen weights matrix using the `shortest_path` function in `scipy.sparse.csgraph`.

8. While every node in a k-nearest neighbor graph has five neighbors, there is a conceptual difference between *in-degree* and *out-degree* of nodes in a graph. The *out-degree* of a node is the number of outgoing links from a node; for a K-Nearest Neighbor graph, this is k for every variable. The *in-degree* of a node in a graph is the number of *incoming* links to that node; for a K-Nearest Neighbor graph, this is the number of other observations that pick the target as their nearest neighbor. The *in-degree* of a node in the K-Nearest Neighbor graph can provide a measure of *hubbiness*, or how central a node is to other nodes.

(a) Using the San Diego tracts data, build a $k = 6$ nearest neighbor weight and call it `knn_6`.

(b) Verify that the $k = 6$ by taking the row sum over the weights matrix in `knn_6.sparse`.

(c) Compute the in-degree of each observation by taking the *column sum* over the weights matrix in `knn_6.sparse`, and divide by 6, the out-degree for all observations.

(d) Make a histogram of the in-degrees for the $k = 6$ weights. How evenly-distributed is the distribution of in-degrees?

(e) Make a new histogram of the in-degree standardized by the out-degree when $k = 26$. Does hubbiness reduce when increasing the number of k-nearest neighbors?

9. Sometimes, graphs are not simple to construct. For the `san_diego_neighborhoods` dataset:

(a) Build the Queen contiguity weights, and plot the graph on top of the neighborhoods themselves. How many connected components does this Queen contiguity graph have?

(b) Build the K-Nearest Neighbor graph for the default, $k = 2$. How many connected components does this K-Nearest Neighbor graph have?

(c) What is the smallest k that you can find for the K-Nearest Neighbor graph to be fully-connected?

(d) In graph theory, a link whose *removal* will increase the number of connected components in a graph is called a *bridge*. In the fully-connected KNN graph with the smallest k, how many bridges are there between the north and south components? *(hint: use the plotting functionality)*

(e) What are the next two values of k required for there to be an *additional* bridge at that k?

4.10 Next steps

For additional reading and further information on the topic of networks and spatial weights matrices, consider chapter 3 of Anselin and Rey, *Modern Spatial Econometrics in Practice: A Guide to GeoDa, GeoDaSpace, and Pysal*. Further, for more general thinking on networks in geography, consider:

Uitermark, Justus and Michiel van Meeteren. 2021. "Geographcial Network Analysis." *Tijdschrift voor economische en sociale geografie* 112: 337-350.

Part II
Spatial Data Analysis

Now that we understand geographic processes and the data that measures them, we will introduce *exploratory spatial data analysis* (ESDA). ESDA augments Tukey's exploratory data analysis, and involves a large collection of techniques used to "orient yourself" (find structure) inside your dataset. For geographical problems, this often involves understanding whether our data displays a *geographical pattern*. We cover such topics in this part.

First, in Chapter 5, we discuss the workhorse of statistical visualization for geographic data: choropleths. In Chapter 6, we introduce *spatial autocorrelation*, the concept that formally connects geographical and statistical similarity. This allows us to characterize the "strength" of a geographical pattern and is at the intellectual core of many explicitly spatial techniques. All patterns have exceptions, however, and Chapter 7 will present *local* methods that can detect observations that are unlike (or too like) their neighbors. To wrap up, Chapter 8 discusses methods for visualizing, characterizing and analyzing points, the raw *locational* data. Taken altogether, this part provides methods to explore most of the fundamental questions involved in geographical analysis: whatever the nature of my data, is there a geographical pattern, and are there places where this pattern does not hold?

Taylor & Francis
Taylor & Francis Group
http://taylorandfrancis.com

5

Choropleth Mapping

Choropleths are geographic maps that display statistical information encoded in a color palette. Choropleth maps play a prominent role in geographic data science as they allow us to display non-geographic attributes or variables on a geographic map. The word choropleth stems from the root "choro", meaning "region". Such choropleth maps represent data at the region level, and are appropriate for areal unit data where each observation combines a value of an attribute and a geometric figure, usually a polygon. Choropleth maps derive from an earlier era where cartographers faced technological constraints that precluded the use of unclassed maps where each unique attribute value could be represented by a distinct symbol or color. Instead, attribute values were grouped into a smaller number of classes, usually not more than 12. Each class was associated with a unique symbol that was in turn applied to all observations with attribute values falling in the class.

Although today these technological constraints are no longer binding, and unclassed mapping is feasible, there are still good reasons for adopting a classed approach. Chief among these is to reduce the cognitive load involved in parsing the complexity of an unclassed map. A choropleth map reduces this complexity by drawing upon statistical and visualization theory to provide an effective representation of the spatial distribution of the attribute values across the areal units.

5.1 Principles

The effectiveness of a choropleth map depends largely on the purpose of the map. Which message you want to communicate will shape what options are preferable over others. In this chapter we consider three dimensions over which putting intentional thinking will pay off. Choropleth mapping thus revolves around: first, selecting a number of groups smaller than n into which all values in our dataset will be mapped to; second, identifying a classification algorithm that executes such mapping, following some principle that is aligned with our interest; and third, once we know into how many groups we are going to reduce all values in our data, which color is assigned to each group to ensure it encodes the information we want to reflect. In broad terms, the classification scheme defines the number of classes as well as the rules for assignment; while a good symbolization conveys information about the value differentiation across classes.

In this chapter we first discuss the approaches used to classify attribute values. This is followed by a (brief) overview of color theory and the implications of different color schemes for effective map design. We combine theory and practice by exploring how these concepts are implemented in different Python packages, including `geopandas`, and the Pysal federation of packages.

```
import seaborn
import pandas
import geopandas
import pysal
import numpy
import matplotlib.pyplot as plt
```

5.2 Quantitative data classification

Selecting the number of groups into which we want to assign the values in our data, and how each value is assigned into a group can be seen as a classification problem. Data classification considers the problem of partitioning the attribute values into mutually exclusive and exhaustive groups. The precise manner in which this is done will be a function of the measurement scale of the attribute in question. For quantitative attributes (ordinal, interval, ratio scales), the classes will have an explicit ordering. More formally, the classification problem is to define class boundaries such that

$$c_j < y_i \leq c_{j+1} \; \forall y_i \in C_j$$

where y_i is the value of the attribute for spatial location i, j is a class index, and c_j represents the lower bound of interval j. Different classification schemes obtain from their definition of the class boundaries. The choice of the classification scheme should

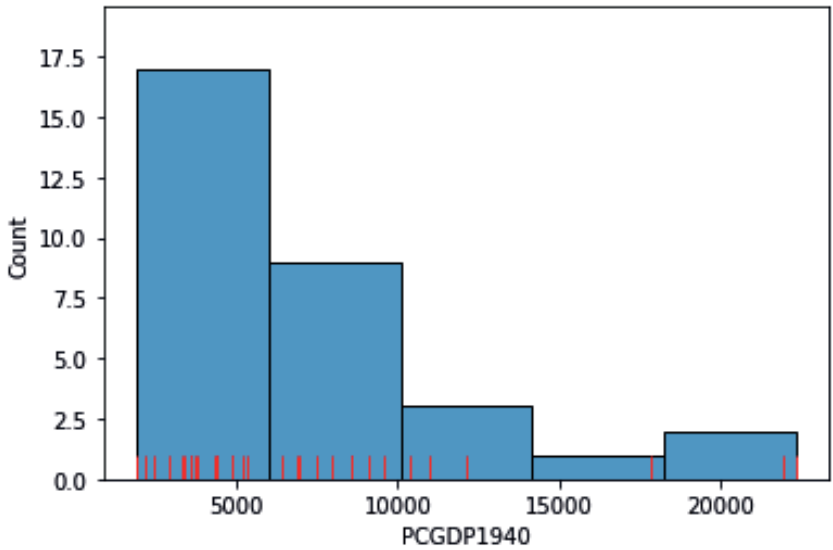


Fig. 5.1: Distribution of per capita GDP across 1940s Mexican states

take into consideration the statistical distribution of the attribute values as well as the goal of our map (e.g., highlight outliers vs. accurately depict the distribution of values).

To illustrate these considerations, we will examine regional income data for 32 Mexican states [RSastreGutierrez10] in this chapter. The variable we focus on is per capita gross domestic product for 1940 (`PCGDP1940`):

```
mx = geopandas.read_file("../data/mexico/mexicojoin.shp")
mx[["NAME", "PCGDP1940"]].head()
```

```
                    NAME  PCGDP1940
0  Baja California Norte    22361.0
1    Baja California Sur     9573.0
2                Nayarit     4836.0
3                Jalisco     5309.0
4         Aguascalientes    10384.0
```

Which displays the following statistical distribution (Figure 5.1).

```
# Plot histogram
ax = seaborn.histplot(mx["PCGDP1940"], bins=5)
# Add rug on horizontal axis
seaborn.rugplot(mx["PCGDP1940"], height=0.05, color="red",␣
↪ax=ax);
```

As we can see, the distribution is positively skewed as is common in regional income studies. In other words, the mean exceeds the median (`50%`, in the table below), leading to the long right tail in the figure. As we shall see, this skewness will have implications for the choice of choropleth classification scheme.

```
mx["PCGDP1940"].describe()
```

```
count       32.000000
mean      7230.531250
std       5204.952883
min       1892.000000
25%       3701.750000
50%       5256.000000
75%       8701.750000
max      22361.000000
Name: PCGDP1940, dtype: float64
```

For quantitative attributes we first sort the data by their value, such that $x_0 \leq x_2 \ldots \leq x_{n-1}$. For a prespecified number of classes k, the classification problem boils down to selecting $k-1$ break points along the sorted values that separate the values into mutually exclusive and exhaustive groups.

In fact, the determination of the histogram above can be viewed as one approach to this selection. The method `seaborn.histplot` uses the matplotlib `hist` function under the hood to determine the class boundaries and the counts of observations in each class. In the figure, we have five classes which can be extracted with an explicit call to the `hist` function:

```
counts, bins, patches = ax.hist(mx["PCGDP1940"], bins=5)
```

The `counts` object captures how many observations each category in the classification has:

```
counts
```

```
array([17.,  9.,  3.,  1.,  2.])
```

The `bin` object stores these break points we are interested in when considering classification schemes (the `patches` object can be ignored in this context, as it stores the geometries of the histogram plot):

```
bins
```

```
array([ 1892. ,  5985.8, 10079.6, 14173.4, 18267.2, 22361. ])
```

This yields five bins, with the first having a lower bound of 1892 and an upper bound of 5985.8 which contains 17 observations. The determination of the interval width (w) and the number of bins in `seaborn` is based on the Freedman-Diaconis rule [FD81]:

$$w = 2 * IQR * n^{-1/3}$$

where IQR is the inter quartile range of the attribute values. Given w, the number of bins (k) is:

$$k = \frac{(max - min)}{w}$$

The choropleth literature has many alternative classification algorithms that follow criteria that can be of interest in different contexts, as they focus on different priorities. Below, we will focus on a few of them. To compute the classification, we will rely on the `mapclassify` package of the Pysal family:

```
import mapclassify
```

5.2.1 Equal intervals

The Freedman-Diaconis approach provides a rule to determine the width and, in turn, the number of bins for the classification. This is a special case of a more general classifier known as "equal intervals", where each of the bins has the same width in the value space. For a given value of k, equal intervals classification splits the range of the attribute space into k equal length intervals, with each interval having a width $w = \frac{x_0 - x_{n-1}}{k}$. Thus the maximum class is $(x_{n-1} - w, x_{n-1}]$ and the first class is $(-\infty, x_{n-1} - (k-1)w]$.

Equal intervals have the dual advantages of simplicity and ease of interpretation. However, this rule only considers the extreme values of the distribution and, in some cases, this can result in one or more classes being sparse. This is clearly the case in our income dataset, as the majority of the values are placed into the first two classes leaving the last three classes rather sparse:

```
ei5 = mapclassify.EqualInterval(mx["PCGDP1940"], k=5)
ei5
```

```
EqualInterval

      Interval          Count
----------------------------
[ 1892.00,  5985.80] |    17
( 5985.80, 10079.60] |     9
(10079.60, 14173.40] |     3
(14173.40, 18267.20] |     1
(18267.20, 22361.00] |     2
```

Note that each of the intervals, however, has equal width of $w = 4093.8$. It should also be noted that the first class is closed on the lower bound, in contrast to the general approach defined above.

5.2.2 Quantiles

To avoid the potential problem of sparse classes, the quantiles of the distribution can be used to identify the class boundaries. Indeed, each class will have approximately $\lfloor \frac{n}{k} \rfloor$ observations using the quantile classifier. If $k = 5$ the sample quintiles are used to define the upper limits of each class resulting in the following classification:

```
q5 = mapclassify.Quantiles(mx.PCGDP1940, k=5)
q5
```

```
Quantiles

      Interval         Count
----------------------------
[ 1892.00,  3576.20] |     7
( 3576.20,  4582.80] |     6
( 4582.80,  6925.20] |     6
( 6925.20,  9473.00] |     6
( 9473.00, 22361.00] |     7
```

Note that while the numbers of values in each class are roughly equal, the widths of the first four intervals are rather different:

```
q5.bins[1:] - q5.bins[:-1]
```

```
array([ 1006.6,  2342.4,  2547.8, 12888. ])
```

While quantile classification avoids the pitfall of sparse classes, this classification is not problem-free. The varying widths of the intervals can be markedly different which can lead to problems of interpretation. A second challenge facing quantiles arises when there are a large number of duplicate values in the distribution such that the limits for one or more classes become ambiguous. For example, if one had a variable with $n = 20$ but 10 of the observations took on the same value which was the minimum observed, then for values of $k > 2$, the class boundaries become ill-defined since a simple rule of splitting at the n/k ranked observed value would depend upon how ties are treated when ranking.

Let us generate a synthetic variable with these characteristics:

```
# Set seed for reproducibility
numpy.random.seed(12345)
```

(continued on next page)

(continued from previous page)

```
# Generate a variable of 20 values randomly
# selected from 0 to 10
x = numpy.random.randint(0, 10, 20)
# Manually ensure the first ten values are 0 (the
# minimum value)
x[0:10] = x.min()
x
```

```
array([0, 0, 0, 0, 0, 0, 0, 0, 0, 9, 7, 6, 0, 2, 9, 1, 2,␣
↪6, 7])
```

And we will now run quantile classification:

```
ties = mapclassify.Quantiles(x, k=5)
ties
```

```
Quantiles

  Interval      Count
--------------------
[0.00, 0.00] |    11
(0.00, 1.40] |     1
(1.40, 6.20] |     4
(6.20, 9.00] |     4
```

For clarity, the unique values in our dataset are:

```
ux = numpy.unique(x)
ux
```

```
array([0, 1, 2, 6, 7, 9])
```

In this case, `mapclassify` will issue a warning alerting the user to the issue that this sample does not contain enough unique values to form the number of well-defined classes requested. It then forms a lower number of classes using pseudo-quantiles, or quantiles defined on the unique values in the sample, and then uses the pseudo-quantiles to classify all the values.

5.2.3 Mean-standard deviation

Our third classifier uses the sample mean $\bar{x} = \frac{1}{n}\sum_{i=1}^{n} x_i$ and sample standard deviation $s = \sqrt{\frac{1}{n-1}\sum_{i=1}^{n}(x_i - \bar{x})}$ to define class boundaries as some distance from the sample mean, with the distance being a multiple of the standard deviation. For example, a common definition for $k = 5$ is to set the upper limit of the first class to two standard deviations ($c_0^u = \bar{x} - 2s$), and the intermediate classes to have upper limits within one standard deviation ($c_1^u = \bar{x} - s$, $c_2^u = \bar{x} + s$, $c_3^u = \bar{x} + 2s$). Any values greater (smaller) than two standard deviations above (below) the mean are placed into the top (bottom) class.

```
msd = mapclassify.StdMean(mx["PCGDP1940"])
msd
```

```
StdMean

      Interval          Count
----------------------------
(    -inf, -3179.37] |     0
(-3179.37,  2025.58] |     1
( 2025.58, 12435.48] |    28
(12435.48, 17640.44] |     0
(17640.44, 22361.00] |     3
```

This classifier is best used when data is normally distributed or, at least, when the sample mean is a meaningful measure to anchor the classification around. Clearly this is not the case for our income data as the positive skew results in a loss of information when we use the standard deviation. The lack of symmetry leads to an inadmissible upper bound for the first class as well as a concentration of the vast majority of values in the middle class.

5.2.4 Maximum breaks

The maximum breaks classifier decides where to set the break points between classes by considering the difference between sorted values. That is, rather than considering a value of the dataset in itself, it looks at how apart each value is from the next one in the sorted sequence. The classifier then places the $k - 1$ break points in between the pairs of values most stretched apart from each other in the entire sequence, proceeding in descending order relative to the size of the breaks:

```
mb5 = mapclassify.MaximumBreaks(mx["PCGDP1940"], k=5)
mb5
```

```
MaximumBreaks

      Interval          Count
----------------------------
[ 1892.00,  5854.00] |    17
( 5854.00, 11574.00] |    11
(11574.00, 14974.00] |     1
(14974.00, 19890.50] |     1
(19890.50, 22361.00] |     2
```

Maximum breaks is an appropriate approach when we are interested in making sure observations in each class are separated from those in neighboring classes. As such, it works well in cases where the distribution of values is not unimodal. In addition, the algorithm is relatively fast to compute. However, its simplicity can sometimes cause unexpected results. To the extent that maximum breaks classification only considers the top $k-1$ differences between consecutive values, other more nuanced within-group differences and dissimilarities can be ignored.

5.2.5 Boxplot

The boxplot classification is a blend of the quantile and standard deviation classifiers. Here k is predefined to six, with the upper limit of the first class set to:

$$q_{0.25} - h\,IQR$$

where $IQR = q_{0.75} - q_{0.25}$ is the inter-quartile range; and h corresponds to the hinge, or the multiplier of the IQR to obtain the bounds of the "whiskers" from a box-and-whisker plot of the data. The lower limit of the sixth class is set to $q_{0.75} + h\,IQR$. Intermediate classes have their upper limits set to the 0.25, 0.50 and 0.75 percentiles of the attribute values.

```
bp = mapclassify.BoxPlot(mx["PCGDP1940"])
bp
```

```
BoxPlot

      Interval          Count
----------------------------
(    -inf, -3798.25] |     0
(-3798.25,  3701.75] |     8
( 3701.75,  5256.00] |     8
( 5256.00,  8701.75] |     8
( 8701.75, 16201.75] |     5
(16201.75, 22361.00] |     3
```

Any values falling into either of the extreme classes are defined as outliers. Note that because the income values are non-negative by definition, the lower outlier class has an inadmissible upper bound meaning that lower outliers would not be possible for this sample.

The default value for the hinge is $h = 1.5$ in `mapclassify`. However, this can be specified by the user for an alternative classification:

```
bp1 = mapclassify.BoxPlot(mx["PCGDP1940"], hinge=1)
bp1
```

```
BoxPlot

      Interval          Count
----------------------------
(    -inf, -1298.25] |     0
(-1298.25,  3701.75] |     8
( 3701.75,  5256.00] |     8
( 5256.00,  8701.75] |     8
( 8701.75, 13701.75] |     5
(13701.75, 22361.00] |     3
```

Doing so will affect the definition of the outlier classes, as well as the neighboring internal classes.

5.2.6 Head-tail breaks

The head tail algorithm [Jia13] is based on a recursive partitioning of the data using splits around iterative means. The splitting process continues until the distributions within each of the classes no longer display a heavy-tailed distribution in the sense that there is a balance between the number of smaller and larger values assigned to each class.

```
ht = mapclassify.HeadTailBreaks(mx["PCGDP1940"])
ht
```

```
HeadTailBreaks

      Interval          Count
----------------------------
[ 1892.00,  7230.53] |    20
( 7230.53, 12244.42] |     9
(12244.42, 20714.00] |     1
(20714.00, 22163.00] |     1
(22163.00, 22361.00] |     1
```

For data with a heavy-tailed distribution, such as power law and log normal distributions, the head tail breaks classifier can be particularly effective.

5.2.7 Jenks-Caspall breaks

This approach, as well as the following two, tackles the classification challenge from a heuristic perspective, rather than from a deterministic one. Originally proposed by [JC71], the Jenks-Caspall classification algorithm aims to minimize the sum of absolute deviations around class means. The approach begins with a prespecified number of classes and an arbitrary initial set of class breaks - for example using quintiles. The algorithm attempts to improve the objective function by considering the movement of observations between adjacent classes. For example, the largest value in the lowest quintile would be considered for movement into the second quintile, while the lowest value in the second quintile would be considered for a possible move into the first quintile. The candidate move resulting in the largest reduction in the objective function would be made, and the process continues until no other improving moves are possible. The Jenks-Caspall algorithm is the one-dimension case of the widely used K-Means algorithm for clustering, which we will see later in this book when we consider *Clustering and Regionalization*.

```
numpy.random.seed(12345)
jc5 = mapclassify.JenksCaspall(mx["PCGDP1940"], k=5)
jc5
```

```
JenksCaspall

      Interval         Count
----------------------------
[ 1892.00,  2934.00] |     4
( 2934.00,  4414.00] |     9
( 4414.00,  6399.00] |     5
( 6399.00, 12132.00] |    11
(12132.00, 22361.00] |     3
```

5.2.8 Fisher-Jenks breaks

The second optimal algorithm adopts a dynamic programming approach to minimize the sum of the absolute deviations around class medians. In contrast to the Jenks-Caspall algorithm, the Fisher-Jenks alorithm is guaranteed to produce an optimal classification for a prespecified number of classes:

```
numpy.random.seed(12345)
fj5 = mapclassify.FisherJenks(mx["PCGDP1940"], k=5)
fj5
```

```
FisherJenks

      Interval         Count
----------------------------
[ 1892.00,  5309.00] |    17
( 5309.00,  9073.00] |     8
( 9073.00, 12132.00] |     4
(12132.00, 17816.00] |     1
(17816.00, 22361.00] |     2
```

5.2.9 Max-p

Finally, the max-p classifier adopts the algorithm underlying the max-p region building method [DAR11] to the case of map classification. It is similar in spirit to Jenks-Caspall in that it considers greedy swapping between adjacent classes to improve the objective function. It is a heuristic, however, so unlike Fisher-Jenks, there is no optimal solution guaranteed:

```
mp5 = mapclassify.MaxP(mx["PCGDP1940"], k=5)
mp5
```

```
MaxP

      Interval         Count
----------------------------
[ 1892.00,  3569.00] |     7
( 3569.00,  5309.00] |    10
( 5309.00,  7990.00] |     5
( 7990.00, 10384.00] |     5
(10384.00, 22361.00] |     5
```

5.2.10 Comparing classification schemes

As a special case of clustering, the definition of the number of classes and the class boundaries pose a problem to the map designer. Recall that the Freedman-Diaconis rule was said to be optimal, however; optimality can only be measured relative to a specified objective function. In the case of Freedman-Diaconis, the objective function is to minimize the difference between the area under estimated kernel density based on

the sample and the area under the theoretical population distribution that generated the sample.

This notion of statistical fit is an important one. However, it is not the only consideration when evaluating classifiers for the purpose of choropleth mapping. Also relevant is the spatial distribution of the attribute values and the ability of the classifier to convey a sense of that spatial distribution. As we shall see, this is not necessarily directly related to the statistical distribution of the attribute values. We will return to a joint consideration of both the statistical and spatial distributions of the attribute values when comparing classifiers later in this chapter.

For map classification, a common optimality criterion is a measure of fit. In `mapclassify`, the absolute deviation around class medians (ADCM) is calculated and provides a measure of fit that allows for comparison of alternative classifiers for the same value of k. The ADCM will give us a sense of how "compact" each group is. To see this, we can compare different classifiers for $k = 5$ on the Mexico data in Figure 5.2:

```
# Bunch classifier objects
class5 = q5, ei5, ht, mb5, msd, fj5, jc5, mp5
# Collect ADCM for each classifier
fits = numpy.array([c.adcm for c in class5])
# Convert ADCM scores to a DataFrame
adcms = pandas.DataFrame(fits)
# Add classifier names
adcms["classifier"] = [c.name for c in class5]
# Add column names to the ADCM
adcms.columns = ["ADCM", "Classifier"]
ax = seaborn.barplot(
    y="Classifier", x="ADCM", data=adcms, palette="Pastel1"
)
```

As is to be expected, the Fisher-Jenks classifier dominates all other k=5 classifiers with an ADCM of 23,729 (remember, lower is better). Interestingly, the equal interval classifier performs well despite the problems associated with being sensitive to the extreme values in the distribution. The mean-standard deviation classifier has a very poor fit due to the skewed nature of the data and the concentrated assignment of the majority of the observations to the central class.

The ADCM provides a global measure of fit which can be used to compare the alternative classifiers. As a complement to this global perspective, it can be revealing to consider how each of the observations in our data was classified across the alternative approaches. To do this, we can add the class bin attribute (`yb`) generated by the `mapclassify` classifiers as additional columns in the dataframe to visualize how they map to observations:

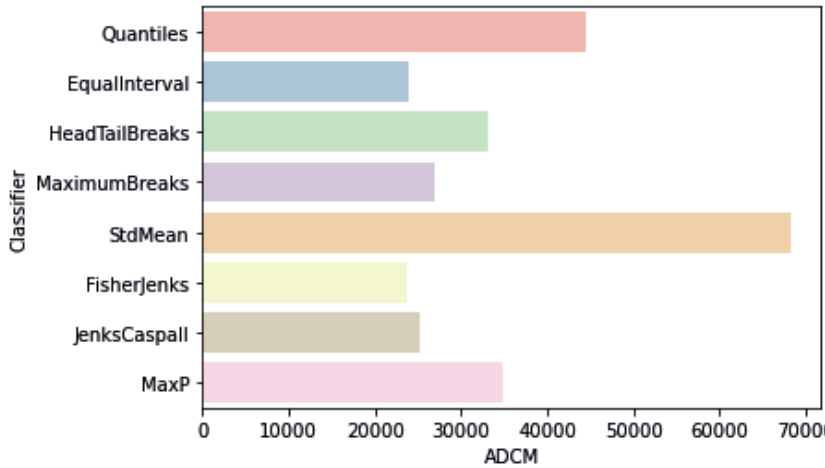


Fig. 5.2: Absolute deviation around class medians. Alternative classification schemes, Mexican state per capita GDP in 1940.

```
# Append class values as a separate column
mx["Quantiles"] = q5.yb
mx["Equal Interval"] = ei5.yb
mx["Head-Tail Breaks"] = ht.yb
mx["Maximum Breaks"] = mb5.yb
mx["Mean-Standard Deviation"] = msd.yb
mx["Fisher-Jenks"] = fj5.yb
mx["Jenks Caspall"] = jc5.yb
mx["MaxP"] = mp5.yb
```

With those in one place, we can display their labels in a heatmap. Note that, since our variable of interest is continuous, we can sort the rows of the table by their value (`.sort_values('PCGDP1940')`) and color each cell according to the label assigned to it by each classifier. To make the heatmap easier to read, we transpose it (`.T`) so that Mexican states are displayed along the horizontal axis and classification schemes are along the vertical one. (see Figure 5.3)

```
f, ax = plt.subplots(1, figsize=(9, 3))
seaborn.heatmap(
    mx.set_index("NAME")
    .sort_values("PCGDP1940")[
        [
            "Head-Tail Breaks",
            "Fisher-Jenks",
            "Maximum Breaks",
            "Equal Interval",
```

(continued on next page)

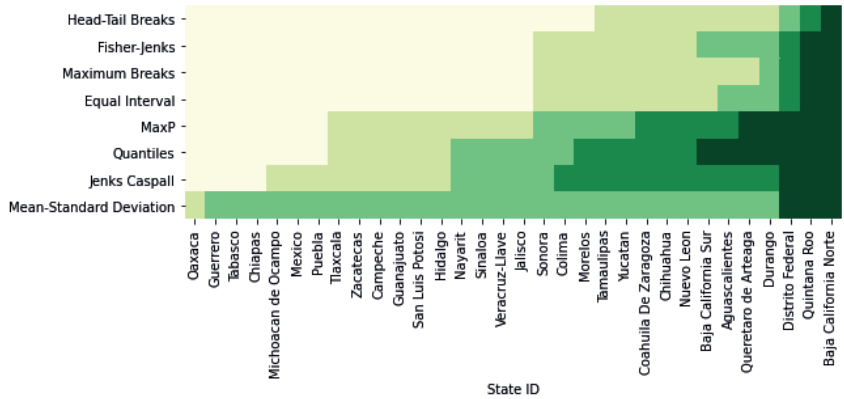


Fig. 5.3: Assignment differences between alternative classification schemes, Mexican state per capita GDP in 1940.

(continued from previous page)

```
            "MaxP",
            "Quantiles",
            "Jenks Caspall",
            "Mean-Standard Deviation",
        ]
    ]
    .T,
    cmap="YlGn",
    cbar=False,
    ax=ax,
)
ax.set_xlabel("State ID");
```

Figure 5.3 can be challenging to read at first but, once you "decode" it, it packs a lot of information. Each row includes a full series of all of our data, classified by an algorithm, with the group to which it has been assigned encoded on a color scale from light yellow (lowest value group) to dark green (largest value group). Conversely, each column represents how a given state is classified across the different schemes considered. Inspection of the table reveals a number of interesting results. For example, the only Mexican state that is treated consistently across the k=5 classifiers is Baja California Norte which is placed in the highest class by all classifiers. Additionally, the mean-standard deviation classifier has an empty first class due to the inadmissible upper bound and the over-concentration of values in the central class (2).

Finally, we can consider a meso-level view of the classification results by comparing the number of values assigned to each class across the different classifiers:

```
pandas.DataFrame(
    {c.name: c.counts for c in class5},
    index=["Class-{}".format(i) for i in range(5)],
)
```

```
         Quantiles  EqualInterval  HeadTailBreaks ␣
↪MaximumBreaks  StdMean  \
Class-0          7              17              20            ␣
↪17        0
Class-1          6               9               9            ␣
↪11        1
Class-2          6               3               1            ␣
↪ 1       28
Class-3          6               1               1            ␣
↪ 1        0
Class-4          7               2               1            ␣
↪ 2        3

         FisherJenks  JenksCaspall  MaxP
Class-0           17             4     7
Class-1            8             9    10
Class-2            4             5     5
Class-3            1            11     5
Class-4            2             3     5
```

Doing so highlights the similarities between Fisher-Jenks and equal intervals as the distribution counts are very similar, with the two approaches agreeing on all 17 states assigned to the first class. Indeed, the only observation that distinguishes the two classifiers is the treatment of Baja California Sur which is kept in class 1 in equal intervals, but assigned to class 2 by Fisher-Jenks.

5.3 Color

Having considered the evaluation of the statistical distribution of the attribute values and the alternative classification approaches, we turn to select the symbolization and color scheme. Together with the choice of classifier, these will determine the overall effectiveness of the choropleth map in representing the spatial distribution of the attribute values.

Prior to examining the attribute values it is important to note that, as we will see in the figures below, the spatial units for these states are far from homogeneous in their shapes and sizes. This can have major impacts on our brain's pattern recognition capabilities, as we tend to be drawn to the larger polygons, even though they might not be the most relevant ones for our analysis. Yet, when we considered the statistical distribution above,

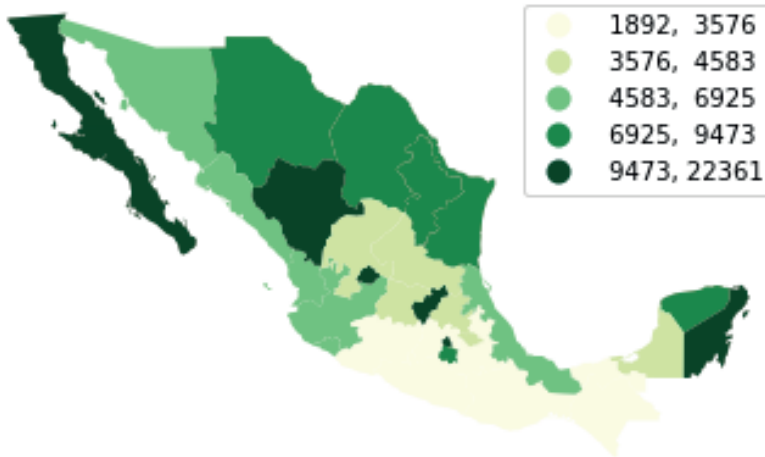


Fig. 5.4: Quantile choropleth, Mexican state per capita GDP in 1940.

each observation was given equal weight. Thus, the spatial distribution becomes more complicated to evaluate from a visual and statistical perspective.

The choice of a color scheme for a choropleth map should be based on the type of variable under consideration [BMPH97]. Generally, a distinction is drawn between three types of numerical attributes: sequential, diverging, and qualitative. We will dig into each below, but we will explore how we can make choropleths in Python first. The mechanics are the same across different types of data, so it is worth spending a bit of time first to get the general idea.

We will illustrate it with a quantile map in Figure 5.4:

```
ax = mx.plot(
    column="PCGDP1940",  # Data to plot
    scheme="Quantiles",  # Classification scheme
    cmap="YlGn",  # Color palette
    legend=True,  # Add legend
    legend_kwds={"fmt": "{:.0f}"},  # Remove decimals in␣
↪legend
)
ax.set_axis_off();
```

Making choropleths on geo-tables is an extension of plotting their geometries. We use the same `.plot()` function, but now we also select the column of data we want to encode with color (in our case, `PCGDP1940`). We can also specify the classification scheme using the same names as we saw above with `mapclassify`. In fact, the

underlying computation is always performed with `mapclassify`. This approach simply dispatches it so it is more convenient and we can make maps in one line of code. Next, we pick the color scheme. The default color map used by `geopandas` is viridis, which is a multi-hue sequential scheme but, for this example, we pick the yellow-to-green scale from Color Brewer. Finally, we specify that we would like to add a legend, and format it for legibility so that there are no decimals and it reads cleaner (Figure 5.4).

5.3.1 Sequential palettes

Sequential color schemes are appropriate for continuous data where the origin is in one end of the series. The `PCGDP1940` column we have been using so far is a good example. In these cases, we want a palette that encodes this feature in its choice of colors. Sequential palettes use a gradient of colors from an origin color to a destination color. The example above, where lowest values are encoded in the lightest yellow and the highest in dark green is a good one. Sequential palettes can also have a shades of a single color. For example, the popular "blues" palette in Color Brewer is a great choice too, shown in Figure 5.5:

```
ax = mx.plot(
    column="PCGDP1940",  # Data to plot
    scheme="Quantiles",  # Classification scheme
    cmap="Blues",  # Color palette
    edgecolor="k",  # Borderline color
    linewidth=0.1,  # Borderline width
    legend=True,  # Add legend
    legend_kwds={
        "fmt": "{:.0f}"
    },  # Remove decimals in legend (for legibility)
)
ax.set_axis_off();
```

Note how, in this case, we switch borderlines to black so that we can distinguish states in the lowest category from the white background.

5.3.2 Diverging palettes

A slightly different pallete from the sequential one is the so-called "diverging" values palette. This is useful with continuous data when one wishes to place equal emphasis on mid-range critical values as well as extremes at both ends of the distribution. Light colors are used to emphasize the mid-range class, while dark colors with contrasting hues are used to distinguish the low and high extremes.

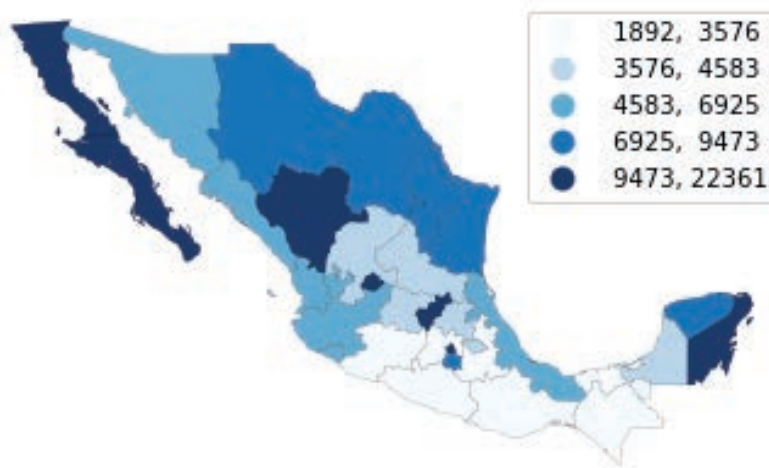


Fig. 5.5: Quantile choropleth with black borderlines, Mexican state per capita GDP in 1940.

To illustrate this with the Mexican income data, we can derive a new variable which measures the change in a state's rank in the income distribution from 1940 to 2000:

```
# Create income-based rank table (Rank 1 is highest)
rnk = mx[["NAME", "PCGDP1940", "PCGDP2000"]].
↪rank(ascending=False)
# Compute change from 1940 to 2000
rnk["change"] = rnk["PCGDP1940"] - rnk["PCGDP2000"]
# Add column with bin class
rnk["class"] = pandas.cut(rnk["change"], [-numpy.inf, -5, 0,␣
↪5, 20])
```

The `rnk` table now contains the change in rank positions of each state between 1940 and 2000, as well as a `class` column that binds together states in the [-inf, -5), [-5, 0), [0, 5), [5, 20] groups. Note that these are descending ranks, so the wealthiest state in any period has a rank of 1, and therefore when considering the change in ranks, a negative change reflects moving down the income distribution. We can use a divergent palette to signify both intensity of the change in ranks, as well as direction in Figure 5.6:

```
ax = (
    mx[["geometry"]]
    .join(rnk)
    .plot("class", legend=True, cmap="RdYlGn")
)
ax.set_axis_off();
```

Fig. 5.6: Divergent palette, Mexican state per capita income rank change.

In the map, the red (green) hues are states that have moved downward (upward) in the income distribution, with the darker hue representing a larger movement.

5.3.3 Qualitative palettes

Qualitative palettes encode categorical data. In this case, colors do *not* follow a gradient but rather imply qualitative differences between classes. That is, observations in one group are not more or less, above or below those in other groups, rather just different.

The Mexico data set also has several variables that are on a nominal measurement scale. One of these is a region definition variable that groups individual states in contiguous clusters of similar characteristics:

```
mx["HANSON98"].head()
```

```
0    1.0
1    2.0
2    2.0
3    3.0
4    2.0
Name: HANSON98, dtype: float64
```

This aggregation scheme partitions Mexico into five regions, recorded with the numbers

Fig. 5.7: (Incorrect) sequential palette, Mexican regions.

1 to 5 in the table. A naive (and incorrect) way to display this would be to treat the region variable as sequential, visualized in Figure 5.7:

```
ax = mx.plot("HANSON98")
ax.set_axis_off();
```

This is not correct because the region variable is not on an interval scale, so the differences between the values have no quantitative significance, but rather the values simply indicate region membership. However, the choropleth in Figure 5.7 gives a clear visual cue that regions in the south have larger values than those in the north, as the color map implies an intensity gradient.

A more appropriate visualization is to use a "qualitative" color palette, which is used if you specify that the variable is categorical (Figure 5.8).

```
ax = mx.plot("HANSON98", categorical=True, legend=True)
ax.set_axis_off();
```

Fig. 5.8: Qualitative palette, Mexican regions.

5.4 Advanced topics

5.4.1 User-defined choropleths

In this last section of the chapter, we consider bespoke partitions of the data that do not follow any particular algorithm but instead are informed by, for example, domain knowledge. Consider the case of classifying income in a policy context. Imagine we wanted to explore the distribution of areas with less than $10,000, then those between $10,000 and $12,500; $12,500 and $15,000; and greater than $15,000. These boundaries are arbitrary but may be tied to specific policies in which the first group is targetted in one particular way, the second and third in different ways, and the fourth is not part of the policy, for example.

To create a choropleth that reflects this partitioning of the data, we can use the `UserDefined` classifier in `mapclassify`:

```
classi = mapclassify.UserDefined(
    mx["PCGDP2000"], [10000, 12500, 15000]
)
classi
```

```
UserDefined

      Interval         Count
```

(continued on next page)

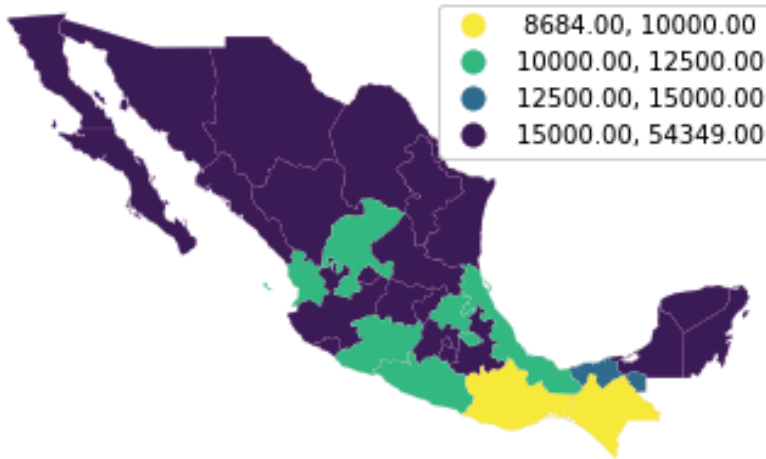


Fig. 5.9: Choropleth map colored to focus on areas of southern Mexico eligible for a target policy, showcasing user-defined map classifications.

(continued from previous page)

```
----------------------------
[ 8684.00, 10000.00] |      2
(10000.00, 12500.00] |      7
(12500.00, 15000.00] |      1
(15000.00, 54349.00] |     22
```

If we now want to display these classes on a map, we can use a similar approach to how we have seen above, or use the built-in plotting method in `mapclassify` to generate Figure 5.9:

```
classi.plot(
    mx,  # Use geometries in the geo-table
    legend=True,  # Add a legend
    legend_kwds={
        "loc": "upper right"
    },  # Place legend on top right corner
    axis_on=False,  # Remove axis
    cmap="viridis_r",  # Use reverse Viridis
);
```

Since we want to draw attention to the classes at the bottom of the scale, we use the reverse viridis (`viridis_r`) palette. Thus, Figure 5.9 shows in purple the areas not targeted by our hypothetical policy.

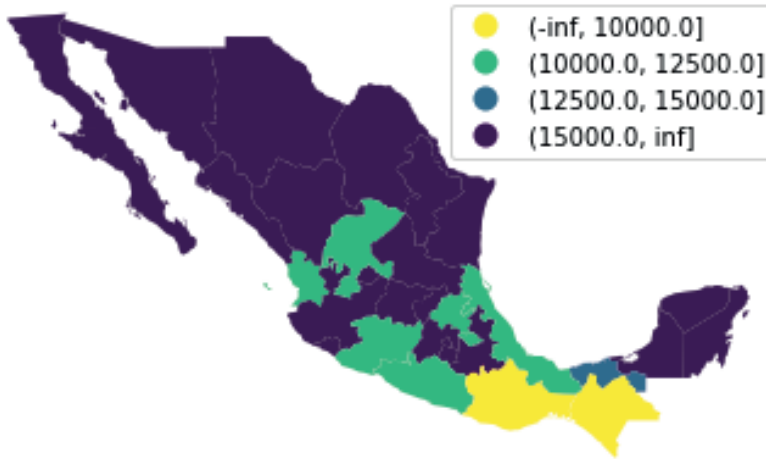


Fig. 5.10: User-defined palette, `pandas` approach.

The approach above is useful in that it is based on `mapclassify` and thus provides a unified interface shared with all the algorithms seen above. An alternative one involves using the `pandas.cut` method, which allows us to easily include a legend too in Figure 5.10:

```
# Classify values specifying bins
lbls = pandas.cut(
    mx["PCGDP2000"], [-numpy.inf, 10000, 12500, 15000, numpy.
↪inf]
)
# Dynamically assign to geo-table and plot with a legend
ax = mx.plot(lbls, cmap="viridis_r", legend=True)
# Remove axis
ax.set_axis_off();
```

5.4.2 Pooled classifications

Sometimes choropleths exist as part of larger figures that may include more choropleths. In some cases, each of them can be best considered as an independent map, and then everything we have seen so far applies directly. In other instances, we may want to create a single classification of values *across* the maps and use it consistently. For those situations, we can create *pooled* classifications that consider all the values across the series.

To illustrate this approach, we will create a figure with choropleths of GDP per capita in 1940, 1960, 1980, and 2000; and we will use the same classification across the four maps.

```
# List the years we want of pc GDP
years = ["PCGDP1940", "PCGDP1960", "PCGDP1980", "PCGDP2000"]
# Create pooled classification
pooled = mapclassify.Pooled(mx[years], classifier="Quantiles",
↪ k=5)
```

The `pooled` object contains a lot of information on the classification and we can use it to generate a figure with the maps. To do that, we rely also on the `UserDefined` classifier we have just seen in the previous section to create a multi-pane figure showing the per capita income as it changes over time (Figure 5.11).

```
# Set up figure with four axis
f, axs = plt.subplots(2, 2, figsize=(12, 12))
# Flatten the array of axis so you can loop over
# in one dimension
axs = axs.flatten()
# Loop over each year
for i, y in enumerate(years):
    mx.plot(
        y,  # Year to plot
        scheme="UserDefined",  # Use our own bins
        classification_kwds={
            "bins": pooled.global_classifier.bins
        },  # Use global bins
        legend=True,  # Add a legend
        ax=axs[i],  # Plot on the corresponding axis
    )
    # Remove axis
    axs[i].set_axis_off()
    # Name the subplot with the name of the column
    axs[i].set_title(y)
# Tight layout to better use space
plt.tight_layout()
# Display figure
plt.show()
```

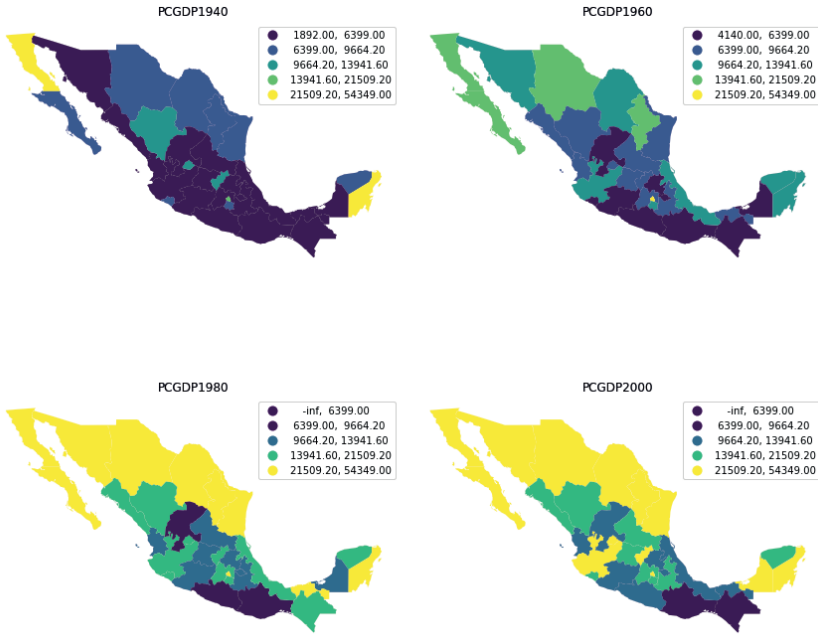


Fig. 5.11: Pooled quantile classification of per capita GDP for 1940, 1960, 1980, and 2000, Mexican states.

5.5 Conclusion

In this chapter we have considered the construction of choropleth maps for spatial data visualization. The key issues of the choice of classification scheme, variable measurement scale, spatial configuration and color palettes were illustrated using Pysal's map classification module together with other related packages in the Python data stack.

Choropleth maps are a central tool in the geographic data science toolkit, as they provide powerful visualizations of the spatial distribution of attribute values. We have only touched on the basic concepts in this chapter, as there is much more that can be said about cartographic theory and the design of effective choropleth maps. Readers interested in pursuing this literature are encouraged to see the references cited.

At the same time, given the philosophy underlying Pysal, the methods we cover here are sufficient for exploratory data analysis where the rapid and flexible generation of views is critical to the work flow. Once the analysis is complete, and the final presentation quality maps are to be generated, there are excellent packages in the data stack that the user can turn to.

5.6 Questions

1. A variable (such as population density measured for census tracts in a metropolitan area) can display a high degree of skewness. That is, the distribution may be very asymmetric, either with a few very high values and a bulk of low ones; or a few very low values with a bulk of high values. What is an appropriate choice for a choropleth classification for a skewed variable?
2. Provide two solutions to the problem of ties when applying quantile classification to the following series: $y = [2, 2, 2, 2, 2, 2, 4, 7, 8, 9, 20, 21]$ and $k = 4$. Discuss the merits of each approach.
3. Which classifiers are appropriate for data that displays a high degree of multi-modality in its statistical distribution?
4. Are there any colormaps that work well for multi-modal data?
5. Contrast and compare classed choropleth maps with class-less (i.e., continuous-scale) choropleth maps? What are the strengths and limitations of each type of visualization for spatial data?
6. In what ways do choropleth classifiers treat intra-class and inter-class heterogeneity differently? What are the implications of these choices?
7. To what extent do most commonly employed choropleth classification methods take the geographical distribution of the variable into consideration? Can you think of ways to incorporate the spatial features of a variable into a classification for a choropleth map?
8. Discuss the similarities between the choice of the number of classes in choropleth mapping, on the one hand, and the determination of the number of clusters in a data set on the other. What aspects of choropleth mapping differentiate the former from the latter?
9. The Fisher-Jenks classifier will always have more internally homogeneous classes than other k-classifiers. Given this, why might one decide on choosing a different k-classifier for a particular data set?

5.7 Next steps

We have but touched the surface of the large literature on choropleth mapping in particular, and geovisualization more generally. Readers interested in delving deeper into these topices are directed to the following:

- Slocum Terry A., Robert B. McMaster, Fritz C. Kessler, and Hugh H. Howard. 2009. Thematic Cartography and Geovisualization. Pearson.

- Cromley, Robert G. 2009. “Choropleth map legend design for visualizing community health disparities.” *International Journal of Health Geographics*, 8: 1-11.
- Cromely, Robert G. 1996. “A comparison of optimal classification strategies for choroplethic displays of spatiall aggregated data.” *International Journal of Geographc Information Systems*, 10: 405-424.
- Brewer, Cynthia A. 2015. Designing better maps: A guide for GIS Users. ESRI Press.

6 Global Spatial Autocorrelation

The notion of spatial autocorrelation relates to the existence of a "functional relationship between what happens at one point in space and what happens elsewhere" [Ans88]. Spatial autocorrelation thus has to do with the degree to which the similarity in values between observations in a dataset is related to the similarity in locations of such observations. This is similar to the traditional idea of correlation between two variables, which informs us about how the values in one variable change as a function of those in the other, albeit with some key differences discussed in this chapter. In a similar fashion, spatial autocorrelation is also related (but distinct) to temporal counterpart, *temporal autocorrelation*, which relates the value of a variable at a given point in time with those in previous periods. In contrast to these other ideas of correlation, *spatial autocorrelation* relates the value of the variable of interest in a given location, with values of the same variable in other locations. An alternative way to understand the concept is as the degree of information contained in the value of a variable at a given location about the value of that same variable in other locations.

6.1 Understanding spatial autocorrelation

In order to better understand the notion of spatial autocorrelation, it is useful to begin by considering what the world looks like in its absence. A key idea in this context is that of spatial randomness: a situation in which the location of an observation gives no information whatsoever about its value. In other words, a variable is spatially random if its distribution follows no discernible spatial pattern. Spatial autocorrelation can thus be defined as the "absence of spatial randomness".

This definition is still too vague, though. So, to get more specific, spatial autocorrelation is typically categorized along two main dimensions: sign and scale. Similar to the traditional, non-spatial case, spatial autocorrelation can adopt two main forms: *positive*

and *negative*. The former relates to a situation where similarity and geographical closeness go hand-in-hand. In other words, similar values are located near each other, while different values tend to be scattered and further away. It is important that the sign of these values is not relevant for the presence of spatial autocorrelation: it may be high values close to high values, *or* low values close to low values. The important bit in this context is the relationship between closeness and statistical similarity is positive. This is a fairly common case in many social contexts and, in fact, several human phenomena display clearly positive spatial autocorrelation. For example, think of the distribution of income, or poverty, over space: it is common to find similar values located nearby (wealthy areas close to other wealthy areas, poor population concentrated in space too). In contrast, *negative* spatial autocorrelation reflects a situation where similar values tend to be located away from each other. In this case, statistical similarity is associated with distance. This is somewhat less common in the social sciences, but it still exists. An example can be found in phenomena that follow processes of spatial competition or situations where the location of a set of facilities aims at the highest spatial coverage. The distribution of supermarkets of different brands, or of hospitals, usually follows a pattern of negative spatial dependence.

It can also help to understand spatial autocorrelation using the scale at which it is considered. We generally talk of global or local processes. **Global** spatial autocorrelation, on which this chapter is focused, considers the overall trend that the location of values follows. In doing this, the study of global spatial autocorrelation makes possible statements about the degree of *clustering* in the dataset. Do values generally follow a particular pattern in their geographical distribution? Are similar values closer to other similar values than we would expect from pure chance? These are some of the questions that relate to global spatial autocorrelation. **Local** autocorrelation focuses on deviations from the global trend at much more focused levels than the entire map, and it is the subject of the next chapter.

We will explore these concepts with an applied example, interrogating the data about the presence, nature, and strength of global spatial autocorrelation. To do this, we will use a set of tools collectively known as Exploratory Spatial Data Analysis (ESDA). Analogous to its non-spatial counterpart (EDA; [Tuk77]), ESDA has been specifically designed for this purpose, and puts space and the relative location of the observations in a dataset at the forefront of the analysis. The range of ESDA methods is wide and spans from simpler approaches like choropleth maps (previous chapter), to more advanced and robust methodologies that include statistical inference and an explicit recognition of the geographical arrangement of the data. The purpose of this chapter is to dip our toes into the latter group.

6.2 An empirical illustration: the EU Referendum

To illustrate the notion of spatial autocorrelation and its different variants, let us turn to an example with real world data. Before the data, let us import all the relevant libraries that we will use throughout the chapter:

```
# Graphics
import matplotlib.pyplot as plt
import seaborn
from pysal.viz import splot
from splot.esda import plot_moran
import contextily

# Analysis
import geopandas
import pandas
from pysal.explore import esda
from pysal.lib import weights
from numpy.random import seed
```

In 2016, the United Kingdom ran a referendum to decide whether to remain in the European Union or to leave the club, the so-called "Brexit" vote. We will use the official data from the Electoral Commission at the local authority level on percentage of votes for the Remain and Leave campaigns. There are two distinct datasets we will combine:

- Electoral Commission data on vote percentages at the local authority level. [CSV]
- ONS Local Authority Districts (December 2016) Generalized Clipped Boundaries in the UK WGS84. [SHP]

The vote results are stored in a `csv` file which we read into a dataframe:

```
brexit_data_path = "../data/brexit/brexit_vote.csv"
ref = pandas.read_csv(brexit_data_path, index_col="Area_Code")
ref.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 382 entries, E06000031 to E08000036
Data columns (total 20 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   id                      382 non-null    int64
 1   Region_Code             382 non-null    object
 2   Region                  382 non-null    object
 3   Area                    382 non-null    object
 4   Electorate              382 non-null    int64
 5   ExpectedBallots         382 non-null    int64
 6   VerifiedBallotPapers    382 non-null    int64
```

(continued on next page)

(continued from previous page)

```
 7   Pct_Turnout               382 non-null    float64
 8   Votes_Cast                382 non-null    int64
 9   Valid_Votes               382 non-null    int64
 10  Remain                    382 non-null    int64
 11  Leave                     382 non-null    int64
 12  Rejected_Ballots          382 non-null    int64
 13  No_official_mark          382 non-null    int64
 14  Voting_for_both_answers   382 non-null    int64
 15  Writing_or_mark           382 non-null    int64
 16  Unmarked_or_void          382 non-null    int64
 17  Pct_Remain                382 non-null    float64
 18  Pct_Leave                 382 non-null    float64
 19  Pct_Rejected              382 non-null    float64
dtypes: float64(4), int64(13), object(3)
memory usage: 62.7+ KB
```

While the shapes of the geographical units (local authority districts, in this case) are stored in a compressed GeoJSON file, we can read it directly from the `.zip` file as follows:

```
lads = geopandas.read_file(
    "../data/brexit/local_authority_districts.geojson"
).set_index("lad16cd")
lads.info()
```

```
<class 'geopandas.geodataframe.GeoDataFrame'>
Index: 391 entries, E06000001 to W06000023
Data columns (total 10 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   objectid    391 non-null    int64
 1   lad16nm     391 non-null    object
 2   lad16nmw    22 non-null     object
 3   bng_e       391 non-null    int64
 4   bng_n       391 non-null    int64
 5   long        391 non-null    float64
 6   lat         391 non-null    float64
 7   st_areasha  391 non-null    float64
 8   st_lengths  391 non-null    float64
 9   geometry    391 non-null    geometry
dtypes: float64(4), geometry(1), int64(3), object(2)
memory usage: 33.6+ KB
```

Although there are several variables that could be considered, we will focus on `Pct_Leave`, which measures the proportion of votes for the Leave alternative. For convenience, let us merge the vote results with the spatial data and project the output

into the Spherical Mercator coordinate reference system (CRS), the preferred choice of web maps, which will allow us to combine them with contextual tiles later:

```
db = (
    geopandas.GeoDataFrame(
        lads.join(ref[["Pct_Leave"]]), crs=lads.crs
    )
    .to_crs(epsg=3857)[
        ["objectid", "lad16nm", "Pct_Leave", "geometry"]
    ]
    .dropna()
)
db.info()
```

```
<class 'geopandas.geodataframe.GeoDataFrame'>
Index: 380 entries, E06000001 to W06000023
Data columns (total 4 columns):
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 0   objectid   380 non-null    int64
 1   lad16nm    380 non-null    object
 2   Pct_Leave  380 non-null    float64
 3   geometry   380 non-null    geometry
dtypes: float64(1), geometry(1), int64(1), object(1)
memory usage: 14.8+ KB
```

And with these elements, we can generate a choropleth map to get a quick sense of the spatial distribution of the data we will be analyzing. Note how we use some visual tweaks (e.g., transparency through the `alpha` attribute) to make the final plot in Figure 6.1 easier to read:

```
f, ax = plt.subplots(1, figsize=(9, 9))
db.plot(
    column="Pct_Leave",
    cmap="viridis",
    scheme="quantiles",
    k=5,
    edgecolor="white",
    linewidth=0.0,
    alpha=0.75,
    legend=True,
    legend_kwds={"loc": 2},
    ax=ax,
)
contextily.add_basemap(
    ax,
    crs=db.crs,
    source=contextily.providers.Stamen.TerrainBackground,
```

(continued on next page)

(continued from previous page)

```
)
ax.set_axis_off()
```

The final piece we need before we can delve into autocorrelation is the spatial weights matrix. We will use eight nearest neighbors for the sake of the example, but our earlier discussion of spatial weights in Chapter 4 applies in this context, and other criteria would be valid too. We also row-standardize them:

```
# Generate W from the GeoDataFrame
w = weights.KNN.from_dataframe(db, k=8)
# Row-standardization
w.transform = "R"
```

6.3 Global spatial autocorrelation

The map above is a good way to begin exploring the main spatial patterns in the data. At first sight, it appears to display a fair amount of positive spatial autocorrelation: local authorities with high percentages of votes to leave the EU tend to be next to each other (see, for instance, the eastern region), as are those where a much smaller proportion of their population voted to leave (with Scotland being a good example in the north). Humans however are very good pattern detectors. All over our history since we started as a species, life has rewarded pattern recognition abilities and punished individuals lacking it. Think of the advantage our ancestors had if they were able to spot particular shapes or movement when hunting, or the trouble they could get into if they were not able to recognize certain others in the darkness of night. This extraordinary capability to spot trends, patterns and associations tends to also create many false positives: cases where we think there is a pattern, but in fact what we are seeing is largely random [She08]. This is particularly accentuated in the case of maps where, as we have seen in choropleth maps from Chapter 5, the shape and size of geometries can also significantly distort our perception of the underlying pattern. By looking at the map above, for example, we can have an educated guess about the presence of spatial autocorrelation; but actually determining whether what we are seeing could have come from pure chance or not is usually easier said than done.

That is exactly the purpose of indicators of global spatial autocorrelation: to leverage the power of statistics to help us first summarize the spatial distribution of values present in a map, and second obtain a formal quantification of the departure from randomness. These are statistics to characterize a map in terms of its degree of clustering and summarize it, either in a visual or numerical way. However, before we can delve into the statistics, we need to understand a core building block: the spatial lag. With that concept under the belt, we are in a position to build a good understanding of global

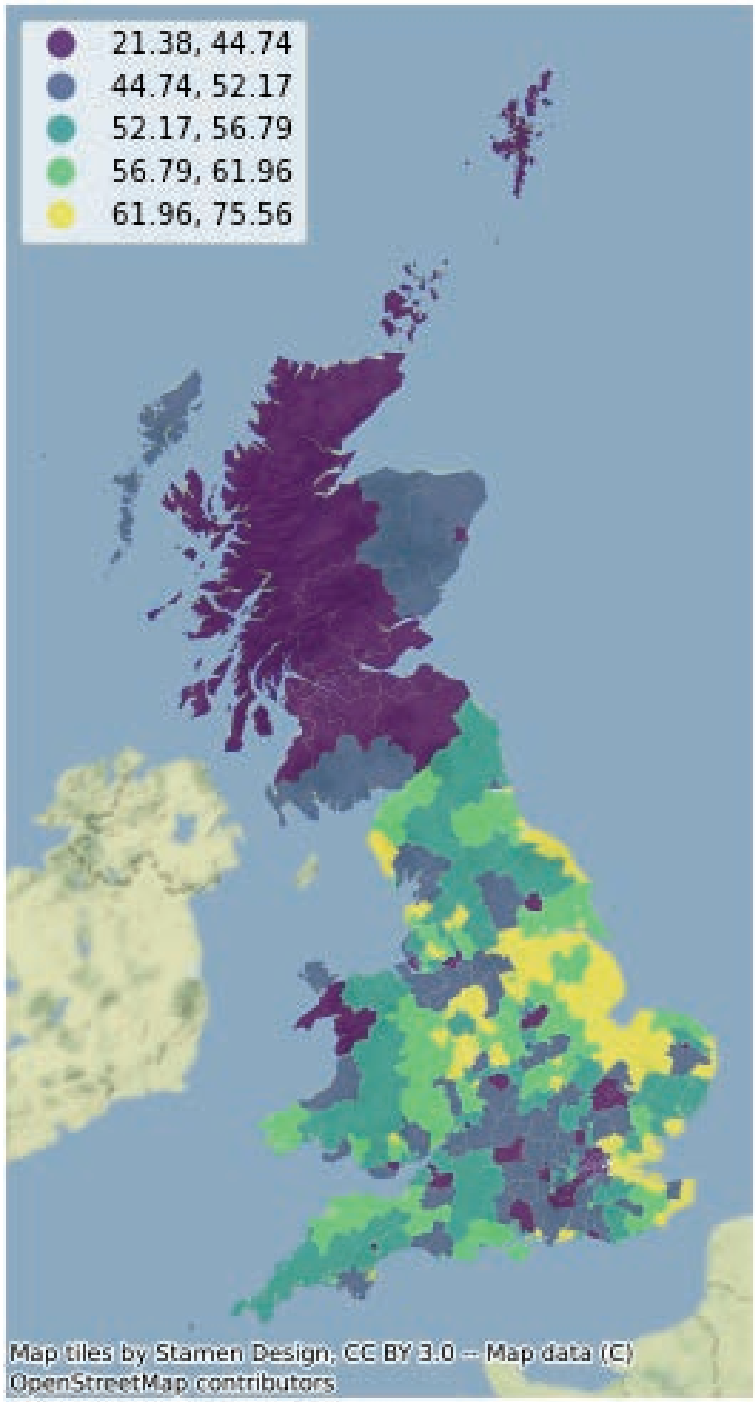


Fig. 6.1: Percentage of voters wanting to leave the EU in the 2016 UK Referendum known as the 'Brexit' vote.

spatial autocorrelation. We will gently enter it with the binary case, when observations can only take two (potentially categorical) values, before we cover the two workhorses of the continuous case: the Moran Plot and Moran's I.

6.3.1 Spatial lag

The spatial lag operator is one of the most common and direct applications of spatial weights matrices (called **W** formally) in spatial analysis. The mathematical definition is the product of **W** and the vector of a given variable. Conceptually, the spatial lag captures the behavior of a variable in the immediate surroundings of each location; in that respect, it is akin to a local smoother of a variable.

We can formally express it in matrix notation as:

$$Y_{sl} = \mathbf{W}Y$$

or, in individual notation as:

$$y_{sl-i} = \sum_j w_{ij} y_j$$

where w_{ij} is the cell in **W** on the i-th row and j-th column, thus capturing the spatial relationship between observations i and j. y_{sl-i} thus captures the product of the values and weights of each observation other than i in the dataset. Because non-neighbors receive a weight of zero, y_{sl-i} really captures the product of values and weights for i's neighbors. If **W** is binary, this will amount to the sum of the values of i's neighbors (useful in some contexts, such as studies of market potential); if W is row standardized, a common transformation, then w_{ij} is bounded between zero and one; the spatial lag thus then becomes a "local average," the average value of Y in the neighborhood of each observation i. This latter meaning is the one that will enable our analysis of spatial autocorrelation below.

As we will discover throughout this book, the spatial lag is a key element of many spatial analysis techniques and, as such, it is fully supported in Pysal. To compute the spatial lag of a given variable, `Pct_Leave` for example, we can do it as follows:

```
db["Pct_Leave_lag"] = weights.spatial_lag.lag_spatial(
    w, db["Pct_Leave"]
)
```

Let us peek into two local authority districts to get a better intuition of what is behind the spatial lag:

```
db.loc[["E08000012", "S12000019"], ["Pct_Leave", "Pct_Leave_
↪lag"]]
```

```
           Pct_Leave  Pct_Leave_lag
lad16cd
E08000012      41.81       54.61375
S12000019      37.94       38.01875
```

The first row (`E08000012`) represents Liverpool, which was a notorious "Remainer" island among the mostly-Leave North of England. Outside of London and Scotland, it was one of the few locations with less than majority to Leave. The second row (`S12000019`) represents Midlothian, in Scotland, where no local authority voted to leave. Although both Liverpool and Midlothian display a similar percentage of population who voted to leave (42% and 38%, respectively), the difference in their spatial lags captures the wider geographical context, which are quite different.

To end this section visually, the smoothing nature of the lag can be appreciated in the following map comparison in Figure 6.2.

```
f, axs = plt.subplots(1, 2, figsize=(12, 6))
ax1, ax2 = axs

db.plot(
    column="Pct_Leave",
    cmap="viridis",
    scheme="quantiles",
    k=5,
    edgecolor="white",
    linewidth=0.0,
    alpha=0.75,
    legend=True,
    ax=ax1,
)
ax1.set_axis_off()
ax1.set_title("% Leave")
contextily.add_basemap(
    ax1,
    crs=db.crs,
    source=contextily.providers.Stamen.TerrainBackground,
)

db.plot(
    column="Pct_Leave_lag",
    cmap="viridis",
    scheme="quantiles",
    k=5,
    edgecolor="white",
    linewidth=0.0,
    alpha=0.75,
    legend=True,
```

(continued on next page)

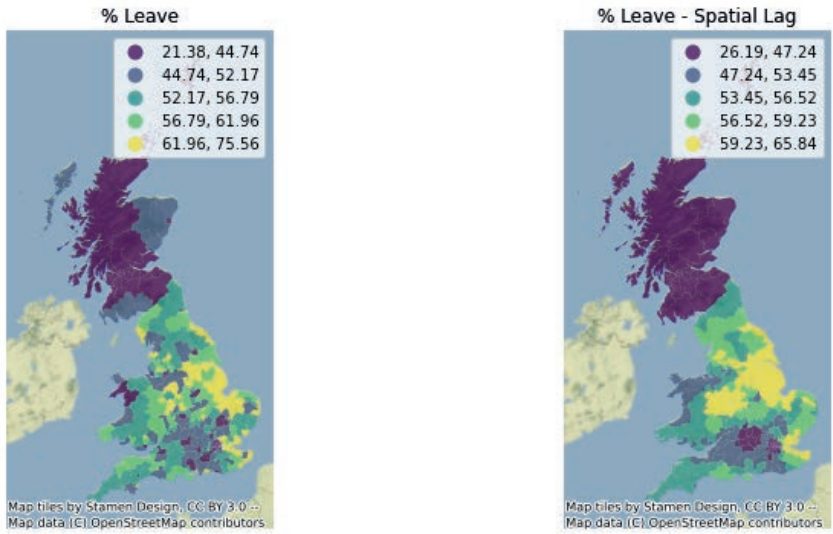


Fig. 6.2: Vote to leave the EU and its spatial lag.

(continued from previous page)

```
    ax=ax2,
)
ax2.set_axis_off()
ax2.set_title("% Leave - Spatial Lag")
contextily.add_basemap(
    ax2,
    crs=db.crs,
    source=contextily.providers.Stamen.TerrainBackground,
)

plt.show()
```

The stark differences on the left between immediate neighbors (as in the case of Liverpool, in the NW of England) are diminished in the map on the right. Thus, as discussed above, the spatial lag can also smooth out the differences between nearby observations.

6.3.2 Binary case: join counts

The spatial lag plays an important role in quantifying spatial autocorrelation. Using it, we can begin to relate the behavior of a variable at a given location to its pattern in the immediate neighborhood. Measures of global spatial autocorrelation will then use each observation to construct overall measures about the general trend in a given dataset.

Our first dip into these measures considers a simplified case: binary values. This occurs when the variable we are interested in only takes two values. In this context, we are interested in whether a given observation is surrounded by others within the same category. For example, returning to our dataset, we want to assess the extent to which local authorities who voted to Leave tend to be surrounded by others who also voted to leave. To proceed, let us first calculate a binary variable (`Leave`) that indicates 1 if the local authority voted to leave, and zero otherwise:

```
db["Leave"] = (db["Pct_Leave"] > 50).astype(int)
db[["Pct_Leave", "Leave"]].tail()
```

lad16cd	Pct_Leave	Leave
W06000018	57.63	1
W06000019	62.03	1
W06000021	49.56	0
W06000022	55.99	1
W06000023	53.74	1

Which we can visualize readily in Figure 6.3:

```
f, ax = plt.subplots(1, figsize=(9, 9))
db.plot(
    ax=ax,
    column="Leave",
    categorical=True,
    legend=True,
    edgecolor="0.5",
    linewidth=0.25,
    cmap="Set3",
    figsize=(9, 9),
)
ax.set_axis_off()
ax.set_title("Leave Majority")
plt.axis("equal")
plt.show()
```

Visually, it appears that the map represents a clear case of positive spatial autocorrelation: overall, there are few visible cases where a given observation is surrounded by

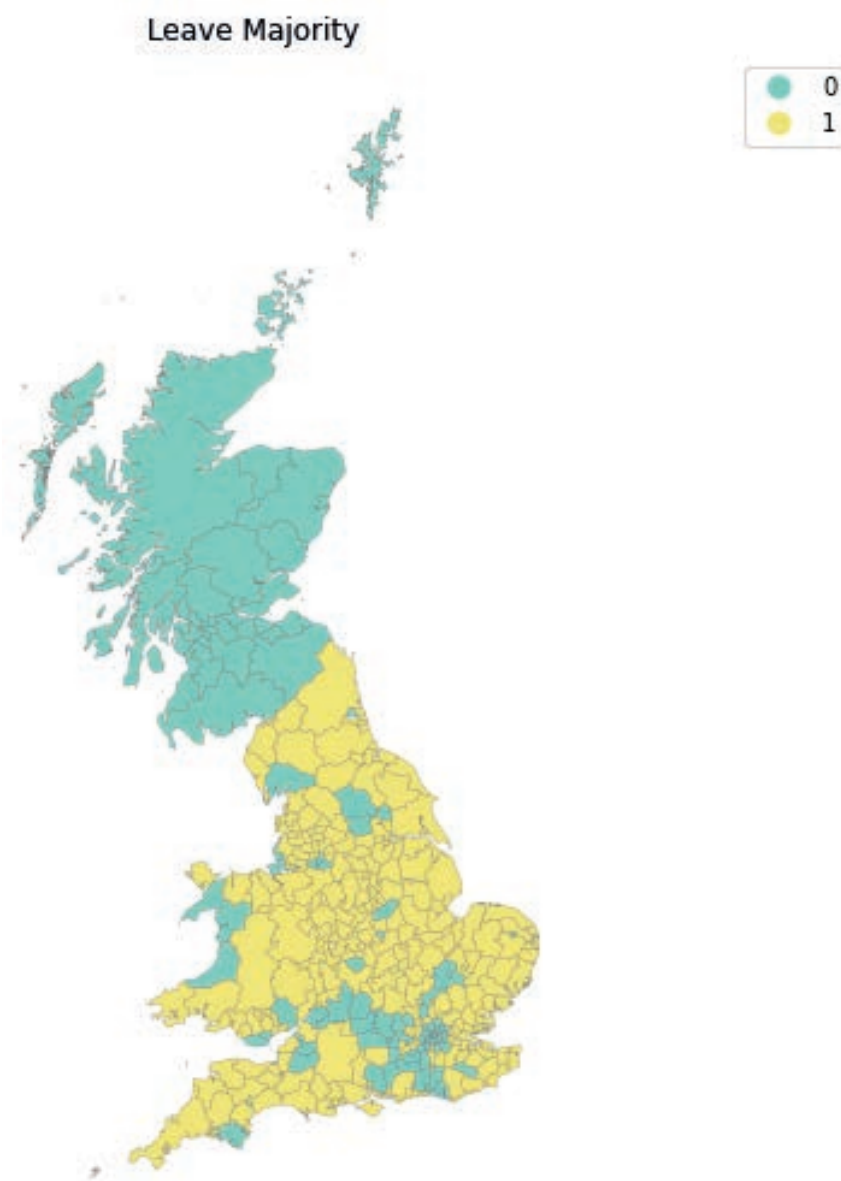


Fig. 6.3: Places with a majority voting leave in the Brexit vote

others in the opposite category. To formally explore this initial assessment, we can use what is called a "join count" statistic (JC; [CO81]). Imagine a checkerboard with green (G, value 0) and yellow (Y, value 1) squares. The idea of the statistic is to count occurrences of green-green (GG), yellow-yellow (YY), or green-yellow/yellow-green (GY)

joins (or neighboring pairs) on the map. In this context, both GG and YY reflect positive spatial autocorrelation, while GY captures its negative counterpart. The intuition of the statistic is to provide a baseline of how many GG, YY, and GY one would expect under the case of complete spatial randomness, and to compare this with the observed counts in the dataset. A situation where we observe more GG/YY than expected and less GY than expected would suggest positive spatial autocorrelation; while the opposite, more GY than GG/YY, would point towards negative spatial autocorrelation.

Since the spatial weights are only used here to delimit who is a neighbor or not, the join count statistic requires binary weights. Let us thus transform `w` back to a non-standardized state:

```
w.transform
```

```
'R'
```

```
w.transform = "O"
```

```
w.transform
```

```
'O'
```

We can compute the statistic as:

```
seed(1234)
jc = esda.join_counts.Join_Counts(db["Leave"], w)
jc
```

```
<esda.join_counts.Join_Counts at 0x7fcf54653c40>
```

As it is common throughout Pysal, we are creating an object (`jc`) that holds a lot of information beyond the value of the statistic calculated. For example, we can check how many occurrences of GG we have (note the attribute is `bb`, which originates from the original reference where the two considered classes were black and white):

```
jc.bb
```

```
871.0
```

how many YY occurrences our map has:

```
jc.ww
```

```
302.0
```

and how many GY/YG we find:

```
jc.bw
```

```
347.0
```

The sum of the three types of joins gives us the total number of comparisons:

```
jc.bb + jc.ww + jc.bw
```

```
1520.0
```

```
jc.J
```

```
1520.0
```

The statistic is based on comparing the actual number of joins of each class (`bb`, `ww`, `bc`) with what one would expect in a case of spatial randomness. Those expectations can be accessed as well, for the GG/YY case:

```
jc.mean_bb
```

```
727.4124124124124
```

and for GY joins:

```
jc.mean_bw
```

```
649.3233233233233
```

Statistical inference to obtain a sense of whether these values are likely to come from random chance or not can be accessed using random spatial permutations of the observed values to create synthetic maps under the null hypothesis of complete spatial randomness. `esda` generates 999 such synthetic patterns and then uses the distribution of join counts from these patterns to generate pseudo-p-values for our observed join count statistics:

```
jc.p_sim_bb
```

```
0.001
```

```
jc.p_sim_bw
```

```
1.0
```

These results point to a clear presence of positive spatial autocorrelation, as there are a lot more joins of pairs in the same category than one would expect (`p_sim_bb`) and significantly less of opposite joins (`p_sim_bw`). We will discuss the generation of the pseudo-p-values in more detail in the next section.

6.3.3 Continuous case: Moran Plot and Moran's I

Once we have built some intuition around how spatial autocorrelation can be formally assessed in the binary case, let us move to situations where the variable of interest does not only take two values, but is instead continuous. Probably the most commonly used statistic in this context is Moran's I [Mor48], which can be written as:

$$I = \frac{n}{\sum_i \sum_j w_{ij}} \frac{\sum_i \sum_j w_{ij} z_i z_j}{\sum_i z_i^2}$$

where n is the number of observations, z_i is the standardized value of the variable of interest at location i, and w_{ij} is the cell corresponding to the i-th row and j-th column of a W spatial weights matrix.

In order to understand the intuition behind its math, it is useful to begin with a graphical interpretation: the Moran Plot. The Moran Plot is a way of visualizing a spatial dataset to explore the nature and strength of spatial autocorrelation. It is essentially a traditional scatterplot in which the variable of interest is displayed against its *spatial lag*. In order to be able to interpret values as above or below the mean, the variable of interest is usually standardized by subtracting its mean:

```
db["Pct_Leave_std"] = db["Pct_Leave"] - db["Pct_Leave"].mean()
db["Pct_Leave_lag_std"] = weights.lag_spatial(
    w, db["Pct_Leave_std"]
)
```

Technically speaking, creating a Moran Plot is very similar to creating any other scatterplot in Python. We will make one for Figure 6.4.

```
f, ax = plt.subplots(1, figsize=(6, 6))
seaborn.regplot(
    x="Pct_Leave_std",
    y="Pct_Leave_lag_std",
    ci=None,
    data=db,
    line_kws={"color": "r"},
)
ax.axvline(0, c="k", alpha=0.5)
```

(continued on next page)

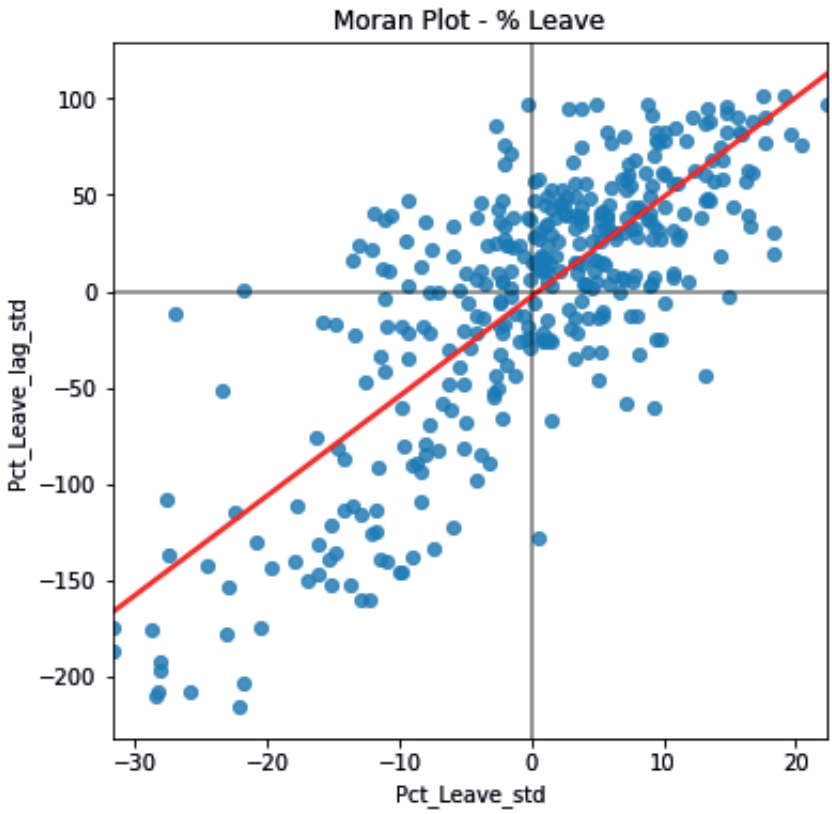


Fig. 6.4: Brexit vote, % leave Moran Scatterplot.

(continued from previous page)

```
ax.axhline(0, c="k", alpha=0.5)
ax.set_title("Moran Plot - % Leave")
plt.show()
```

Figure 6.4 displays the relationship between the standardized "Leave" voting percentage in a local authority and its spatial lag which, because the W used is row-standardized, can be interpreted as the average standardized density of the percent Leave vote in the neighborhood of each observation. In order to guide the interpretation of the plot, a linear fit is also included. This line represents the best linear fit to the scatterplot or, in other words, what is the best way to represent the relationship between the two variables as a straight line.

The plot displays a positive relationship between both variables. This is indicates the

presence of positive spatial autocorrelation: similar values tend to be located close to each other. This means that the overall trend is for high values to be close to other high values, and for low values to be surrounded by other low values. This, however, does not mean that this is the only case in the dataset: there can of course be particular situations where high values are surrounded by low ones, and vice versa. But it means that, if we had to summarize the main pattern of the data in terms of how clustered similar values are, the best way would be to say they are positively correlated and, hence, clustered over space. In the context of the example, this can be interpreted along the lines of: local authorities where people voted in high proportion to leave the EU tend to be located nearby other regions that also registered high proportions of Leave vote. In other words, we can say the percentage of Leave votes is spatially autocorrelated in a positive way.

The Moran Plot is an excellent tool to explore the data and get a good sense of how much values are clustered over space. However, because it is a graphical device, it is sometimes hard to condense its insights into a more concise way. For these cases, a good approach is to come up with a statistical measure that summarizes the figure. This is exactly what Moran's I, as formally expressed above, is meant to do.

Very much in the same way the mean summarizes a crucial element of the distribution of values in a non-spatial setting, so does Moran's I for a spatial dataset. Continuing the comparison, we can think of the mean as a single numerical value summarizing a histogram or a kernel density plot. Similarly, Moran's I captures much of the essence of the Moran Plot. In fact, there is a close connection between the two: the value of Moran's I corresponds with the slope of the linear fit overlayed on top of the Moran Plot.

In order to calculate Moran's I in our dataset, we can call a specific function in `esda` directly (before that, let us row standardize the `w` object again):

```
w.transform = "R"
moran = esda.moran.Moran(db["Pct_Leave"], w)
```

The method `Moran` creates an object that contains much more information than the actual statistic. If we want to retrieve the value of the statistic, we can do it this way:

```
moran.I
```

```
0.6454521298096587
```

The other bit of information we will extract from Moran's I relates to statistical inference: could the pattern we observe in the map (and that measured by Moran's I) have arisen purely from randomness? If we considered the same variable but shuffled its locations randomly, would we obtain a map with similar characteristics? To obtain insight into these questions, `esda` performs a simulation and returns a measure of certainty about how likely it is to obtain a pattern like the one we observe under a spatially random process. This is summarized in the `p_sim` attribute:

```
moran.p_sim
```

```
0.001
```

The value is calculated as an empirical p-value that represents the proportion of realizations in the simulation under spatial randomness that are more extreme than the observed value. A small enough p-value associated with the Moran's I of a map allows to reject the hypothesis that the map is random. In other words, we can conclude that the map displays more spatial pattern than we would expect if the values had been randomly allocated to a locations.

That is a very low value, particularly considering it is actually the minimum value we could have obtained given the simulation behind it used 999 permutations (default in `esda`) and, by standard terms, it would be deemed statistically significant. We can elaborate a bit further on the intuition behind the value of `p_sim`. If we generated a large number of maps with the same values but randomly allocated over space, and calculated the Moran's I statistic for each of those maps, only 0.01% of them would display a larger (absolute) value than the one we obtain from the observed data, and the other 99.99% of the random maps would receive a smaller (absolute) value of Moran's I. If we remember again that the value of Moran's I can also be interpreted as the slope of the Moran Plot, what we have is that, in this case, the particular spatial arrangement of values over space we observe for the percentage of Leave votes is more concentrated than if we were to randomly shuffle the vote proportions among the map, hence the statistical significance. As a first step, the global autocorrelation analysis can teach us that observations do seem to be positively autocorrelated over space. Indeed, the overall spatial pattern in the EU Referendum vote was highly marked: nearby areas tended to vote alike.

Thanks to the `splot` visualization module in Pysal, we can obtain a quick representation of the statistic that combines the Moran scatterplot we saw before with a graphic of the empirical test that we carry out to obtain `p_sim`. This is shown in Figure 6.5.

```
plot_moran(moran);
```

On the left panel we can see in grey the empirical distribution generated from simulating 999 random maps with the values of the `Pct_Leave` variable and then calculating Moran's I for each of those maps. The blue rug signals the mean. In contrary, the red rug shows Moran's I calculated for the variable using the geography observed in the dataset. It is clear the value under the observed pattern is significantly higher than under randomness. This insight is confirmed on the right panel, which shows an equivalent plot to the Moran Scatterplot we created above.

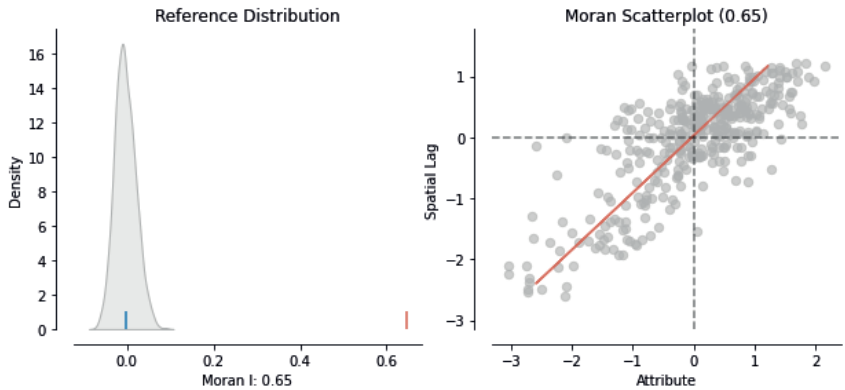


Fig. 6.5: Brexit vote, Moran's I replicate distribution and Scatterplot.

6.3.4 Other global indices

Moran's I is probably the most widely used statistic for global spatial autocorrelation; however, it is not the only one. In this final part of the chapter, we introduce two additional measures that are common in applied work. Although they all consider spatial autocorrelation, they differ in how the concept is tackled in the specification of each test.

6.3.4.1 Geary's C

The contiguity ratio c, proposed by [Gea54], is given by:

$$C = \frac{(n-1)}{2\sum_i \sum_j w_{ij}} \frac{\sum_i \sum_j w_{ij}(y_i - y_j)^2}{\sum_i (y_i - \bar{y})^2}$$

where n is the number of observations, w_{ij} is the cell in a binary matrix W expressing whether i and j are neighbors ($w_{ij} = 1$) or not ($w_{ij} = 1$), y_i is the i-th observation of the variable of interest, and $\bar{y}$ is its sample mean. When compared to Moran's I, it is apparent both measures compare the relationship of Y within each observation's local neighborhood to that over the entire sample. However, there are also subtle differences. While Moran's I takes cross-products on the standardized values, Geary's C uses differences on the values without any standardization.

Computationally, Geary's C is more demanding, but it can be easily computed using `esda`:

```
geary = esda.geary.Geary(db["Pct_Leave"], w)
```

Which has a similar way of accessing its estimate:

```
geary.C
```

```
0.4080233215854691
```

Inference is performed in a similar way as with Moran's I. We can perform a simulation that allows us to draw an empirical distribution of the statistic under the null of spatial randomness, and then compare it with the statistic obtained when using the observed geographical distribution of the data. To access the pseudo-p-value, calculated as in the Moran case, we can call `p_sim`:

```
geary.p_sim
```

```
0.001
```

In this case, Geary's C points in the same direction as Moran's I: there is clear indication that the statistic we calculate on the observed dataset is different from what would be expected in a situation of pure spatial randomness. Hence, from this analysis, we can also conclude spatial autocorrelation is present.

6.3.4.2 Getis and Ord's G

Originally proposed by [GO92], the G is the global version of a family of statistics of spatial autocorrelation based on distance. The G class of statistics is conceived for points, hence the use of a distance W, but it can also be applied to polygon data if a binary spatial weights matrix can be constructed. Additionally, it is designed for the study of positive variables with a natural origin. The G can be expressed as follows:

$$G(d) = \frac{\sum_i \sum_j w_{ij}(d)\, y_i\, y_j}{\sum_i \sum_j y_i\, y_j}$$

where $w_{ij}(d)$ is the binary weight assigned on the relationship between observations i and j following a distance band criterion. G was originally proposed as a measure of concentration rather than of spatial autocorrelation. As such, it is well suited to test to what extent similar values (either high or low) tend to co-locate. In other words, the G is a statistic of *positive* spatial autocorrelation. This is usually the interest in most Geographic Data Science applications. However, it is important to note that, because G can be understood as a measure of the intensity with which Y is concentrated, the statistic is not able to pick up cases of *negative* spatial autocorrelation.

To illustrate its computation, let us calculate a binary distance band W. To make sure every observation has at least one neighbor, we will use the `min_threshold_distance` method and project the dataset into the Ordnance Survey CRS (`EPSG` code 27700), expressed in meters:

```
db_osgb = db.to_crs(epsg=27700)
pts = db_osgb.centroid
xys = pandas.DataFrame({"X": pts.x, "Y": pts.y})
min_thr = weights.util.min_threshold_distance(xys)
min_thr
```

```
180878.91800926204
```

For every local authority to have a neighbor, the distance band needs to at least be about 181 kilometers. This information can then be passed to the `DistanceBand` constructor:

```
w_db = weights.DistanceBand.from_dataframe(db_osgb, min_thr)
```

At this point, we are ready to calculate the global G statistic:

```
gao = esda.getisord.G(db["Pct_Leave"], w_db)
```

Access to the statistic (`gao.G`) and additional attributes can be gained in the same way as with the previous statistics:

```
print(
    "Getis & Ord G: %.3f | Pseudo P-value: %.3f" % (gao.G,␣
↪gao.p_sim)
)
```

```
Getis & Ord G: 0.434 | Pseudo P-value: 0.003
```

Similarly, inference can also be carried out by relying on computational simulations that replicate several instances of spatial randomness using the values in the variable of interest, but shuffling their locations. In this case, the pseudo-p-value computed suggests a clear departure from the hypothesis of no concentration.

6.4 Questions

1. Return to the original `ref` table and pull out the `Pct_Rejected` variable. Let us explore patterns in rejected votes:
   (a) Create a choropleth displaying the spatial distribution of `Pct_Rejected`.
   (b) Build a spatial weights matrix with eight nearest neighbors for the Local Authorities.
   (c) Create a Moran Scatterplot relating `Pct_Rejected` to its spatial lag.

(d) Calculate Moran's I for `Pct_Rejected`.

(e) Interpret what you find through this Moran's analysis. What do we learn about the geography of vote rejection?

2. Sometimes referendums require more than 50% to make the change they ask about. Let us imagine the EU referendum required 60% to succeed on leaving the EU.

   (a) Use `Pct_Leave` to create a binary variable that takes a value of 1 if the percentage was larger than 60, 0 otherwise.

   (b) Create a choropleth with the newly created variable. Are there any differences in the geographical pattern of the vote to leave the EU?

   (c) Recompute the Join Counts statistic for this new variable. What can we conclude? Are there any notable changes in the extent to which "Leave" votes were distributed spatially?

3. Let us explore the effect of different weights matrices by returning to the global analysis we performed for the `Leave` variable.

   (a) Create two additional KNN weights to those already built, one with four neighbors (you may call it `wk4`) and one with 12 neighbors (`wk12`)

   (b) Create a choropleth that displays the spatial lag of `Pct_Leave` using each of the two new matrices. How are they different? Why?

   (c) Now generate Moran Scatterplots using `wk4` and `wk12`. Do they differ from the one we created earlier in the chapter? How? Why?

   (d) Calculate Moran's I using all of the matrices and similarly compare results.

4. Using the same spatial weights matrix throughout, calculate the following statistics of global spatial autocorrelation for the `Pct_Rejected` variable:

   (a) Moran's I

   (b) Geary's C

   (c) Getis and Ord's G

   Describe the results. Do you draw substantially different conclusions from each statistic? If so, why?

5. Drawing from the results found in Question 3 and your intuition, try to generalize the effect of a larger number of neighbors (i.e., a more densely connected graph) in the spatial weights matrix when exploring global spatial autocorrelation.

6. Think whether it is possible to find cases when Moran's I and Getis and

Ord's G disagree substantially. What could drive such a result? What does that mean for the use and interpretation of both statistics?

7. Using k-nearest neighbor weights, can you find the k where Moran's I is largest? Make a plot of the Moran's I for each k you evaluate to show the relationship between the two.

8. As in the previous question, at what value of k is the Geary's C largest?

6.5 Next steps

For a timeless conceptual overview to the approaches of spatial data analysis, consult [Ans89]:

"What is special about spatial data? Alternative perspectives on spatial data analysis." UC Santa Barbara: National Center for Geographic Information and Analysis.

The GIS body of knowledge represents a large set of collected knowledge by geographers across many different domains. Thus, the GISBoK, as it's called, has a very good introductory discussion of global measures of spatial association, too. [WKW19]:

Global Measures of Spatial Association. *The Geographic Information Science & Technology Body of Knowledge (1st Quarter 2019 Edition)*, John P. Wilson (Ed.). DOI: 10.22224/gistbok/2019.1.12

Finally, a more personal, reflective perspective is offered by longstanding quantitative geographer Art Getis in his piece [Get07]:

"Reflections on spatial autocorrelation." *Regional Science & Urban Economics*, 37: 491-496. DOI: 10.1016/j.regsciurbeco.2007.04.005

Taylor & Francis
Taylor & Francis Group
http://taylorandfrancis.com

7

Local Spatial Autocorrelation

In the previous chapter, we explored how global measures of spatial autocorrelation can help us determine whether the overall spatial distribution of our phenomenon of interest is compatible with a geographically random process. These statistics are useful: the presence of spatial autocorrelation has important implications for subsequent statistical analysis. From a substantive perspective, spatial autocorrelation could reflect the operation of processes that generate association between the values in nearby locations. This could represent *spillovers*, where outcomes at one site influence other sites; or it could indicate *contagion*, where outcomes at one site *causally influence* other sites. As we will see later in *Chapter 11*, it could simply be the result of systematic spatial variation (or, as we will call it then, *heterogeneity*). Spatial autocorrelation also sometimes arises from data measurement and processing. In this case, the dependence is a form of non-random noise rather than due to substantive processes. For example, when "down-sampling" geographic data, sometimes large patches of identical values can be created. These may only be artifacts of the interpolation, rather than substantive autocorrelation. Regardless of whether the spatial autocorrelation is due to substantive or nuisance sources, it is a form of non-randomness that complicates statistical analysis. For these reasons, the ability to determine whether spatial autocorrelation is present in a geographically referenced data set is a critical component of the geographic data science toolbox.

Despite their importance, global measures of spatial autocorrelation are "whole map" statistics. They provide a single summary for an entire data set. For example, Moran's I is a good tool to summarize a dataset into a single value that captures the degree of geographical clustering (or dispersion, if negative). However, Moran's I does not indicate areas within the map where specific types of values (e.g., high, low) are clustered, or instances of explicit dispersion. In other words, Moran's I can tell us whether values in our map *cluster* together (or disperse) overall, but it will not inform us about where specific *clusters* (or outliers) are.

In this chapter, we introduce local measures of spatial autocorrelation. Local measures of spatial autocorrelation focus on the relationships between *each* observation and its surroundings, rather than providing a single summary of these relationships across the map. In this sense, they are not summary statistics but scores that allow us to learn more about the spatial structure in our data. The general intuition behind the metrics however is similar to that of global ones. Some of them are even mathematically connected, where the global version can be decomposed into a collection of local ones. One such example are Local Indicators of Spatial Association (LISAs) [Ans95], which we use to build the understanding of local spatial autocorrelation, and on which we spend a good part of the chapter. Once such concepts are clarified, we introduce a couple of alternative statistics that present complementary information or allow us to obtain similar insights for categorical data. Although very often these statistics are used with data expressed in geo-tables, there is nothing fundamentally connecting the two. In fact, the application of these methods to large surfaces is a promising area of work. For that reason, we close the chapter with an illustration of how one can run these statistics on data stored as surfaces.

7.1 An empirical illustration: the EU Referendum

We continue with the same dataset about Brexit voting that we examined in the previous chapter, and thus we utilize the same imports and initial data preparation steps:

```
import matplotlib.pyplot as plt  # Graphics
from matplotlib import colors
import seaborn  # Graphics
import geopandas  # Spatial data manipulation
import pandas  # Tabular data manipulation
import rioxarray  # Surface data manipulation
import xarray  # Surface data manipulation
from pysal.explore import esda  # Exploratory Spatial␣
↪analytics
from pysal.lib import weights  # Spatial weights
import contextily  # Background tiles
```

We read the vote data as a non-spatial table:

```
ref = pandas.read_csv(
    "../data/brexit/brexit_vote.csv", index_col="Area_Code"
)
```

And the spatial geometries for the local authority districts in Great Britain:

```
lads = geopandas.read_file(
    "../data/brexit/local_authority_districts.geojson"
).set_index("lad16cd")
```

Then, we "trim" the `DataFrame` so it retains only what we know we will need, reproject it to spherical mercator, and drop rows with missing data:

```
db = (
    geopandas.GeoDataFrame(
        lads.join(ref[["Pct_Leave"]]), crs=lads.crs
    )
    .to_crs(epsg=3857)[
        ["objectid", "lad16nm", "Pct_Leave", "geometry"]
    ]
    .dropna()
)

db.info()
```

```
<class 'geopandas.geodataframe.GeoDataFrame'>
Index: 380 entries, E06000001 to W06000023
Data columns (total 4 columns):
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 0   objectid   380 non-null    int64
 1   lad16nm    380 non-null    object
 2   Pct_Leave  380 non-null    float64
 3   geometry   380 non-null    geometry
dtypes: float64(1), geometry(1), int64(1), object(1)
memory usage: 14.8+ KB
```

Although there are several variables that could be considered, we will focus on `Pct_Leave`, which measures the proportion of votes in the UK Local Authority that wanted to Leave the European Union. With these elements, we can generate a choropleth to get a quick sense of the spatial distribution of the data we will be analyzing. Note how we use some visual tweaks (e.g., transparency through the `alpha` attribute) to make the final plot easier to read in Figure 7.1:

```
# Set up figure and a single axis
f, ax = plt.subplots(1, figsize=(9, 9))
# Build choropleth
db.plot(
    column="Pct_Leave",
    cmap="viridis",
    scheme="quantiles",
    k=5,
    edgecolor="white",
    linewidth=0.0,
    alpha=0.75,
    legend=True,
    legend_kwds=dict(loc=2),
```

(continued on next page)

(continued from previous page)

```
    ax=ax,
)
# Add basemap
contextily.add_basemap(
    ax,
    crs=db.crs,
    source=contextily.providers.CartoDB.VoyagerNoLabels,
)
# Remove axes
ax.set_axis_off();
```

As in the previous chapter, we require a spatial weights matrix to implement our statistic. Here, we will use eight nearest neighbors for the sake of the example, but the discussion in the earlier chapter on weights applies in this context, and other criteria would be valid too. We also row-standardize them:

```
# Generate W from the GeoDataFrame
w = weights.distance.KNN.from_dataframe(db, k=8)
# Row-standardization
w.transform = "R"
```

7.2 Motivating local spatial autocorrelation

To better understand the underpinnings of local spatial autocorrelation, we return to the Moran Plot as a graphical tool. In this context, it is more intuitive to represent the data in a standardized form, as it will allow us to more easily discern a typology of spatial structure. Let us first calculate the spatial lag of our variable of interest:

```
db["w_Pct_Leave"] = weights.lag_spatial(w, db['Pct_Leave'])
```

And their respective centered versions, where we subtract the average off of every value:

```
db["Pct_Leave_std"] = db["Pct_Leave"] - db["Pct_Leave"].mean()
db["w_Pct_Leave_std"] = weights.lag_spatial(w, db['Pct_Leave_
↪std'])
```

Technically speaking, creating a Moran scatterplot is very similar to creating any other scatterplot. We have also done this before in Chapter 6. To see this again, we can make Figure 7.2:

```
# Set up the figure and axis
f, ax = plt.subplots(1, figsize=(6, 6))
```

(continued on next page)

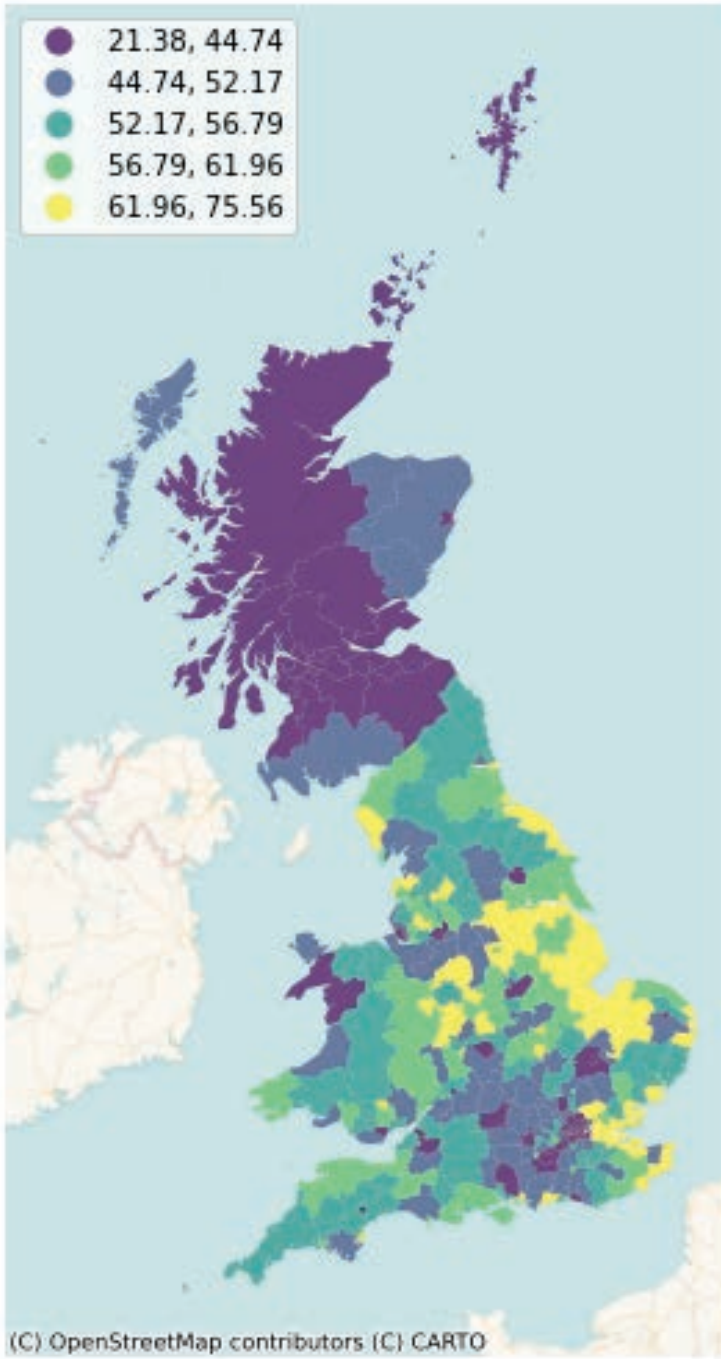


Fig. 7.1: Percentage of voters wanting to leave the EU in the 2016 UK Referendum known as the 'Brexit' vote.

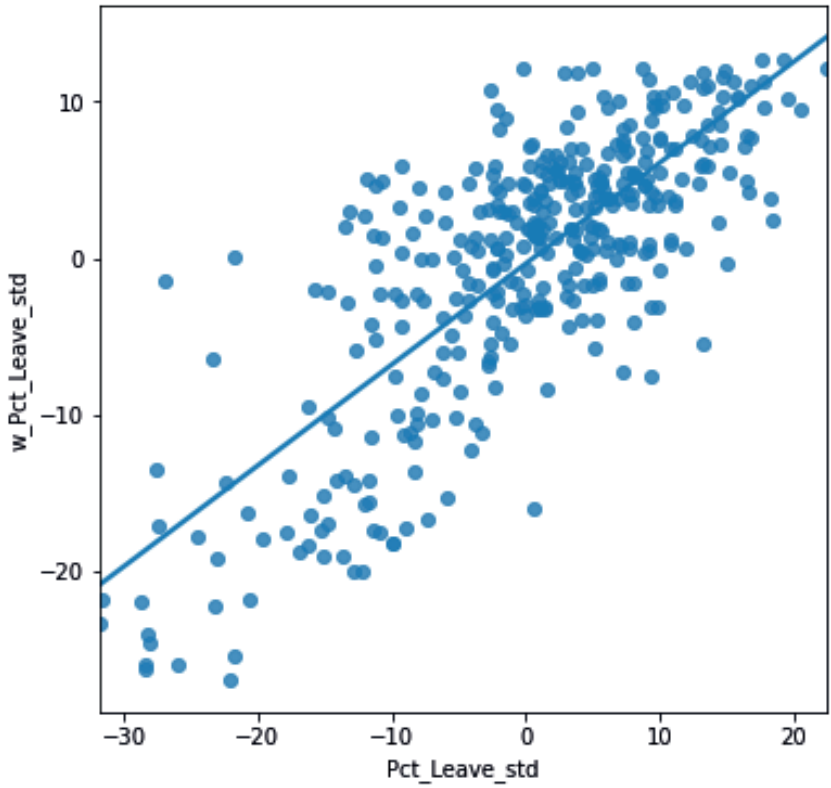


Fig. 7.2: Brexit % leave Moran scatterplot.

(continued from previous page)

```
# Plot values
seaborn.regplot(
    x="Pct_Leave_std", y="w_Pct_Leave_std", data=db, ci=None
)
plt.show()
```

Using standardized values, we can immediately divide each variable (percentage that voted to leave, and its spatial lag) in two groups: those with above-average leave voting, which have positive standardized values; and those with below-average leave voting, which feature negative standardized values. Applying this thinking to both the percentage to leave and its spatial lag, divides a Moran scatterplot into four quadrants. Each of them captures a situation based on whether a given area displays a value above the mean (high) or below (low) in either the original variable (`Pct_Leave`) or its spatial lag (`w_Pct_Leave_std`). Using this terminology, we name the four quadrants

as follows: high-high (HH) for the top-right, low-high (LH) for the top-left, low-low (LL) for the bottom-left, and high-low (HL) for the bottom right. Graphically, we can express this in Figure 7.3:

```
# Set up the figure and axis
f, ax = plt.subplots(1, figsize=(6, 6))
# Plot values
seaborn.regplot(
    x="Pct_Leave_std", y="w_Pct_Leave_std", data=db, ci=None
)
# Add vertical and horizontal lines
plt.axvline(0, c="k", alpha=0.5)
plt.axhline(0, c="k", alpha=0.5)
# Add text labels for each quadrant
plt.text(20, 5, "HH", fontsize=25, c="r")
plt.text(12, -11, "HL", fontsize=25, c="r")
plt.text(-20, 8.0, "LH", fontsize=25, c="r")
plt.text(-25, -11.0, "LL", fontsize=25, c="r")
# Display
plt.show()
```

7.3 Local Moran's I_i

One way to look at the figure above is as a classification of each observation in the dataset depending on its value and that of its neighbors. Furthermore, this classification is exhaustive: every point is assigned a label. But remember local measures help us to identify areas of *unusual* concentration of values. Clusters will represent values of one type that are *unlikely* to appear under the assumption of spatial randomness. To know whether each location belongs to a *statistically significant* cluster of a given kind, we thus need to compare it with what we would expect if the data were allocated over space in a completely random way. However, what we are interested in is whether the strength with which the values are concentrated is unusually high. This is exactly what LISAs are designed to do. A detailed description of the statistical underpinnings of LISAs is beyond the scope of this chapter. If you would like to delve deeper into the math and probability challenges arising, a good recent reference is [SORW21]. In this context, we will provide some intuition about how they work in one LISA statistic, the local Moran's I_i.

The core idea of a local Moran's I_i is to identify cases in which the value of an observation and the average of its surroundings is either more similar (HH or LL in the scatterplot from Figure 7.3) or dissimilar (HL, LH) than we would expect from pure chance. The mechanism to do this is similar to the one in the global Moran's I, but it is applied in this case to each observation. This results in as many statistics as original

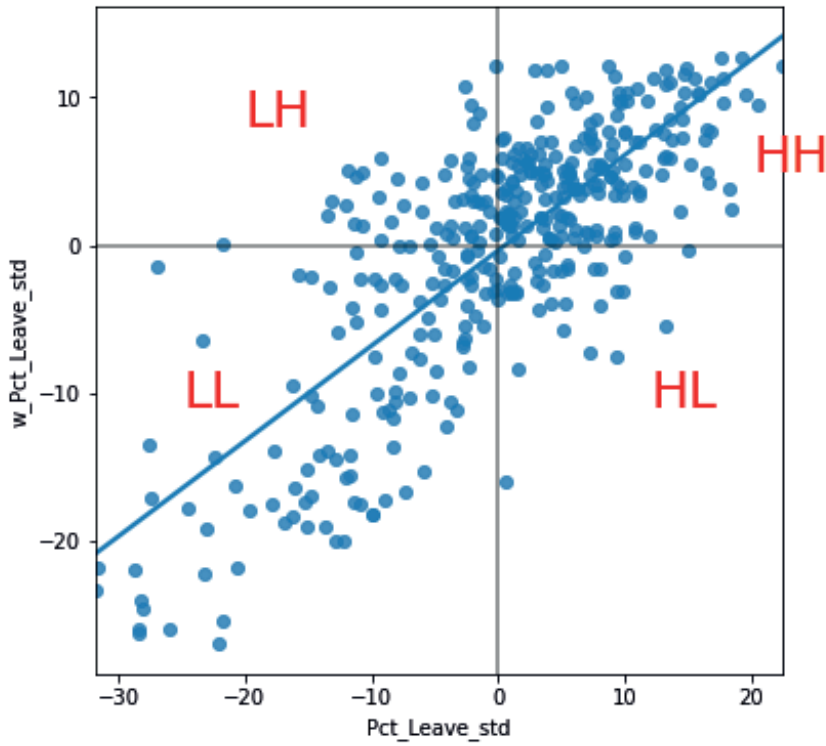


Fig. 7.3: Brexit % leave Moran scatterplot with labelled quadrants.

observations. The formal representation of the statistic can be written as:

$$I_i = \frac{z_i}{m_2} \sum_j w_{ij} z_j \; ; \; m_2 = \frac{\sum_i z_i^2}{n}$$

where m_2 is the second moment (variance) of the distribution of values in the data, $z_i = y_i - \bar{y}$, $w_{i,j}$ is the spatial weight for the pair of observations i and j, and n is the number of observations.

LISAs are widely used in many fields to identify geographical clusters of values or find geographical outliers. They are a useful tool that can quickly return areas in which values are concentrated and provide suggestive evidence about the processes that might be at work. For these reasons, they have a prime place in the geographic data science toolbox. Among many other applications, LISAs have been used to identify geographical clusters of poverty [DSSC18], map ethnic enclaves [JPF10], delineate areas of particularly high/low economic activity [TPPGTZ14], or identify clusters of contagious disease [ZRW+20]. The Local Moran's I_i statistic is only one of a wide variety of LISAs that can be used on many different types of spatial data.

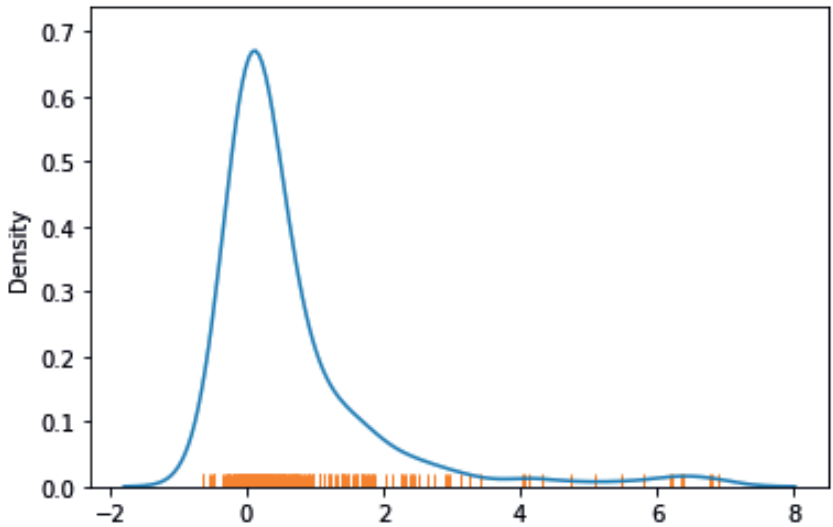


Fig. 7.4: Brexit % Leave vote, observed distribution LISA statistics for all sites.

In Python, we can calculate LISAs in a very streamlined way thanks to `esda`. To compute local Moran statistics, we use the `Moran_Local` function:

```
lisa = esda.moran.Moran_Local(db["Pct_Leave"], w)
```

We need to pass the variable of interest—proportion of Leave votes in this context—and the spatial weights that describes the neighborhood relations between the different areas that make up the dataset. This creates a LISA object (`lisa`) that has a number of attributes of interest. The local indicators themselves are in the `Is` attribute and we can get a sense of their distribution using `seaborn`'s kernel density estimate plotting, as in Figure 7.4:

```
# Draw KDE line
ax = seaborn.kdeplot(lisa.Is)
# Add one small bar (rug) for each observation
# along horizontal axis
seaborn.rugplot(lisa.Is, ax=ax);
```

The figure reveals a rather skewed distribution of local Moran's I_i statistics. This outcome is due to the dominance of positive forms of spatial association, implying most of the local statistic values will be positive. Here it is important to keep in mind that the high positive values arise from value similarity in space, and this can be due to either high values being next to high values *or* low values next to low values. The local I_i values alone cannot distinguish these two cases.

The values in the left tail of the density represent locations displaying negative spatial association. There are also two forms, a high value surrounded by low values, or a low value surrounded by high-valued neighboring observations. And, again, the I_i statistic cannot distinguish between the two cases.

Because of their very nature, looking at the numerical result of LISAs is not always the most useful way to exploit all the information they can provide. Remember we are calculating a statistic for every single observation in the data so, if we have many of them, it will be difficult to extract any meaningful pattern. In this context, a choropleth can help. At first glance, this may seem to suggest that a choropleth of the I_i values would be a useful way to visualize the spatial distribution. We can see such map in the top-left panel of the figure below and, while it tells us whether the local association is positive (HH/LL) or negative (HL/LH), it cannot tell, for example, whether the yellow areas in Scotland are similar to those in the eastern cluster of yellow areas. Are the two experiencing similar patterns of spatial association, or is one of them HH and the other LL? Also, we know that values around zero will not be statistically significant. Which local statistics are thus significant and which ones non-significant from a statistical point of view? In other words, which ones can be considered statistical clusters and which ones mere noise?

To answer these questions, we need to bring in additional information that we have computed when calculating the LISA statistics. We do this in four acts. The first one we have already mentioned: a straighforward choropleth of the local statistic of each area. The other three include information on the quadrant each area is assigned into, whether the statistic is considered significant or not, and a combination of those two in a single so-called *cluster* map. A handy tool in this context is the `splot` library, part of the Pysal family, which provides a lightweight visualization layer for spatial statistics:

```
from splot import esda as esdaplot
```

With all pieces in place, let's first get busy building the figure:

```
# Set up figure and axes
f, axs = plt.subplots(nrows=2, ncols=2, figsize=(12, 12))
# Make the axes accessible with single indexing
axs = axs.flatten()

# Subplot 1 #
# Choropleth of local statistics
# Grab first axis in the figure
ax = axs[0]
# Assign new column with local statistics on-the-fly
db.assign(
    Is=lisa.Is
    # Plot choropleth of local statistics
).plot(
    column="Is",
```

(continued on next page)

(continued from previous page)

```
    cmap="plasma",
    scheme="quantiles",
    k=5,
    edgecolor="white",
    linewidth=0.1,
    alpha=0.75,
    legend=True,
    ax=ax,
)

# Subplot 2 #
# Quadrant categories
# Grab second axis of local statistics
ax = axs[1]
# Plot Quadrant colors (note to ensure all polygons are␣
↪assigned a
# quadrant, we "trick" the function by setting significance␣
↪level to
# 1 so all observations are treated as "significant" and thus␣
↪assigned
# a quadrant color
esdaplot.lisa_cluster(lisa, db, p=1, ax=ax)

# Subplot 3 #
# Significance map
# Grab third axis of local statistics
ax = axs[2]
#
# Find out significant observations
labels = pandas.Series(
    1 * (lisa.p_sim < 0.05),  # Assign 1 if significant, 0␣
↪otherwise
    index=db.index  # Use the index in the original data
    # Recode 1 to "Significant and 0 to "Non-significant"
).map({1: "Significant", 0: "Non-Significant"})
# Assign labels to `db` on the fly
db.assign(
    cl=labels
    # Plot choropleth of (non-)significant areas
).plot(
    column="cl",
    categorical=True,
    k=2,
    cmap="Paired",
    linewidth=0.1,
    edgecolor="white",
    legend=True,
```

(continued on next page)

(continued from previous page)

```
    ax=ax,
)


# Subplot 4 #
# Cluster map
# Grab second axis of local statistics
ax = axs[3]
# Plot Quadrant colors In this case, we use a 5% significance
# level to select polygons as part of statistically␣
↪significant
# clusters
esdaplot.lisa_cluster(lisa, db, p=0.05, ax=ax)

# Figure styling #
# Set title to each subplot
for i, ax in enumerate(axs.flatten()):
    ax.set_axis_off()
    ax.set_title(
        [
            "Local Statistics",
            "Scatterplot Quadrant",
            "Statistical Significance",
            "Moran Cluster Map",
        ][i],
        y=0,
    )
# Tight layout to minimize in-between white space
f.tight_layout()

# Display the figure
plt.show()
```

The purple and yellow locations in the top-left map in Figure 7.5 display the largest magnitude (positive and negative values) for the local statistics I_i. Yet, remember this signifies positive spatial autocorrelation, which can be of high *or* low values. This map thus cannot distinguish between areas with low support for the Brexit vote and those highly in favour.

To distinguish between these two cases, the map in the top-right of Figure 7.5 shows the location of the LISA statistic in the quadrant of the Moran scatterplot. This indicates whether the positive (or negative) local association exists within a specific *quadrant*, such as the HH quadrant. This information is recorded in the `q` attribute of the `lisa` object:

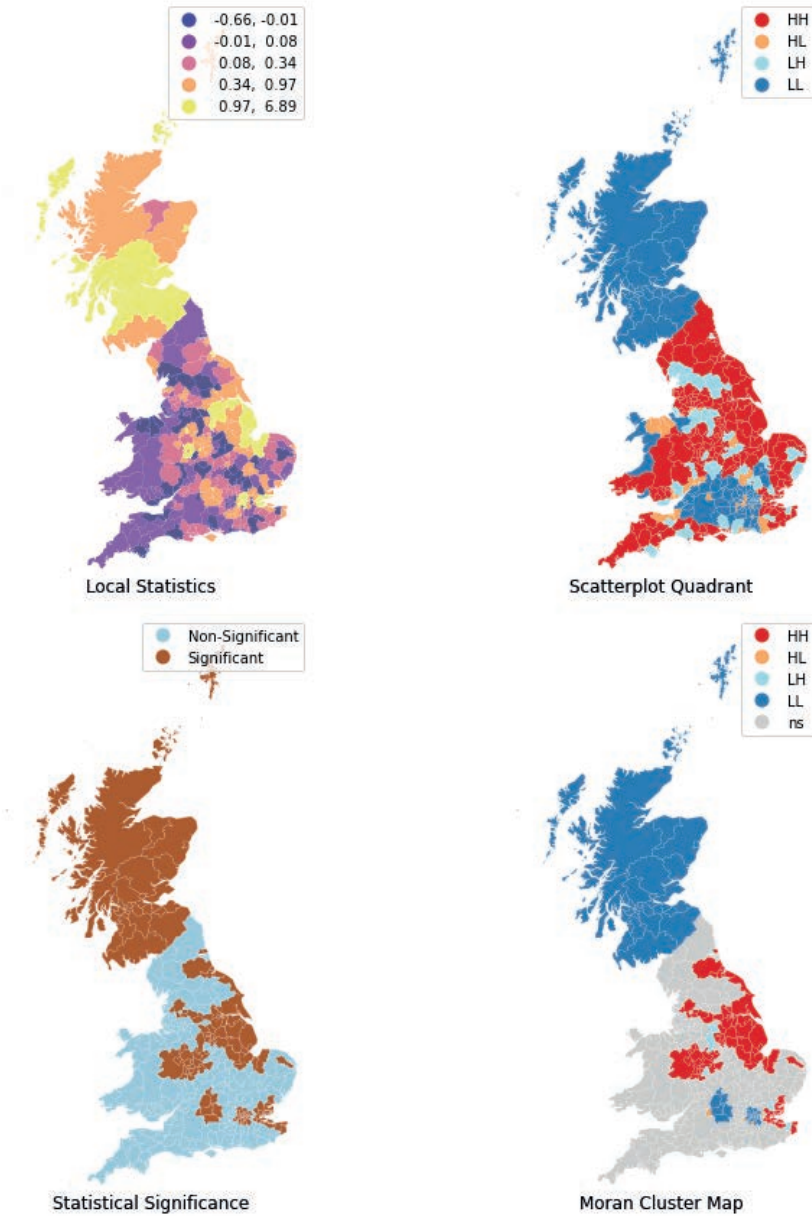


Fig. 7.5: Brexit % Leave vote, Pct_Leave. LISA (top-left), Quadrant (top-right), Signficance (bottom-left), Cluster Map (bottom-right).

```
lisa.q[:10]
```

```
array([1, 1, 1, 1, 1, 1, 4, 1, 4, 1])
```

The correspondence between the numbers in the `q` attribute and the actual quadrants is as follows: `1` represents observations in the HH quadrant, `2` those in the LH one, `3` in the LL region, and `4` in the HL quadrant. Comparing the two maps in the top row reveals that the positive local association in Scotland is due to low support for Brexit, while the positive local association in the south is among local authorities that strongly support Brexit. Overall, we can obtain counts of areas in each quadrant as follows:

```
counts = pandas.value_counts(lisa.q)
counts
```

```
1    183
3    113
2     50
4     34
dtype: int64
```

Showing that the high-high (1), and low-low (3), values are predominant. Care must be taken, however, in the interpretation of these first two maps, as the underlying statistical significance of the local values has not been considered. We have simply mapped the raw LISA value alongside the quadrant in which the local statistic resides. To statistical significance, the bottom-left map distinguishes those polygons whose pseudo-p-value is above (*"Non-Significant"*) or below (*"Significant"*) the threshold value of 5% we use in this context. An examination of the map suggests that quite a few local authorities have local statistics that are small enough so as to be compatible with pure chance.

Therefore, in order to focus on the areas that are most promising, we need to include significance information alongside the quadrant and local statistic. Together, this "cluster map" (as it is usually called) extracts significant observations -those that are highly unlikely to have come from pure chance- and plots them with a specific color depending on their quadrant category. All of the needed pieces are contained inside the `lisa` object we have created above and, if passed in tandem with the geo-table containing the geographies it relates to, `splot` will make a cluster map for us.

Reading the clustermap reveals a few interesting aspects that would have been hard to grasp by looking at the other maps only and that are arguably more relevant for an analysis of the data. First, fewer than half of polygons have degrees of local spatial association strong enough to reject the idea of pure chance:

```
(lisa.p_sim < 0.05).sum() * 100 / len(lisa.p_sim)
```

```
40.26315789473684
```

A little over 41% of the local authorities are considered, by this analysis, to be part of a spatial cluster. Second, we identify three clear areas of low support for leaving the EU: Scotland, London, and the area around Oxford (North-West of London). And third, although there appeared to be many areas with concentrated values indicating high support, it is only the region in the North-East and West of England whose spatial concentration shows enough strength to reasonably rule out pure chance.

Before we move on from the LISA statistics, let's dive into a bit of the data engineering required to "export" significance levels and other information, as well as dig a bit further into what these numbers represent. The latter are useful if we need to work with them as part of a broader data pipeline. So far, cluster maps have been handled by `splot`, but there is quite a bit that happens under the hood. If we needed to recreate one of its maps, or to use this information in a different context, we would need to extract them out of our `lisa` object, and link them up to the original `db` table. Here is one way you can do this.

First, we pull the information computed in `lisa` and insert it in the main data table:

```
# Assign pseudo P-values to `db`
db["p-sim"] = lisa.p_sim
# `1` if significant (at 5% confidence level), `0` otherwise
sig = 1 * (lisa.p_sim < 0.05)
# Assign significance flag to `db`
db["sig"] = sig
# Print top of the table to inspect
db[["sig", "p-sim"]].head()
```

```
           sig  p-sim
lad16cd
E06000001    1  0.016
E06000002    1  0.014
E06000003    1  0.019
E06000004    1  0.016
E06000010    1  0.022
```

```
# Print bottom of the table to inspect
db[["sig", "p-sim"]].tail()
```

```
           sig  p-sim
lad16cd
W06000018    0  0.448
W06000019    0  0.435
W06000021    0  0.427
W06000022    0  0.392
W06000023    0  0.257
```

Thus, the first five values are statistically significant, while the last five observations are not.

Let us stop for a second on these two steps. First, we consider the `sig` column. Akin to global Moran's I, `esda` automatically computes a pseudo-p-value for each LISA. Because some instances of the LISA statistics may not be statistically significant, we want to identify those with a p-value small enough that rules out the possibility of obtaining a similar value in random maps. A few different ways of generating random maps are considered by `esda`, but we focus on a strategy that actually simulates hundreds of thousands of random maps to get a rough idea of the possible local statistic values at each local authority given the data we saw. In addition, we follow a similar reasoning as with global Moran's I and use 5% as the threshold for statistical significance. To identify these values, we create a variable, `sig`, that contains `True` if the p-value of the observation satisfies the condition, and `False` otherwise.

Next, we construct our quadrant values using the `q` attribute which records the Moran Scatterplot quadrant for each local value. However, we now mask these values using the newly created binary significance measure `sig`, so only observations in a quadrant that are considered significant are labeled as part of that given quadrant. The remainder are labeled as non-significant.

```
# Pick as part of a quadrant only significant polygons,
# assign `0` otherwise (Non-significant polygons)
spots = lisa.q * sig
# Mapping from value to name (as a dict)
spots_labels = {
    0: "Non-Significant",
    1: "HH",
    2: "LH",
    3: "LL",
    4: "HL",
}
# Create column in `db` with labels for each polygon
db["labels"] = pandas.Series(
    # First initialise a Series using values and `db` index
    spots,
    index=db.index
    # Then map each value to corresponding label based
    # on the `spots_labels` mapping
).map(spots_labels)
# Print top for inspection
db["labels"].head()
```

```
lad16cd
E06000001    HH
E06000002    HH
E06000003    HH
```

(continued on next page)

(continued from previous page)

```
E06000004    HH
E06000010    HH
Name: labels, dtype: object
```

These cluster labels are meaningful if you know of the Moran Plot. To help making them a bit more intuitive, a terminology that is sometimes used goes as follows. Positive forms of local spatial autocorrelation are of two types. First, HH observations, which we can term as "hot spots", represent areas where values at the site and its surroundings are larger than average. Second, LL observations, significant clusters of low values surrounded by low values, are sometimes referred to as "cold spots". Negative forms of local spatial autocorrelation also include two cases. When the focal observation displays low values but its surroundings have high values (LH), we call them "doughnuts". Conversely, areas with high values but neighbored by others with low values (HL) can be referred to as "diamonds in the rough". We note this terminology is purely mnemonic, but recognize in some cases it can help in remembering the interpretation of local statistics.

After building these new columns, analysis on the overall trends of LISA statistics is more straightforward than from the `lisa` object. For example, an overview of the distribution of labels is one line away:

```
db["labels"].value_counts()
```

```
Non-Significant    227
HH                  74
LL                  69
LH                   6
HL                   4
Name: labels, dtype: int64
```

This shows, for one, that most local statistics are *not* statistically significant. Among those that are, we see many more hotspots/coldspots than doughnuts/diamonds-in-the-rough. This is consistent with the skew we saw in the distribution of local statistics earlier.

7.4 Getis and Ord's local statistics

Similar to the global case, there are more local indicators of spatial correlation than the local Moran's I. `esda` includes Getis and Ord's G_i-type statistics. These are a different kind of local statistic that are commonly used in two forms: the G_i statistic, which omits the value at a site in its local summary, and the G_i^*, which includes the site's own value in the local summary. The way to calculate them also follows similar patterns as with the Local Moran's I_i statistics above. Let us see how that would look like for our Brexit example:

```
# Gi
go_i = esda.getisord.G_Local(db["Pct_Leave"], w)
# Gi*
go_i_star = esda.getisord.G_Local(db["Pct_Leave"], w,␣
↪star=True)
```

Like all local statistics, it is best to explore Getis and Ord statistics by plotting them on a map. Unlike with LISA though, the G statistics only allow to identify positive spatial autocorrelation. When standardized, positive values imply clustering of high values, while negative implies grouping of low values. Unfortunately, it is not possible to discern spatial outliers.

Unlike with LISAs, `splot` does not support visualization of G statistics at this point. To visualize their output, we will instead write a little function that generates the map from the statistic's output object and its set of associated geometries.

```
def g_map(g, db, ax):
    """
    Create a cluster map
    ...

    Arguments
    ---------
    g       : G_Local
              Object from the computation of the G statistic
    db      : GeoDataFrame
              Table aligned with values in `g` and containing
              the geometries to plot
    ax      : AxesSubplot
              `matplotlib` axis to draw the map on

    Returns
    -------
    ax      : AxesSubplot
              Axis with the map drawn
    """
    ec = "0.8"

    # Break observations into significant or not
    sig = g.p_sim < 0.05

    # Plot non-significant clusters
    ns = db.loc[sig == False, "geometry"]
    ns.plot(ax=ax, color="lightgrey", edgecolor=ec,␣
↪linewidth=0.1)
    # Plot HH clusters
    hh = db.loc[(g.Zs > 0) & (sig == True), "geometry"]
```

(continued on next page)

(continued from previous page)

```
    hh.plot(ax=ax, color="red", edgecolor=ec, linewidth=0.1)
    # Plot LL clusters
    ll = db.loc[(g.Zs < 0) & (sig == True), "geometry"]
    ll.plot(ax=ax, color="blue", edgecolor=ec, linewidth=0.1)
    # Style and draw
    contextily.add_basemap(
        ax,
        crs=db.crs,
        source=contextily.providers.Stamen.TerrainBackground,
    )
    # Flag to add a star to the title if it's G_i*
    st = ""
    if g.star:
        st = "*"
    # Add title
    ax.set_title(f"G{st} statistic for Pct of Leave votes",␣
↪size=15)
    # Remove axis for aesthetics
    ax.set_axis_off()
    return ax
```

With this function at hand, generating $G_i^{(*)}$ cluster maps is as straightforward as it is for LISA outputs through `splot` in Figure 7.6:

```
# Set up figure and axes
f, axs = plt.subplots(1, 2, figsize=(12, 6))
# Loop over the two statistics
for g, ax in zip([go_i, go_i_star], axs.flatten()):
    # Generate the statistic's map
    ax = g_map(g, db, ax)
# Tight layout to minimise blank spaces
f.tight_layout()
# Render
plt.show()
```

In this case, the results are virtually the same for G_i and G_i^*. Also, at first glance, these maps appear to be visually similar to the final LISA map from above. Naturally, this leads to the question: *why use the G_i statistics at all?* The answer to this question is that the two sets of local statistics, local I and local G_i, are complementary statistics. The local I_i statistic (on its own) gives an indication of cluster/outlier status, and the local G_i shows which side of the hotspot/coldspot divide the observation is on. Alternatively, the local Moran's I_i cluster map provides both pieces of information, but it can be more challenging to visualize all at once. Thus, it depends on your analytical preferences and the point of the analysis at hand.

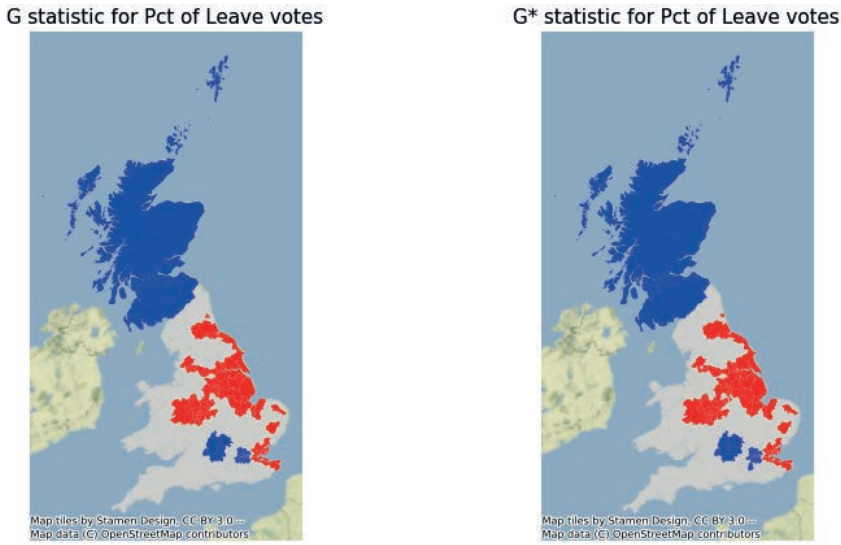


Fig. 7.6: Brexit Leave vote, Pct_Leave, Getis-Ord G (left) and G* (right) statistics.

7.5 Bonus: local statistics on surfaces

Before we wrap up the chapter, we are going to cover an illustration that, conceptually, is very similar to the topics we have seen above but, from a technical standpoint, has a bit of a different spin. We will learn how to compute local Moran's I_i on data that are stored as a surface, rather than as a geo-table (as we have seen above). As we have seen earlier in the book, more and more data for which we might want to explore local spatial autocorrelation are being provided as surfaces rather than geo-tables. The trick to follow this illustration is to realize that, despite the data structure, surfaces also provide data spatially arranged and that, as such, we can apply the battery of tools we have learned in this chapter to better understand their spatial structure.

Before we start, a note of caution. The functionality required to handle LISA on surfaces is still experimental and a bit rough around the edges. This is because, unlike the case of geo-tables, it has not been a common use-case for geographic data scientists and the tooling ecosystem is not as evolved. Nevertheless, it is an exciting time to get started on this, because a lot is happening in this space, and the basic building blocks to develop a full-fledged ecosystem are already in place. For this reason, we think it is important to cover in this chapter, even though some of the code we will use below is a bit more sophisticated than what we have seen above. Be patient and do not worry if you have to read things twice (or thrice!) before they start making sense. This is getting into *geographic data scientist pro* territory!

For this case, we will use the GHSL dataset that contains an extract of gridded population for the metropolitan region of Sao Paulo (Brazil). Let us read the data first into a `DataArray` object:

```
# Open GeoTIFF file and read into `xarray.DataArray`
pop = xarray.open_rasterio("../data/ghsl/ghsl_sao_paulo.tif")
```

Next is building a weights matrix that represents the spatial configuration of pixels with values in `pop`. We will use the same approach as we saw in the chapter on weights:

```
w_surface_sp = weights.Queen.from_xarray(pop)
```

So far, so good. Now comes the first hairy bit. The weights builder for surfaces automatically generates a matrix with integers (`int8` in this case which, roughly speaking, are numbers without a decimal component):

```
w_surface_sp.sparse.dtype
```

```
dtype('int8')
```

For the LISA computation, we will need two changes in `w_surface_sp`. First, the matrix needs to be expressed as floats (roughly speaking, numbers with a decimal component) so we can multiply values and obtain the correct result. Second, we need a `W` object and, so far, we have a `WSP`:

```
type(w_surface_sp)
```

```
libpysal.weights.weights.WSP
```

`WSP` objects are a thin version of spatial weights matrices that are optimised for certain computations and are more lightweight in terms of memory requirements (they are great, for example, for spatial econometrics). Unfortunately, to calculate LISA statistics we require a few more bits of information, so we have to convert it into a `W` object.

We take both steps in the following code snippet:

```
w_surface = weights.WSP2W(  # 3.Convert `WSP` object to `W`
    weights.WSP(  # 2.Build `WSP` from the float sparse matrix
        w_surface_sp.sparse.astype(
            float
        )  # 1.Convert sparse matrix to floats
    )
)
w_surface.index = w_surface_sp.index  # 4.Assign index to new␣
↪`W`
```

There is quite a bit going on in those lines of code, so let's unpack them:

1. The first step (line 3) is to convert the values from integers into floats. To do this, we access the sparse matrix at the core of `w_surface_sp` (which holds all the main data) and convert it to floats using `astype`.
2. Then we convert that sparse matrix into a `WSP` object (line 2), which is a thin wrapper, so the operation is quick.
3. Once represented as a `WSP`, we can use Pysal again to convert it into a full-fledged `W` object using the `WSP2W` utility. This step may take a bit more of computing muscle.
4. Finally, spatial weights from surfaces include an `index` object that will help us later return data into a surface data structure. Since this is lost with the transformations, we reattach it in the final line (line 6) from the original object.

This leaves us with a weights object (`w_surface`) we can work with for the LISA. Next is to recast the values from the original data structure to one that `Moran_Local` will understand. This happens in the next code snippet:

```
# Convert `DataArray` to a `pandas.Series`
pop_values = pop.to_series()
# Subset to keep only values that aren't missing
pop_values = pop_values[pop_values != pop.rio.nodata]
```

Note that we do two operations: one is to convert the two-dimensional `DataArray` surface into a one-dimensional vector in the form of a `Series` object (`pop_values`); the second is to filter out values in which, in the surface, contain missing data. In surfaces, this is usually expressed with a rare value rather than with another data type. We can check that in `pop`, this is negative:

```
pop.rio.nodata
```

```
-200.0
```

At this point, we are ready to run a LISA the same way we have done in the previous chapter when using geo-tables:

```
# NOTE: this may take a bit longer to run depending on␣
↪hardware
pop_lisa = esda.moran.Moran_Local(
    pop_values.astype(float), w_surface, n_jobs=-1
)
```

Note that, before computing the LISA, we ensure the population values are *also* expressed as floats and thus in line with those in our spatial weights.

Now that we have computed the LISA, on to visualization. For this, we need to express the results as a surface rather than as a table, for which we will use the bridge built in `pysal`:

```
from libpysal.weights import raster
```

We are aiming to create a cluster plot. This means we want to display values that are statistically significant in a color aligned with the quadrant of the Moran plot in which they lie. For this, we will create a new `Series` that intersects the quadrant information with significance. We use a 1% level for the example:

```
sig_pop = pandas.Series(
    pop_lisa.q
    * (
        pop_lisa.p_sim < 0.01
    ),  # Quadrant of significant at 1% (0 otherwise)
    index=pop_values.index,  # Index from the Series and␣
↪aligned with `w_surface`
)
```

The `sig_pop` object, expressed as a one-dimensional vector, contains the information we would like to recast into a `DataArray` object. For this conversion, we can use the `w2da` function, which derives the spatial configuration of each value in `sig_pop` from `w_surface`:

```
# Build `DataArray` from a set of values and weights
lisa_da = raster.w2da(
    sig_pop,  # Values
    w_surface,  # Spatial weights
    attrs={
        "nodatavals": pop.attrs["nodatavals"]
    }  # Value for missing data
    # Add CRS information in a compliant manner
).rio.write_crs(pop.rio.crs)
```

The resulting `DataArray` only contains missing data pixels (expressed with the same value as the original `pop` feature), `0` for non-significant pixels, and 1-4 depending on the quadrant for HH, LH, LL, HL significant clusters, same as with the Brexit example before:

```
lisa_da.to_series().unique()
```

```
array([-200,    0,    3,    1,    4,    2])
```

We have all the data in the right shape to build the figure. Before we can do that, we need to hardwire the coloring scheme on our own. This is something that we do not

Fig. 7.7: Colormap for Local Moran's I Maps, starting with non-significant local scores in grey,and proceeding through high-high local statistics, low-high, low-low, then high-low.

have to pay attention to when working with geo-tables thanks to `splot`. For surfaces, we are not that lucky.

First, we create the colormap to encode clusters with the same colors that `splot` uses for geo-tables. For that, we need the method in `matplotlib` that builds a color map from a list of colors:

```
from matplotlib.colors import ListedColormap
```

We express the colors we will use as a dictionary mapping the key to the color code:

```
# LISA colors
lc = {
    "ns": "lightgrey",   # Values of 0
    "HH": "#d7191c",   # Values of 1
    "LH": "#abd9e9",   # Values of 2
    "LL": "#2c7bb6",   # Values of 3
    "HL": "#fdae61",   # Values of 4
}
```

With these pieces, we can create the colormap object (shown in Figure 7.7) that replicates our original local Moran cluster map colors.

```
lisa_cmap = ListedColormap(
    [lc["ns"], lc["HH"], lc["LH"], lc["LL"], lc["HL"]]
)
lisa_cmap
```

At this point, we have all the pieces we need to build our cluster map. Let's put them together to build Figure 7.8:

```
# Set up figure and axis
f, axs = plt.subplots(1, 2, figsize=(12, 6))
# Subplot 1 #
# Select pixels that do not have the `nodata` value
# (ie. they are not missing data)
pop.where(
    pop
```

(continued on next page)

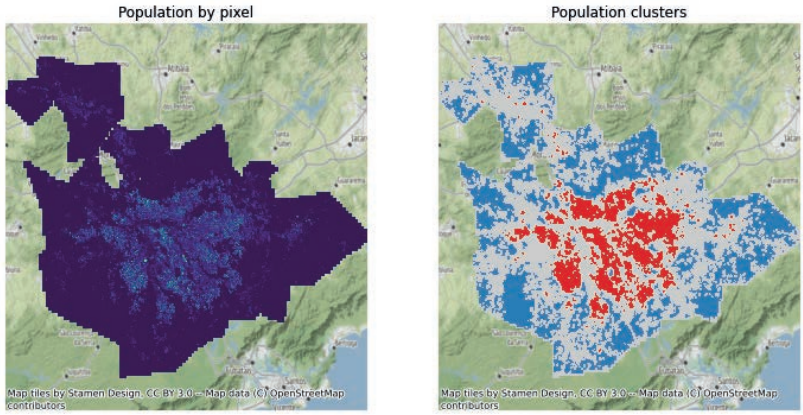


Fig. 7.8: LISA map for Sao Paulo population surface.

(continued from previous page)

```
    != pop.rio.nodata
    # Plot surface with a horizontal colorbar
).plot(
    ax=axs[0],
    add_colorbar=False,  # , cbar_kwargs={"orientation":
↪"horizontal"}
)
# Subplot 2 #
# Select pixels with no missing data and rescale to [0, 1] by
# dividing by 4 (maximum value in `lisa_da`)
(
    lisa_da.where(lisa_da != -200)
    / 4
    # Plot surface without a colorbar
).plot(cmap=lisa_cmap, ax=axs[1], add_colorbar=False)
# Aesthetics #
# Subplot titles
titles = ["Population by pixel", "Population clusters"]
# Apply the following to each of the two subplots
for i in range(2):
    # Keep proportion of axes
    axs[i].axis("equal")
    # Remove axis
    axs[i].set_axis_off()
    # Add title
    axs[i].set_title(titles[i])
    # Add basemap
    contextily.add_basemap(axs[i], crs=lisa_da.rio.crs)
```

7.6 Conclusion

Local statistics are one of the most commonly-used tools in the geographic data science toolkit. When used properly, local statistics provide a powerful way to analyze and visualize the structure of geographic data. The Local Moran's I_i statistic, as Local Indicator of Spatial Association, summarizes the co-variation between observations and their immediate surroundings. The Getis-Ord local G statistics, on the other hand, compare the sum of values in the area around each site. Regardless, learning to use local statistics effectively is important for any geographic data scientist, as they are the most common "first brush" geographic statistic for many analyses.

7.7 Questions

1. Do the same Local Moran analysis done for `Pct_Leave`, but using `Pct_Turnout`. Is there a geography to how involved people were in different places? Where was turnout percentage (relatively) higher or lower?
2. Do the same Getis-Ord analysis done for `Pct_Leave`, but using `Pct_Turnout`.
3. Local Moran statistics are premised on a few distributional assumptions. One well-recognized concern with Moran statistics is when they are estimated for *rates*. Rate data is distinct from other kinds of data because it embeds the relationship between two quantities: the event and the population. For instance, in the case of Leave voting, the "event" is a person voting leave, and the "population" could be the number of eligible voters, the number of votes cast, or the total number of people. This usually only poses a problem for analysis when the event outcome is somehow dependent on the population.
    - Using our past analytical steps, build a new `db` dataframe from `ref` and `lads` that contains the `Electorate`, `Votes_Cast`, and `Leave` columns.
    - From this new dataframe, make scatterplots of:
        - the number of votes cast and the percent leave vote
        - the size of the electorate and the percent of leave vote
    - Based on your answers to the previous point, does it appear that there is a relationship between the event and the population size? Use `scipy.stats.kendalltau` or `scipy.stats.pearsonr` to confirm your visual intuition.

- Using `esda.moran.Moran_Rate`, estimate a global Moran's I that takes into account the rate structure of `Pct_Leave`, using the `Electorate` as the population. Is this estimate different from the one obtained without taking into account the rate structure? What about when `Votes_Cast` is used for the population?
- Using `esda.moran.Moran_Local_Rate`, estimate *local* Moran's I treating Leave data as a rate.
  - does any site's local I change? Make a scatterplot of the `lisa.Is` you estimated before and this new rate-based local Moran.
  - does any site's local I change their outlier/statistical significance classifications? Use `pandas.crosstab` to examine how many classifications change between the two kinds of statistic. Make sure to consider observations' statistical significances in addition to their quadrant classification.
- Make two maps, side-by-side, of the local statistics without rate correction and with rate correction. Does your interpretation of the maps change depending on the correction?

4. Local statistics use *permutation-based* inference for their significance testing. This means that, to test the statistical significance of a local relationship, values of the observed variable are *shuffled* around the map. These large numbers of *random* maps are then used to compare against the observed map. Local inference requires some restrictions on how each shuffle occurs, since each observation must be "fixed" and compared to randomly-shuffled neighboring observations. The distribution of local statistics for each "shuffle" is contained in the `.rlisas` attribute of a Local Moran object.
   - For the first observation, make a `seaborn.distplot` of its shuffled local statistics. Add a vertical line to the histogram using `plt.axvline()`.
   - Do the same for the last observation as well.
   - Looking only at their permutation distributions, do you expect the first LISA statistic to be statistically-significant? Do you expect the last?
5. LISAs have some amount of fundamental uncertainty due to their estimation. This is called the `standard error` of the statistic.
   - The standard errors are contained in the `.seI_sim` attribute. Make a map of the standard errors. Are there any areas of the map that appear to be more uncertain about their local statistics?
   - compute the standard deviation of each observation's "shuffle" distribution, contained in the `.rlisas` attribute. Verify that the standard

deviation of this shuffle distribution is the same as the standard errors in `seI_sim`.

6. Local Getis-Ord statistics come in two forms. As discussed above, Getis-Ord G_i statistics *omit* each site from their own local statistic. In contrast, G_i^* statistics *include* the site in its own local statistic.

   - make a scatterplot of the two types of statistic, contained in `gostats.Zs` and `gostars.Zs` to examine how similar the two forms of the Getis-Ord statistic are.

   - the two forms of the Getis-Ord statistic differ by their inclusion of the *site* value, y_i, in the value for the G_i statistic at that site. So, make a scatterplot of the percent leave variable and the *difference* of the two statistics. Is there a relationship between the percent leave vote and the difference in the two forms of the Getis-Ord statistic? Confirm this for yourself using `scipy.stats.kendalltau` or `scipy.stats.pearsonr`.

7.8 Next Steps

For more thinking on the foundational methods and concepts in local testing, Fotheringham is a classic:

Fotheringham, A. Stewart. 1997. "Trends in Quantitative Methods I: Stressing the local." *Progress in Human Geography* 21(1): 88-96.

More recent discussion on local statistics (in the context of spatial statistics more generally) is provided by Nelson:

Nelson, Trisalyn. "Trends in Spatial Statistics." *The Professional Geographer* 64(1): 83-94.

8

Point Pattern Analysis

Points are spatial entities that can be understood in two fundamentally different ways. On the one hand, points can be seen as fixed objects in space, which is to say their location is taken as given (*exogenous*). In this interpretation, the location of an observed point is considered as secondary to the value observed at the point. Think of this like measuring the number of cars traversing a given road intersection; the location is fixed, and the data of interest comes from the measurement taken at that location. The analysis of this kind of point data is very similar to that of other types of spatial data such as polygons and lines. On the other hand, an observation occurring at a point can also be thought of as a site of measurement from an underlying geographically-continuous process. In this case, the measurement could theoretically take place anywhere, but was only carried out or conducted in certain locations. Think of this as measuring the length of birds' wings: the location at which birds are measured reflects the underlying geographical process of bird movement and foraging, and the length of the birds' wings may reflect an underlying ecological process that varies by bird. This kind of approach means that both the location and the measurement matter. This is the perspective we will adopt in the rest of the chapter.

When points are seen as events that could take place in several locations but only happen in a few of them, a collection of such events is called a *point pattern*. In this case, the location of points is one of the key aspects of interest for analysis. A good example of a point pattern is geo-tagged photographs: they could technically happen in many locations, but we usually find photos tend to concentrate only in a handful of them. Point patterns can be *marked*, if more attributes are provided with the location, or *unmarked*, if only the coordinates of where the event occurred are provided. Continuing the photo example, an unmarked pattern would result if only the location where the photos are taken is used for analysis; while we would be speaking of a marked point pattern if other attributes, such as the time, camera model, or a "image quality score" was provided with the location.

8.1 Introduction

Point pattern analysis is thus concerned with the visualization, description, statistical characterization, and modeling of point patterns, trying to understand the generating process that gives rise and explains the observed data. Common questions in this domain include:

- *What does the pattern look like?*
- *What is the nature of the distribution of points?*
- *Is there any structure in the way locations are arranged over space? That is, are events clustered? or are they dispersed?*
- *Why do events occur in those places and not in others?*

At this point, it is useful to remind ourselves of an important distinction between process and pattern. The former relates to the underlying mechanism that is at work to generate the outcome we end up observing. Because of its abstract nature, we do not get to see it. However, in many contexts, the key focus of any analysis is to learn about *what* determines a given phenomenon and *how* those factors combine to generate it. In this context, "process" is associated with the *how*. "Pattern," on the other hand, relates to the result of that process. In some cases, it is the only trace of the process we can observe and thus the only input we have to work with in order to reconstruct it. Although directly observable and, arguably, easier to tackle, pattern is only a reflection of process. The real challenge is not to characterize the former but to use it to work out the latter.

In this chapter, we provide an introduction to point patterns through geo-tagged Flickr photos from Tokyo. We will treat the phenomena represented in the data as events: photos could be taken of any place in Tokyo, but only certain locations are captured. Keep in mind this understanding of Tokyo photos is not immutable: one could conceive cases where it makes sense to take those locations as given and look at the properties of each of them ignoring their "event" aspect. However, in this context, we will focus on those questions that relate to location and the collective shape of locations. The use of these tools will allow us to transform a long list of unintelligible XY coordinates into tangible phenomena with a characteristic spatial structure, and to answer questions about the center, dispersion, and clustering of attractions in Tokyo for Flickr users.

8.2 Patterns in Tokyo photographs

The rise of new forms of data such as geo-tagged photos uploaded to online services is creating new ways for researchers to study and understand cities. Where do people take pictures? When are those pictures taken? Why do certain places attract many more photographers than others? All these questions and more become more than just rhetorical ones when we consider, for example, online photo hosting services as volunteered geographic information (VGI, [Goo07]). In this chapter we will explore metadata from a

sample of geo-referenced images uploaded to Flickr and extracted thanks to the 100m Flickr dataset. In doing so, we will introduce a few approaches that help us better understand the distribution and characteristics of a point pattern.

To get started, let's load the packages we will need in this example.

```
import numpy
import pandas
import geopandas
import pysal
import seaborn
import contextily
import matplotlib.pyplot as plt
from sklearn.cluster import DBSCAN
```

Then, let us load some data about picture locations from Flickr:

```
db = pandas.read_csv("../data/tokyo/tokyo_clean.csv")
```

The table contains the following information about the sample of 10,000 photographs: the ID of the user who took the photo, the location expressed as latitude and longitude columns, a transformed version of those coordinates expressed in Pseudo Mercator, the timestamp when the photo was taken, and the URL where the picture they refer to is stored online:

```
db.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 7 columns):
 #   Column                Non-Null Count  Dtype
---  ------                --------------  -----
 0   user_id               10000 non-null  object
 1   longitude             10000 non-null  float64
 2   latitude              10000 non-null  float64
 3   date_taken            10000 non-null  object
 4   photo/video_page_url  10000 non-null  object
 5   x                     10000 non-null  float64
 6   y                     10000 non-null  float64
dtypes: float64(4), object(3)
memory usage: 547.0+ KB
```

Note that the data is provided as a `.csv` file, so the spatial information is encoded as separate columns, one for each coordinate. This is in contrast to how we have consumed spatial data in previous chapters, where spatial information was stored in a single column and encoded in geometry objects.

8.3 Visualizing point patterns

There are many ways to visualize geographic point patterns, and the choice of method depends on the intended message.

8.3.1 Showing patterns as dots on a map

The first step to get a sense of what the spatial dimension of this dataset looks like is to plot it. At its most basic level, we can generate a scatterplot with `seaborn` in Figure 8.1:

```
# Generate scatterplot
seaborn.jointplot(x="longitude", y="latitude", data=db, s=0.
↪5);
```

This is a good start: we can see dots tend to be concentrated in the center of the covered area in a non-random pattern. Furthermore, within the broad pattern, we can also see there seems to be more localized clusters. However, the plot above has two key drawbacks: one, it lacks geographical context; and two, there are areas where the density of points is so large that it is hard to tell anything beyond a blue blurb.

Start with the context. The easiest way to provide additional context is by overlaying a tile map from the internet. Let us quickly call `contextily` for that, and integrate it with `jointplot` to create Figure 8.2:

```
# Generate scatterplot
joint_axes = seaborn.jointplot(
    x="longitude", y="latitude", data=db, s=0.5
)
contextily.add_basemap(
    joint_axes.ax_joint,
    crs="EPSG:4326",
    source=contextily.providers.CartoDB.PositronNoLabels,
);
```

Note how we can pull out the axis where the points are plotted and add the basemap there, specifying the CRS as WGS84, since we are plotting longitude and latitude. Compared to the previous plot, adding a basemap to our initial plot makes the pattern of Flickr data clearer.

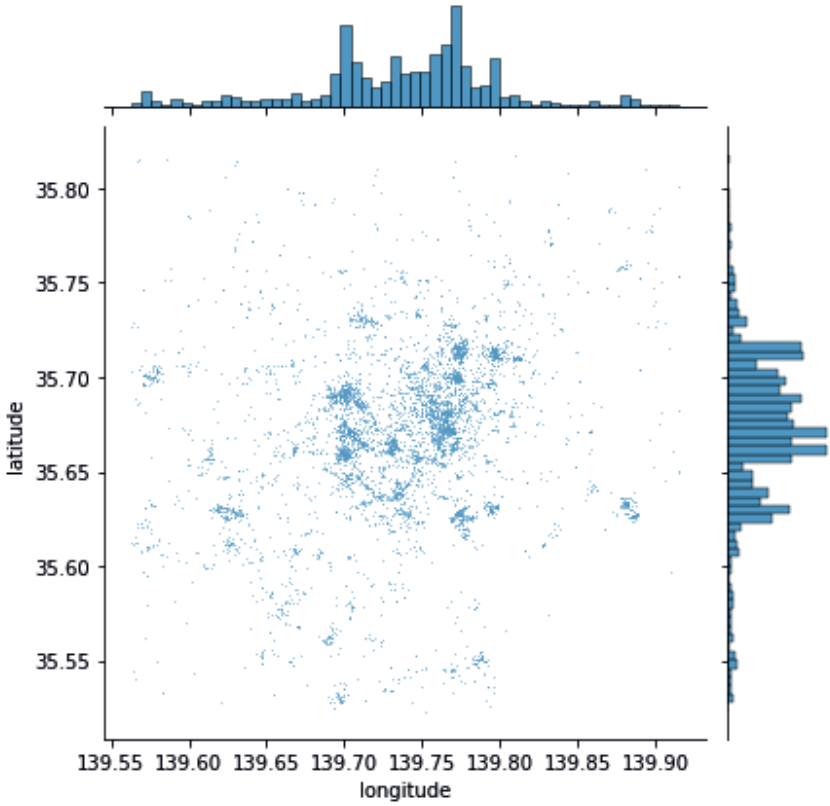


Fig. 8.1: Tokyo photographs jointplot showing the longitude and latitude where photographs were taken.

8.3.2 Showing density with hexbinning

Consider our second problem: cluttering. When too many photos are concentrated in some areas, plotting opaque dots on top of one another can make it hard to discern any pattern and explore its nature. For example, in the middle of the map in Figure 8.2, toward the right, there appears to be the highest concentration of pictures taken; the sheer amount of dots on the maps in some parts obscures whether all of that area receives as many pics or whether, within there, some places receive a particularly high degree of attention.

One solution to get around cluttering relates to what we referred to earlier as moving from "tables to surfaces". We can now recast this approach as a *spatial* or *two-dimensional histogram*. Here, we generate a regular grid (either squared or hexagonal), count how many dots fall within each grid cell, and present it as we would any other choropleth. This is attractive because it is simple, intuitive and, if fine enough, the

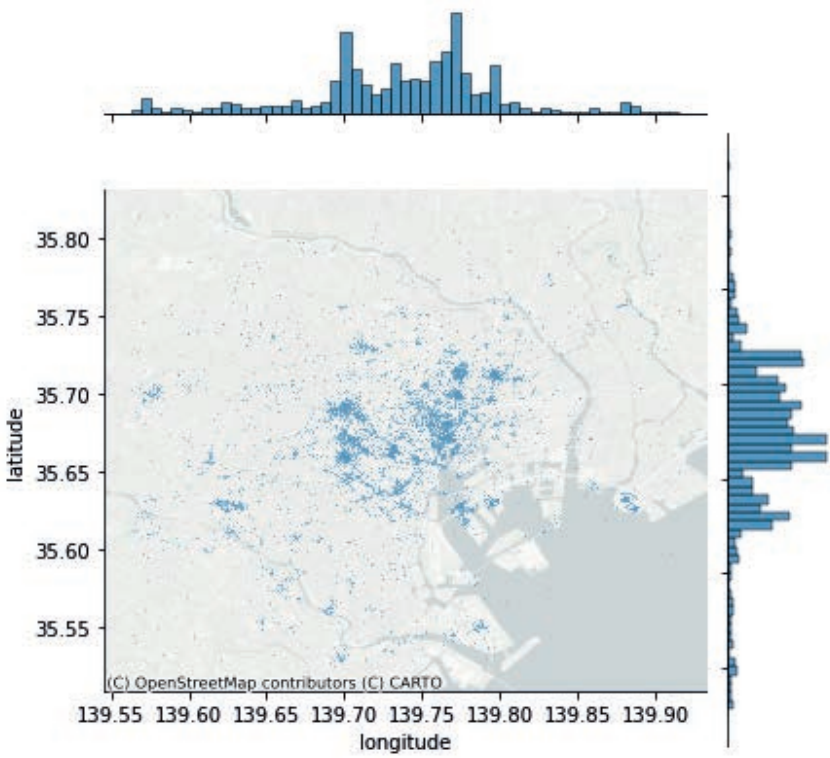


Fig. 8.2: Tokyo jointplot showing longitude and latitude of photographs with a basemap via contextily.

regular grid removes some of the area distortions choropleth maps may induce. For this illustration, let us use use hexagonal binning (sometimes called hexbin) because it has slightly nicer properties than squared grids, such as less shape distortion and more regular connectivity between cells. Creating a hexbin two-dimensional histogram is straightforward in Python using the `hexbin` function to create Figure 8.3:

```
# Set up figure and axis
f, ax = plt.subplots(1, figsize=(12, 9))
# Generate and add hexbin with 50 hexagons in each
# dimension, no borderlines, half transparency,
# and the reverse viridis colormap
hb = ax.hexbin(
    db["x"],
    db["y"],
    gridsize=50,
    linewidths=0,
    alpha=0.5,
```

(continued on next page)

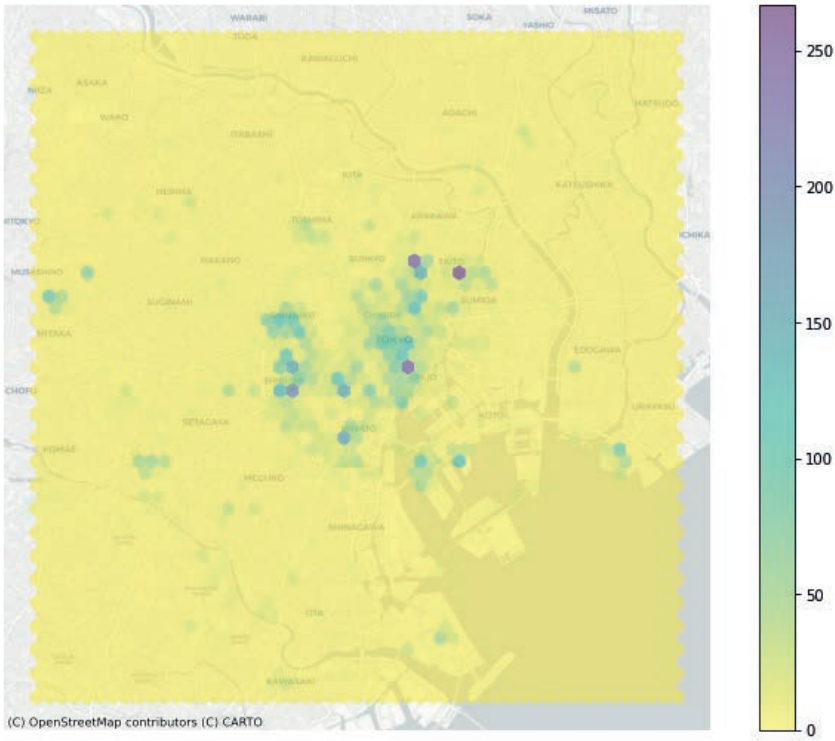


Fig. 8.3: Tokyo photographs two-dimensional histogram built with hexbinning.

(continued from previous page)

```
    cmap="viridis_r",
)
# Add basemap
contextily.add_basemap(
    ax, source=contextily.providers.CartoDB.Positron
)
# Add colorbar
plt.colorbar(hb)
# Remove axes
ax.set_axis_off()
```

Voila, this allows a lot more detail! It is now clear that the majority of photographs relate to much more localized areas, and that the previous map was obscuring this.

8.3.3 Another kind of density: kernel density estimation

Grids are the spatial equivalent of a histogram: the user decides how many "buckets", and the points are counted within them in a discrete fashion. This is fast, efficient, and potentially very detailed (if many bins are created). However, it does represent a discretization of an essentially contiguous phenomenon and, as such, it may introduce distortions (e.g., the modifiable areal unit problem [Won04]). An alternative approach is to instead create what is known as a kernel density estimation (KDE): an empirical approximation of the probability density function. This approach is covered in detail elsewhere (e.g., [Sil86]), but we can provide the intuition here. Instead of overlaying a grid of squares of hexagons and count how many points fall within each, a KDE lays a grid of points over the space of interest on which it places kernel functions that count points around them with a different weight based on the distance. These counts are then aggregated to generate a global surface with probability. The most common kernel function is the Gaussian one, which applies a normal distribution to weight points. The result is a continuous surface with a probability function that may be evaluated at every point. Creating a Gaussian kernel map in Python is rather straightforward, using the `seaborn.kdeplot()` function to create Figure 8.4:

```
# Set up figure and axis
f, ax = plt.subplots(1, figsize=(9, 9))
# Generate and add KDE with a shading of 50 gradients
# coloured contours, 75% of transparency,
# and the reverse viridis colormap
seaborn.kdeplot(
    x="x",
    y="y",
    data=db,
    n_levels=50,
    shade=True,
    alpha=0.55,
    cmap="viridis_r",
)
# Add basemap
contextily.add_basemap(
    ax, source=contextily.providers.CartoDB.Positron
)
# Remove axes
ax.set_axis_off()
```

The result is a smoother output that captures the same structure of the hexbin but "eases" the transitions between different areas. This provides a better generalization of the theoretical probability distribution over space. Technically, the continuous nature of the KDE function implies that for any given point the probability of an event is 0. However, as the area around a point increases, the probability of an event within

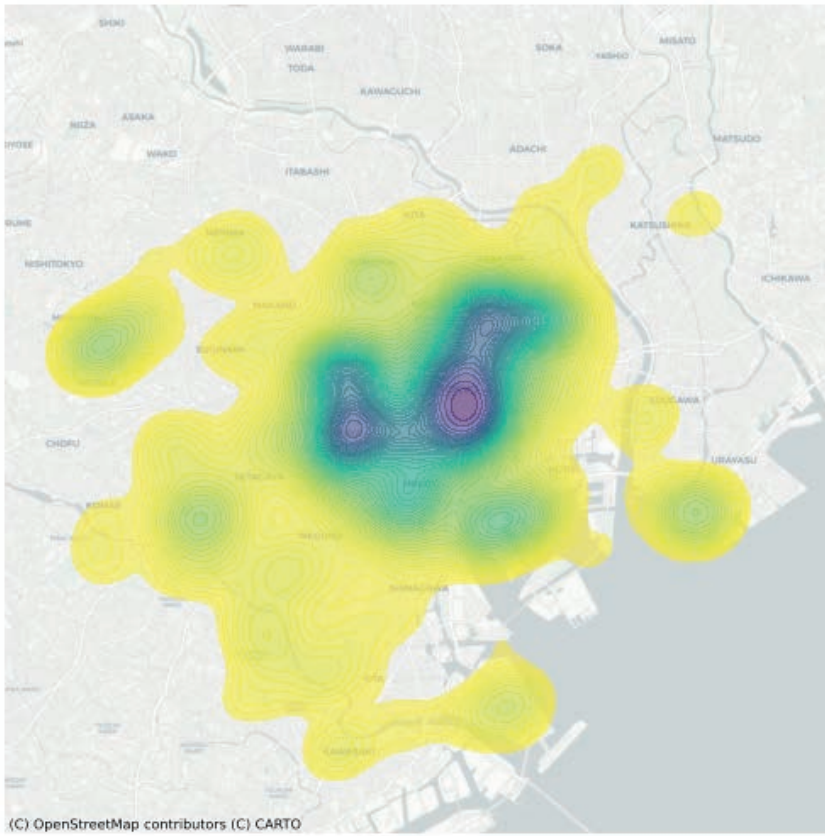


Fig. 8.4: Tokyo photographs kernel density map.

that area can be obtained. This is useful in some cases, but it is mainly of use to escape the restrictions imposed by a regular grid of hexagons or squares.

8.4 Centrography

Centrography is the analysis of centrality in a point pattern. By "centrality," we mean the general location and dispersion of the pattern. If the hexbin above can be seen as a "spatial histogram", centrography is the point pattern equivalent of measures of central tendency such as the mean. These measures are useful because they allow us to summarize spatial distributions in smaller sets of information (e.g., a single point). Many different indices are used in centrography to provide an indication of "where" a point pattern is, how tightly the point pattern clusters around its center, or how irregular its shape is.

8.4.1 Tendency

A common measure of central tendency for a point pattern is its *center of mass*. For marked point patterns, the center of mass identifies a central point close to observations that have higher values in their marked attribute. For unmarked point patterns, the center of mass is equivalent to the *mean center*, or average of the coordinate values. In addition, the *median center* is analogous to the *median* elsewhere, and represents a point where half of the data is above or below the point and half is to its left or right. We can analyze the mean center with our Flickr point pattern using the `pointpats` package in Python.

```
from pointpats import centrography
```

```
mean_center = centrography.mean_center(db[["x", "y"]])
med_center = centrography.euclidean_median(db[["x", "y"]])
```

It is easiest to visualize this by plotting the point pattern and its mean center alongside one another, as done to create Figure 8.5:

```
# Generate scatterplot
joint_axes = seaborn.jointplot(
    x="x", y="y", data=db, s=0.75, height=9
)
# Add mean point and marginal lines
joint_axes.ax_joint.scatter(
    *mean_center, color="red", marker="x", s=50, label="Mean␣
↪Center"
)
joint_axes.ax_marg_x.axvline(mean_center[0], color="red")
joint_axes.ax_marg_y.axhline(mean_center[1], color="red")
# Add median point and marginal lines
joint_axes.ax_joint.scatter(
    *med_center,
    color="limegreen",
    marker="o",
    s=50,
    label="Median Center"
)
joint_axes.ax_marg_x.axvline(med_center[0], color="limegreen")
joint_axes.ax_marg_y.axhline(med_center[1], color="limegreen")
# Legend
joint_axes.ax_joint.legend()
# Add basemap
contextily.add_basemap(
    joint_axes.ax_joint, source=contextily.providers.CartoDB.
↪Positron
)
```

(continued on next page)

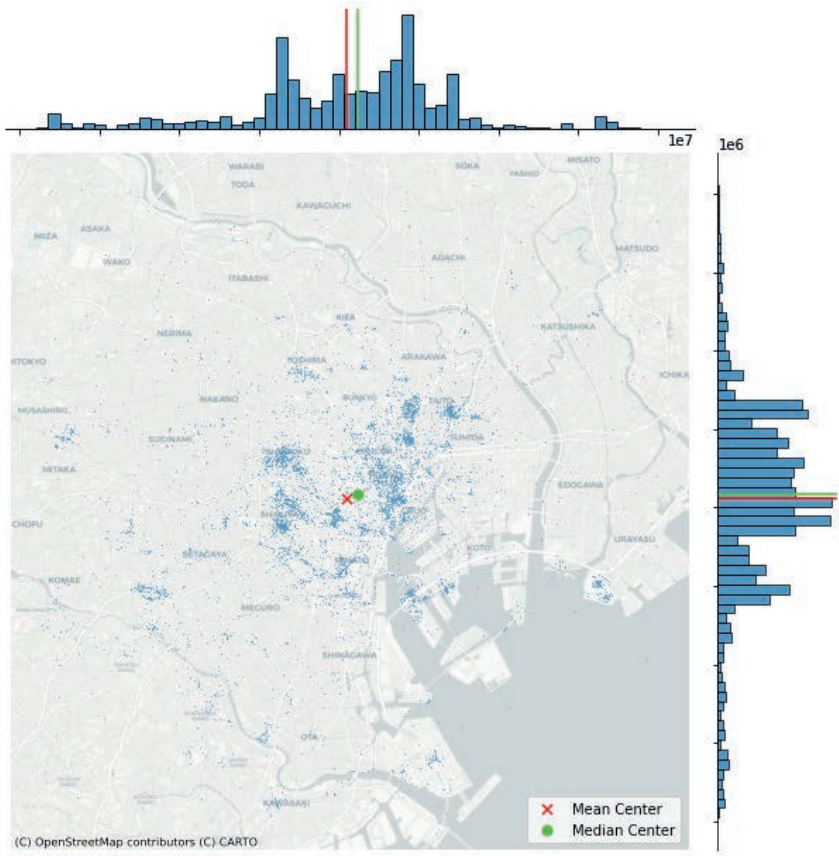


Fig. 8.5: Tokyo photographs mean and median centers.

(continued from previous page)

```
# Clean axes
joint_axes.ax_joint.set_axis_off()
# Display
plt.show()
```

The discrepancy between the two centers is caused by the skew; there are many "clusters" of pictures far out in West and South Tokyo, whereas North and East Tokyo is densely packed, but drops off very quickly. Thus, the far out clusters of pictures pulls the mean center to the west and south, relative to the median center.

8.4.2 Dispersion

A measure of dispersion that is common in centrography is the *standard distance*. This measure provides the average distance away from the center of the point cloud (such as measured by the center of mass). This is also simple to compute using `pointpats`, using the `std_distance` function:

```
centrography.std_distance(db[["x", "y"]])
```

```
8778.218564382098
```

This means that, on average, pictures are taken around 8800 meters away from the mean center.

Another helpful visualization is the *standard deviational ellipse*, or *standard ellipse*. This is an ellipse drawn from the data that reflects its center, dispersion, and orientation. To visualize this, we first compute the axes and rotation using the `ellipse` function in `pointpats`:

```
major, minor, rotation = centrography.ellipse(db[["x", "y"]])
```

Then, we will visualize this in Figure 8.6:

```
from matplotlib.patches import Ellipse

# Set up figure and axis
f, ax = plt.subplots(1, figsize=(9, 9))
# Plot photograph points
ax.scatter(db["x"], db["y"], s=0.75)
ax.scatter(*mean_center, color="red", marker="x", label="Mean␣
↪Center")
ax.scatter(
    *med_center, color="limegreen", marker="o", label="Median␣
↪Center"
)

# Construct the standard ellipse using matplotlib
ellipse = Ellipse(
    xy=mean_center,  # center the ellipse on our mean center
    width=major * 2,  # centrography.ellipse only gives half␣
↪the axis
    height=minor * 2,
    angle=numpy.rad2deg(
        rotation
    ),  # Angles for this are in degrees, not radians
    facecolor="none",
    edgecolor="red",
```

(continued on next page)

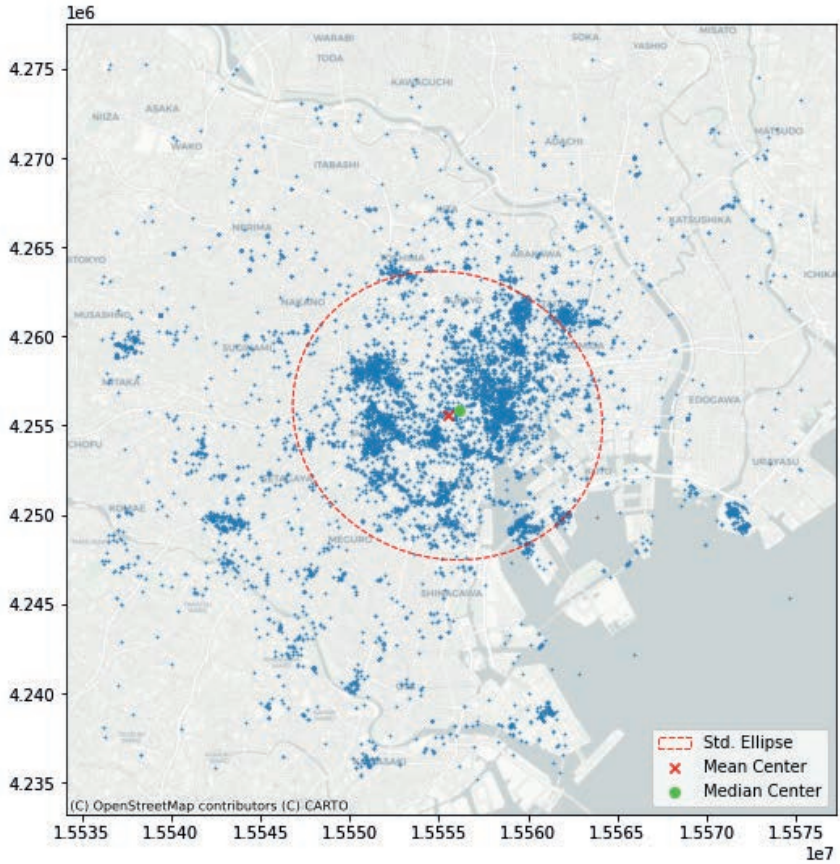


Fig. 8.6: Tokyo photographs standard deviational ellipse.

(continued from previous page)

```
    linestyle="--",
    label="Std. Ellipse",
)
ax.add_patch(ellipse)

ax.legend()
# Display
# Add basemap
contextily.add_basemap(
    ax, source=contextily.providers.CartoDB.Positron
)
plt.show()
```

8.4.3 Extent

The last collection of centrography measures we will discuss characterizes the extent of a point cloud. Four shapes are useful, and they reflect varying levels of how "tightly" they bind the pattern (Figure 8.7).

Below, we'll walk through how to construct each example and visualize all of them together at the end. To make things more clear, we'll use the Flickr photos for the most prolific user in the dataset (ID: `95795770`) to show how different these results can be.

```
user = db.query('user_id == "95795770@N00"')
coordinates = user[["x", "y"]].values
```

First, we'll compute the **convex hull**, which is the tightest *convex* shape that encloses the user's photos. By *convex*, we mean that the shape never "doubles back" on itself; it has no divets, valleys, crenulations, or holes. All of its interior angles are smaller than 180 degrees. This is computed using the `centrography.hull` method.

```
convex_hull_vertices = centrography.hull(coordinates)
```

Second, we'll compute the **alpha shape**, which can be understood as a "tighter" version of the convex hull. One way to think of a convex hull is that it's the space left over when rolling a **really** large ball or circle all the way around the shape. The ball is so large relative to the shape, its radius is actually infinite, and the lines forming the convex hull are actually just straight lines!

In contrast, you can think of an alpha shape as the space made from rolling a *small* ball around the shape. Since the ball is smaller, it rolls into the dips and valleys created between points. As that ball gets bigger, the alpha shape becomes the convex hull. But, for small balls, the shape can get very tight indeed. In fact, if alpha gets too small, it "slips" through the points, resulting in *more than one hull!* As such, the `libpysal` package has an `alpha_shape_auto` function to find the smallest *single* alpha shape, so that you don't have to guess at how big the ball needs to be.

```
import libpysal

alpha_shape, alpha, circs = libpysal.cg.alpha_shape_auto(
    coordinates, return_circles=True
)
```

```
f, ax = plt.subplots(1, 1, figsize=(9, 9))

# Plot a green alpha shape
geopandas.GeoSeries(
    [alpha_shape]
```

(continued on next page)

(continued from previous page)

```
).plot(
    ax=ax,
    edgecolor="green",
    facecolor="green",
    alpha=0.2,
    label="Tightest single alpha shape",
)

# Include the points for our prolific user in black
ax.scatter(
    *coordinates.T, color="k", marker=".", label="Source␣
↪Points"
)

# plot the circles forming the boundary of the alpha shape
for i, circle in enumerate(circs):
    # only label the first circle of its kind
    if i == 0:
        label = "Bounding Circles"
    else:
        label = None
    ax.add_patch(
        plt.Circle(
            circle,
            radius=alpha,
            facecolor="none",
            edgecolor="r",
            label=label,
        )
    )

# add a blue convex hull
ax.add_patch(
    plt.Polygon(
        convex_hull_vertices,
        closed=True,
        edgecolor="blue",
        facecolor="none",
        linestyle=":",
        linewidth=2,
        label="Convex Hull",
    )
)

# Add basemap
contextily.add_basemap(
    ax, source=contextily.providers.CartoDB.Positron
```

(continued on next page)

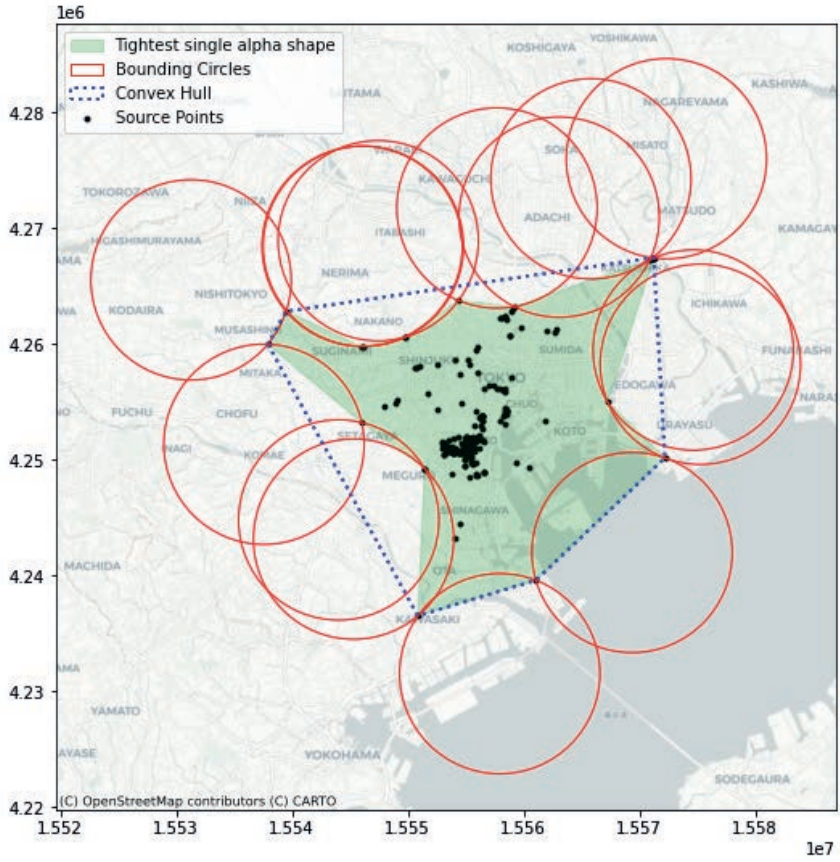


Fig. 8.7: Concave hull and (green) and convex hull (blue) for a subset of Tokyo photographs, with the bounding circles for the concave hull (red).

(continued from previous page)

```
)

plt.legend();
```

We will cover three more bounding shapes, all of them rectangles or circles. First, two kinds of **minimum bounding rectangles**. They both are constructed as the tightest *rectangle* that can be drawn around the data that contains all of the points. One kind of minimum bounding rectangle can be drawn just by considering vertical and horizontal lines. However, diagonal lines can often be drawn to construct a rectangle with a smaller area. This means that the **minimum rotated rectangle** provides a tighter rectangular bound on the point pattern, but the rectangle is askew or rotated.

For the minimum rotated rectangle, we will use the `minimum_rotated_rectangle` function from the `pygeos` module, which constructs the minimum rotated rectangle for an input *multi-point* object. This means that we will need to collect our points together into a single multi-point object and then compute the rotated rectangle for that object.

```
from pygeos import minimum_rotated_rectangle, from_shapely,␣
↪to_shapely

point_array = geopandas.points_from_xy(x=user.x, y=user.y)

min_rot_rect = minimum_rotated_rectangle(
    from_shapely(
        point_array.unary_union()
    )
)
min_rot_rect = to_shapely(min_rot_rect)
```

And, for the minimum bounding rectangle without rotation, we will use the `minimum_bounding_rectangle` function from the `pointpats` package.

```
min_rect_vertices = centrography.minimum_bounding_rectangle(
    coordinates
)
```

Finally, the **minimum bounding circle** is the smallest circle that can be drawn to enclose the entire dataset. Often, this circle is bigger than the minimum bounding rectangle. It is implemented in the `minimum_bounding_circle` function in `pointpats`.

```
(center_x, center_y), radius = centrography.minimum_bounding_
↪circle(
    coordinates
)
```

Now, to visualize these, we'll convert the raw vertices into `matplotlib` patches:

```
from matplotlib.patches import Polygon, Circle, Rectangle

# Make a blue convex hull
convex_hull_patch = Polygon(
    convex_hull_vertices,
    closed=True,
    edgecolor="blue",
    facecolor="none",
    linestyle=":",
    linewidth=2,
```

(continued on next page)

(continued from previous page)

```
    label="Convex Hull",
)

# compute the width and height of the mbr
min_rect_width = min_rect_vertices[2] - min_rect_vertices[0]
min_rect_height = min_rect_vertices[2] - min_rect_vertices[0]

# Make a goldenrod minimum bounding rectangle
min_rect_patch = Rectangle(
    min_rect_vertices[0:2],
    width=min_rect_width,
    height=min_rect_height,
    edgecolor="goldenrod",
    facecolor="none",
    linestyle="dashed",
    linewidth=2,
    label="Min Bounding Rectangle",
)

# and make a red minimum bounding circle
circ_patch = Circle(
    (center_x, center_y),
    radius=radius,
    edgecolor="red",
    facecolor="none",
    linewidth=2,
    label="Min Bounding Circle",
)
```

Finally, we'll plot the patches together with the photograph locations in Figure 8.8:

```
f, ax = plt.subplots(1, figsize=(10, 10))

# a purple alpha shape
geopandas.GeoSeries(
    [alpha_shape]
).plot(
    ax=ax,
    edgecolor="purple",
    facecolor="none",
    linewidth=2,
    label="Alpha Shape",
)

# a green minimum rotated rectangle
geopandas.GeoSeries(
    [min_rot_rect]
```

(continued on next page)

(continued from previous page)

```
).plot(
    ax=ax,
    edgecolor="green",
    facecolor="none",
    linestyle="--",
    label="Min Rotated Rectangle",
    linewidth=2,
)

# add the rest of the patches
ax.add_patch(convex_hull_patch)
ax.add_patch(min_rect_patch)
ax.add_patch(circ_patch)

ax.scatter(db.x, db.y, s=0.75, color="grey")
ax.scatter(user.x, user.y, s=100, color="r", marker="x")
ax.legend(ncol=1, loc="center left")

# Add basemap
contextily.add_basemap(
    ax, source=contextily.providers.CartoDB.Positron
)
plt.show()
```

Each gives a different impression of the area enclosing the user's range of photographs. In this, you can see that the alpha shape is much tighter than the rest of the shapes. The minimum bounding rectangle and circle are the "loosest" shapes, in that they contain the most area outside of the user's typical area. But, they're also the simplest shapes to draw and understand.

8.5 Randomness and clustering

Beyond questions of centrality and extent, spatial statistics on point patterns are often concerned with how *even* a distribution of points is. By this, we mean whether points tend to all cluster near one another or disperse evenly throughout the problem area. Questions like this refer to the *intensity* or *dispersion* of the point pattern overall. In the jargon of the last two chapters, this focus resembles the goals we examined when we introduced *global spatial autocorrelation*: what is the overall degree of *clustering* we observe in the pattern? Spatial statistics has devoted plenty of effort to understand this kind of clustering. This section will cover methods useful for identifying clustering in point patterns.

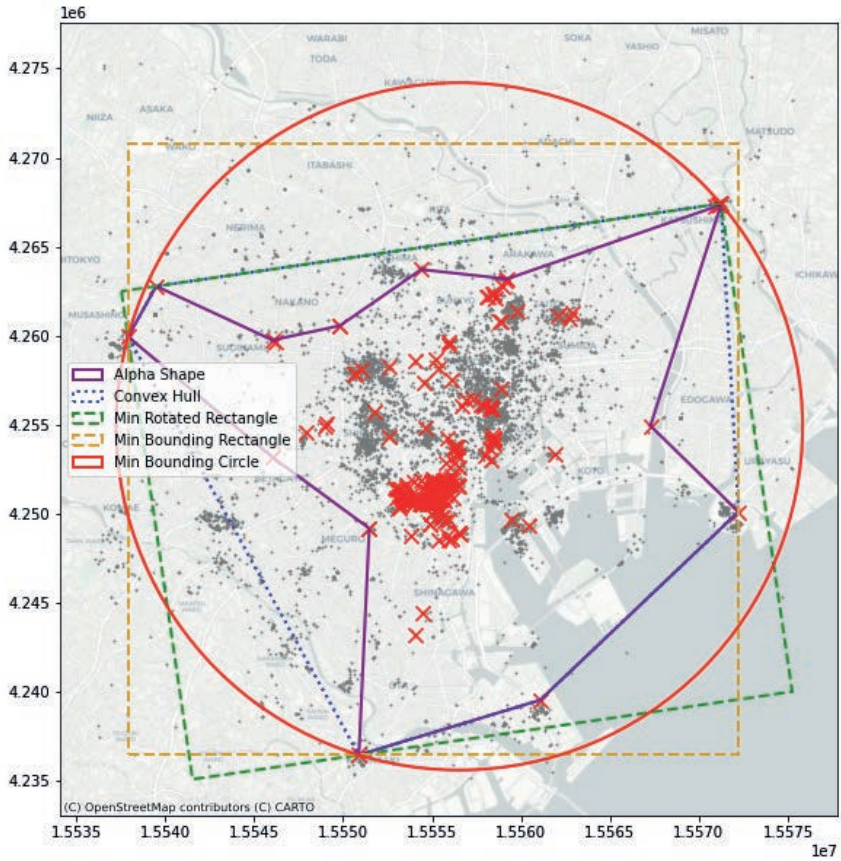


Fig. 8.8: Alpha shape/concave hull, convex hull, minimum rotated rectangle, minimum bounding rectangle, and minimum bounding circle for the Tokyo photographs.

The first set of techniques, **quadrat** statistics, receive their name after their approach to split the data up into small areas (quadrants). Once created, these "buckets" are used to examine the uniformity of counts across them. The second set of techniques all derive from Ripley (1988) and involve measurements of the distance between points in a point pattern.

```
from pointpats import (
    distance_statistics,
    QStatistic,
    random,
    PointPattern,
)
```

For the purposes of illustration, it also helps to provide a pattern derived from a known

completely spatially random process. That is, the location and number of points is totally random; there is neither clustering nor dispersion. In point pattern analysis, this is known as a *Poisson point process*.

To simulate these processes from a given point set, you can use the `pointpats.random` module.

```
random_pattern = random.poisson(coordinates,␣
↪size=len(coordinates))
```

You can visualize this using the same methods as before, which we show in Figure 8.9:

```
f, ax = plt.subplots(1, figsize=(9, 9))
plt.scatter(
    *coordinates.T,
    color="k",
    marker=".",
    label="Observed photographs"
)
plt.scatter(*random_pattern.T, color="r", marker="x", label=
↪"Random")
contextily.add_basemap(
    ax, source=contextily.providers.CartoDB.Positron
)
ax.legend(ncol=1, loc="center left")
plt.show()
```

As you can see, the simulation (by default) works with the bounding box of the input point pattern. To simulate from more restricted areas formed by the point pattern, pass those hulls to the simulator! For example, to generate a random pattern within the alpha shapes:

```
random_pattern_ashape = random.poisson(
    alpha_shape, size=len(coordinates)
)
```

We can visualize this in Figure 8.10:

```
f, ax = plt.subplots(1, figsize=(9, 9))
plt.scatter(*coordinates.T, color="k", marker=".", label=
↪"Observed")
plt.scatter(
    *random_pattern_ashape.T, color="r", marker="x", label=
↪"Random"
)
contextily.add_basemap(
```

(continued on next page)

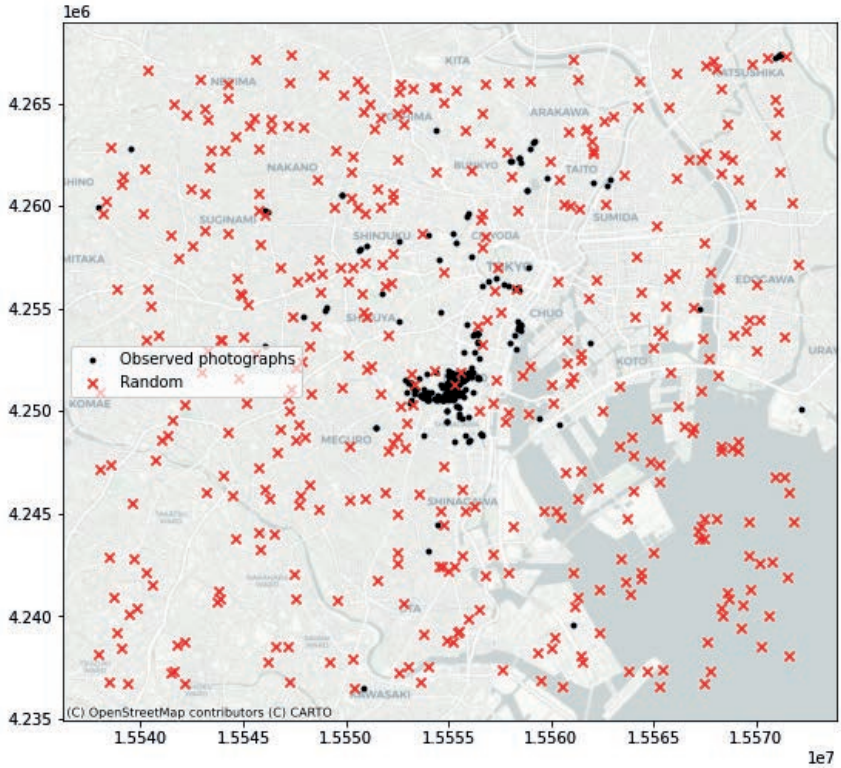


Fig. 8.9: Observed locations for Tokyo Photographs and random locations around Tokyo.

(continued from previous page)

```
    ax, source=contextily.providers.CartoDB.Positron
)
ax.legend(ncol=1, loc="center left")
plt.show()
```

8.5.1 Quadrat statistics

Quadrat statistics examine the spatial distribution of points in an area in terms of the count of observations that fall within a given cell. By examining whether observations are spread *evenly* over cells, the quadrat approach aims to estimate whether points are spread out, or if they are clustered into a few cells. Strictly speaking, quadrat statistics examine the *evenness* of the distribution over cells using a χ^2 statistical test common in the analysis of contingency tables.

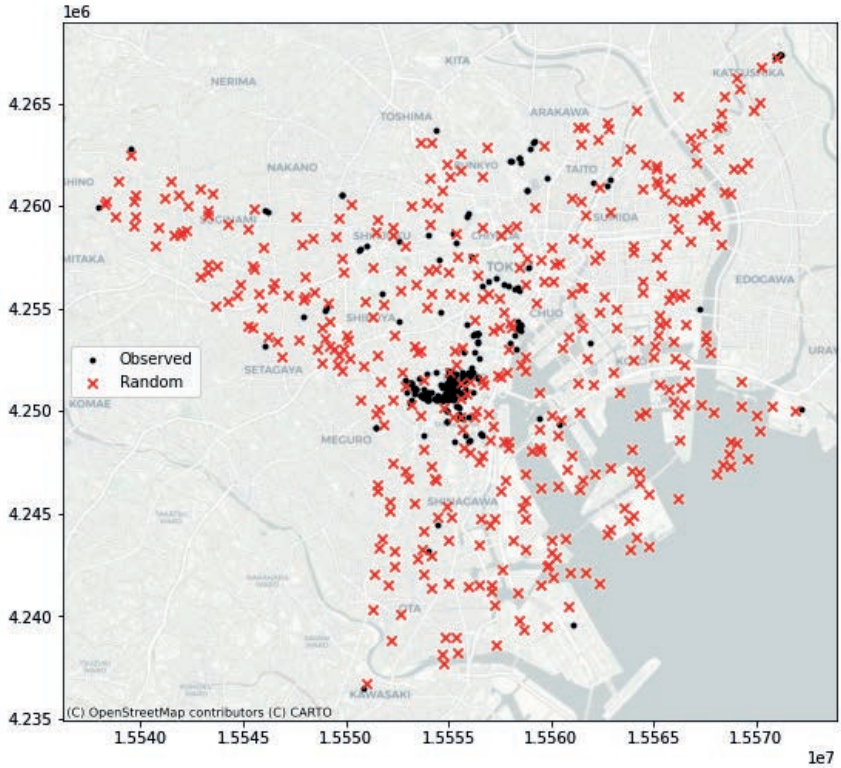


Fig. 8.10: Tokyo points, random and observed patterns within the alpha shape.

In the `pointpats` package, you can visualize the results using the following `QStatistic.plot()` method. This shows the grid used to count the events, as well as the underlying pattern, shown in Figure 8.11:

```
qstat = QStatistic(coordinates)
qstat.plot()
```

In this case, for the default of a three-by-three grid spanning the point pattern, we see that the central square has over 350 observations, but the surrounding cells have many fewer Flickr photographs. This means that the chi-squared test (which compares how likely this distribution is if the cell counts are uniform) will be statistically significant, with a very small p-value:

```
qstat.chi2_pvalue
```

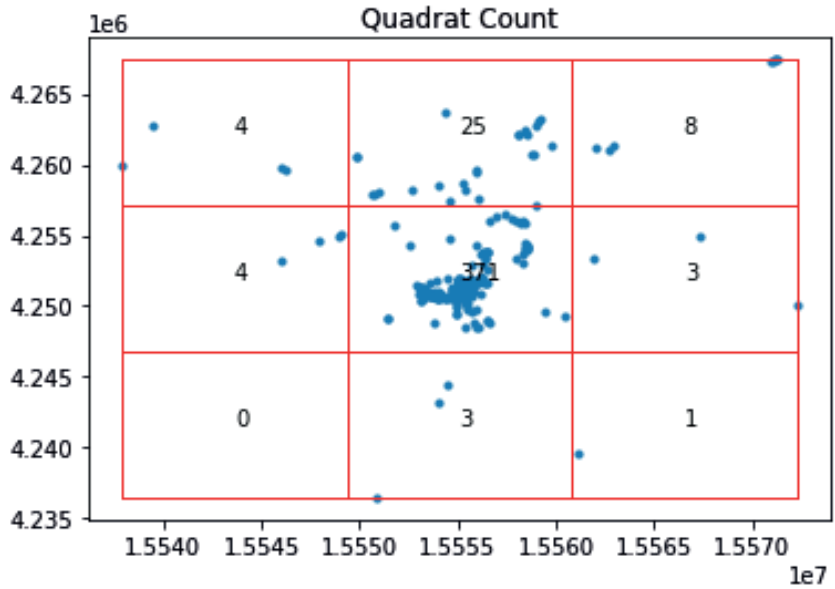


Fig. 8.11: Quadrat counts for the Tokyo photographs.

```
0.0
```

In contrast, our totally random point process will have nearly the same points in every cell, shown in Figure 8.12:

```
qstat_null = QStatistic(random_pattern)
qstat_null.plot()
```

This means its p-value will be large and likely not significant:

```
qstat_null.chi2_pvalue
```

```
0.9598693029768756
```

Be careful, however; the fact that quadrat counts are measured in a *regular tiling* that is overlaid on top of the potentially irregular extent of our pattern can mislead us. In particular, irregular *but random* patterns can be mistakenly found "significant" by this approach. Consider our random set generated within the alpha shape polygon, with the quadrat grid overlaid on top shown in Figure 8.13:

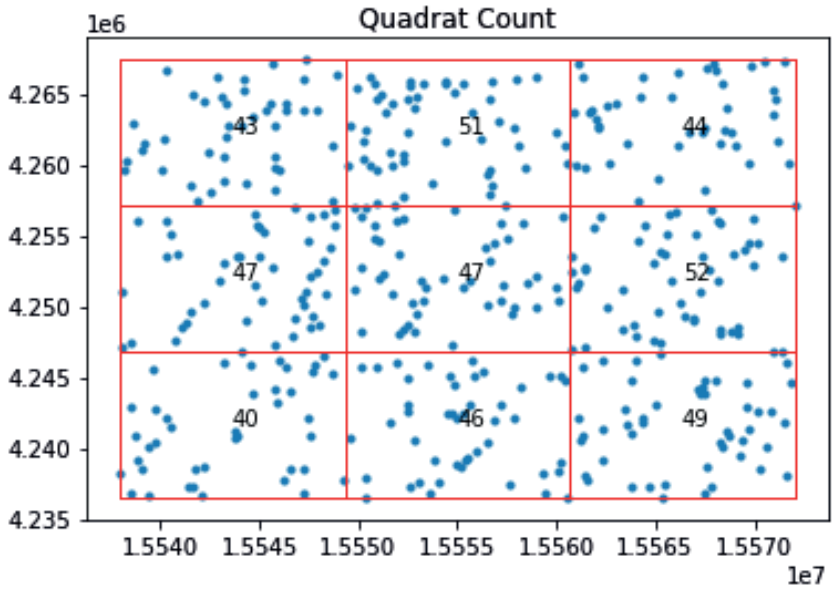


Fig. 8.12: Quadrat counts for the Tokyo photographs.

```
qstat_null_ashape = QStatistic(random_pattern_ashape)
qstat_null_ashape.plot()
```

The quadrat test finds this to be *statistically non-random*, while our simulating process ensured that *within the given study area*, the pattern is a complete spatially-random process.

```
qstat_null_ashape.chi2_pvalue
```

```
1.3191757482682422e-32
```

Thus, quadrat counts can have issues with irregular study areas, and care should be taken to ensure that clustering is not mistakenly identified. One way to interpret the quadrat statistic that reconciles cases like the one above is to think of it as a test that considers both the uniformity of points *and* the shape of their extent to examine whether the resulting pattern is uniform across a regular grid. In some cases, this is a useful tool; in others, this needs to be used with caution.

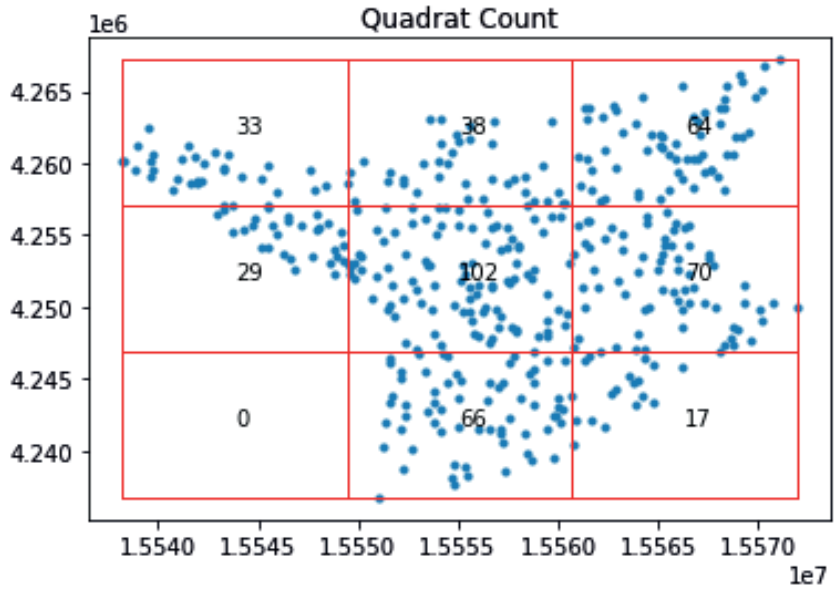


Fig. 8.13: Quadrat statistics for the random points across constrained to alpha shape of the Tokyo photographs.

8.5.2 Ripley's alphabet of functions

The second group of spatial statistics we consider focuses on the distributions of two quantities in a point pattern: nearest neighbor distances and what we will term "gaps" in the pattern. They derive from seminal work by [Rip91] on how to characterize clustering or co-location in point patterns. Each of these characterizes an aspect of the point pattern as we increase the distance range from each point to calculate them.

The first function, Ripley's G, focuses on the distribution of nearest neighbor distances. That is, the G function summarizes the distances between each point in the pattern and its nearest neighbor. In Figure 8.14, this nearest neighbor logic is visualized with the red dots being a detailed view of the point pattern and the black arrows indicating the nearest neighbor to each point. Note that sometimes two points are *mutual* nearest neighbors (and so have arrows going in both directions), but some are not.

Ripley's G keeps track of the proportion of points for which the nearest neighbor is within a given distance threshold, and plots that cumulative percentage against the increasing distance radii. The distribution of these cumulative percentages has a distinctive shape under completely spatially random processes. The intuition behind Ripley's G goes as follows: we can learn about how similar our pattern is to a spatially random

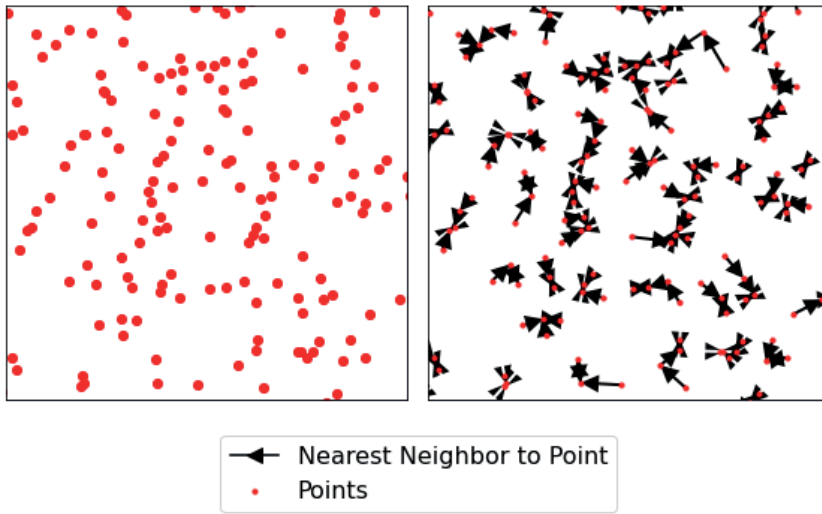


Fig. 8.14: Tokyo points and nearest neighbor graph. Code generated for this figure is available on the web version of the book.

one by computing the cumulative distribution of nearest neighbor distances over increasing distance thresholds, and comparing it to that of a set of simulated patterns that follow a known spatially random process. Usually, a spatial Poisson point process is used as such reference distribution.

To do this in the `pointpats` package, we can use the `g_test` function, which computes both the `G` function for the empirical data *and* these hypothetical replications under a completely spatially random process.

```
g_test = distance_statistics.g_test(
    coordinates, support=40, keep_simulations=True
)
```

Thinking about these distributions of distances, a "clustered" pattern must have more points near one another than a pattern that is "dispersed"; and a completely random pattern should have something in between. Therefore, if the G function increases *rapidly* with distance, we probably have a clustered pattern. If it increases *slowly* with distance, we have a dispersed pattern. Something in the middle will be difficult to distinguish from pure chance.

We can visualize this in Figure 8.15. On the left, we plot the $G(d)$ function, with distance-to-point (d) on the horizontal axis and the fraction of nearest neighbor distances smaller than d on the right axis. The empirical cumulative distribution of nearest neighbor distances is shown in red. In blue, simulations (like the `random` pattern shown in the previous section) are shown. The bright blue line represents the average

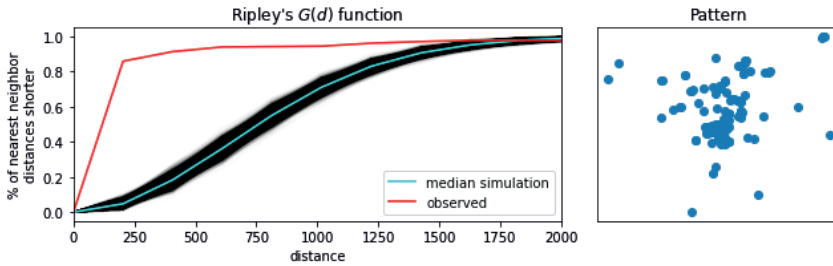


Fig. 8.15: Tokyo points, Ripley's *G* Function. Code generated for this figure is available on the web version of the book.

of all simulations, and the darker blue/black band around it represents the middle 95% of simulations.

In Figure 8.15, we see that the red empirical function rises much faster than simulated completely spatially random patterns. This means that the observed pattern of this user's Flickr photographs are *closer* to their nearest neighbors than would be expected from a completely spatially random pattern. The pattern is *clustered.*

The second function we introduce is Ripley's F. Where the G function works by analyzing the distance *between* points in the pattern, the *F* function works by analyzing the distance *to* points in the pattern from locations in empty space. That is why the F function is called the "the empty space function", since it characterizes the typical distance from arbitrary points in empty space to the point pattern. More explicitly, the F accumulates, for a growing distance range, the percentage of points that can be found within that range from a random point pattern generated within the extent of the observed pattern. If the pattern has large gaps or empty areas, the F function will increase slowly. But, if the pattern is highly dispersed, then the F function will increase rapidly. The shape of this cumulative distribution is then compared to those constructed by calculating the same cumulative distribution between the random pattern and an additional, random one generated in each simulation step.

We can use similar tooling to investigate the F function, since it is so mathematically similar to the G function. This is implemented identically using the `f_test` function in `pointpats`. Since the F function estimated for the observed pattern increases *much* more slowly than the F functions for the simulated patterns, we can be confident that there are many gaps in our pattern; i.e., the pattern is *clustered*.

```
f_test = distance_statistics.f_test(
    coordinates, support=40, keep_simulations=True
)
```

We can visualize this as before in Figure 8.16.

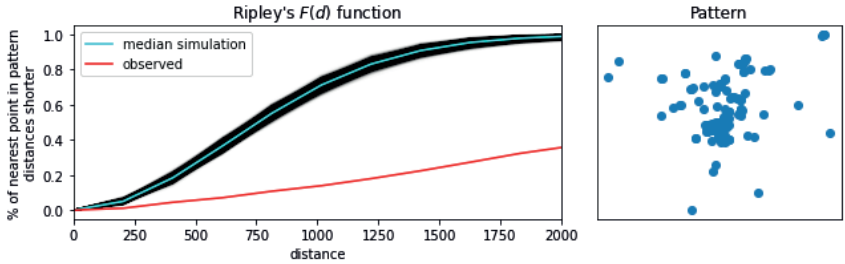


Fig. 8.16: Tokyo points, Cluster vs. non-cluster points. Code generated for this figure is available on the web version of the book.

Ripley's "alphabet" extends to several other letter-named functions that can be used for conducting point pattern analysis in this vein. Good "next steps" in your point pattern analysis journey include the book by [BRT15], and the `pointpats` documentation for guidance on how to run these in Python.

8.6 Identifying clusters

The previous two sections on exploratory spatial analysis of point patterns provide methods to characterize whether point patterns are dispersed or clustered in space. Another way to see the content in those sections is that they help us explore the degree of overall *clustering*. However, knowing that a point pattern is clustered does not necessarily give us information about where that (set of) cluster(s) resides. To do this, we need to switch to a method able to identify areas of high density of points within our pattern. In other words, in this section we focus on the existence and location of *clusters*. This distinction between cluster*ing* and cluster*s* of points is analogue to that discussed in the context of spatial autocorrelation (Chapters *6* and *7*). The notion is the same, the differences in the techniques we examine in each part of the book relate to the unique nature of points we referred to in the beginning of the book. Remember that, while the methods we explored in the earlier chapters take the location of the spatial objects (points, lines, polygons) as given and focus on understanding the configurations of values within those locations, the methods discussed in this chapter understand points as events that happen in particular locations but that could happen in a much broader set of places. Factoring in this underlying relevance of the location of an object itself is what makes the techniques in this chapter distinct.

From the many spatial point clustering algorithms, we will cover one called DBSCAN (Density-Based Spatial Clustering of Applications) [EKS+96]. DBSCAN is a widely used algorithm that originated in the area of knowledge discovery and machine learning and that has since spread into many areas, including the analysis of spatial points. In part,

its popularity resides in its intellectual simplicity and computational tractability. In some ways, we can think of DBSCAN as a point pattern counterpart of the local statistics we explored in *Chapter 7*. They do, however, differ in fundamental ways. Unlike the local statistics we have seen earlier, DBSCAN is not based on an inferential framework, but it is instead a deterministic algorithm. This implies that, unlike the measures seen before, we will not be able to estimate a measure of the degree to which the clusters found are compatible with cases of spatial randomness.

From the point of view of DBSCAN, a cluster is a concentration of at least `m` points, each of them within a distance of `r` of at least another point in the cluster. Following this definition, the algorithm classifies each point in our pattern into three categories:

- *Noise*, for those points outside a cluster.
- *Cores*, for those points inside a cluster with at least `m` points in the cluster within distance `r`.
- *Borders*, for points inside a cluster with less than `m` other points in the cluster within distance `r`.

The flexibility (but also some of the limitations) of the algorithm resides in that both `m` and `r` need to be specified by the user before running DBSCAN. This is a critical point, as their value can influence significantly the final result. Before exploring this in greater depth, let us get a first run at computing `DBSCAN` in Python:

```
# Define DBSCAN
clusterer = DBSCAN()
# Fit to our data
clusterer.fit(db[["x", "y"]])
```

```
DBSCAN()
```

Following the standard interface in scikit-learn, we first define the algorithm we want to run (creating the `clusterer` object), and then we *fit* it to our data. Once fit, `clusterer` contains the required information to access all the results of the algorithm. The `core_sample_indices_` attribute contains the indices (order, starting from zero) of each point that is classified as a *core*. We can have a peek into it to see what it looks like:

```
# Print the first 5 elements of `cs`
clusterer.core_sample_indices_[:5]
```

```
array([ 1, 22, 30, 36, 42])
```

The printout above tells us that the second (remember, Python starts counting at zero!) point in the dataset is a core, as are the 23rd, 31st, 36th, and 43rd points. This attribute has a variable length, depending on how many cores the algorithm finds.

The second attribute of interest is `labels_`:

```
clusterer.labels_[:5]
```

```
array([-1,  0, -1, -1, -1])
```

The `labels_` attribute always has the same length as the number of points used to run DBSCAN. Each value represents the index of the cluster a point belongs to. If the point is classified as *noise*, it receives a −1. Above, we can see that the second point belongs to cluster 1, while the others in the list are effectively not part of any cluster. To make things easier later on, let us turn the labels into a `Series` object that we can index in the same way as our collection of points:

```
lbls = pandas.Series(clusterer.labels_, index=db.index)
```

Now that we already have the clusters, we can proceed to visualize them. There are many ways in which this can be done. We will start just by coloring points in a cluster in red and noise in grey, as done in Figure 8.17.

```
# Setup figure and axis
f, ax = plt.subplots(1, figsize=(9, 9))
# Subset points that are not part of any cluster (noise)
noise = db.loc[lbls == -1, ["x", "y"]]
# Plot noise in grey
ax.scatter(noise["x"], noise["y"], c="grey", s=5, linewidth=0)
# Plot all points that are not noise in red
# NOTE how this is done through some fancy indexing, where
#      we take the index of all points (tw) and substract from
#      it the index of those that are noise
ax.scatter(
    db.loc[db.index.difference(noise.index), "x"],
    db.loc[db.index.difference(noise.index), "y"],
    c="red",
    linewidth=0,
)
# Add basemap
contextily.add_basemap(
    ax, source=contextily.providers.CartoDB.Positron
)
# Remove axes
ax.set_axis_off()
# Display the figure
plt.show()
```

Although informative, the result of this run is not particularly satisfactory. There are *way* too many points that are classified as “noise”.

Fig. 8.17: Tokyo points, DBSCAN clusters.

This is because we have run DBSCAN with the default parameters: a radius of 0.5 and a minimum of five points per cluster. Since our data is expressed in meters, a radius of half a meter will only pick up hyper local clusters. This might be of interest in some cases but, in others, it can result in odd outputs.

If we change those parameters, we can pick up more general patterns. For example, let us say a cluster needs to, at least, have roughly 1% of all the points in the dataset:

```
# Obtain the number of points 1% of the total represents
minp = numpy.round(db.shape[0] * 0.01)
minp
```

```
100.0
```

At the same time, let us expand the maximum radius to, say, 500 meters. Then we can re-run the algorithm and plot the output, all in the same cell this time to create Figure 8.18:

```
# Rerun DBSCAN
clusterer = DBSCAN(eps=500, min_samples=int(minp))
clusterer.fit(db[["x", "y"]])
# Turn labels into a Series
lbls = pandas.Series(clusterer.labels_, index=db.index)
# Setup figure and axis
f, ax = plt.subplots(1, figsize=(9, 9))
# Subset points that are not part of any cluster (noise)
noise = db.loc[lbls == -1, ["x", "y"]]
# Plot noise in grey
ax.scatter(noise["x"], noise["y"], c="grey", s=5, linewidth=0)
# Plot all points that are not noise in red
# NOTE how this is done through some fancy indexing, where
#      we take the index of all points (db) and substract from
#      it the index of those that are noise
ax.scatter(
    db.loc[db.index.difference(noise.index), "x"],
    db.loc[db.index.difference(noise.index), "y"],
    c="red",
    linewidth=0,
)
# Add basemap
contextily.add_basemap(
    ax, source=contextily.providers.CartoDB.Positron
)
# Remove axes
ax.set_axis_off()
# Display the figure
plt.show()
```

8.7 Conclusion

Overall, this chapter has provided an overview of methods to analyze point patterns. We have begun our point journey by visualizing their location and learning a way to overcome the "cluttering" challenge that large point patterns present us with. From a graphical display, we have moved to statistical characterization of their spatial distribution. In this context, we have learned about central tendency dispersion and extent, and we have positioned these measures as the point pattern counterparts of traditional statistics such as the mean or the standard deviation. These measures provide a summary of an entire pattern, but they tell us little about the spatial organization of each

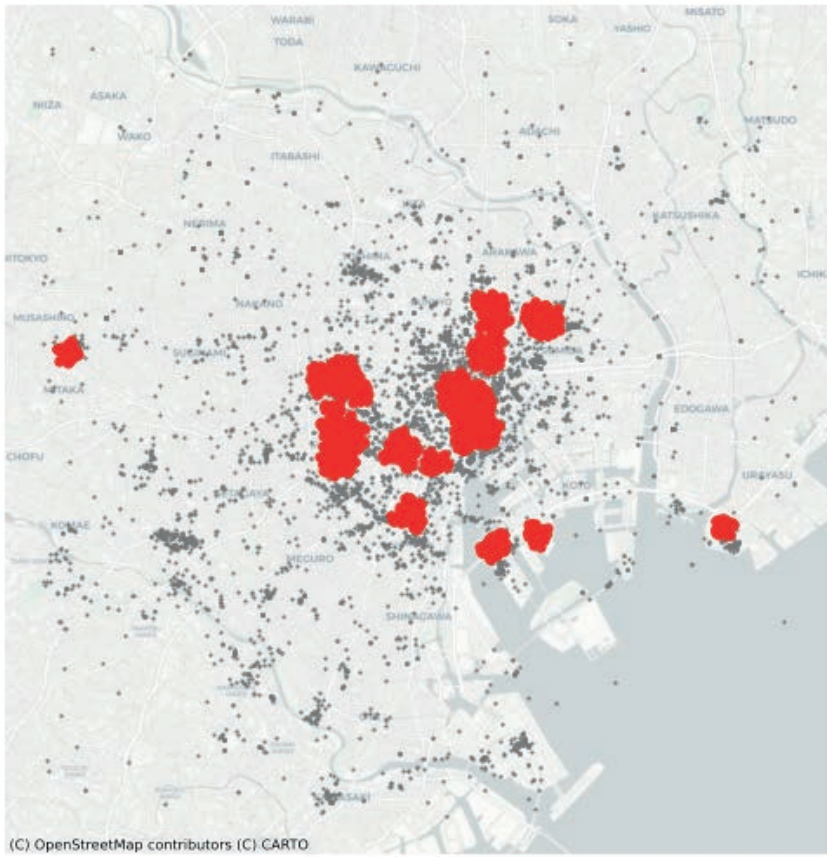


Fig. 8.18: Tokyo points, clusters with DBSCAN and minp=0.01.

point. To that end, we have introduced the quadrat and Ripley's functions. These statistical devices help us in characterizing whether a point pattern is spatially clustered or dispersed. We have wrapped up the chapter going one step further and exploring methods to identify the location of clusters: areas of the map with high density of points. Taken altogether, point pattern analysis has many applications across classical statistical fields as well as in data science. Using the techniques discussed here, you should be able to answer fundamental questions about point patterns that represent widely varied phenomena in the world, from the location where photographs were taken, to the distribution of bird nests, to the clustering of bike crashes in a city.

8.8 Questions

1. What is the trade-off when picking the hexagon granularity when "hexbinning"? Put another way, can we pick a "good" number of bins for all problems? If not, how would you recommend to select a specific number of bins?
2. Kernel Density Estimation (KDE) gets around the need to partition space in "buckets" to count points inside each of them. But, can you think of the limitations of applying this technique? To explore them, reproduce the KDE map from Figure 8.4, but change the arguments of the type of kernel (`kernel`) and the size of the bandwidth (`bw`). Consult the documentation of `seaborn.kdeplot` to learn what each of them controls. What happens when the bandwidth is very small? How does that relate to the number of bins in the hexbin plot?
3. Given a hypothetical point pattern, what characteristics would it need to meet for the mean and median centers to coincide?
4. Using `libpysal.cg.alpha_shape`, plot what happens to the alpha hull for $\alpha = 0, .2, .4, .6, .8, 1, 1.5, 2, 4$. What happens as `alpha` increases?
5. The choice of extent definition you adopt may influence your final results significantly. To further internalize this realization, compute the density of photographs in the example we have seen using each of the extent definitions covered (minimum bounding/rotate circle/rectangle, convex hull and alpha shape). Remember, the density can be obtained by dividing the number of photographs by the area of the extent.
6. Given the discussions in questions 1 and 2, how do you think the density of quadrats affect quadrat statistics?
7. Can you use information from Ripley's functions to inform the choice of DBSCAN parameters? How? Use the example with Tokyo photographs covered above to illustrate your ideas.

8.9 Next steps

For a much deeper and conceptual discussion of the analysis of spatial point patterns, consult Baddeley, Rubak and Turner. Their coverage is often the canonical resource for people interested in this topic:

Baddeley, Adrian, Ege Rubak, and Rolf Turner. 2015. *Spatial Point Patterns: Methodology and Applications with R*. Boca Raton, FL: CRC Press.

Taylor & Francis
Taylor & Francis Group
http://taylorandfrancis.com

Part III
Advanced Topics

This final part of the book shows how the ideas, tools, and building blocks explored in the previous two parts can be put together in a coherent way. We show this using a few specific applications. Part of our intention here is to take traditional themes in data science (un/supervised learning, feature engineering), and to ``sprinkle a bit of geo-dust'' to show how the way of thinking we have advanced in so far in this book can be used to obtain further insight beyond what traditional methods offer. The other part of our intention here is to show examples of *genuinely geographic* data science. Together, this will make the rest of the knowledge in the book useful for most readers.

Chapter 9 presents an explicitly spatial perspectice on inequality. Be it economic, social, or of other nature, inequality often manifests itself in an explicitly geographical way. For isntance, inequality between people is often most clearly recognized as inequality *between places*. This chapter shows how one can apply a ``geographic mind'' to traditional, aspatial measures; in doing so, it also illustrates a broader approach to explicitly spatialize non-spatial measures that can be deployed in a variety of contexts. Chapter 10 considers unsupervised statistical learning and puts it through the lens of Geography. We divide this in two sections: the non-spatial clustering of geographic units, and explicitly spatial algorithms of clustering, or regionalization methods. Chapter 11 moves from unsupervised to supervised learning. We consider the case of regression and present both the intuition and technicalities of building space into a regression framework as a ``first-class'' citizen. Finally, in Chapter 12, we flip the approach and, instead of showing how to build space into the model, we explore how geography can be embedded into the data we feed our supervised algorithms. In the machine learning jargon, polishing data into a shape that can be used in modelling is called ``feature engineering''. Since we present techniques to do this taking advantage of Geography and spatial relationships, we call it ``Spatial Feature Engineering''.

Taylor & Francis
Taylor & Francis Group
http://taylorandfrancis.com

9 Spatial Inequality Dynamics

This chapter uses economic inequality to illustrate how the study of the evolution of social disparities can benefit from an explicitly spatial treatment. Social and economic inequality is often at the top of policymakers' agendas. Its study has always drawn considerable attention in academic circles. Much of the focus has been on *interpersonal income inequality*, on differences between individuals irrespective of the geographical area where they live. Yet there is a growing recognition that the question of *inter-regional income inequality* requires further attention as the growing gaps between poor and rich regions have been identified as key drivers of civil unrest [Ezc19] and political polarization in developing and developed countries [RP18].

9.1 Introduction

Much of the study of inequalities has focused at the individual level: how do outcomes differ across individuals? This approach does not group individuals geographically. In other words, it is not concerned with whether those differences follow a pattern, for example, at the regional level (e.g., *is most of the more disadvantaged population located in a particular section of the map?*). Indeed, while the two literatures (personal and regional inequality) are related, they have developed in a largely parallel fashion with limited cross-fertilization. In this chapter, we examine how a spatially explicit focus can provide insights on the study of inequality and its dynamics. We hope this illustration can be useful in itself but also inspire the use of these methods in the study of other phenomena for which the regional perspective can bring value. This is also the only chapter where we explicitly deal with time as an additional dimension. Our presentation of inequalities takes an inherently temporal view, considering how different indices evolve over time the extent to which a spatial pattern changes. Again, we hope the illustration

we show here with inequality indices has value in itself, but also as inspiration for how to tackle time and dynamics in broader contexts.

After discussing the data we employ, we begin with an introduction to classic methods for interpersonal income inequality analysis and how they have been adopted to the question of regional inequalities. These include a number of graphical tools alongside familiar indices of inequality. As we discuss more fully, the use of these classical methods in spatially referenced data, while useful in providing insights on some of the aspects of spatial inequality, fails to fully capture the nature of geographical disparities and their dynamics. Thus, we next move to spatially explicit measures for regional inequality analysis. The chapter closes with some recent extensions of some classical measures to more fully examine the spatial dimensions of regional inequality dynamics.

```
import seaborn
import pandas
import geopandas
import pysal
import numpy
import mapclassify
import matplotlib.pyplot as plt
from pysal.explore import esda
from pysal.lib import weights
```

9.2 Data: U.S. state per capita income 1969-2017

For this chapter, we use data on average income per capita over time. Specifically, we consider the United States counties from 1969 to 2017. The U.S. counties are small regions that fit hierarchically within states. This perspective will allow us to examine trends for individual observations (counties), or regions containing several of them in a geographically consistent way (states or census regions which are collections of states). The temporal approach will reveal whether these entities get richer or poorer, as well as how the overall distribution of income moves, skews, or spreads out.

```
pci_df = geopandas.read_file(
    "../data/us_county_income/uscountypcincome.gpkg"
)
pci_df.columns
```

```
Index(['STATEFP', 'COUNTYFP', 'COUNTYNS', 'GEOID', 'NAME',
↪'NAMELSAD', 'LSAD',
       'CLASSFP', 'MTFCC', 'CSAFP', 'CBSAFP', 'METDIVFP',
↪'FUNCSTAT', 'ALAND',
       'AWATER', 'INTPTLAT', 'INTPTLON', 'GeoFIPS', 'GeoName',
↪ 'Region',
       'TableName', 'LineCode', 'Descriptio', 'Unit', '1969',
↪'1970', '1971',
```

(continued on next page)

(continued from previous page)

```
       '1972', '1973', '1974', '1975', '1976', '1977', '1978',
↪ '1979', '1980',
       '1981', '1982', '1983', '1984', '1985', '1986', '1987',
↪ '1988', '1989',
       '1990', '1991', '1992', '1993', '1994', '1995', '1996',
↪ '1997', '1998',
       '1999', '2000', '2001', '2002', '2003', '2004', '2005',
↪ '2006', '2007',
       '2008', '2009', '2010', '2011', '2012', '2013', '2014',
↪ '2015', '2016',
       '2017', 'index', 'IndustryCl', 'Descript_1', 'geometry
↪'],
      dtype='object')
```

Inspection of the column names reveals that the table is organized around one row per county, and with the years as columns, together with information about the particular record. This format is an example of a *wide* longitudinal dataset. In wide-format data, each column represents a different time period, meaning that each row represents a set of measurements made about the same "entity" over time (as well as any unique identifying information about that entity.) This contrasts with a *narrow*, *long* format, where each row describes an entity at a specific point in time. Long data results in significant duplication for records and is generally worse for data storage, particularly in the geographic case. However, long form data is sometimes a more useful format when manipulating and analyzing data, as [W+14] discusses. Nonetheless, when analyzing *trajectories*, that is, the paths that entities take over time, wide data is more useful, and we will use that here.

In this dataset, we have 3076 counties across 49 years, as well as 28 extra columns that describe each county.

```
pci_df.shape
```

```
(3076, 77)
```

As an example, we can see the first ten years for Jackson County, Mississippi (state code 28) below:

```
pci_df.query('NAME == "Jackson" & STATEFP == "28"').loc[
    :, "1969":"1979"
]
```

```
      1969  1970  1971  1972  1973  1974  1975  1976  1977 ␣
↪1978  1979
1417  2957  3121  3327  3939  4203  4547  5461  5927  6315 ␣
↪6619  6967
```

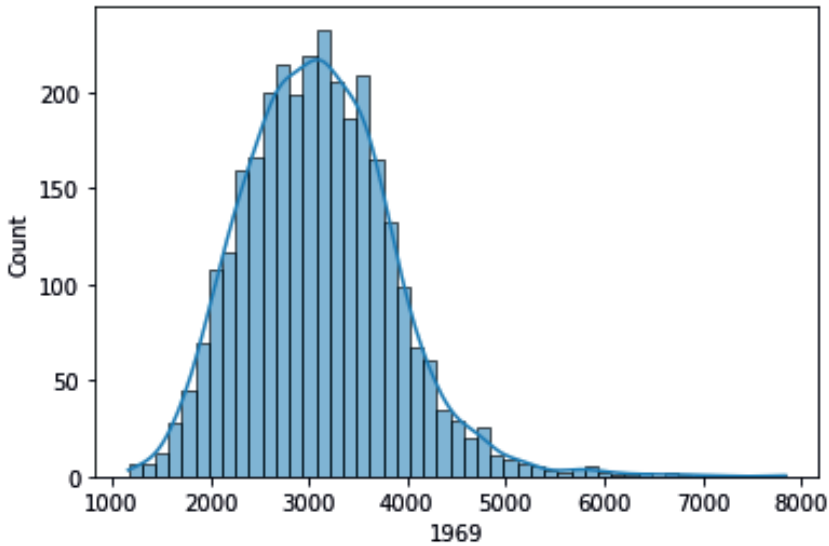


Fig. 9.1: Distribution of U.S. per capita income at county level in 1969.

9.3 Global inequality

We begin our examination of inequality by focusing on several global measures of income inequality. Here, "global" means that the measure is concerned with the overall nature of inequality within the income distribution. That is, these measures focus on the direct disparity between rich and poor, considering nothing about where the rich and poor live. Several classic measures of inequality are available for this purpose.

In general terms, measures of inequality focus on the dispersion present in an income distribution. In the case of regional or spatial inequality, the distributions describe the average or per capita incomes for spatial units, such as for counties, census tracts, or regions. For our U.S. county data, we can visualize (Figure 9.1) the distribution of per capita incomes for the first year in the sample as follows:

```
seaborn.histplot(x=pci_df["1969"], kde=True);
```

Looking at this distribution, notice that the right side of the distribution is much longer than the left side. This long right tail is a prominent feature, and is common in the study of incomes and many other societal phenomena, as it reflects the fact that within a single income distribution, the super-rich are generally much more wealthy than the super-poor are deprived, compared to the average.

A key point to keep in mind here is that the unit of measurement in this data is a spatial aggregate of individual incomes. Here, we are using the per capita incomes for each

county. By contrast, in the wider inequality literature, the observational unit is typically a household or individual. In the latter distributions, the degree of skewness is often more pronounced. This difference arises from the smoothing that is intrinsic to aggregation: the regional distributions are based on averages obtained from the individual distributions, and so the extremely high-income individuals are averaged with the rest of their county. The regional approach implies that, to avoid falling into the so-called "ecological fallacy", whereby individual conclusions are drawn from geographical aggregates, our conclusions will hold at the area level (*county*) rather than the individual one (*person*).

The kernel density estimate (or histogram) is a powerful visualization device that captures the overall morphology of the *feature* distribution for this measure of income. At the same time, the plot is silent on the underlying *geographic distribution* of county incomes. We can look at this second view of the distribution using a choropleth map. To construct this, we can use the standard `geopandas` plotting tools.

Before we can get to mapping, we change the CRS to a suitable one for mapping, the Albers Equal Area projection for North America:

```
pci_df = pci_df.to_crs(
    # Albers Equal Area North America
    epsg=5070
)
```

And the quantile choropleth for 1969 (Figure 9.2) can be generated by:

```
ax = pci_df.plot(
    column="1969",
    scheme="Quantiles",
    legend=True,
    edgecolor="none",
    legend_kwds={"loc": "lower left"},
    figsize=(12, 12),
)
ax.set_axis_off()
plt.show()
```

The choropleth and the kernel density provide different visual depictions of the distribution of county incomes. The kernel density estimate is a *feature*-based representation, and the map is a *geographic*-based representation. Both are useful for developing a more comprehensive understanding. To gain insights on the level of inequality in the distribution, we'll discuss a few indices common in the statistical and econometric literatures.

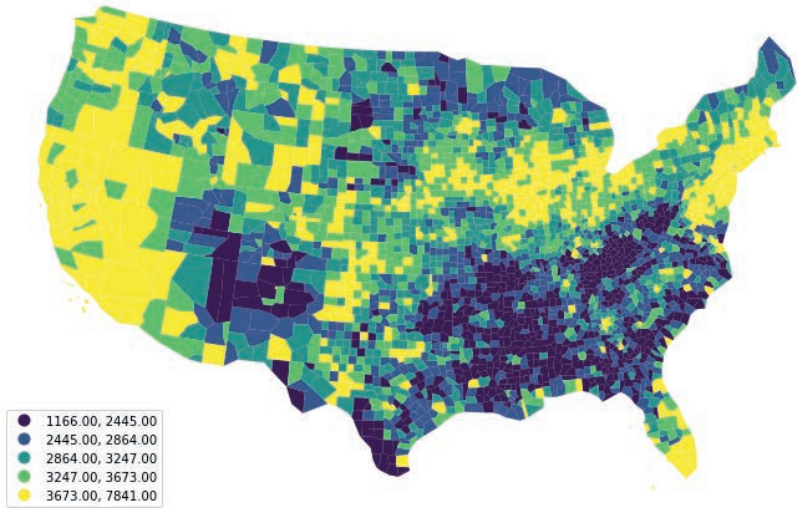


Fig. 9.2: Quintiles of per capita income by county, 1969.

9.3.1 20:20 ratio

One commonly used measure of inequality in a distribution is the so-called 20:20 ratio, which is defined as the ratio of the incomes at the 80th percentile over that at the 20th percentile:

```
top20, bottom20 = pci_df["1969"].quantile([0.8, 0.2])
```

The `top20` (`bottom20`) objects contain the boundary value that separates the series between the top (bottom) 20% of the distribution and the rest. With these, we can generate the ratio:

```
top20 / bottom20
```

```
1.5022494887525562
```

In 1969 the richest 20% of the counties had an income that was 1.5 times the poorest 20% of the counties. The 20:20 ratio has the advantage of being robust to outliers at the top and the bottom of the distribution. To look at the dynamics of this global inequality measure, one way is to create a function that calculates it for a given year, and apply it to all years in our time series, which we can then plot as a time series (Figure 9.3):

```
def ineq_20_20(values):
    top20, bottom20 = values.quantile([0.8, 0.2])
```

(continued on next page)

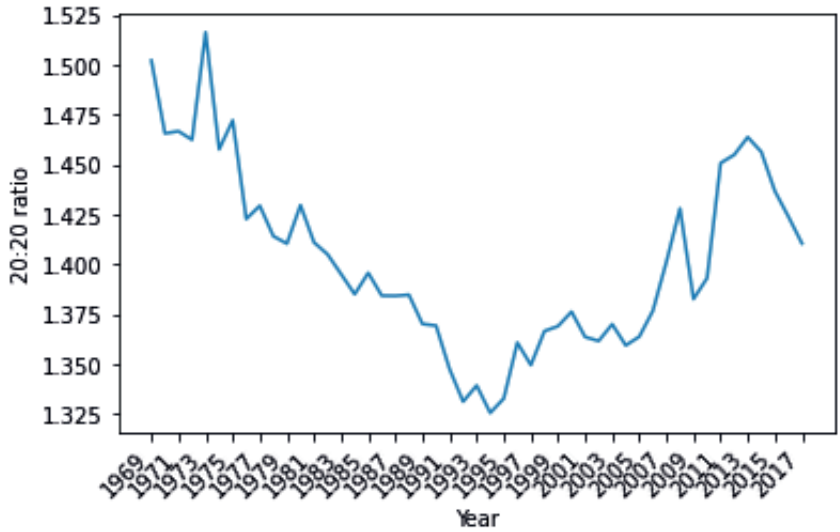


Fig. 9.3: The 20-20 ratio for US county incomes.

(continued from previous page)

```
    return top20 / bottom20


# Generate range of strings from 1969 to 2018
years = numpy.arange(1969, 2018).astype(str)
# Compute 20:20 ratio for every year
ratio_2020 = pci_df[years].apply(ineq_20_20, axis=0)
# Plot evolution of 20:20 ratio
ax = plt.plot(years, ratio_2020)

# Grab figure generated in the plot
figure = plt.gcf()
# Replace tick labels with every other year
plt.xticks(years[::2])
# Set vertical label
plt.ylabel("20:20 ratio")
# Set horizontal label
plt.xlabel("Year")
# Rotate year labels
figure.autofmt_xdate(rotation=45)

plt.show()
```

The evolution of the ratio has a U-shaped pattern over time, bottoming out around 1994 after a long decline. Post-1994, however, the 20:20 ratio indicates there is increasing inequality up until 2013, where there is a turn toward lower income inequality between the counties.

In addition to the 20:20 ratio, we will explore two more traditional measures of inequality: the Gini index and Theil's index. For these, we will use the `inequality` package from Pysal.

```
from pysal.explore import inequality
```

9.3.2 Gini index

The Gini index is a longstanding measure of inequality based on the notion of cumulative wealth distribution [MM21]. The Gini is closely linked to another popular device called the Lorenz curve. To construct a Lorenz curve, the cumulative share of wealth is plotted against the share of the population that owns that wealth. For example, in an extremely unequal society where few people own nearly all the wealth, the Lorenz curve increases very slowly at first, then skyrockets once the wealthiest people are included.

In contrast, a "perfectly equal" society would look like a straight line connecting $(0, 0)$ and $(1, 1)$. This is called the *line of perfect equality*, and represents the case where $p\%$ of the population owns exactly $p\%$ of the wealth. For example, this might mean that 50% of the population earns exactly 50% of the income, or 90% of the population owns 90% of the wealth. The main idea is that the share of wealth or income is exactly proportional to the share of population that owns that wealth or earns that income, which occurs only when everyone has the same income or owns the same amount of wealth.

With these notions in mind, we can define the Gini index as the ratio of the area between the line of perfect equality and the Lorenz curve for a given income or wealth distribution, standardized by the area under the line of perfect equality (which is always $\frac{1}{2}$). Thus, the Gini index is a measure of the gap between a perfectly equal society and the observed society over every level of wealth/income.

We can construct the Lorenz curve for 1969 by first computing the share of our population of counties that is below each observation. For that, we generate a cumulative series:

```
n = len(pci_df)
share_of_population = numpy.arange(1, n + 1) / n
```

Then, we consider the cumulative evolution of income. For this, we need to find out the proportion of total income owned by each share of the population. Empirically, this can be computed in the following fashion. First, we sort county incomes:

```
incomes = pci_df["1969"].sort_values()
```

Second, we find the overall percentage of income accumulated at each data point. To do this, we compute what percentage of the total income each county represents:

```
shares = incomes / incomes.sum()
```

and construct the *cumulative sum* of these shares, which reflects the sum of all of the shares of income up to the current one:

$$\texttt{CumSum(v, k)} = \sum_{i=1}^{k} v_i$$

This starts at 0 and reaches 1 once the last share is included:

```
cumulative_share = shares.cumsum()
```

With this, we can plot both the Lorenz curve and the line of perfect equality (Figure 9.4):

```
# Generate figure with one axis
f, ax = plt.subplots()
# Plot Lorenz Curve
ax.plot(share_of_population, cumulative_share, label="Lorenz␣
↪Curve")
# Plot line of perfect equality
ax.plot((0, 1), (0, 1), color="r", label="Perfect Equality")
# Label horizontal axis
ax.set_xlabel("Share of population")
# Label vertical axis
ax.set_ylabel("Share of income")
# Add legend
ax.legend()

plt.show()
```

The blue line is the Lorenz curve for county incomes in 1969. The Gini index is the area between it and the 45-degree line of equality shown in red, all standardized by the area underneath the line of equality.

A first approach to examine how inequality has evolved is to plot the Lorenz curves for each year. One way to do this in Python involves creating a function that will compute the Lorenz curve for an arbitrary set of incomes. The following function encapsulates the steps shown above into a single shot:

```
def lorenz(y):
    y = numpy.asarray(y)
```

(continued on next page)

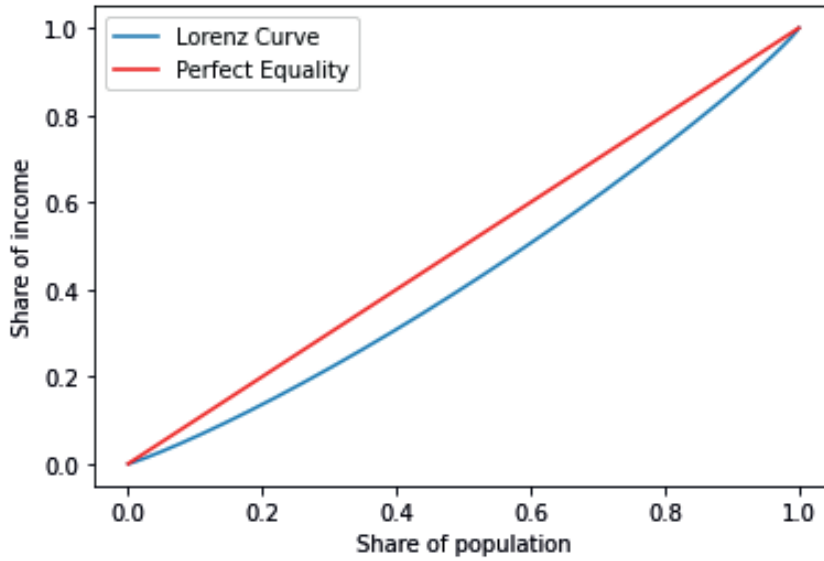


Fig. 9.4: The Lorenz curve for county per capita income 1969.

(continued from previous page)

```
    incomes = numpy.sort(y)
    income_shares = (incomes / incomes.sum()).cumsum()
    N = y.shape[0]
    pop_shares = numpy.arange(1, N + 1) / N
    return pop_shares, income_shares
```

For a single year, say 1969, our function would return a tuple with two arrays, one for each axis in the Lorenz curve plot:

```
lorenz(pci_df["1969"])
```

```
(array([3.25097529e-04, 6.50195059e-04, 9.75292588e-04, ...,
        9.99349805e-01, 9.99674902e-01, 1.00000000e+00]),
 array([1.22486441e-04, 2.52956561e-04, 3.83636778e-04, ...,
        9.98429316e-01, 9.99176315e-01, 1.00000000e+00]))
```

We can now use the same strategy as above to calculate the Lorenz curves for all the years in our datasets:

```
lorenz_curves = pci_df[years].apply(lorenz, axis=0)
```

Practically, this becomes a dataframe with columns for each year. Rows contain the population shares (or income shares) as lists. We can then iterate over the columns

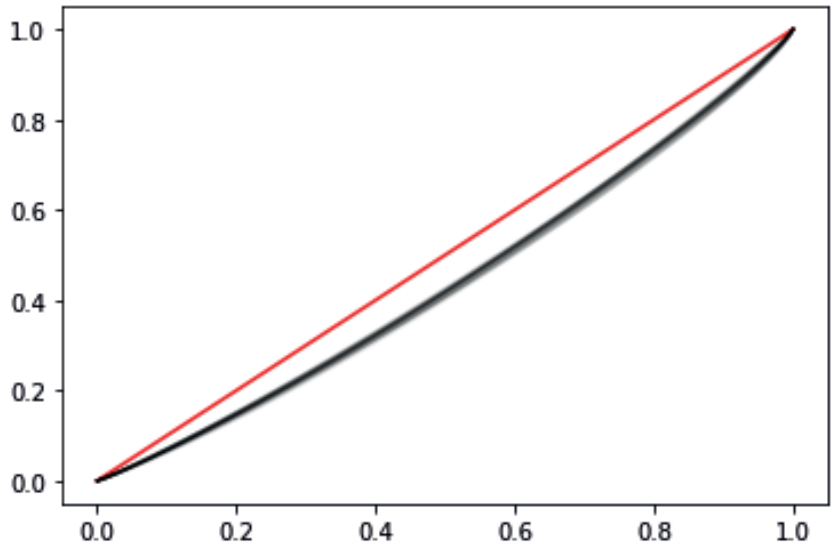


Fig. 9.5: Lorenz curves for county per capita incomes since 1969.

(years) of this dataframe, generating a plot of the Lorenz curve for each year (Figure 9.5):

```
# Set up figure with one axis
f, ax = plt.subplots()
# Plot line of perfect equality
ax.plot((0, 1), (0, 1), color="r")
# Loop over every year in the series
for year in lorenz_curves.columns:
    # Extract the two arrays or each dimension
    year_pop_shares, year_inc_shares = lorenz_curves[year].
↪values
    # Plot Lorenz curve for a given year
    ax.plot(year_pop_shares, year_inc_shares, color="k",␣
↪alpha=0.05)
```

The compression of the Lorenz curves makes it difficult to ascertain the temporal pattern in inequality. Focusing explicitly on the Gini coefficients may shed more light on the evolution of inequality over time.

Remember the Gini coefficient represents the area in between the Lorenz curve and that of perfect equality. The measure can be calculated directly through the `Gini` class in `inequality`. For 1969, this implies:

```
g69 = inequality.gini.Gini(pci_df["1969"].values)
```

To extract the coefficient, we retrieve the `g` property of `g69`:

```
g69.g
```

```
0.13556175504269904
```

Here, the Gini coefficient in 1969 was 0.13. To compute this for every year, we can use a similar pattern as we have before. First, define a function to compute the quantity of interest; then, apply the function across the table with all years:

```
def gini_by_col(column):
    return inequality.gini.Gini(column.values).g
```

`inequality`'s Gini requires an `numpy.ndarray` rather than a `pandas.Series` object, which we can pull out through the `values` attribute. This is passed to the `Gini` class, and we only return the value of the coefficient as a `DataFrame` object.

```
inequalities = (
    pci_df[years].apply(gini_by_col, axis=0).to_frame("gini")
)
```

This results in a series of Gini values, one for each year:

```
inequalities.head()
```

```
          gini
1969  0.135562
1970  0.130076
1971  0.128540
1972  0.129126
1973  0.142166
```

Which we can turn into a graphical representation through standard `pandas` plotting. The resulting pattern (Figure 9.6) is similar to that of the 20:20 ratio above:

```
inequalities.plot(figsize=(10, 3));
```

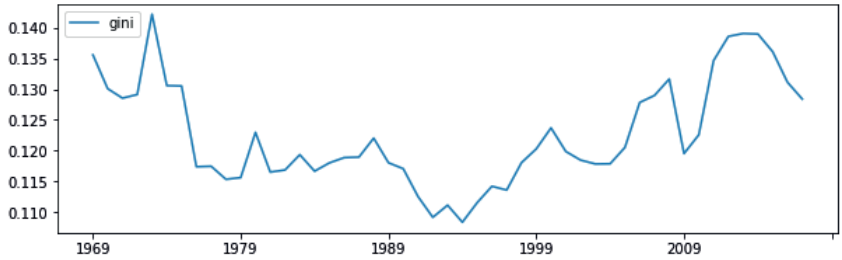


Fig. 9.6: Gini coefficients for per capita income since 1969.

9.3.3 Theil's index

A third commonly used measure of inequality is Theil's T [MM21] given as:

$$T = \sum_{i=1}^{m} \left(\frac{y_i}{\sum_{i=1}^{m} y_i} \ln \left[m \frac{y_i}{\sum_{i=1}^{m} y_i} \right] \right)$$

where y_i is per capita income in area i among m areas. Conceptually, this metric is related to the entropy of the income distribution, measuring how evenly distributed incomes are across the population.

The Theil index is also available in Pysal's `inequality`, so we can take a similar approach as above to calculate it for every year:

```
def theil(column):
    return inequality.theil.Theil(column.values).T


inequalities["theil"] = pci_df[years].apply(theil, axis=0)
```

And generate a plot of its evolution over time (Figure 9.7):

```
inequalities["theil"].plot(color="orange", figsize=(10, 3));
```

The time paths of the Gini and the Theil coefficients appear to show striking similarities. At first glance, this might suggest that the indices are substitutes for one another. However, if we plot them against each other (Figure 9.8), we can see they are not perfectly correlated:

```
_ = seaborn.regplot(x="theil", y="gini", data=inequalities)
```

Indeed, as we shall see below, each index has properties that lend themselves to particular spatial extensions that work in complementary ways. We need both (and more) for a complete picture.

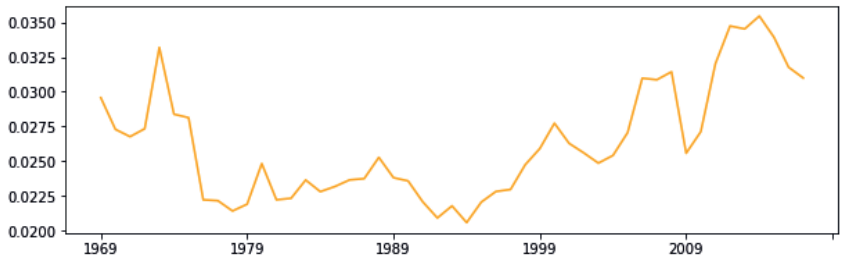


Fig. 9.7: Theil index for county per capita income distributions since 1969.

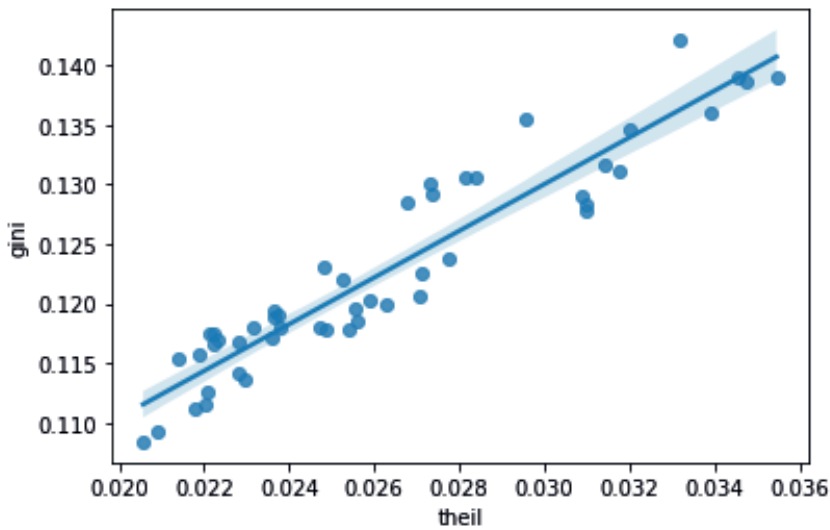


Fig. 9.8: Relationship between Gini and Theil indices for county per capita income distributions since 1969.

9.4 Personal vs. regional income

There is a subtle but important distinction between the study of personal and regional income inequality. To see this, we first need to express the relationships between the two types of inequality. Consider a country composed of N individuals who are distributed over m regions. Let Y_l denote the income of individual l. Total personal income in region i is given as $Y_i = \sum_{l \in i} Y_l$, and per capita income in region i is $y_i = \frac{Y_i}{N_i}$, where N_i is the number of individuals in region i.

At the national level, the coefficient of variation in incomes could be used as an index of interpersonal income inequality. This would be:

$$CV_{nat} = \sqrt{\frac{\sum_{l=1}^{N}(Y_l - \bar{y})^2}{N}}$$

where $\bar{y}$ is the national average for per capita income. The key component here is the sum of squares term, and unpacking this sheds light on the personal versus regional inequality question:

$$TSS = \sum_{l=1}^{N}(Y_l - \bar{y})^2$$

An individual deviation, $\delta_l = Y_l - \bar{y}$, is the contribution to inequality associated with individual l. We can break this into two components:

$$\delta_l = (Y_l - y_i) + (y_i - \bar{y})$$

The first term is the difference between the individual's income and per capita income in the individual's region of residence, while the second term is the difference between the region's per capita income and average national per capita income.

In regional studies, the intra-regional personal income distribution is typically not available. As a result, the assumption is often made that intra-regional personal inequality is zero. In other words, all individuals in the same region have identical incomes. With this assumption in hand, the first term vanishes: $Y_l - y_i = 0$, leading to:[1]

$$\begin{aligned}
TSS &= \sum_{l=1}^{N}(Y_l - \bar{y})^2 \\
&= \sum_{l=1}^{N}\delta_l^2 \\
&= \sum_{l=1}^{N}((Y_l - y_i) + (y_i - \bar{y}))^2 \\
&= \sum_{l=1}^{N}(0 + (y_i - \bar{y}))^2 \\
&= \sum_{i=1}^{m}\sum_{l\in i}(y_i - \bar{y})^2 \\
&= \sum_{i=1}^{m}[N_i(y_i - \bar{y})]^2
\end{aligned}$$

[1] It should also be noted that even at the national scale, the analysis of interpersonal income inequality also relies on aggregate data grouping individuals into income cohorts. See, for example, [PS03].

This means that each individual in a region has an equal contribution to the overall level of national interpersonal inequality, given by $(y_i - \bar{y})$, while the region in question contributes $N_i(y_i - \bar{y})$. While it may seem that the assumption of zero intra-regional interpersonal income inequality is overly restrictive, it serves to isolate the nature of inter-regional income inequality. That is, inequality between places, rather than inequality between people within those places. In essence, this strategy shifts the question up one level in the spatial hierarchy by aggregating micro-level individual data to areal units.

9.5 Spatial inequality

The analysis of regional income inequality differs from the analysis of national interpersonal income inequality in its focus on spatial units. Since regional incomes are explicitly embedded in geographical space, we can take advantage of their spatial configuration to learn more about the nature of the inequality. In the regional inequality literature, this has been approached in a number of ways. Three are considered in this chapter: one that links the discussion to that of spatial autocorrelation in Chapters *6* and *7*, a second one based on decomposing global indices regionally, and a third one that embeds space in traditional global measures.

9.5.1 Spatial autocorrelation

This approach helps us shed light on the properties of the spatial pattern of regional income data. We return to global measures of spatial autocorrelation that we encountered earlier in the book. The essence of this approach is to examine to what extent the spatial distribution of incomes is concentrated over space. For this, we use a queen spatial weights matrix and calculate Moran's I for each year in the sample:

```
wq = weights.Queen.from_dataframe(pci_df)
```

Following the same pattern to "broadcast" a function, we create a function that returns the results we need from each statistic. Here, we will also keep the pseudo p-value for the Moran statistic which, as we saw in *Chapter 6*, helps us identify whether the index is statistically significant under the null hypothesis that incomes are randomly distributed geographically.

```
def moran_by_col(y, w=wq):
    mo = esda.Moran(y, w=w)
    mo_s = pandas.Series(
        {"I": mo.I, "I-P value": mo.p_sim},
    )
    return mo_s
```

This time, our function returns a `Series` object so that when we pass it through `apply`, we get a well-formatted table:

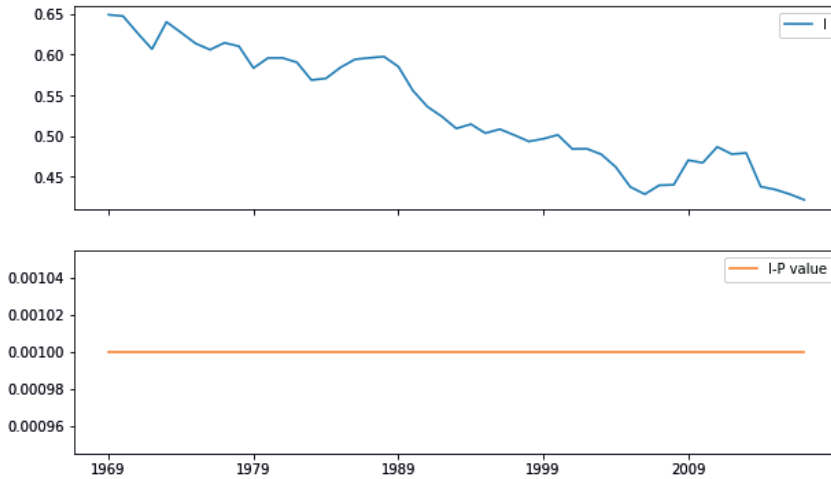


Fig. 9.9: Moran's I, a measure of spatial autocorrelation, for per capita incomes since 1969 together with pseudo p-values.

```
moran_stats = pci_df[years].apply(moran_by_col, axis=0).T

moran_stats.head()
```

```
             I  I-P value
1969  0.649090      0.001
1970  0.647438      0.001
1971  0.626546      0.001
1972  0.606760      0.001
1973  0.640226      0.001
```

For further comparison, the results are attached to the `inequalities` table:

```
inequalities = inequalities.join(moran_stats)
```

Which can be visualized (Figure 9.9) by:

```
inequalities[["I", "I-P value"]].plot(subplots=True,␣
↪figsize=(10, 6))
plt.show()
```

Several patterns emerge from the time series of Moran's I. Before delving into the details, it is worth noting that, while Gini and Theil indices from previous figures follow a similar path, Moran's I displays a distinct trajectory. There is a long-term decline in

the value of Moran's I. This suggests a gradual decline in the geographic structure of inequality with two implications: (a) per capita incomes are now less similar between nearby counties and (b), this has been consistently declining, regardless of whether inequality is high or low. Second, despite this decline, there is never a year in which the spatial autocorrelation is not statistically significant. In other words, there is a strong geographic structure in the distribution of regional incomes that needs to be accounted for when focusing on inequality questions.

9.5.2 Regional decomposition of inequality

One common objection to the analysis of inequality in aggregate relates to lack of detail about the scale at which inequality is most important. Inequality can be driven by differences between groups and not by discrepancies in income between similar individuals. That is, there is always the possibility that observed inequality can be "explained" by a confounding variate, such as age, sex, or education. For example, income differences between older and younger people can "explain" a large part of the societal inequality in wealth: older people have much longer to acquire experience, and thus are generally paid more for that experience. Younger people do not have as much experience, so young people (on average) have lower incomes than older people.

To tackle this issue, it is often useful to *decompose* inequality indices into constituent groups. This allows us to understand how much of inequality is driven by aggregate group differences and how much is driven by observation-level inequality. This also allows us to characterize how unequal each group is separately. In geographic applications, these groups are usually spatially defined, in that *regions* are contiguous geographic groups of observations [SW05]. This section discusses regional inequality decompositions as a way to introduce geography into the study of inequality.

Let's illustrate these ideas with our income dataset. The table records the United States Census Bureau region a county belongs to in the `Region` variable. These divide the country into eight regions, each assigned a number that relates to its name as specified below:

```
region_names = {
    1: "New England",
    2: "Mideast",
    3: "Great Lakes",
    4: "Plains",
    5: "Southeast",
    6: "Southwest",
    7: "Rocky Mountain",
    8: "Far West",
}
```

We can visualize the regions with the names on the legend (Figure 9.10) by first mapping the name to each region number, and then rendering a qualitative choropleth:

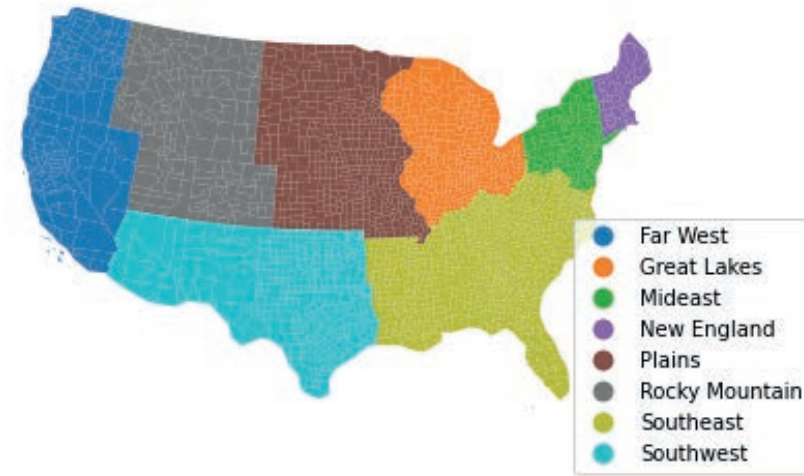


Fig. 9.10: Map of census regions in the United States.

```
ax = pci_df.assign(Region_Name=pci_df.Region.map(region_
↪names)).plot(
    "Region_Name",
    linewidth=0,
    legend=True,
    categorical=True,
    legend_kwds=dict(bbox_to_anchor=(1.2, 0.5)),
)
ax.set_axis_off();
```

Let's peak into income changes for each region. To do that, we can apply a split-apply-combine pattern that groups counties by region, calculates its mean, and combines it into a table:

```
rmeans = (
    pci_df.assign(
        # Create column with region name for each county
        Region_Name=pci_df.Region.map(region_names)
    )
    .groupby(
        # Group counties by region name
        by="Region_Name"
        # Calculate mean by region and save only year columns
    )
    .mean()[years]
)
```

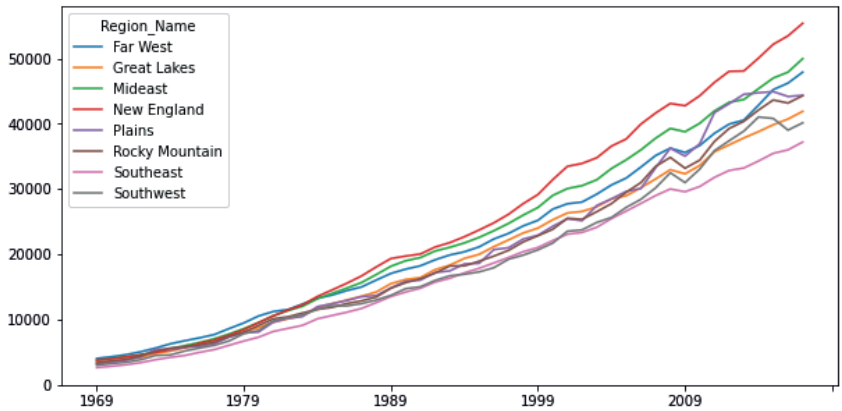


Fig. 9.11: Average county per capita incomes among census regions since 1969.

The resulting table has a row for each region and a column for each year. We can visualize these means to get a sense of their temporal trajectory (Figure 9.11):

```
rmeans.T.plot.line(figsize=(10, 5));
```

One way to introduce geography into the analysis of inequality is to use geographical delineations to define groups for decompositions. For example, Theil's T, which we encountered previously, can be decomposed using regions into so-called *between* and *within* regional inequality components. To proceed in this direction, we first reconceptualize our observations of per capita incomes for m regional economies as $y = (y_1, y_2, \ldots, y_m)$. These are grouped into ω mutually exclusive regions. Formally, this means that when m_g represents the number of areas assigned to region g, the total number of areas must be equal to the count of all the areas in each region: $\sum_{g=1}^{\omega} m_g = m$.[2] With this notation, Theil's index from above can be rewritten to emphasize its between and within components:

$$
\begin{aligned}
T &= \sum_{i=1}^{m} \left(\frac{y_i}{\sum_{i=1}^{m} y_i} \ln \left[m \frac{y_i}{\sum_{i=1}^{m} y_i} \right] \right) \\
&= \left[\sum_{g=1}^{\omega} s_g \ln \left(\frac{m}{m_g} s_g \right) \right] + \left[\sum_{g=1}^{\omega} s_g \sum_{i \in g} s_{i,g} \ln \left(m_g s_{i,g} \right) \right] \\
&= B + W
\end{aligned}
$$

where $s_g = \frac{\sum_{i \in g} y_i}{\sum_i y_i}$, and $s_{i,g} = y_i / \sum_{i \in g} y_i$.

The first term is the between regions inequality component, and the second is the within

[2] This would be violated, for example, if one area were in two regions. This area would get "double counted" in this total.

regions inequality component. The within regions term is a weighted average of inequality between economies belonging to the same region. Similar to what is done above for the case of interpersonal inequality, the estimate of the between-region (group) component of the decomposition is based on setting the incomes of all economies (individuals) belonging to a region (group) equal to that of the regional (group) average of these per capita incomes. Now, however, intra-regional inequality between economies within the same region is explicitly considered in the second component.[3]

Once we have covered the decomposition conceptually, the technical implementation is straightforward thanks to the `inequality` package of Pysal, and the `TheilD` class:

```
theil_dr = inequality.theil.TheilD(
    pci_df[years].values, pci_df.Region
)
```

The `theil_dr` object has the between and within group components stored in the `bg` and `wg` attributes, respectively. For example, the "between" component for each year is computed as:

```
theil_dr.bg
```

```
array([0.00914353, 0.00822696, 0.00782675, 0.00768201, 0.
↪01022634,
       0.0081274 , 0.00783943, 0.00572543, 0.00560271, 0.
↪0054971 ,
       0.00511791, 0.00566001, 0.00486877, 0.00466134, 0.
↪00474425,
       0.00424528, 0.00428434, 0.00453503, 0.00465829, 0.
↪00456699,
       0.00467363, 0.00412391, 0.00366334, 0.00342112, 0.
↪00327131,
       0.00312475, 0.00326071, 0.00359733, 0.00327591, 0.
↪00363014,
       0.00382409, 0.00436261, 0.00399156, 0.00402506, 0.
↪00397   ,
       0.00394649, 0.00353368, 0.00362698, 0.00400508, 0.
↪00449814,
       0.0043533 , 0.00470988, 0.0063954 , 0.00642426, 0.
↪00694236,
       0.00644971, 0.00591871, 0.00554072, 0.00528702])
```

If we store these components in our results table as we have been doing:

```
inequalities["theil_between"] = theil_dr.bg
inequalities["theil_within"] = theil_dr.wg
```

[3] The regional decomposition does not involve weighting the regions by their respective population. See [Glu18] for further details.

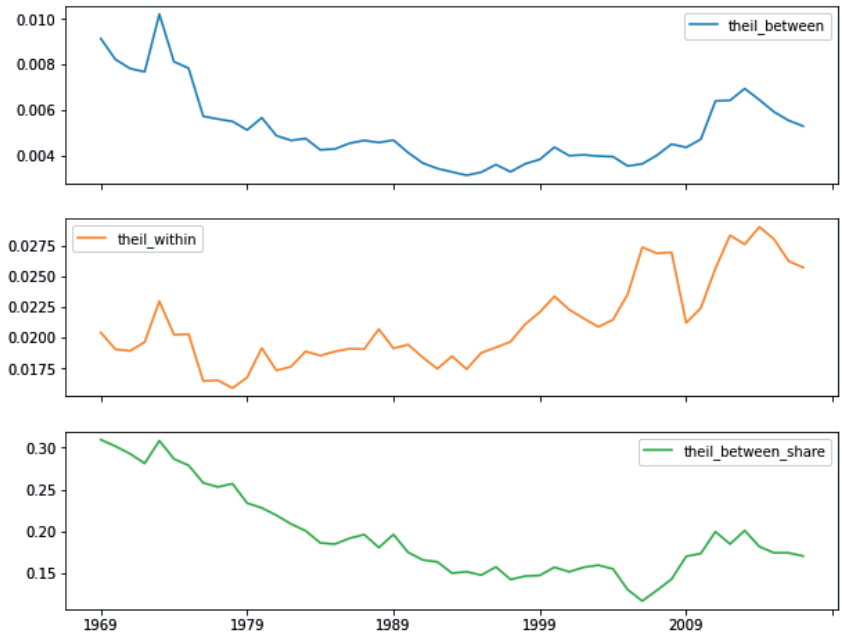


Fig. 9.12: Inequality indices (Gini, Theil), shown alongside Moran's I, with the Theil decomposition into between-region and within-region components at bottom.

Inference on these decompositions can be done using the `inequality.theil.TheilDSim` class, but we omit that here for brevity and report that, like the Moran's I, all of the Theil decompositions are statistically significant.

Since the within and between components are interpreted as shares of the overall Theil index, we can compute the share of the Theil index due to the between-region inequality.

```
inequalities["theil_between_share"] = (
    inequalities["theil_between"] / inequalities["theil"]
)
```

We can visualize the three time series (Figure 9.12):

```
inequalities[
    ["theil_between", "theil_within", "theil_between_share"]
].plot(subplots=True, figsize=(10, 8));
```

The between-region share of inequality is at its lowest in the mid-2000s, not in the mid-1990s. This suggests that regional differences were very important in the 1970s and 1980s, but this importance has been waning, relative to the inequality *within* U.S.

Census Regions. The ratio also generally shares the same pattern, but it does not see minima in the same places.

9.5.3 Spatializing classic measures

While regional decompositions are useful, they do not tell the whole story. Indeed, a "region" is just a special kind of group; its "geography" is only made manifest through group membership (*is the county "in" the region or not?*). This kind of "place-based" thinking, while geographic, is not necessarily *spatial*. It does not incorporate the notions of distance or proximity into the study of inequality. The geographical locations of the regions could be rearranged without impact, so long as the group membership structure is maintained. While, arguably, shuffling regions around means they are no longer "regions," the statistical methods would be no different.

The final approach we review here is an explicit integration of space within traditional, non-spatial measure. In particular, we consider a *spatialized* version of the Gini coefficient, introduced by [RS12]. The spatial Gini is designed to consider the role of spatial adjacency in a decomposition of the traditional Gini. The original index can be formulated focusing on the set of pairwise absolute differences in incomes:

$$G = \frac{\sum_i \sum_j |y_i - y_j|}{2n^2 \bar{y}}$$

where n is the number of observations, and $\bar{y}$ is the mean regional income. Focusing on the set of pairwise absolute differences in income, we can de-compose this into the set of differences between "nearby" observations and the set of differences among "distant" observations. This is the main conceptual point of the "spatial Gini" coefficient. This decomposition works similarly to the regional decomposition of the Theil index:

$$\sum_i \sum_j |y_i - y_j| = \sum_i \sum_j \underbrace{(w_{ij} |y_i - y_j|)}_{\text{near differences}} + \underbrace{((1 - w_{ij}) |y_i - y_j|)}_{\text{far differences}}$$

In this decomposition, w_{ij} is a binary variable that is 1 when i and j are neighbors, and is zero otherwise. Recalling the spatial weights matrices from *Chapter 4*, this can be used directly from a spatial weights matrix.[4] Thus, with this decomposition, the spatial Gini can be stated as

$$G = \frac{\sum_i \sum_j w_{i,j} |x_i - x_j|}{2n^2 \bar{x}} + \frac{\sum_i \sum_j (1 - w_{i,j}) |x_i - x_j|}{2n^2 \bar{x}}$$

with the first term being the component among neighbors and the second term being the component among non-neighbors. The "spatial Gini", then, is the first component that describes the differences between nearby observations.

[4] However, non-binary spatial weights matrices require a correction factor and are not discussed here.

The spatial Gini allows for a consideration of spatial dependence in inequality. If spatial dependence is very strong and positive, incomes are very similar among nearby observations, so the inequality of "near" differences will be small. Most of the inequality in the society will be driven by disparities in income between distant places. In contrast, when dependence is very weak (or even negative), then the two components may equalize. Inference on the spatial Gini can be based on random spatial permutations of the income values, as we have seen elsewhere in this book. This tests whether the distribution of the components is different from that obtained when incomes are randomly distributed across the map.

The spatial Gini also provides a useful complement to the regional decomposition used in the Theil statistic. The latter does not consider pairwise relationships between observations, while the spatial Gini does. By considering the pairwise relationships between observations, the Gini coefficient is more sensitive and can also be more strongly affected by small groups of significantly wealthy observations.

We can estimate spatial Gini coefficients using the `Gini_Spatial` class:

```
from inequality.gini import Gini_Spatial
```

First, since the spatial Gini requires binary spatial weights, we will ensure this is so before proceeding:

```
wq.transform = "B"
```

Then, the spatial Gini can be computed from an income vector and the spatial weights describing adjacency among the observations.

```
gs69 = Gini_Spatial(pci_df["1969"], wq)
```

The aspatial Gini is stored in the `g` attribute, just like for the aspatial class:

```
gs69.g
```

```
0.13556175504269904
```

The share of the overall Gini coefficient that is due to the "far" differences is stored in the `wcg` share:

```
gs69.wcg_share
```

```
0.13541750749645268
```

The p-value for this tests whether the component measuring inequality among neighbors is larger (or smaller) than that would have occurred if incomes were shuffled randomly around the map:

```
gs69.p_sim
```

```
0.01
```

The value is statistically significant for 1969, indicating that inequality between neighboring pairs of counties is different from the inequality between county pairs that are not geographically proximate.

We can apply the same statistic over each year in the sample using the function-by-column approach as before. In this case, we want to return the statistic itself, as well as the decomposition between variation in the neighbors and that for non-neighbors, and the pseudo-P-values:

```
def gini_spatial_by_col(incomes, weights):
    gs = Gini_Spatial(incomes, weights)
    denom = 2 * incomes.mean() * weights.n ** 2
    near_diffs = gs.wg / denom
    far_diffs = gs.wcg / denom
    out = pandas.Series(
        {
            "gini": gs.g,
            "near_diffs": near_diffs,
            "far_diffs": far_diffs,
            "p_sim": gs.p_sim,
        }
    )
    return out
```

Inference on this estimator is computationally demanding, since the pairwise differences have to be recomputed every permutation, so the following cell takes some time to complete execution:

```
%%time
spatial_gini_results = (
    pci_df[years].apply(gini_spatial_by_col, weights=wq).T
)
```

```
CPU times: user 1min 43s, sys: 52.8 ms, total: 1min 43s
Wall time: 1min 43s
```

```
spatial_gini_results.head()
```

```
          gini  near_diffs  far_diffs  p_sim
1969  0.135562    0.000144   0.135418   0.01
1970  0.130076    0.000141   0.129935   0.01
1971  0.128540    0.000142   0.128398   0.01
```

(continued on next page)

(continued from previous page)

```
1972  0.129126    0.000140   0.128985   0.01
1973  0.142166    0.000145   0.142021   0.01
```

The p-values are always small, suggesting that the contribution of the local ties is always smaller than what would be expected if incomes were distributed randomly in the map.[5] We can compute the percent of times the p-value is smaller than a threshold using the mean:

```
(spatial_gini_results.p_sim < 0.05).mean()
```

```
1.0
```

While it may appear that the component due to "near differences" is quite small, this has two reasons. First, the number of "nearby" pairs is less than 0.2% of all pairs of observations:

```
wq.pct_nonzero
```

```
0.19385366975502275
```

Second, when spatial dependence is high, nearby observations will be similar. So, each "near difference" will also be small. Adding together a small number of small observations will generally be small, relative to the large differences between distant observations. Thus, small values of the "near" distances are indicative of spatial dependence. Indeed, we can see visually (Figure 9.13) that as the spatial dependence weakens, the `near_diffs` get larger:

```
inequalities["near_diffs"] = spatial_gini_results.near_diffs

inequalities[["near_diffs", "I"]].plot.line(
    subplots=True, figsize=(15, 6)
);
```

9.6 Conclusion

Inequality is an important social phenomenon, and its geography is a growing concern for social scientists. Geographical disparities in well-being have been pointed to as a major driver behind the rise of right-wing populist movements in the U.S. and Europe [RP18]. Thus, understanding the nature of these disparities and their evolution is a challenge for both science and policy.

[5] While it is possible that the "near differences" component could be *larger* than expected, that would imply negative spatial dependence, which is generally rare in empirical work.

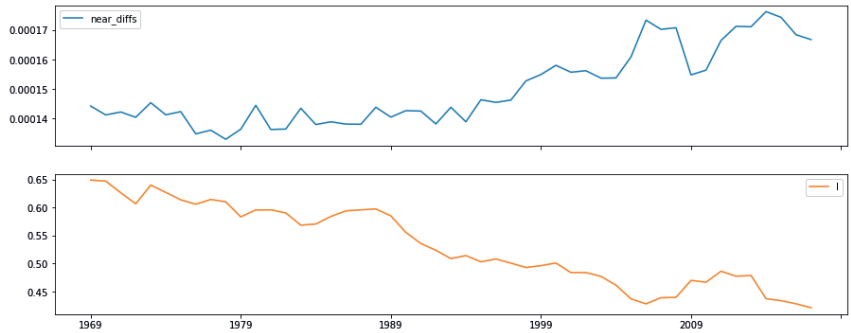


Fig. 9.13: Relationship between the 'near differences' term of the spatial Gini coefficient and Moran's I. The top, as a measure of spatial dissimilarity, should move in an opposite direction to the bottom, which measures spatial similarity (albeit in a different fashion).

This chapter discusses methods to assess inequality, as well as to examine its spatial and regional structure. We have seen the Gini coefficient and Theil index as examples of global measures to summarize the overall level of inequality. As is often the case in many areas of spatial analysis, the straightforward adoption of methods from economics and sociology to spatial data can often be fruitful but, at the same time, can miss key elements of the spatial story. In the context of spatial income disparities, we have highlighted the differences between personal and regional inequality. From this vantage, we have reviewed three approaches to incorporate geography and space in the study of inequality. Together, this gives us a good sense of how inequality manifests geographically, and how it is (possibly) distinct from other kinds of spatial measures, such as those for spatial autocorrelation discussed in Chapters *6* and *7*. Furthermore, in this chapter we have dipped our toes in spatiotemporal data, exploring how spatial patterns change and evolve over time.

Before leaving the topic of spatial inequality, we note that there is much more that can be said about inequality and related concepts. Inequality is generally concerned with the static snapshot of the regional income distribution and the shares of that distribution that each region holds. Those shares are reflected in the variance or spread of the distribution. However, this is only one moment of the distribution, and a comprehensive understanding of disparities requires analysis of the distribution's location (mean) and shape (modes, kurtosis, skewness) as well as dispersion. Moreover, movements of individual regions within the distribution over time, or what is referred to as *spatial income mobility*, are critical to our understanding of the dynamics of spatial disparities. Full consideration of these concepts is beyond the scope of this chapter. Interested readers are directed to [Rey14] as an entry point to these more advanced topics.

9.7 Questions

1. Why is the study of regional income inequality important? In what ways is the study of regional income inequality different from the study of personal income inequality?
2. Given that the Theil and Gini statistics appear to have similar time paths, why would a researcher choose to use both measures when analyzing the dynamics of regional disparities? Why not just one or the other?
3. What aspects of a regional income distribution are not captured by a Theil or Gini coefficient? Why are these omissions important, and what approaches might be used to address these limitations?
4. How might the measure of inter-regional income inequality be affected by the choice of the regionalization scheme (i.e., how the different spatial units are grouped to form regions)?
5. What is the relationship between spatial income inequality and the spatial dependence of regional incomes?

9.8 Next steps

The literature on regional inequality has exploded over the last several decades. For recent reviews of the causes and policy responses, see the following:

Cörvers, Frank and Ken Mayhew. 2021. "Regional inequalities: causes and cures." *Oxford Review of Economic Policy* 37(1): 1-16.[CorversM21]

Rodríguez-Pose, Andrés. 2018. "The revenge of the places that don't matter (and what to do about it)." *Cambridge Journal of Regions, Economy and Society*, 11(1): 189-20.[RP18]

Methodologically, spatial analysis of regional disparities is generally covered in two strands of the literature. For work on spatial econometric modeling of convergence and divergence, see:

Arbia, Giuseppe. 2006. *Spatial econometrics: statistical foundations and applications to regional convergence*. Springer Science & Business Media.[Arb06]

The second branch of the methodological literature focuses on exploratory spatial data analysis of inequality and is reviewed in:

Rey, Sergio J. and Julie Le Gallo. 2009. "Spatial anlaysis of economic convergence." In Terry C. Mills and Kerry Patterson (eds.) *Palgrave Handbook of Econometrics*, Palgrave, pages 1251-1290.[RLG09]

10

Clustering and Regionalization

The world's hardest questions are complex and multi-faceted. Effective methods to learn from data recognize this. Many questions and challenges are inherently multidimensional; they are affected, shaped, and defined by many different components all acting simultaneously. In statistical terms, these processes are called *multivariate processes*, as opposed to *univariate processes*, where only a single variable acts at once. Clustering is a fundamental method of geographical analysis that draws insights from large, complex multivariate processes. It works by finding similarities among the many dimensions in a multivariate process, condensing them down into a simpler representation. Thus, through clustering, a complex and difficult to understand process is recast into a simpler one that even non-technical audiences can use.

10.1 Introduction

Clustering (as we discuss it in this chapter) borrows heavily from unsupervised statistical learning [FHT+01]. Often, clustering involves sorting observations into groups without any prior idea about what the groups are (or, in machine learning jargon, without any labels, hence the *unsupervised* name). These groups are delineated so that members of a group should be more similar to one another than they are to members of a different group. Each group is referred to as a *cluster* while the process of assigning objects to groups is known as *clustering*. If done well, these clusters can be characterized by their *profile*, a simple summary of what members of a group are like in terms of the original multivariate phenomenon.

Since a good cluster is more similar internally than it is to any other cluster, these cluster-level profiles provide a convenient shorthand to describe the original complex multivariate phenomenon we are interested in. Observations in one group may have consistently high scores on some traits but low scores on others. The analyst only needs

to look at the profile of a cluster in order to get a good sense of what all the observations in that cluster are like, instead of having to consider all of the complexities of the original multivariate process at once. Throughout data science, and particularly in geographic data science, clustering is widely used to provide insights on the (geographic) structure of complex multivariate (spatial) data.

In the context of explicitly spatial questions, a related concept, the *region*, is also instrumental. A *region* is similar to a *cluster*, in the sense that all members of a region have been grouped together, and the region should provide a shorthand for the original data within the region. For a region to be analytically useful, its members also should display stronger similarity to each other than they do to the members of other regions. However, regions are more complex than clusters because they combine this similarity in profile with additional information about the location of their members: they should also describe a clear geographic area. In short, regions are like clusters (since they have a consistent profile) where all their members are geographically consistent.

The process of creating regions is called regionalization [DRSurinach07]. A regionalization is a special kind of clustering where the objective is to group observations which are similar in their statistical attributes, but also in their spatial location. In this sense, regionalization embeds the same logic as standard clustering techniques, but also it applies a series of geographical constraints. Often, these constraints relate to connectivity: two candidates can only be grouped together in the same region if there exists a path from one member to another member that never leaves the region. These paths often model the spatial relationships in the data, such as contiguity or proximity. However, connectivity does not always need to hold for all regions, and in certain contexts it makes sense to relax connectivity or to impose different types of geographic constraints.

In this chapter we consider clustering techniques and regionalization methods. In the process, we will explore the socioeconomic characteristics of neighborhoods in San Diego. We will extract common patterns from the cloud of multi-dimensional data that the Census Bureau produces about small areas through the American Community Survey. We begin with an exploration of the multivariate nature of our dataset by suggesting some ways to examine the statistical and spatial distribution before carrying out any clustering. Focusing on the individual variables, as well as their pairwise associations, can help guide the subsequent application of clusterings or regionalizations. We then consider geodemographic approaches to clustering—the application of multivariate clustering to spatially referenced demographic data. Two popular clustering algorithms are employed: k-means and Ward's hierarchical method. As we will see, mapping the spatial distribution of the resulting clusters reveals interesting insights on the socioeconomic structure of the San Diego metropolitan area. We also see that in many cases, clusters are spatially fragmented. That is, a cluster may actually consist of different areas that are not spatially connected. Indeed, some clusters will have their members strewn all over the map. This will illustrate why connectivity might be important when building insight about spatial data, since these clusters will not at all provide intelligible regions. With this insight in mind, we will move on to regionalization, exploring different approaches that incorporate geographical constraints into the

exploration of the social structure of San Diego. Applying a regionalization approach is not always required, but it can provide additional insights into the spatial structure of the multivariate statistical relationships that traditional clustering is unable to articulate.

```
from esda.moran import Moran
from libpysal.weights import Queen, KNN
import seaborn
import pandas
import geopandas
import numpy
import matplotlib.pyplot as plt
```

10.2 Data

We return to the San Diego tracts dataset we have used earlier in the book. In this case, we will not only rely on its polygon geometries, but also on its attribute information. The data comes from the American Community Survey (ACS) from 2017. Let us begin by reading in the data.

```
# Read file
db = geopandas.read_file("../data/sandiego/sandiego_tracts.
↪gpkg")
```

To make things easier later on, let us collect the variables we will use to characterize census tracts. These variables capture different aspects of the socioeconomic reality of each area and, taken together, provide a comprehensive characterization of San Diego as a whole. We thus create a list with the names of the columns we will use later on:

```
cluster_variables = [
    "median_house_value",  # Median house value
    "pct_white",  # % tract population that is white
    "pct_rented",  # % households that are rented
    "pct_hh_female",  # % female-led households
    "pct_bachelor",  # % tract population with a Bachelors␣
↪degree
    "median_no_rooms",  # Median n. of rooms in the tract's␣
↪households
    "income_gini",  # Gini index measuring tract wealth␣
↪inequality
    "median_age",  # Median age of tract population
    "tt_work",  # Travel time to work
]
```

Let's start building up our understanding of this dataset through both visual and statistical summaries. The first stop is considering the spatial distribution of each variable alone. This will help us draw a picture of the multi-faceted view of the tracts we want

to capture with our clustering. Let's use (quantile) choropleth maps for each attribute and compare them side-by-side (Figure 10.1):

```
f, axs = plt.subplots(nrows=3, ncols=3, figsize=(12, 12))
# Make the axes accessible with single indexing
axs = axs.flatten()
# Start a loop over all the variables of interest
for i, col in enumerate(cluster_variables):
    # select the axis where the map will go
    ax = axs[i]
    # Plot the map
    db.plot(
        column=col,
        ax=ax,
        scheme="Quantiles",
        linewidth=0,
        cmap="RdPu",
    )
    # Remove axis clutter
    ax.set_axis_off()
    # Set the axis title to the name of variable being plotted
    ax.set_title(col)
# Display the figure
plt.show()
```

Several visual patterns jump out from the maps, revealing both commonalities as well as differences across the spatial distributions of the individual variables. Several variables tend to increase in value from the east to the west (`pct_rented`, `median_house_value`, `median_no_rooms`, and `tt_work`), while others have a spatial trend in the opposite direction (`pct_white`, `pct_hh_female`, `pct_bachelor`, `median_age`). This will help show the strengths of clustering; when variables have different spatial distributions, each variable contributes distinct information to the profiles of each cluster. However, if all variables display very similar spatial patterns, the amount of useful information across the maps is actually smaller than it appears, so cluster profiles may be much less useful as well. It is also important to consider whether the variables display any spatial autocorrelation, as this will affect the spatial structure of the resulting clusters.

Recall from *Chapter 6* that Moran's I is a commonly used measure for global spatial autocorrelation. We can use it to formalize some of the intuitions built from the maps. Recall from earlier in the book that we will need to represent the spatial configuration of the data points through a spatial weights matrix. We will start with queen contiguity:

```
w = Queen.from_dataframe(db)
```

Now let's calculate Moran's I for the variables being used. This will measure the extent to which each variable contains spatial structure:

Fig. 10.1: The complex, multi-dimensional human geography of San Diego.

```
# Set seed for reproducibility
numpy.random.seed(123456)
# Calculate Moran's I for each variable
mi_results = [
    Moran(db[variable], w) for variable in cluster_variables
]
# Structure results as a list of tuples
mi_results = [
    (variable, res.I, res.p_sim)
    for variable, res in zip(cluster_variables, mi_results)
]
# Display on table
table = pandas.DataFrame(
    mi_results, columns=["Variable", "Moran's I", "P-value"]
).set_index("Variable")
table
```

```
                    Moran's I  P-value
Variable
median_house_value   0.646618    0.001
pct_white            0.602079    0.001
pct_rented           0.451372    0.001
pct_hh_female        0.282239    0.001
pct_bachelor         0.433082    0.001
median_no_rooms      0.538996    0.001
income_gini          0.295064    0.001
median_age           0.381440    0.001
tt_work              0.102748    0.001
```

Each of the variables displays significant positive spatial autocorrelation, suggesting clear spatial structure in the socioeconomic geography of San Diego. This means it is likely the clusters we find will have a non-random spatial distribution.

Spatial autocorrelation only describes relationships between observations for a single attribute at a time. So, the fact that all of the clustering variables are positively autocorrelated does not say much about how attributes co-vary over space. To explore cross-attribute relationships, we need to consider the spatial correlation between variables. We will take our first dip in this direction exploring the bivariate correlation in the maps of covariates themselves. This would mean that we would be comparing each pair of choropleths to look for associations and differences. Given there are nine attributes, there are 36 pairs of maps that must be compared.

This would be too many maps to process visually. Instead, we focus directly on the bivariate relationships between each pair of attributes, devoid for now of geography, and use a scatterplot matrix (Figure 10.2).

```
_ = seaborn.pairplot(
    db[cluster_variables], kind="reg", diag_kind="kde"
)
```

Two different types of plots are contained in the scatterplot matrix. On the diagonal are the density functions for the nine attributes. These allow for an inspection of the univariate distribution of the values for each attribute. Examining these we see that our selection of variables includes some that are negatively skewed (`pct_white` and `pct_hh_female`) as well as positively skewed (`median_house_value`, `pct_bachelor`, and `tt_work`).

The second type of visualization lies in the off-diagonal cells of the matrix; these are bivariate scatterplots. Each cell shows the association between one pair of variables. Several of these cells indicate positive linear associations (`median_age` vs. `median_house_value`, `median_house_value` vs. `median_no_rooms`) while other cells display negative correlation (`median_house_value` vs. `pct_rented`, `median_no_rooms` vs. `pct_rented`, and `median_age` vs. `pct_rented`). The one variable that tends to have consistently weak association

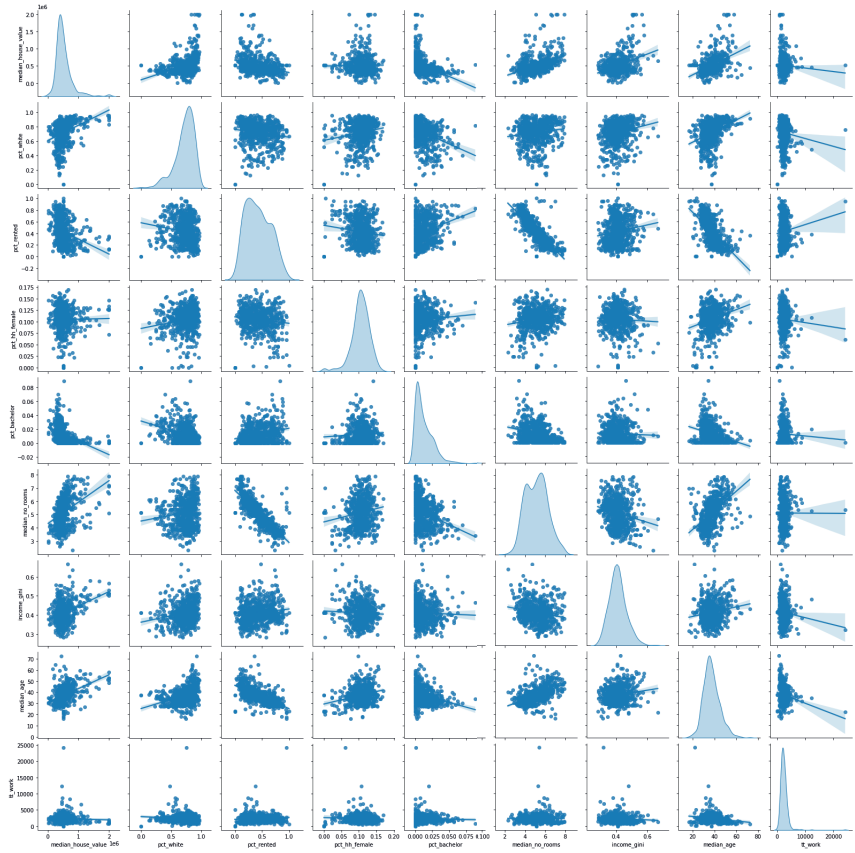


Fig. 10.2: A scatter matrix demonstrating the various pair-wise dependencies between each of the variables considered in this section. Each 'facet', or little scatterplot, shows the relationship between the vairable in that column (as its horizontal axis) and that row (as its vertical axis). Since the diagonal represents the situation where the row and column have the same variable, it instead shows the univariate distribution of that variable.

with the other variables is `tt_work`, and in part this appears to reflect its rather concentrated distribution as seen on the lower right diagonal corner cell.

Indeed, this kind of concentration in values is something you need to be very aware of in clustering contexts. *Distances between datapoints* are of paramount importance in clustering applications. In fact, (dis)similarity between observations is calculated as the statistical distance between themselves. Because distances are sensitive to the units of measurement, cluster solutions can change when you re-scale your data.

For example, say we locate an observation based on only two variables: house price and Gini coefficient. In this case:

```
db[["income_gini", "median_house_value"]].head()
```

```
   income_gini  median_house_value
0       0.5355       732900.000000
1       0.4265       473800.000000
2       0.4985       930600.000000
3       0.4003       478500.000000
4       0.3196       515570.896382
```

The distance between observations in terms of these variates can be computed easily using `scikit-learn`:

```
from sklearn import metrics
```

```
metrics.pairwise_distances(
    db[["income_gini", "median_house_value"]].head()
).round(4)
```

```
array([[     0.    , 259100.    , 197700.    , 254400.    ,␣
↪217329.1036],
       [259100.    ,      0.    , 456800.    ,   4700.    ,␣
↪41770.8964],
       [197700.    , 456800.    ,      0.    , 452100.    ,␣
↪415029.1036],
       [254400.    ,   4700.    , 452100.    ,      0.    ,␣
↪37070.8964],
       [217329.1036,  41770.8964, 415029.1036,  37070.8964,␣
↪    0.    ]])
```

In this case, we know that the housing values are in the hundreds of thousands, but the Gini coefficient (which we discussed in the previous chapter) is constrained to fall between zero and one. So, for example, the distance between the first two observations is nearly totally driven by the difference in median house value (which is 259100 dollars) and ignores the difference in the Gini coefficient (which is about .11). Indeed, a change of a single dollar in median house value will correspond to *the maximum possible* difference in Gini coefficients. So, a clustering algorithm that uses this distance to determine classifications will pay a lot of attention to median house value, but very little to the Gini coefficient!

Therefore, *as a rule*, we standardize our data when clustering. There are many different methods of standardization offered in the `sklearn.preprocessing` module, and these map onto the main methods common in applied work. We review a small subset of them here. The `scale()` method subtracts the mean and divides by the standard deviation:

$$z = \frac{x_i - \bar{x}}{\sigma_x}$$

This "normalizes" the variate, ensuring the rescaled variable has a mean of zero and a variance of one. However, the variable can still be quite skewed, bimodal, etc. and insofar as the mean and variance may be affected by outliers in a given variate, the scaling can be too dramatic. One alternative intended to handle outliers better is `robust_scale()`, which uses the median and the inter-quartile range in the same fashion:

$$z = \frac{x_i - \tilde{x}}{\lceil x \rceil_{75} - \lceil x \rceil_{25}}$$

where $\lceil x \rceil_p$ represents the value of the pth percentile of x. Alternatively, sometimes it is useful to ensure that the maximum of a variate is 1 and the minimum is zero. In this instance, the `minmax_scale()` is appropriate:

$$z = \frac{x - min(x)}{max(x - min(x))}$$

In most clustering problems, the `robust_scale()` or `scale()` methods are useful. Further, transformations of the variate (such as log-transforming or Box-Cox transforms) can be used to non-linearly rescale the variates, but these generally should be done before the above kinds of scaling. Here, we will analyze robust-scaled variables. To detach the scaling from the analysis, we will perform the former now, creating a scaled view of our data which we can use later for clustering. For this, we import the scaling method:

```
from sklearn.preprocessing import robust_scale
```

And create the `db_scaled` object which contains only the variables we are interested in, scaled:

```
db_scaled = robust_scale(db[cluster_variables])
```

In conclusion, exploring the univariate and bivariate relationships is a good first step into building a fully multivariate understanding of a dataset. To take it to the next level, we would want to know to what extent these pair-wise relationships hold across different attributes, and whether there are patterns in the "location" of observations within the scatterplots. For example, do nearby dots in each scatterplot of the matrix represent the *same* observations? These types of questions are exactly what clustering helps us explore.

10.3 Geodemographic clusters in san diego census tracts

Geodemographic analysis is a form of multivariate clustering where the observations represent geographical areas [WB18]. The output of these clusterings is nearly always mapped. Altogether, these methods use multivariate clustering algorithms to construct

a known number of clusters (k), where the number of clusters is typically much smaller than the number of observations to be clustered. Each cluster is given a unique label, and these labels are mapped. Using the clusters' profile and label, the map of labels can be interpreted to get a sense of the spatial distribution of socio-demographic traits. The power of (geodemographic) clustering comes from taking statistical variation across several dimensions and compressing it into a single categorical one that we can visualize through a map. To demonstrate the variety of approaches in clustering, we will show two distinct but very popular clustering algorithms: k-means and Ward's hierarchical method.

10.3.1 K-means

K-means is probably the most widely used approach to cluster a dataset. The algorithm groups observations into a pre-specified number of clusters so that each observation is closer to the mean of its own cluster than it is to the mean of any other cluster. The k-means problem is solved by iterating between an assignment step and an update step. First, all observations are randomly assigned one of the k labels. Next, the multivariate mean over all covariates is calculated for each of the clusters. Then, each observation is reassigned to the cluster with the closest mean. If the observation is already assigned to the cluster whose mean it is closest to, the observation remains in that cluster. This assignment-update process continues until no further reassignments are necessary.

The nature of this algorithm requires us to select the number of clusters we want to create. The right number of clusters is unknown in practice. For illustration, we will use $k = 5$ in the `KMeans` implementation from `scikit-learn`.

```
# Initialize KMeans instance
from sklearn.cluster import KMeans
```

This illustration will also be useful as virtually every algorithm in `scikit-learn`, the (Python) standard library for machine learning, can be run in a similar fashion. To proceed, we first create a `KMeans` clusterer object that contains the description of all the parameters the algorithm needs (in this case, only the number of clusters):

```
# Initialize KMeans instance
kmeans = KMeans(n_clusters=5)
```

Next, we set the seed for reproducibility and call the `fit` method to compute the algorithm specified in `kmeans` to our scaled data:

```
# Set the seed for reproducibility
numpy.random.seed(1234)
# Run K-Means algorithm
k5cls = kmeans.fit(db_scaled)
```

Now that the clusters have been assigned, we can examine the label vector, which records the cluster to which each observation is assigned:

```
# Print first five labels
k5cls.labels_[:5]
```

```
array([2, 1, 3, 1, 4], dtype=int32)
```

In this case, the first observation is assigned to cluster 2, the second and fourth ones are assigned to cluster 1, the third to number 3 and the fifth receives the label 4. It is important to note that the integer labels should be viewed as denoting membership only — the numerical differences between the values for the labels are meaningless. The profiles of the various clusters must be further explored by looking at the values of each dimension. But, before we do that, let's make a map.

10.3.2 Spatial distribution of clusters

Having obtained the cluster labels, Figure 10.3 displays the spatial distribution of the clusters by using the labels as the categories in a choropleth map. This allows us to quickly grasp any sort of spatial pattern the clusters might have. Since clusters represent areas with similar characteristics, mapping their labels allows to see to what extent similar areas tend to have similar locations. Thus, this gives us one map that incorporates the information from all nine covariates.

```
# Assign labels into a column
db["k5cls"] = k5cls.labels_
# Set up figure and ax
f, ax = plt.subplots(1, figsize=(9, 9))
# Plot unique values choropleth including
# a legend and with no boundary lines
db.plot(
    column="k5cls", categorical=True, legend=True,␣
↪linewidth=0, ax=ax
)
# Remove axis
ax.set_axis_off()
# Display the map
plt.show()
```

The map provides a useful view of the clustering results; it allows for a visual inspection of the extent to which Tobler's first law of geography is reflected in the multivariate clusters. Recall that the law implies that nearby tracts should be more similar to one another than tracts that are geographically more distant from each other. We can see evidence of this in our cluster map, since clumps of tracts with the same color emerge. However, this visual inspection is obscured by the complexity of the underlying spatial units. Our eyes are drawn to the larger polygons in the eastern part of the county, giving

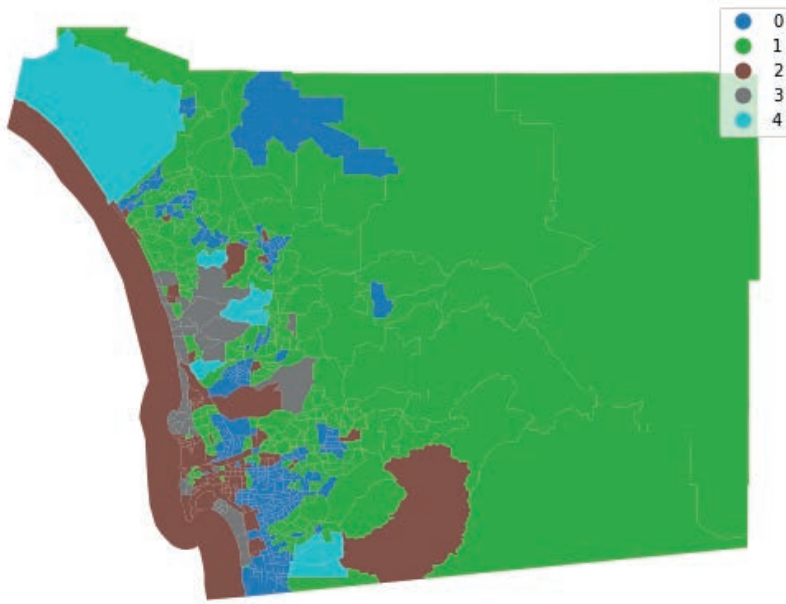


Fig. 10.3: Clusters in the socio-demographic data, found using K-means with k=5. Note that the large eastern part of San Diego actually contains few observations, since those tracts are larger.

the impression that more observations fall into that cluster. While this seems to be true in terms of land area (and we will verify this below), there is more to the cluster pattern than this. Because the tract polygons are all different sizes and shapes, we cannot solely rely on our eyes to interpret the spatial distribution of clusters.

10.3.3 Statistical analysis of the cluster map

To complement the geovisualization of the clusters, we can explore the statistical properties of the cluster map. This process allows us to delve into what observations are part of each cluster and what their characteristics are. This gives us the profile of each cluster so we can interpret the meaning of the labels we've obtained. We can start, for example, by considering cardinality, or the count of observations in each cluster:

```
# Group data table by cluster label and count observations
k5sizes = db.groupby("k5cls").size()
k5sizes
```

```
k5cls
0    248
1    244
2     88
3     39
4      9
dtype: int64
```

There are substantial differences in the sizes of the five clusters, with two very large clusters (0,1), one medium-sized cluster (2), and two small clusters (3, 4). Cluster 0 is the largest when measured by the number of assigned tracts, but cluster 1 is not far behind. This confirms our discussion from the map above, where we got the visual impression that tracts in cluster 1 seemed to have the largest area by far, but we missed exactly how large cluster 0 would be.

Let's see if this is the case. One way to do so involves using the `dissolve` operation in `geopandas`, which combines all tracts belonging to each cluster into a single polygon object. After we have dissolved all the members of the clusters, we report the total land area of the cluster:

```
# Dissolve areas by Cluster, aggregate by summing,
# and keep column for area
areas = db.dissolve(by="k5cls", aggfunc="sum")["area_sqm"]
areas
```

```
k5cls
0     739.184478
1    8622.481814
2    1335.721492
3     315.428301
4     708.956558
Name: area_sqm, dtype: float64
```

We can then use cluster shares to show visually in Figure 10.4 a comparison of the two membership representations (based on land and tracts):

```
# Bind cluster figures in a single table
area_tracts = pandas.DataFrame({"No. Tracts": k5sizes, "Area
↪": areas})
# Convert raw values into percentages
area_tracts = area_tracts * 100 / area_tracts.sum()
# Bar plot
ax = area_tracts.plot.bar()
# Rename axes
ax.set_xlabel("Cluster labels")
ax.set_ylabel("Percentage by cluster");
```

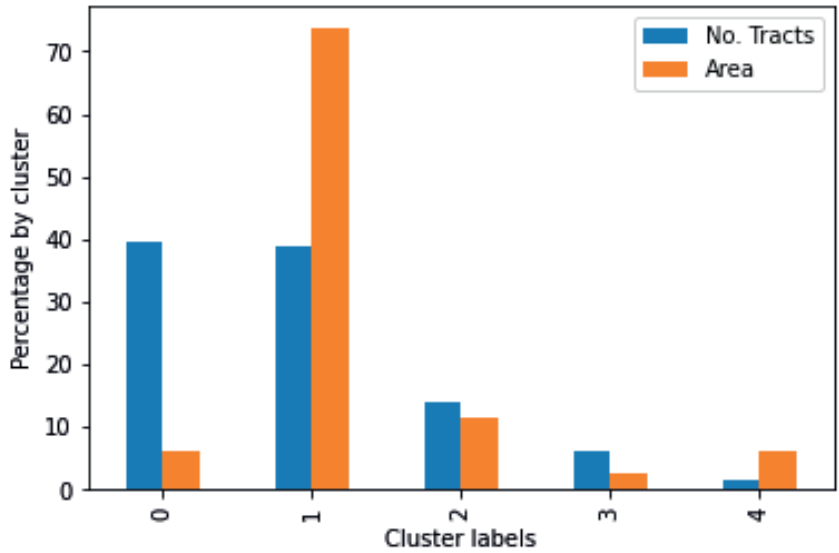


Fig. 10.4: Measuring cluster size by the number of tracts per cluster and land area per cluster.

Our visual impression from the map is confirmed: cluster 1 contains tracts that together comprise 8622 square miles (about 22,330 square kilometers) which accounts for well over half of the total land area in the county:

```
areas[1] / areas.sum()
```

```
0.7355953810798616
```

Let's move on to build the profiles for each cluster. Again, the profiles is what provides the conceptual shorthand, moving from the arbitrary label to a meaningful collection of observations with similar attributes. To build a basic profile, we can compute the (unscaled) means of each of the attributes in every cluster:

```
# Group table by cluster label, keep the variables used
# for clustering, and obtain their mean
k5means = db.groupby("k5cls")[cluster_variables].mean()
# Transpose the table and print it rounding each value
# to three decimals
k5means.T.round(3)
```

```
k5cls                          0           1           2           ␣
↪     3   \
```

(continued on next page)

(continued from previous page)

```
median_house_value  356997.331  538463.934  544888.738 ␣
↪1292905.256
pct_white                0.620       0.787       0.741       ␣
↪0.874
pct_rented               0.551       0.270       0.596       ␣
↪0.275
pct_hh_female            0.108       0.114       0.065       ␣
↪0.109
pct_bachelor             0.023       0.007       0.005       ␣
↪0.002
median_no_rooms          4.623       5.850       4.153       ␣
↪6.100
income_gini              0.400       0.397       0.449       ␣
↪0.488
median_age              32.783      42.057      32.590       ␣
↪46.356
tt_work               2238.883    2244.320    2349.511     ␣
↪1746.410

k5cls                        4
median_house_value  609385.655
pct_white                0.583
pct_rented               0.377
pct_hh_female            0.095
pct_bachelor             0.007
median_no_rooms          5.800
income_gini              0.391
median_age              33.500
tt_work                9671.556
```

Note in this case we do not use scaled measures. This is to create profiles that are easier to interpret and relate to. We see that cluster 3, for example, is composed of tracts that have the highest average `median_house_value`, and also the highest level of inequality (`income_gini`); and cluster 0 contains a younger population (`median_age`) who tend to live in housing units with fewer rooms (`median_no_rooms`). For interpretability, it is useful to consider the raw features, rather than scaled versions that the clusterer sees. However, you can also give profiles in terms of rescaled features.

Average values, however, can hide a great deal of detail and, in some cases, give wrong impressions about the type of data distribution they represent. To obtain more detailed profiles, we could use the `describe` command in `pandas`, after grouping our observations by their clusters:

```
#-------------------------------------------------------------#
# Illustrative code only, not executed
```

(continued on next page)

(continued from previous page)

```
#---------------------------------------------------------------#
# Group table by cluster label, keep the variables used
# for clustering, and obtain their descriptive summary
k5desc = db.groupby('k5cls')[cluster_variables].describe()
# Loop over each cluster and print a table with descriptives
for cluster in k5desc.T:
    print('\n\t---------\n\tCluster %i'%cluster)
    print(k5desc.T[cluster].unstack())
#---------------------------------------------------------------#
```

However, this approach quickly gets out of hand: more detailed profiles can simply return to an unwieldy mess of numbers. A better way of constructing cluster profiles is to draw the distributions of cluster members' data. To do this, we need to "tidy up" the dataset. A tidy dataset [W+14] is one where every row is an observation, and every column is a variable. This is akin to the long-format referred to in *Chapter 9*, and contrasts with the wide-format we used when looking at inequality over time. A few steps are required to tidy up our labeled data:

```
# Index db on cluster ID
tidy_db = db.set_index("k5cls")
# Keep only variables used for clustering
tidy_db = tidy_db[cluster_variables]
# Stack column names into a column, obtaining
# a "long" version of the dataset
tidy_db = tidy_db.stack()
# Take indices into proper columns
tidy_db = tidy_db.reset_index()
# Rename column names
tidy_db = tidy_db.rename(
    columns={"level_1": "Attribute", 0: "Values"}
)
# Check out result
tidy_db.head()
```

	k5cls	Attribute	Values
0	2	median_house_value	732900.000000
1	2	pct_white	0.916988
2	2	pct_rented	0.373913
3	2	pct_hh_female	0.052896
4	2	pct_bachelor	0.000000

Now we are ready to plot. Figure 10.5, generated with the code below, shows the distribution of each cluster's values for each variable. This gives us the full distributional profile of each cluster:

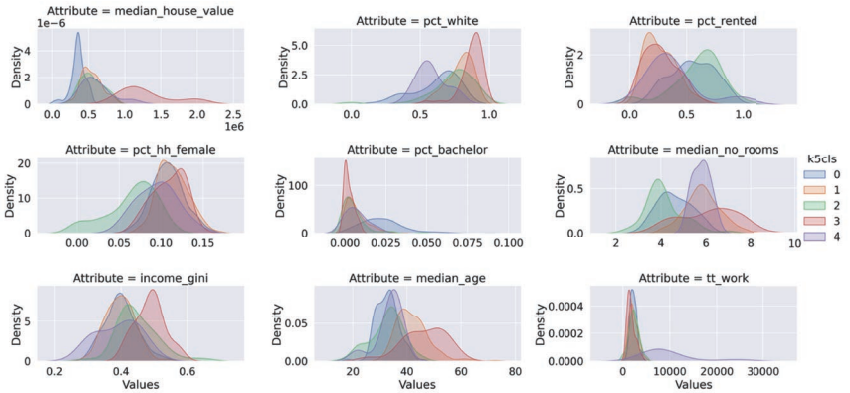


Fig. 10.5: Distributions of each variable for the different clusters.

```
# Scale fonts to make them more readable
seaborn.set(font_scale=1.5)
# Setup the facets
facets = seaborn.FacetGrid(
    data=tidy_db,
    col="Attribute",
    hue="k5cls",
    sharey=False,
    sharex=False,
    aspect=2,
    col_wrap=3,
)
# Build the plot from `sns.kdeplot`
_ = facets.map(seaborn.kdeplot, "Values", shade=True).add_
↪legend()
```

Note that we create the figure using the facetting functionality in `seaborn`, which streamlines notably the process to create multi-plot figures whose dimensions and content are data-driven. This happens in two steps: first, we set up the frame (`facets`), and then we "map" a function (`seaborn.kdeplot`) to the data, within such frame.

The figure allows us to see that, while some attributes such as the percentage of female households (`pct_hh_female`) display largely the same distribution for each cluster, others paint a much more divided picture (e.g., `median_house_value`). Taken altogether, these graphs allow us to start delving into the multi-dimensional complexity of each cluster and the types of areas behind them.

10.4 Hierarchical Clustering

As mentioned above, k-means is only one clustering algorithm. There are plenty more. In this section, we will take a similar look at the San Diego dataset using another staple of the clustering toolkit: agglomerative hierarchical clustering (AHC). Agglomerative clustering works by building a hierarchy of clustering solutions that starts with all singletons (each observation is a single cluster in itself) and ends with all observations assigned to the same cluster. These extremes are not very useful in themselves. But, in between, the hierarchy contains many distinct clustering solutions with varying levels of detail. The intuition behind the algorithm is also rather straightforward:

1. begin with everyone as part of its own cluster;
2. find the two closest observations based on a distance metric (e.g., Euclidean);
3. join them into a new cluster;
4. repeat steps (2) and (3) until reaching the degree of aggregation desired.

The algorithm is thus called "agglomerative" because it starts with individual clusters and "agglomerates" them into fewer and fewer clusters containing more and more observations each. Also, like with k-means, AHC requires the user to specify a number of clusters in advance. This is because, following from the mechanism the method has to build clusters, AHC can provide a solution with as many clusters as observations ($k = n$), or with only one ($k = 1$).

Enough of theory, let's get coding! In Python, AHC can be run with `scikit-learn` in very much the same way we did for k-means in the previous section. First we need to import it:

```
from sklearn.cluster import AgglomerativeClustering
```

In this case, we use the `AgglomerativeClustering` class and again use the `fit` method to actually apply the clustering algorithm to our data:

```
# Set seed for reproducibility
numpy.random.seed(0)
# Initialize the algorithm
model = AgglomerativeClustering(linkage="ward", n_clusters=5)
# Run clustering
model.fit(db_scaled)
# Assign labels to main data table
db["ward5"] = model.labels_
```

As above, we can check the number of observations that fall within each cluster:

```
ward5sizes = db.groupby("ward5").size()
ward5sizes
```

```
ward5
0    198
1     10
2     48
3    287
4     85
dtype: int64
```

Further, we can check the simple average profiles of our clusters:

```
ward5means = db.groupby("ward5")[cluster_variables].mean()
ward5means.T.round(3)
```

```
ward5                          0           1            2       ␣
↪    3  \
median_house_value  365932.350  625607.090  1202087.604 ␣
↪503608.711
pct_white                0.589       0.598        0.871       ␣
↪0.770
pct_rented               0.573       0.360        0.285       ␣
↪0.287
pct_hh_female            0.105       0.098        0.107       ␣
↪0.112
pct_bachelor             0.023       0.006        0.002       ␣
↪0.009
median_no_rooms          4.566       5.860        6.010       ␣
↪5.738
income_gini              0.405       0.394        0.480       ␣
↪0.394
median_age              31.955      34.250       45.196      ␣
↪40.695
tt_work               2181.970    9260.400     1766.354    ␣
↪2268.718

ward5                          4
median_house_value  503905.198
pct_white                0.766
pct_rented               0.657
pct_hh_female            0.076
pct_bachelor             0.006
median_no_rooms          3.904
income_gini              0.442
median_age              33.540
tt_work               2402.671
```

And again, we can tidy our dataset:

```
# Index db on cluster ID
tidy_db = db.set_index("ward5")
# Keep only variables used for clustering
tidy_db = tidy_db[cluster_variables]
# Stack column names into a column, obtaining
# a "long" version of the dataset
tidy_db = tidy_db.stack()
# Take indices into proper columns
tidy_db = tidy_db.reset_index()
# Rename column names
tidy_db = tidy_db.rename(
    columns={"level_1": "Attribute", 0: "Values"}
)
# Check out result
tidy_db.head()
```

	ward5	Attribute	Values
0	4	median_house_value	732900.000000
1	4	pct_white	0.916988
2	4	pct_rented	0.373913
3	4	pct_hh_female	0.052896
4	4	pct_bachelor	0.000000

And create a plot of the profiles' distributions (Figure 10.6):

```
# Setup the facets
facets = seaborn.FacetGrid(
    data=tidy_db,
    col="Attribute",
    hue="ward5",
    sharey=False,
    sharex=False,
    aspect=2,
    col_wrap=3,
)
# Build the plot as a `sns.kdeplot`
facets.map(seaborn.kdeplot, "Values", shade=True).add_
↪legend();
```

For the sake of brevity, we will not spend much time on the plots above. However, the interpretation is analogous to that of the k-means example.

On the spatial side, we can explore the geographical dimension of the clustering solution by making a map of the clusters. To make the comparison with k-means simpler, Figure 10.7, generated with the code below, displays both side-by-side:

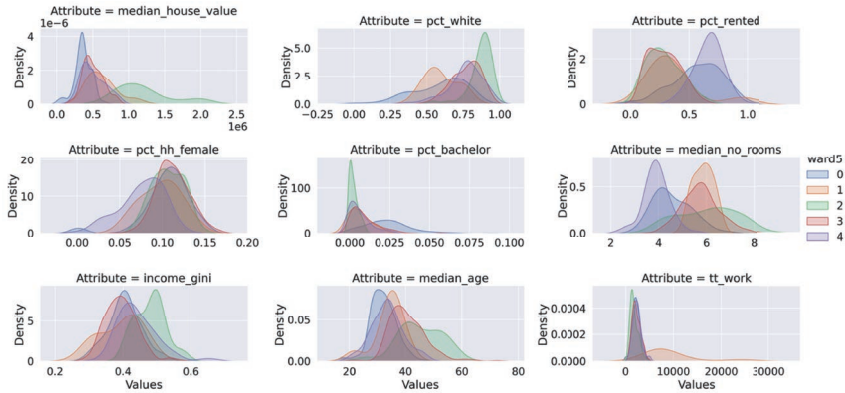


Fig. 10.6: Distributions of each variable in clusters obtained from Ward's hierarchical clustering.

```
db["ward5"] = model.labels_
# Set up figure and ax
f, axs = plt.subplots(1, 2, figsize=(12, 6))

### K-Means ###
ax = axs[0]
# Plot unique values choropleth including
# a legend and with no boundary lines
db.plot(
    column="ward5",
    categorical=True,
    cmap="Set2",
    legend=True,
    linewidth=0,
    ax=ax,
)
# Remove axis
ax.set_axis_off()
# Add title
ax.set_title("K-Means solution ($k=5$)")

### AHC ###
ax = axs[1]
# Plot unique values choropleth including
# a legend and with no boundary lines
db.plot(
    column="k5cls",
    categorical=True,
    cmap="Set3",
```

(continued on next page)

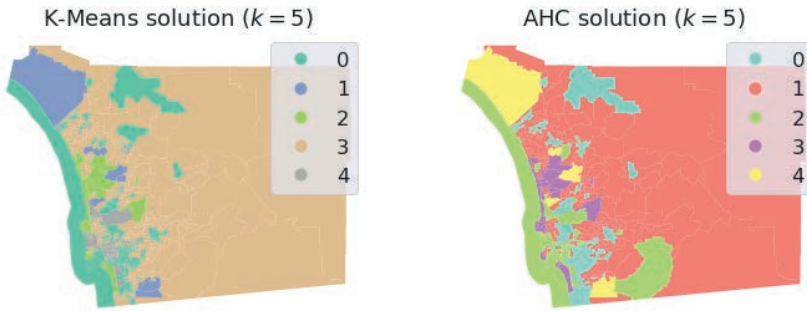


Fig. 10.7: Two clustering solutions, one for the K-means solution, and the other for Ward's hierarchical clustering. Note that colorings cannot be directly compared between the two maps.

(continued from previous page)

```
    legend=True,
    linewidth=0,
    ax=ax,
)
# Remove axis
ax.set_axis_off()
# Add title
ax.set_title("AHC solution ($k=5$)")

# Display the map
plt.show()
```

While we must remember our earlier caveat about how irregular polygons can baffle our visual intuition, a closer visual inspection of the cluster geography suggests a clear pattern: although they are not identical, both clustering solutions capture very similar overall spatial structure. Furthermore, both solutions slightly violate Tobler's law in the sense all of the clusters have disconnected components. The five multivariate clusters in each case are actually composed of many disparate geographical areas, strewn around the map according only to the structure of the data and not its geography. That is, in order to travel to every tract belonging to a cluster, we would have to journey through other clusters as well.

10.5 Regionalization: spatially constrained hierarchical clustering

10.5.1 Contiguity constraint

Fragmented clusters are not intrinsically invalid, particularly if we are interested in exploring the overall structure and geography of multivariate data. However, in some

cases, the application we are interested in might require that all the observations in a class be spatially connected. For example, when detecting communities or neighborhoods (as is sometimes needed when drawing electoral or census boundaries), they are nearly always distinct self-connected areas, unlike our clusters shown above. To ensure that clusters are not spatially fragmented, we turn to regionalization.

Regionalization methods are clustering techniques that impose a spatial constraint on clusters. In other words, the result of a regionalization algorithm contains clusters with areas that are geographically coherent, in addition to having coherent data profiles. Effectively, this means that regionalization methods construct clusters that are all internally connected; these are the *regions*. Thus, a region's members must be geographically *nested* within the region's boundaries.

This type of nesting relationship is easy to identify in the real world. Census geographies provide good examples: counties nest within states in the U.S.; or local super output areas (LSOAs) nest within middle super output areas (MSOAs) in the UK. The difference between these real-world nestings and the output of a regionalization algorithm is that the real-world nestings are aggregated according to administrative principles, while regions' members are aggregated according to statistical similarity. In the same manner as the clustering techniques explored above, these regionalization methods aggregate observations that are similar in their attributes; the profiles of regions are useful in a similar manner as the profiles of clusters. But, in regionalization, the clustering is also spatially constrained, so the region profiles and members will likely be different from the unconstrained solutions.

As in the non-spatial case, there are many different regionalization methods. Each has a different way to measure (dis)similarity, how the similarity is used to assign labels, how these labels are iteratively adjusted, and so on. However, as with clustering algorithms, regionalization methods all share a few common traits. In particular, they all take a set of input attributes and a representation of spatial connectivity in the form of a binary spatial weights matrix. Depending on the algorithm, they also require the desired number of output regions. For illustration, we will take the AHC algorithm we have just used above and apply an additional spatial constraint. In `scikit-learn`, this is done using our spatial weights matrix as a `connectivity` option. This parameter will force the agglomerative algorithm to only allow observations to be grouped in a cluster if they are also spatially connected:

```
# Set the seed for reproducibility
numpy.random.seed(123456)
# Specify cluster model with spatial constraint
model = AgglomerativeClustering(
    linkage="ward", connectivity=w.sparse, n_clusters=5
)
# Fit algorithm to the data
model.fit(db_scaled)
```

```
AgglomerativeClustering(connectivity=<628x628 sparse matrix␣
↪of type '<class 'numpy.float64'>'
        with 4016 stored elements in Compressed Sparse Row␣
↪format>,
                        n_clusters=5)
```

Let's inspect the output visually (Figure 10.8):

```
db["ward5wq"] = model.labels_
# Set up figure and ax
f, ax = plt.subplots(1, figsize=(9, 9))
# Plot unique values choropleth including a legend and with␣
↪no boundary lines
db.plot(
    column="ward5wq",
    categorical=True,
    legend=True,
    linewidth=0,
    ax=ax,
)
# Remove axis
ax.set_axis_off()
# Display the map
plt.show()
```

Introducing the spatial constraint results in fully connected clusters with much more concentrated spatial distributions. From an initial visual impression, it might appear that our spatial constraint has been violated: there are tracts for both cluster 0 and cluster 1 that appear to be disconnected from the rest of their clusters. However, closer inspection reveals that each of these tracts is indeed connected to another tract in its own cluster by very narrow shared boundaries.

10.5.2 Changing the spatial constraint

The spatial constraint in regionalization algorithms is structured by the spatial weights matrix we use. An interesting question is thus how the choice of weights influences the final region structure. Fortunately, we can directly explore the impact that a change in the spatial weights matrix has on regionalization. To do so, we use the same attribute data but replace the Queen contiguity matrix with a spatial k-nearest neighbor matrix, where each observation is connected to its four nearest observations, instead of those it touches.

```
w = KNN.from_dataframe(db, k=4)
```

With this matrix connecting each tract to the four closest tracts, we can run another AHC regionalization:

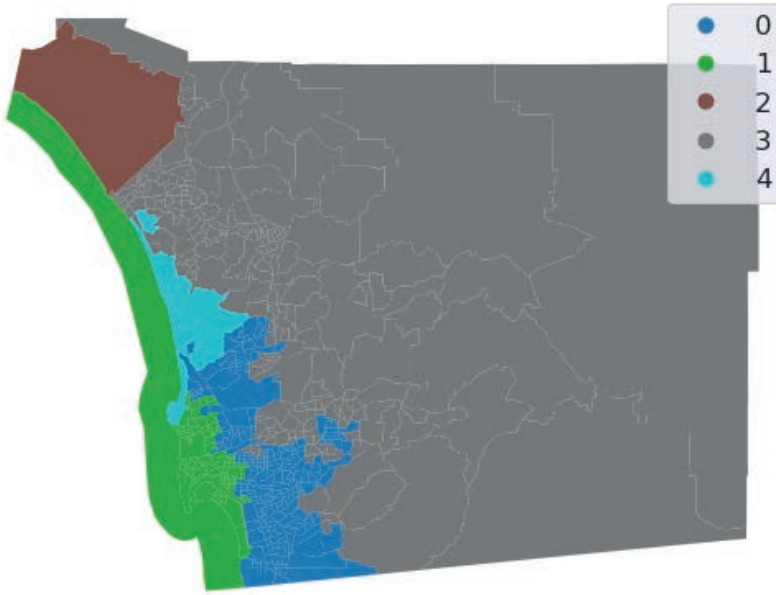


Fig. 10.8: Spatially constrained clusters, or 'regions', of San Diego using Ward's hierarchical clustering.

```
# Set the seed for reproducibility
numpy.random.seed(123456)
# Specify cluster model with spatial constraint
model = AgglomerativeClustering(
    linkage="ward", connectivity=w.sparse, n_clusters=5
)
# Fit algorithm to the data
model.fit(db_scaled)
```

```
AgglomerativeClustering(connectivity=<628x628 sparse matrix␣
↪of type '<class 'numpy.float64'>'
        with 2512 stored elements in Compressed Sparse Row␣
↪format>,
                        n_clusters=5)
```

And plot the final regions (Figure 10.9).

Even though we have specified a spatial constraint, the constraint applies to the connectivity graph modeled by our weights matrix. Therefore, using k-nearest neighbors to constrain the agglomerative clustering may not result in regions that are connected according to a different connectivity rule, such as the queen contiguity rule used in

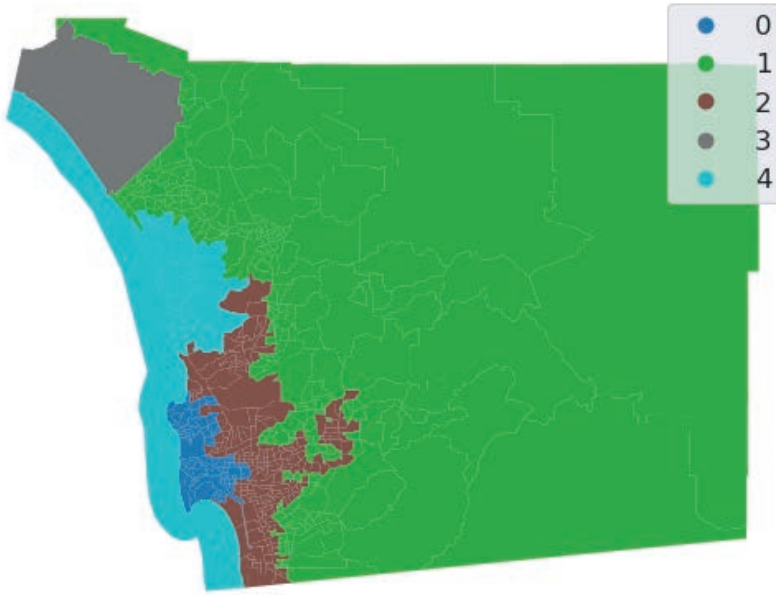


Fig. 10.9: Regions from a spatially constrained socio-demographic clustering, using a different connectivity constraint. Code generated for this figure is available on the web version of the book.

the previous section. However, the regionalization here is fortuitous; even though we used the 4-nearest tracts to constrain connectivity, all of our clusters are also connected according to the Queen contiguity rule.

So, which one is a "better" regionalization? Well, regionalizations are often compared based on measures of *geographical coherence*, as well as measures of *cluster coherence*. The former involves measures of cluster *shape* that can answer to questions like "are clusters evenly sized, or are they very differently sized? Are clusters very strangely shaped, or are they compact?"; while the latter generally focuses on whether cluster observations are more similar to their current clusters than to other clusters. This *goodness of fit* is usually better for unconstrained clustering algorithms than for the corresponding regionalizations. We'll show this next.

10.5.3 Geographical coherence

One very simple measure of geographical coherence involves the "compactness" of a given shape. The most common of these measures is the isoperimetric quotient

[HHV93]. This compares the area of the region to the area of a circle with the same perimeter as the region. To obtain the statistic, we can recognize that the circumference of the circle c is the same as the perimeter of the region i, so $P_i = 2\pi r_c$. Then, the area of the isoperimetric circle is $A_c = \pi r_c^2 = \pi \left(\frac{P_i}{2\pi}\right)^2$. Simplifying, we get:

$$IPQ_i = \frac{A_i}{A_c} = \frac{4\pi A_i}{P_i^2}$$

For this measure, more compact shapes have an IPQ closer to 1, whereas very elongated or spindly shapes will have IPQs closer to zero. For the clustering solutions, we would expect the IPQ to be very small indeed, since the perimeter of a cluster/region gets smaller the more boundaries that members share.

Computing this, then, can be done directly from the area and perimeter of a region:

```
results = []
for cluster_type in ("k5cls", "ward5", "ward5wq", "ward5wknn
↪"):
    # compute the region polygons using a dissolve
    regions = db[[cluster_type, "geometry"]].
↪dissolve(by=cluster_type)
    # compute the actual isoperimetric quotient for these␣
↪regions
    ipqs = (
        regions.area * 4 * numpy.pi / (regions.boundary.
↪length ** 2)
    )
    # cast to a dataframe
    result = ipqs.to_frame(cluster_type)
    results.append(result)
# stack the series together along columns
pandas.concat(results, axis=1)
```

	k5cls	ward5	ward5wq	ward5wknn
0	0.022799	0.018571	0.074571	0.139216
1	0.063241	0.149260	0.129419	0.245486
2	0.038124	0.046086	0.542491	0.050342
3	0.048154	0.064358	0.281874	0.542491
4	0.161781	0.027323	0.112495	0.123690

From this, we can see that the *shape* measures for the clusters are much better under the regionalizations than under the clustering solutions. As we'll show in the next section, this comes at the cost of goodness of fit. Alternatively, the two spatial solutions have different compactness values; the knn-based regions are much more compact than the queen weights-based solutions. The most compact region in the Queen regionalization is about at the median of the knn solutions.

Many other measures of shape regularity exist. Most of the well-used ones are implemented in the `esda.shapestats` module, which also documents the sensitivity of the different measures of shape.

10.5.4 Feature coherence (goodness of fit)

Many measures of the feature coherence, or *goodness of fit*, are implemented in scikit-learn's `metrics` module, which we used earlier to compute distances. This metrics module also contains a few goodness of fit statistics that measure, for example:

- `metrics.calinski_harabasz_score()` (CH): the within-cluster variance divided by the between-cluster variance.
- `metrics.silhouette_score()`: the average standardized distance from each observation to its "next best fit" cluster—the most similar cluster to which the observation is *not* currently assigned.

To compute these, each scoring function requires both the original data and the labels which have been fit. We'll compute the CH score for all the different clusterings below:

```
ch_scores = []
for cluster_type in ("k5cls", "ward5", "ward5wq", "ward5wknn
↪"):
    # compute the CH score
    ch_score = metrics.calinski_harabasz_score(
        # using scaled variables
        robust_scale(db[cluster_variables]),
        # using these labels
        db[cluster_type],
    )
    # and append the cluster type with the CH score
    ch_scores.append((cluster_type, ch_score))

# re-arrange the scores into a dataframe for display
pandas.DataFrame(
    ch_scores, columns=["cluster type", "CH score"]
).set_index("cluster type")
```

```
                CH score
cluster type
k5cls         115.118055
ward5          98.529245
ward5wq        62.518714
ward5wknn      54.378576
```

For all functions in `metrics` that end in "score", higher numbers indicate greater fit, whereas functions that end in `loss` work in the other direction. Thus, the K-means

solution has the highest Calinski-Harabasz score, while the ward clustering comes second. The regionalizations both come *well* below the clusterings, too. As we said before, the improved geographical coherence comes at a pretty hefty cost in terms of feature goodness of fit. This is because regionalization is *constrained*, and mathematically *cannot* achieve the same score as the unconstrained K-means solution, unless we get lucky and the k-means solution *is* a valid regionalization.

10.5.5 Solution similarity

The `metrics` module also contains useful tools to compare whether the labelings generated from different clustering algorithms are similar, such as the Adjusted Rand Score or the Mutual Information Score. To show that, we can see how similar clusterings are to one another:

```
ami_scores = []
# for each cluster solution
for i_cluster_type in ("k5cls", "ward5", "ward5wq", "ward5wknn
↪"):
    # for every other clustering
    for j_cluster_type in ("k5cls", "ward5", "ward5wq",
↪"ward5wknn"):
        # compute the adjusted mutual info between the two
        ami_score = metrics.adjusted_mutual_info_score(
            db[i_cluster_type], db[j_cluster_type]
        )
        # and save the pair of cluster types with the score
        ami_scores.append((i_cluster_type, j_cluster_type,␣
↪ami_score))
# arrange the results into a dataframe
results = pandas.DataFrame(
    ami_scores, columns=["source", "target", "similarity"]
)
# and spread the dataframe out into a square
results.pivot("source", "target", "similarity")
```

target	k5cls	ward5	ward5wknn	ward5wq
source				
k5cls	1.000000	0.574792	0.267554	0.302755
ward5	0.574792	1.000000	0.250029	0.258057
ward5wknn	0.267554	0.250029	1.000000	0.648272
ward5wq	0.302755	0.258057	0.648272	1.000000

From this, we can see that the K-means and Ward clusterings are the most self-similar, and the two regionalizations are slightly less similar to one another than the clusterings. The regionalizations are generally *not* very similar to the clusterings, as would be expected from our discussions above.

10.6 Conclusion

Overall, clustering and regionalization are two complementary tools to reduce complexity in multivariate data and build better understandings of their spatial structure. Often, there is simply too much data to examine every variable's map and its relation to all other variable maps. Thus, clustering reduces this complexity into a single conceptual shorthand by which people can easily describe complex and multi-faceted data. Clustering constructs groups of observations (called *clusters*) with coherent *profiles*, or distinct and internally consistent distributional/descriptive characteristics. These profiles are the conceptual shorthand, since members of each cluster should be more similar to the cluster at large than they are to any other cluster. Many different clustering methods exist; they differ on how the cluster is defined, and how "similar" members must be to clusters, or how these clusters are obtained. Regionalization is a special kind of clustering that imposes an additional geographic requirement. Observations should be grouped so that each spatial cluster, or *region*, is spatially coherent as well as data-coherent. Thus, regionalization is often concerned with connectivity in a contiguity graph for data collected in areas; this ensures that the regions that are identified are fully internally connected. However, since many regionalization methods are defined for an arbitrary connectivity structure, these graphs can be constructed according to different rules as well, such as the k-nearest neighbor graph. Finally, while regionalizations are usually more geographically coherent, they are also usually worse-fit to the features at hand. This reflects an intrinsic tradeoff that, in general, cannot be removed.

In this chapter, we discussed the conceptual basis for clustering and regionalization, as well as showing why clustering is done. Further, we have demonstrated how to build clusters using a combination of (geographic) data science packages, and how to interrogate the meaning of these clusters as well. More generally, clusters are often used in predictive and explanatory settings, in addition to being used for exploratory analysis in their own right. Clustering and regionalization are intimately related to the analysis of spatial autocorrelation as well, since the spatial structure and covariation in multivariate spatial data is what determines the spatial structure and data profile of discovered clusters or regions. Thus, clustering and regionalization are essential tools for the geographic data scientist.

10.7 Questions

1. What disciplines employ regionalization? Cite concrete examples for each discipline you list.
2. Contrast and compare the concepts of *clusters* and *regions*?
3. In evaluating the quality of the solution to a regionalization problem, how might traditional measures of cluster evaluation be used? In what ways might

those measures be limited and need expansion to consider the geographical dimensions of the problem?

4. Discuss the implications for the processes of regionalization that follow from the number of *connected components* in the spatial weights matrix that would be used.

5. Consider two possible weights matrices for use in a spatially constrained clustering problem. Both form a single connected component for all the areal units. However, they differ in the sparsity of their adjacency graphs (think Rook being less dense than Queen graphs).

   a. How might the sparsity of the weights matrix affect the quality of the clustering solution?

   b. Using `pysal.lib.weights.higher_order`, construct a second-order adjacency matrix of the weights matrix used in this chapter.

   c. Compare the `pct_nonzero` for both matrices.

   d. Rerun the analysis from this chapter using this new second-order weights matrix. What changes?

6. The idea of spatial dependence, that near things tend to be more related than distant things, is an extensively studied property of spatial data. How might solutions to clustering and regionalization problems change if dependence is very strong and positive? very weak? very strong and negative?

7. Using a spatial weights object obtained as `w = pysal.lib.weights.lat2W(20,20)`, what are the number of unique ways to partition the graph into 20 clusters of 20 units each, subject to each cluster being a connected component? What are the unique numbers of possibilities for `w = pysal.lib.weights.lat2W(20,20, rook=False)`?

10.8 Next steps

For a "classical" introduction to clustering methods in arbitrary data science problems, it is difficult to beat the *Introduction to Statistical Learning*:

James, Gareth, Daniela Witten, Trevor Hastie, and Robert Tibshirani. 2021. *Introduction to Statistical Learning* (2nd Edition). Wiley: New York.

For regionalization problems and methods, a useful discussion of the theory and operation of various heuristics and methods is provided by:

Duque, Juan Carlos, Raúl Ramos, and Jordi Suriñach. 2007. "Supervised Regionalization Methods: A survey." *International Regional Science Review* 30(3): 195-220.

Finally, methods for geodemographics are comprehensively covered in the book by:

Harris, Rich, Peter Sleight, and Richard Webber. 2005. *Geodemographics, GIS, and Neighbourhood Targeting.* Wiley.

And a more recent overview and discussion can also be provided by:

Singleton, Alex and Seth Spielman. 2014. "The past, present, and future of geodemographic research in the United States and the United Kingdom." *The Professional Geographer* 66(4): 558-567.

11

Spatial Regression

Regression (and prediction more generally) provides us a perfect case to examine how spatial structure can help us understand and analyze our data. In this chapter, we discuss how spatial structure can be used to both validate and improve prediction algorithms, focusing on linear regression specifically.

11.1 *What* is spatial regression and *why* should I care?

Usually, spatial structure helps regression models in one of two ways. The first (and most clear) way space can have an impact on our data is when the process *generating* the data is itself explicitly spatial. Here, think of something like the prices for single family homes. It's often the case that individuals pay a premium on their house price in order to live in a better school district for the same quality house. Alternatively, homes closer to noise or chemical polluters like waste water treatment plants, recycling facilities, or wide highways, may actually be cheaper than we would otherwise anticipate. In cases like asthma incidence, the locations individuals tend to travel to throughout the day, such as their places of work or recreation, may have more impact on their health than their residential addresses. In this case, it may be necessary to use data *from other sites* to predict the asthma incidence at a given site. Regardless of the specific case at play, here, *geography is a feature*: it directly helps us make predictions about outcomes *because those outcomes are obtained from geographical processes*.

An alternative (and more skeptical understanding) reluctantly acknowledges geography's instrumental value. Often, in the analysis of predictive methods and classifiers, we are interested in analyzing what we get wrong. This is common in econometrics; an analyst may be concerned that the model *systematically* mis-predicts some types of

observations. If we know our model routinely performs poorly on a known set of observations or type of input, we might make a better model if we can account for this. Among other kinds of error diagnostics, geography provides us with an exceptionally useful embedding to assess structure in our errors. Mapping classification/prediction error can help show whether or not there are *clusters of error* in our data. If we *know* that errors tend to be larger in some areas than in other areas (or if error is "contagious" between observations), then we might be able to exploit this structure to make better predictions.

Spatial structure in our errors might arise from when geography *should be* an attribute somehow, but we are not sure exactly how to include it in our model. They might also arise because there is some *other* feature whose omission causes the spatial patterns in the error we see; if this additional feature were included, the structure would disappear. Or, it might arise from the complex interactions and interdependencies between the features that we have chosen to use as predictors, resulting in intrinsic structure in mis-prediction. Most of the predictors we use in models of social processes contain *embodied* spatial information: patterning intrinsic to the feature that we get for free in the model. If we intend to or not, using a spatially patterned predictor in a model can result in spatially patterned errors; using more than one can amplify this effect. Thus, *regardless of whether or not the true process is explicitly geographic*, additional information about the spatial relationships between our observations or more information about nearby sites can make our predictions better.

In this chapter, we build space into the traditional regression framework. We begin with a standard linear regression model, devoid of any geographical reference. From there, we formalize space and spatial relationships in three main ways: first, encoding it in exogenous variables; second, through spatial heterogeneity, or as systematic variation of outcomes across space; third, as dependence, or through the effect associated to the characteristics of spatial neighbors. Throughout, we focus on the conceptual differences each approach entails rather than on the technical details.

```
from pysal.lib import weights
from pysal.explore import esda
import numpy
import pandas
import geopandas
import matplotlib.pyplot as plt
import seaborn
import contextily
```

11.2 Data: San Diego Airbnb

To learn a little more about how regression works, we'll examine information about Airbnb properties in San Diego, CA. This dataset contains house intrinsic characteristics, both continuous (number of beds as in `beds`) and categorical (type of renting or, in Airbnb jargon, property group as in the series of `pg_X` binary variables), but also variables that explicitly refer to the location and spatial configuration of the dataset (e.g., distance to Balboa Park, `d2balboa` or neighborhood id, `neighborhood_cleansed`).

```
db = geopandas.read_file("../data/airbnb/regression_db.geojson
↪")
```

These are the explanatory variables we will use throughout the chapter.

```
variable_names = [
    "accommodates",  # Number of people it accommodates
    "bathrooms",  # Number of bathrooms
    "bedrooms",  # Number of bedrooms
    "beds",  # Number of beds
    # Below are binary variables, 1 True, 0 False
    "rt_Private_room",  # Room type: private room
    "rt_Shared_room",  # Room type: shared room
    "pg_Condominium",  # Property group: condo
    "pg_House",  # Property group: house
    "pg_Other",  # Property group: other
    "pg_Townhouse",  # Property group: townhouse
]
```

11.3 Non-spatial regression, a (very) quick refresh

Before we discuss how to explicitly include space into the linear regression framework, let us show how basic regression can be carried out in Python, and how one can begin to interpret the results. By no means is this a formal and complete introduction to regression so, if that is what you are looking for, we recommend [GH06], in particular chapters 3 and 4, which provide a fantastic, non-spatial introduction.

The core idea of linear regression is to explain the variation in a given (*dependent*) variable as a linear function of a collection of other (*explanatory*) variables. For example, in our case, we may want to express the price of a house as a function of the number of bedrooms it has and whether it is a condominium or not. At the individual level, we

can express this as:

$$P_i = \alpha + \sum_k \mathbf{X}_{ik}\beta_k + \epsilon_i$$

where P_i is the Airbnb price of house i, and X is a set of covariates that we use to explain such price (e.g., No. of bedrooms and condominium binary variable). β is a vector of parameters that give us information about in which way and to what extent each variable is related to the price, and α, the constant term, is the average house price when all the other variables are zero. The term ϵ_i is usually referred to as "error" and captures elements that influence the price of a house but are not included in X. We can also express this relation in matrix form, excluding sub-indices for i, which yields:

$$P = \alpha + \mathbf{X}\beta + \epsilon$$

A regression can be seen as a multivariate extension of bivariate correlations. Indeed, one way to interpret the β_k coefficients in the equation above is as the degree of correlation between the explanatory variable k and the dependent variable, *keeping all the other explanatory variables constant*. When one calculates bivariate correlations, the coefficient of a variable is picking up the correlation between the variables, but it is also subsuming into it variation associated with other correlated variables – also called confounding factors. Regression allows us to isolate the distinct effect that a single variable has on the dependent one, once we *control* for those other variables.

Practically speaking, linear regressions in Python are rather streamlined and easy to work with. There are also several packages which will run them (e.g., `statsmodels`, `scikit-learn`, `pysal`). We will import the `spreg` module in Pysal:

```
from pysal.model import spreg
```

In the context of this chapter, it makes sense to start with `spreg`, as that is the only library that will allow us to move into explicitly spatial econometric models. To fit the model specified in the equation above with X as the list defined, using ordinary least squares (OLS), we only need the following line of code:

```
# Fit OLS model
m1 = spreg.OLS(
    # Dependent variable
    db[["log_price"]].values,
    # Independent variables
    db[variable_names].values,
    # Dependent variable name
    name_y="log_price",
    # Independent variable name
    name_x=variable_names,
)
```

We use the command `OLS`, part of the `spreg` sub-package, and specify the dependent variable (the log of the price, so we can interpret results in terms of percentage change) and the explanatory ones. Note that both objects need to be arrays, so we extract them from the `pandas.DataFrame` object using `.values`.

In order to inspect the results of the model, we can print the `summary` attribute:

```
print(m1.summary)
```

```
REGRESSION
----------
SUMMARY OF OUTPUT: ORDINARY LEAST SQUARES
-----------------------------------------
Data set            :     unknown
Weights matrix      :        None
Dependent Variable  :   log_price                Number of Observations:        6110
Mean dependent var  :      4.9958                Number of Variables   :          11
S.D. dependent var  :      0.8072                Degrees of Freedom    :        6099
R-squared           :      0.6683
Adjusted R-squared  :      0.6678
Sum squared residual:    1320.148                F-statistic           :   1229.0564
Sigma-square        :       0.216                Prob(F-statistic)     :           0
S.E. of regression  :       0.465                Log likelihood        :   -3988.895
Sigma-square ML     :       0.216                Akaike info criterion :    7999.790
S.E of regression ML:      0.4648                Schwarz criterion     :    8073.685
------------------------------------------------------------------------------------
            Variable     Coefficient       Std.Error     t-Statistic     Probability
------------------------------------------------------------------------------------
            CONSTANT       4.3883830       0.0161147     272.3217773       0.0000000
       accommodates        0.0834523       0.0050781      16.4336318       0.0000000
          bathrooms        0.1923790       0.0109668      17.5419773       0.0000000
           bedrooms        0.1525221       0.0111323      13.7009195       0.0000000
               beds       -0.0417231       0.0069383      -6.0134430       0.0000000
     rt_Private_room      -0.5506868       0.0159046     -34.6244758       0.0000000
      rt_Shared_room      -1.2383055       0.0384329     -32.2198992       0.0000000
      pg_Condominium       0.1436347       0.0221499       6.4846529       0.0000000
            pg_House      -0.0104894       0.0145315      -0.7218393       0.4704209
            pg_Other       0.1411546       0.0228016       6.1905633       0.0000000
        pg_Townhouse      -0.0416702       0.0342758      -1.2157316       0.2241342
------------------------------------------------------------------------------------
REGRESSION DIAGNOSTICS
MULTICOLLINEARITY CONDITION NUMBER           11.964
TEST ON NORMALITY OF ERRORS
TEST                             DF        VALUE           PROB
Jarque-Bera                       2        2671.611           0.0000
DIAGNOSTICS FOR HETEROSKEDASTICITY
RANDOM COEFFICIENTS
TEST                             DF        VALUE           PROB
Breusch-Pagan test               10         322.532           0.0000
Koenker-Bassett test             10         135.581           0.0000
================================= END OF REPORT =====================================
```

A full detailed explanation of the output is beyond the scope of this chapter, so we will focus on the relevant bits for our main purpose. This is concentrated on the `Coefficients` section, which gives us the estimates for β_k in our model. In other words, these numbers express the relationship between each explanatory variable and the dependent one, once the effect of confounding factors has been accounted for. Keep in mind however that regression is no magic; we are only discounting the effect of confounding factors that we include in the model, not of *all* potentially confounding factors.

Results are largely as expected: houses tend to be significantly more expensive if they accommodate more people (`accommodates`), if they have more bathrooms and bedrooms, and if they are a condominium or part of the "other" category of house type. Conversely, given a number of rooms, houses with more beds (i.e., listings that are more "crowded") tend to go for cheaper, as it is the case for properties where one does not rent the entire house but only a room (`rt_Private_room`) or even shares it

(rt_Shared_room). Of course, you might conceptually doubt the assumption that it is possible to *arbitrarily* change the number of beds within an Airbnb without eventually changing the number of people it accommodates, but methods to address these concerns using *interaction effects* won't be discussed here.

11.3.1 Hidden structures

In general, our model performs well, being able to predict slightly about two-thirds ($R^2 = 0.67$) of the variation in the mean nightly price using the covariates we've discussed above. But, our model might display some clustering in the errors, which may be a problem as that violates the i.i.d. assumption linear models usually come built-in with. To interrogate this, we can do a few things. One simple concept might be to look at the correlation between the error in predicting an Airbnb and the error in predicting its nearest neighbor. To examine this, we first might want to split our data up by regions and see if we've got some spatial structure in our residuals. One reasonable theory might be that our model does not include any information about *beaches*, a critical aspect of why people live and vacation in San Diego. Therefore, we might want to see whether or not our errors are higher or lower depending on whether or not an Airbnb is in a "beach" neighborhood, a neighborhood near the ocean. We use the code below to generate Figure 11.1, which looks at prices between the two groups of houses, "beach" and "no beach".

```
# Create a Boolean (True/False) with whether a
# property is coastal or not
is_coastal = db.coastal.astype(bool)
# Split residuals (m1.u) between coastal and not
coastal = m1.u[is_coastal]
not_coastal = m1.u[~is_coastal]
# Create histogram of the distribution of coastal residuals
plt.hist(coastal, density=True, label="Coastal")
# Create histogram of the distribution of non-coastal␣
↪residuals
plt.hist(
    not_coastal,
    histtype="step",
    density=True,
    linewidth=4,
    label="Not Coastal",
)
# Add line on 0
plt.vlines(0, 0, 1, linestyle=":", color="k", linewidth=4)
# Add legend
plt.legend()
# Display
plt.show()
```

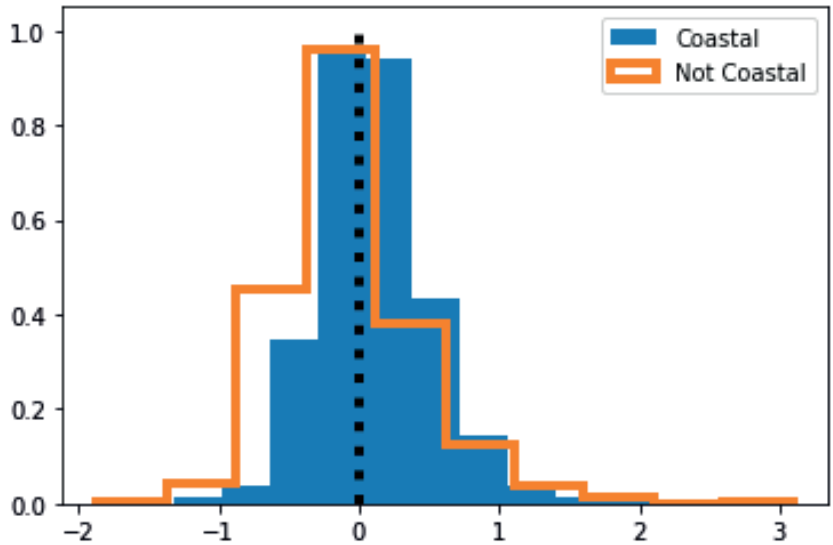


Fig. 11.1: Distributions of prediction errors (residuals) for the basic linear model. Residuals for coastal Airbnbs are generally positive, meaning that the model under-predicts their prices.

While it appears that the neighborhoods on the coast have only slightly higher average errors (and have lower variance in their prediction errors), the two distributions are significantly distinct from one another when compared using a classic t-test:

```
from scipy.stats import ttest_ind

ttest_ind(coastal, not_coastal)
```

```
Ttest_indResult(statistic=array([13.98193858]),␣
↪pvalue=array([9.442438e-44]))
```

There are more sophisticated (and harder to fool) tests that may be applicable for this data, however. We cover them in the *Challenge* section.

Additionally, it might be the case that some neighborhoods are more desirable than other neighborhoods due to unmodeled latent preferences or marketing. For instance, despite its presence close to the sea, living near Camp Pendleton -a Marine base in the North of the city- may incur some significant penalties on area desirability due to noise and pollution. These are questions that domain knowledge provides and data analysis can help us answer. For us to determine whether this is the case, we might be interested in the full distribution of model residuals within each neighborhood.

To make this more clear, we'll first sort the data by the median residual in that neighborhood, and then make a boxplot (Figure 11.2), which shows the distribution of residuals in each neighborhood:

```
# Create column with residual values from m1
db["residual"] = m1.u
# Obtain the median value of residuals in each neighborhood
medians = (
    db.groupby("neighborhood")
    .residual.median()
    .to_frame("hood_residual")
)

# Increase fontsize
seaborn.set(font_scale=1.25)
# Set up figure
f = plt.figure(figsize=(15, 3))
# Grab figure's axis
ax = plt.gca()
# Generate bloxplot of values by neighborhood
# Note the data includes the median values merged on-the-fly
seaborn.boxplot(
    x="neighborhood",
    y="residual",
    ax=ax,
    data=db.merge(
        medians, how="left", left_on="neighborhood", right_
↪index=True
    ).sort_values("hood_residual"),
    palette="bwr",
)
# Rotate the X labels for legibility
f.autofmt_xdate(rotation=-90)
# Display
plt.show()
```

No neighborhood is disjoint from one another, but some do appear to be higher than others, such as the well-known downtown tourist neighborhoods areas of the Gaslamp Quarter, Little Italy, or The Core. Thus, there may be a distinctive effect of intangible neighborhood fashionableness that matters in this model.

Noting that many of the most over- and under-predicted neighborhoods are near one another in the city, it may also be the case that there is some sort of *contagion* or spatial spillovers in the nightly rent price. This often is apparent when individuals seek to price their Airbnb listings to compete with similar nearby listings. Since our model is not aware of this behavior, its errors may tend to cluster. One exceptionally simple way we can look into this structure is by examining the relationship between an observation's residuals and its surrounding residuals.

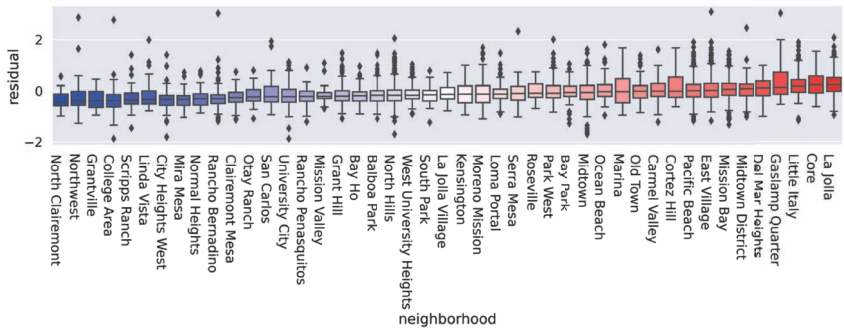


Fig. 11.2: Boxplot of prediction errors by neighborhood in San Diego, showing that the basic model systematically over- (or under-) predicts the nightly price of some neighborhoods' Airbnbs.

To do this, we will use *spatial weights* to represent the geographic relationships between observations. We cover spatial weights in detail in *Chapter 4*, so we will not repeat ourselves here. For this example, we'll start off with a KNN matrix where $k = 1$, meaning we're focusing only on the linkages of each Airbnb to their closest other listing.

```
knn = weights.KNN.from_dataframe(db, k=1)
```

This means that, when we compute the *spatial lag* of that KNN weight and the residual, we get the residual of the Airbnb listing closest to each observation.

```
lag_residual = weights.spatial_lag.lag_spatial(knn, m1.u)
ax = seaborn.regplot(
    x=m1.u.flatten(),
    y=lag_residual.flatten(),
    line_kws=dict(color="orangered"),
    ci=None,
)
ax.set_xlabel("Model Residuals - $u$")
ax.set_ylabel("Spatial Lag of Model Residuals - $W u$");
```

In Figure 11.3, we see that our prediction errors tend to cluster! Above, we show the relationship between our prediction error at each site and the prediction error at the site nearest to it. Here, we're using this nearest site to stand in for the *surroundings* of that Airbnb. This means that, when the model tends to over-predict a given Airbnb's nightly log price, sites around that Airbnb are more likely to *also be over-predicted*. An interesting property of this relationship is that it tends to stabilize as the number of nearest neighbors used to construct each Airbnb's surroundings increases. Consult the *Challenge* section for more on this property.

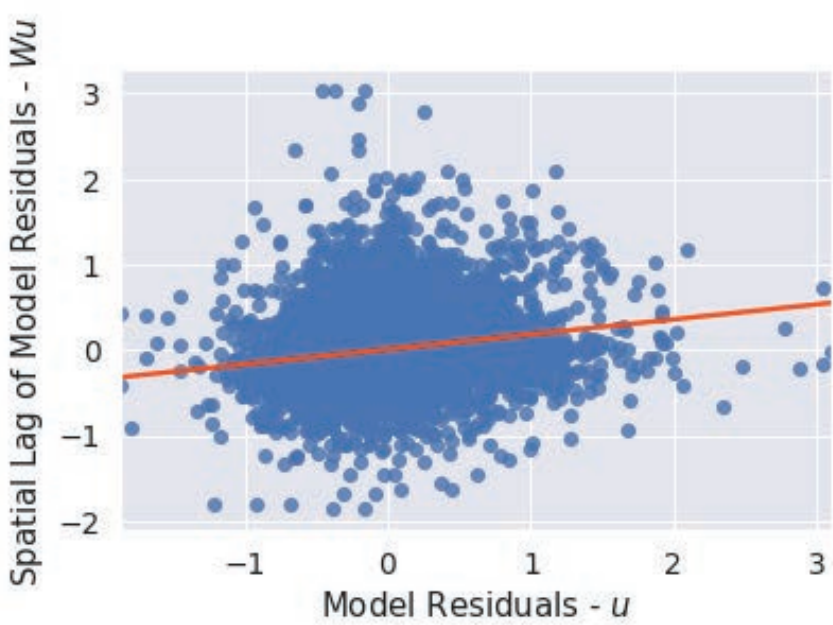


Fig. 11.3: The relationship between prediction error for an Airbnb and the nearest Airbnb's prediction error. This suggests that if an Airbnb's nightly price is over-predicted, its nearby Airbnbs will also be over-predicted.

Given this behavior, let's look at the stable $k = 20$ number of neighbors. Examining the relationship between this stable *surrounding* average and the focal Airbnb, we can even find clusters in our model error. Recalling the *local Moran* statistics in *Chapter 7*, Figure 11.4 is generated from the code below to identify certain areas where our predictions of the nightly (log) Airbnb price tend to be significantly off:

```
# Re-weight W to 20 nearest neighbors
knn.reweight(k=20, inplace=True)
# Row standardize weights
knn.transform = "R"
# Run LISA on residuals
outliers = esda.moran.Moran_Local(m1.u, knn,␣
↪permutations=9999)
# Select only LISA cluster cores
error_clusters = outliers.q % 2 == 1
# Filter out non-significant clusters
error_clusters &= outliers.p_sim <= 0.001
# Add `error_clusters` and `local_I` columns
ax = (
    db.assign(
```

(continued on next page)

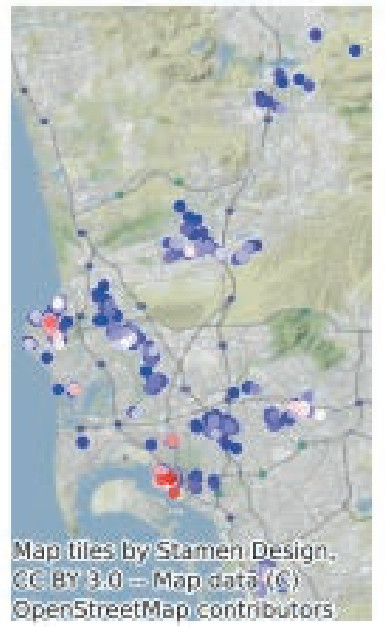


Fig. 11.4: Map of clusters in regression errors, according to the Local Moran's I_i.

(continued from previous page)

```
        error_clusters=error_clusters,
        local_I=outliers.Is
        # Retain error clusters only
    )
    .query(
        "error_clusters"
        # Sort by I value to largest plot on top
    )
    .sort_values(
        "local_I"
        # Plot I values
    )
    .plot("local_I", cmap="bwr", marker=".")
)
# Add basemap
contextily.add_basemap(ax, crs=db.crs)
# Remove axes
ax.set_axis_off();
```

Thus, these areas tend to be locations where our model significantly under-predicts the nightly Airbnb price both for that specific observation and observations in its immediate

surroundings. This is critical since, if we can identify how these areas are structured — if they have a *consistent geography* that we can model — then we might make our predictions even better, or at least not systematically mis-predict prices in some areas while correctly predicting prices in other areas. Since significant under- and over-predictions do appear to cluster in a highly structured way, we might be able to use a better model to fix the geography of our model errors.

11.4 Bringing space into the regression framework

There are many different ways that spatial structure shows up in our models, predictions, and our data, even if we do not explicitly intend to study it. Fortunately, there are nearly as many techniques, called *spatial regression* methods, that are designed to handle these sorts of structures. Spatial regression is about *explicitly* introducing space or geographical context into the statistical framework of a regression. Conceptually, we want to introduce space into our model whenever we think it plays an important role in the process we are interested in, or when space can act as a reasonable proxy for other factors that we cannot but should include in our model. As an example of the former, we can imagine how houses at the seafront are probably more expensive than those in the second row, given their better views. To illustrate the latter, we can think of how the character of a neighborhood is important in determining the price of a house; however, it is very hard to identify and quantify "character" per se, although it might be easier to get at its spatial variation, hence a case of space as a proxy.

Spatial regression is a large field of development in the econometrics and statistics literatures. In this brief introduction, we will consider two related but very different processes that give rise to spatial effects: spatial heterogeneity and spatial dependence. Before diving into them, we begin with another approach that introduces space in a regression model without modifying the model itself but rather creates spatially explicit independent variables. For more rigorous treatments of the topics introduced here, we refer you to [Ans03][AR14][GH06].

11.4.1 Spatial feature engineering: proximity variables

Using geographic information to "construct" new data is a common approach to bring in spatial information into data analysis. Often, this reflects the fact that processes are not the same everywhere in the map of analysis, or that geographical information may be useful to predict our outcome of interest. In this section, we will briefly present how to insert *spatial features*, or X variables that are constructed from geographical relationships, in a standard linear model. We discuss spatial feature engineering extensively in *Chapter 12*, though, and the depth and extent of spatial feature engineering is difficult to over-state. Rather than detail, this section will show how spatially explicit variables

Fig. 11.5: A map showing the 'Distance to Balboa Park' variable.

you engineer can be "plugged" into a model to improve its performance or help you explain the underlying process of interest with more accuracy.

One relevant proximity-driven variable that could influence our San Diego model is based on the listings proximity to Balboa Park. A common tourist destination, Balboa Park is a central recreation hub for the city of San Diego, containing many museums and the San Diego Zoo. Thus, it could be the case that people searching for Airbnbs in San Diego are willing to pay a premium to live closer to the park. If this were true *and* we omitted this from our model, we may indeed see a significant spatial pattern caused by this distance decay effect.

Therefore, this is sometimes called a *spatially patterned omitted covariate*: geographic information our model needs to make good predictions which we have left out of our model. Therefore, let's build a new model containing this distance to Balboa Park covariate. First, though, it helps to visualize (Figure 11.5) the structure of this distance covariate itself:

```
ax = db.plot("d2balboa", marker=".", s=5)
contextily.add_basemap(ax, crs=db.crs)
ax.set_axis_off();
```

To run a linear model that includes the additional variable of distance to the park, we add the name to the list of variables we included originally:

```
balboa_names = variable_names + ["d2balboa"]
```

And then fit the model using the OLS class in Pysal's `spreg`:

```
m2 = spreg.OLS(
    db[["log_price"]].values,
    db[balboa_names].values,
    name_y="log_price",
    name_x=balboa_names,
)
```

When you inspect the regression diagnostics and output, you see that this covariate is not quite as helpful as we might anticipate:

```
pandas.DataFrame(
    [[m1.r2, m1.ar2], [m2.r2, m2.ar2]],
    index=["M1", "M2"],
    columns=["R2", "Adj. R2"],
)
```

```
          R2   Adj. R2
M1  0.668345  0.667801
M2  0.668502  0.667904
```

It is not statistically significant at conventional significance levels, the model fit does not substantially change:

```
# Set up table of regression coefficients
pandas.DataFrame(
    {
        # Pull out regression coefficients and
        # flatten as they are returned as Nx1 array
        "Coeff.": m2.betas.flatten(),
        # Pull out and flatten standard errors
        "Std. Error": m2.std_err.flatten(),
        # Pull out P-values from t-stat object
        "P-Value": [i[1] for i in m2.t_stat],
    },
    index=m2.name_x,
)
```

```
                  Coeff.  Std. Error        P-Value
CONSTANT         4.379624    0.016915   0.000000e+00
accommodates     0.083644    0.005079   1.156896e-59
bathrooms        0.190791    0.011005   9.120139e-66
bedrooms         0.150746    0.011179   7.418035e-41
beds            -0.041476    0.006939   2.394322e-09
```

(continued on next page)

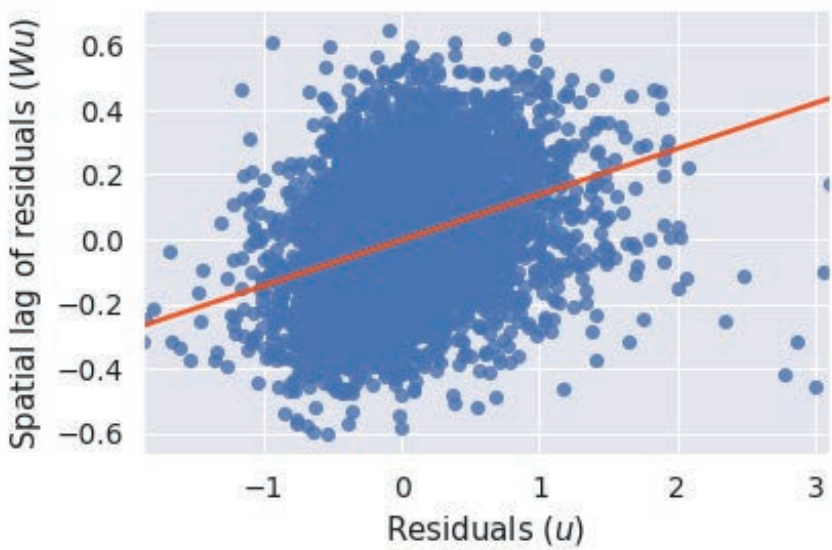


Fig. 11.6: The relationship between prediction error and the nearest Airbnb's prediction error for the model including the 'Distance to Balboa Park' variable. Note the much stronger relationship here than before.

(continued from previous page)

```
rt_Private_room -0.552996    0.015960  2.680270e-240
rt_Shared_room  -1.235521    0.038462  2.586867e-209
pg_Condominium   0.140459    0.022225   2.803765e-10
pg_House        -0.013302    0.014623   3.630396e-01
pg_Other         0.141176    0.022798   6.309880e-10
pg_Townhouse    -0.045784    0.034356   1.826992e-01
d2balboa         0.001645    0.000967   8.902052e-02
```

And, there still appears to be spatial structure in our model's errors, as we can see in Figure 11.6, generated by the code below:

```
lag_residual = weights.spatial_lag.lag_spatial(knn, m2.u)
ax = seaborn.regplot(
    x=m2.u.flatten(),
    y=lag_residual.flatten(),
    line_kws=dict(color="orangered"),
    ci=None,
)
ax.set_xlabel("Residuals ($u$)")
ax.set_ylabel("Spatial lag of residuals ($Wu$)");
```

Finally, the distance to Balboa Park variable does not fit our theory about how distance

to amenity should affect the price of an Airbnb; the coefficient estimate is *positive*, meaning that people are paying a premium to be *further* from Balboa Park. We will revisit this result later on, when we consider spatial heterogeneity and will be able to shed some light on this. Further, the next chapter is an extensive treatment of spatial fixed effects, presenting many more spatial feature engineering methods. Here, we have only showed how to include these engineered features in a standard linear modeling framework.

11.4.2 Spatial heterogeneity

So far we have assumed that our proximity variable might stand in for a difficult-to-measure premium individuals pay when they're close to a recreational zone (a park in this case). Our approach in that case was to incorporate space through a very specific channel, that is the distance to an amenity we thought might be influencing the final price. However, not all neighborhoods have the same house prices; some neighborhoods may be systematically more expensive than others, regardless of their proximity to Balboa Park. If this is our case, we need some way to account for the fact that each neighborhood may experience these kinds of *gestalt*, unique effects. One way to do this is by capturing *spatial heterogeneity* (SH). At its most basic, *spatial heterogeneity* means that parts of the model may vary systematically with geography, change in different places. For example, changes to the intercept, α, may reflect the fact that different areas have different baseline exposures to a given process. Changes to the slope terms, β, may indicate some kind of geographical mediating factor that makes the relationship between the independent and dependent variables vary across space, such as when a governmental policy is not consistently applied across jurisdictions. Finally, changes to the variance of the residuals, commonly denoted σ^2, can introduce spatial heteroskedasticity. We deal with the first two in this section.

11.4.2.1 Spatial fixed effects

The literature commonly refers to geographic variations of α as "spatial fixed effects (FE)". To illustrate them, let us consider the house price example from the previous section. Sometimes, spatial fixed effects are said to capture "space as a proxy", in that we know the outcome varies over space, we (hope to) know the pattern it follows (in our case, by neighborhood) and we can thus incorporate that knowledge into the model by letting α vary accordingly. The rationale goes as follows. Given we are only including a few explanatory variables in the model, it is likely we are missing some important factors that play a role at determining the price at which a house is sold. Some of them, however, are likely to vary systematically over space (e.g., different neighborhood characteristics). If that is the case, we can control for those unobserved factors by using traditional binary variables but basing their creation on a spatial rule. For example, let us include a binary variable for every neighborhood, indicating whether a given house is located within such area (`1`) or not (`0`). Mathematically, we are now fitting the following

equation:

$$\log P_i = \alpha_r + \sum_k \mathbf{X}_{ik}\beta_k + \epsilon_i$$

where the main difference is that we are now allowing the constant term, α, to vary by neighborhood r, α_r.

Programmatically, we will show two different ways we can estimate this: one, using `statsmodels`; and two, with `spreg`. First, we will use `statsmodels`, the econometrician's toolbox in Python.

```
import statsmodels.formula.api as sm
```

This package provides a formula-like API, which allows us to express the *equation* we wish to estimate directly:

```
f = (
    "log_price ~ "
    + " + ".join(variable_names)
    + " + neighborhood - 1"
)
print(f)
```

```
log_price ~ accommodates + bathrooms + bedrooms + beds + rt_
↪Private_room + rt_Shared_room + pg_Condominium + pg_House +␣
↪pg_Other + pg_Townhouse + neighborhood - 1
```

The *tilde* operator in this statement is usually read as "log price is a function of …", to account for the fact that many different model specifications can be fit according to that functional relationship between `log_price` and our covariate list. Critically, note that the trailing `-1` term means that we are fitting this model without an intercept term. This is necessary, since including an intercept term alongside unique means for every neighborhood would make the underlying system of equations underspecified.

Using this expression, we can estimate the unique effects of each neighborhood, fitting the model in `statsmodels` (note how the specification of the model, formula and data is separated from the fitting step):

```
m3 = sm.ols(f, data=db).fit()
```

We could rely on the `summary2()` method to print a similar summary report from the regression but, given it is a lengthy one in this case, we will illustrate how you can extract the spatial fixed effects into a table for display.

```
# Store variable names for all the spatial fixed effects
sfe_names = [i for i in m3.params.index if "neighborhood[" in␣
↪i]
```

(continued on next page)

(continued from previous page)

```
# Create table
pandas.DataFrame(
    {
        "Coef.": m3.params[sfe_names],
        "Std. Error": m3.bse[sfe_names],
        "P-Value": m3.pvalues[sfe_names],
    }
)
```

	Coef.	Std. Error	P-Value
neighborhood[Balboa Park]	4.280766	0.033292	0.0
neighborhood[Bay Ho]	4.198251	0.076878	0.0
neighborhood[Bay Park]	4.329223	0.050987	0.0
neighborhood[Carmel Valley]	4.389261	0.056553	0.0
neighborhood[City Heights West]	4.053518	0.058378	0.0
neighborhood[Clairemont Mesa]	4.095259	0.047699	0.0
neighborhood[College Area]	4.033697	0.058258	0.0
neighborhood[Core]	4.726186	0.052643	0.0
neighborhood[Cortez Hill]	4.608090	0.051526	0.0
neighborhood[Del Mar Heights]	4.496910	0.054337	0.0
neighborhood[East Village]	4.545469	0.029373	0.0
neighborhood[Gaslamp Quarter]	4.775799	0.047304	0.0
neighborhood[Grant Hill]	4.306742	0.052365	0.0
neighborhood[Grantville]	4.053298	0.071396	0.0
neighborhood[Kensington]	4.302671	0.077176	0.0
neighborhood[La Jolla]	4.682084	0.025809	0.0
neighborhood[La Jolla Village]	4.330311	0.077237	0.0
neighborhood[Linda Vista]	4.191149	0.056916	0.0
neighborhood[Little Italy]	4.666742	0.046838	0.0
neighborhood[Loma Portal]	4.301909	0.033236	0.0
neighborhood[Marina]	4.558298	0.047994	0.0
neighborhood[Midtown]	4.366661	0.028394	0.0
neighborhood[Midtown District]	4.584938	0.065087	0.0
neighborhood[Mira Mesa]	3.989562	0.056101	0.0
neighborhood[Mission Bay]	4.515479	0.022422	0.0
neighborhood[Mission Valley]	4.275960	0.074231	0.0
neighborhood[Moreno Mission]	4.400942	0.056730	0.0
neighborhood[Normal Heights]	4.097400	0.049022	0.0
neighborhood[North Clairemont]	3.984440	0.069149	0.0
neighborhood[North Hills]	4.253425	0.025478	0.0
neighborhood[Northwest]	4.173752	0.069728	0.0
neighborhood[Ocean Beach]	4.437164	0.030088	0.0
neighborhood[Old Town]	4.420160	0.041893	0.0
neighborhood[Otay Ranch]	4.185941	0.081597	0.0
neighborhood[Pacific Beach]	4.438829	0.022417	0.0
neighborhood[Park West]	4.440907	0.044768	0.0
neighborhood[Rancho Bernadino]	4.180906	0.072010	0.0
neighborhood[Rancho Penasquitos]	4.162428	0.061776	0.0
neighborhood[Roseville]	4.386992	0.058623	0.0
neighborhood[San Carlos]	4.334991	0.083040	0.0
neighborhood[Scripps Ranch]	4.082380	0.076244	0.0
neighborhood[Serra Mesa]	4.312967	0.059925	0.0
neighborhood[South Park]	4.225311	0.053643	0.0
neighborhood[University City]	4.193718	0.036965	0.0
neighborhood[West University Heights]	4.297672	0.043134	0.0

The approach above shows how spatial FE are a particular case of a linear regression with a categorical variable. Neighborhood membership is modeled using binary dummy variables. Thanks to the formula grammar used in `statsmodels`, we can express the model abstractly, and Python parses it, appropriately creating binary variables as required.

The second approach leverages `spreg` Regimes functionality. We will see regimes below but, for now, think of them as a generalization of spatial fixed effects where not only α can vary. This framework allows the user to specify which variables are to be

estimated separately for each group. In this case, instead of describing the model in a formula, we need to pass each element of the model as separate arguments.

```
# spreg spatial fixed effect implementation
m4 = spreg.OLS_Regimes(
    # Dependent variable
    db[["log_price"]].values,
    # Independent variables
    db[variable_names].values,
    # Variable specifying neighborhood membership
    db["neighborhood"].tolist(),
    # Allow the constant term to vary by group/regime
    constant_regi="many",
    # Variables to be allowed to vary (True) or kept
    # constant (False). Here we set all to False
    cols2regi=[False] * len(variable_names),
    # Allow separate sigma coefficients to be estimated
    # by regime (False so a single sigma)
    regime_err_sep=False,
    # Dependent variable name
    name_y="log_price",
    # Independent variables names
    name_x=variable_names,
)
```

Similarly as above, we could rely on the `summary` attribute to print a report with all the results computed. For simplicity here, we will only confirm that, to the 12th decimal, the parameters estimated are indeed the same as those we get from `statsmodels`:

```
import numpy

numpy.round(m4.betas.flatten() - m3.params.values,␣
↪decimals=12)
```

```
array([ 0.e+00, -0.e+00,  0.e+00,  0.e+00,  0.e+00,  0.e+00, ␣
↪0.e+00,
        0.e+00,  0.e+00, -1.e-12,  0.e+00,  0.e+00,  0.e+00, ␣
↪0.e+00,
        0.e+00,  0.e+00,  0.e+00,  0.e+00,  0.e+00,  0.e+00, ␣
↪0.e+00,
        0.e+00,  0.e+00,  0.e+00, -0.e+00,  0.e+00,  0.e+00, ␣
↪0.e+00,
        0.e+00,  0.e+00,  0.e+00,  0.e+00,  0.e+00,  0.e+00, ␣
↪0.e+00,
        0.e+00,  0.e+00,  0.e+00,  0.e+00,  0.e+00,  0.e+00, ␣
↪0.e+00,
```

(continued on next page)

(continued from previous page)

```
        0.e+00,  0.e+00,  0.e+00,  0.e+00, -0.e+00, -0.e+00,
↪0.e+00,
       -0.e+00, -0.e+00,  0.e+00,  0.e+00,  0.e+00,  0.e+00])
```

Econometrically speaking, what the neighborhood FEs we have introduced imply is that, instead of comparing all house prices across San Diego as equal, we only derive variation from within each postcode. Remember that the interpretation of β_k is the effect of variable k, *given all the other explanatory variables included remain constant*. By including a single variable for each area, we are effectively forcing the model to compare as equal only house prices that share the same value for each variable; or, in other words, only houses located within the same area. Introducing FE affords a higher degree of isolation of the effects of the variables we introduce in the model because we can control for unobserved effects that align spatially with the distribution of the FE introduced (by neighborhood, in our case). To make a map of neighborhood fixed effects, we need to process the results from our model slightly.

First, we extract only the effects pertaining to the neighborhoods:

```
neighborhood_effects = m3.params.filter(like="neighborhood")
neighborhood_effects.head()
```

```
neighborhood[Balboa Park]          4.280766
neighborhood[Bay Ho]               4.198251
neighborhood[Bay Park]             4.329223
neighborhood[Carmel Valley]        4.389261
neighborhood[City Heights West]    4.053518
dtype: float64
```

Then, we need to extract just the neighborhood name from the index of this Series. A simple way to do this is to strip all the characters that come before and after our neighborhood names:

```
# Create a sequence with the variable names without
# `neighborhood[` and `]`
stripped = neighborhood_effects.index.str.strip(
    "neighborhood["
).str.strip("]")
# Reindex the neighborhood_effects Series on clean names
neighborhood_effects.index = stripped
# Convert Series to DataFrame
neighborhood_effects = neighborhood_effects.to_frame("fixed_
↪effect")
# Print top of table
neighborhood_effects.head()
```

```
                    fixed_effect
Balboa Park             4.280766
Bay Ho                  4.198251
Bay Park                4.329223
Carmel Valley           4.389261
City Heights West       4.053518
```

Good, we're back to our raw neighborhood names. These allow us to join it to an auxillary file with neighborhood boundaries that is indexed on the same names. Let's read the boundaries first:

```
sd_path = "../data/airbnb/neighbourhoods.geojson"
neighborhoods = geopandas.read_file(sd_path)
```

And we can then merge the spatial FE and plot them on a map (Figure 11.7):

```
# Plot base layer with all neighborhoods in grey
ax = neighborhoods.plot(
    color="k", linewidth=0, alpha=0.5, figsize=(12, 6)
)
# Merge SFE estimates (note not every polygon
# receives an estimate since not every polygon
# contains Airbnb properties)
neighborhoods.merge(
    neighborhood_effects,
    how="left",
    left_on="neighbourhood",
    right_index=True
    # Drop polygons without a SFE estimate
).dropna(
    subset=["fixed_effect"]
    # Plot quantile choropleth
).plot(
    "fixed_effect",  # Variable to display
    scheme="quantiles",  # Choropleth scheme
    k=7,  # No. of classes in the choropleth
    linewidth=0.1,  # Polygon border width
    cmap="viridis",  # Color scheme
    ax=ax,  # Axis to draw on
)
# Add basemap
contextily.add_basemap(
    ax,
    crs=neighborhoods.crs,
    source=contextily.providers.CartoDB.PositronNoLabels,
)
# Remove axis
```

(continued on next page)

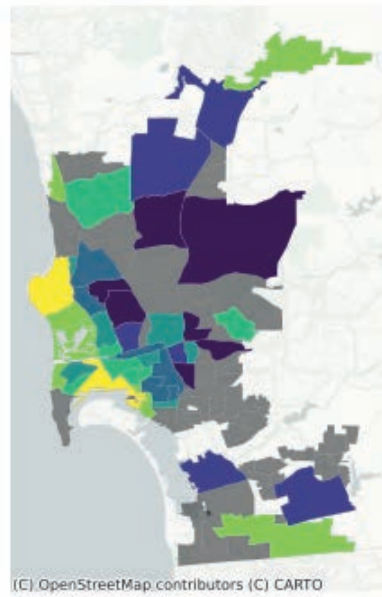


Fig. 11.7: Neighborhood effects on Airbnb nightly prices. Neighborhoods shown in grey are 'not statistically significant' in their effect on Airbnb prices.

(continued from previous page)

```
ax.set_axis_off()
# Display
plt.show()
```

We can see a clear spatial structure in the SFE estimates. The most expensive neighborhoods tend to be located near the coast, while the cheapest ones are more inland.

11.4.2.2 Spatial regimes

At the core of estimating spatial FEs is the idea that, instead of assuming the dependent variable behaves uniformly over space, there are systematic effects following a geographical pattern that affect its behavior. In other words, spatial FEs introduce econometrically the notion of spatial heterogeneity. They do this in the simplest possible form: by allowing the constant term to vary geographically. The other elements of the regression are left untouched and hence apply uniformly across space. The idea of spatial regimes (SRs) is to generalize the spatial FE approach to allow not only the constant term to vary but also any other explanatory variable. This implies that the equation we

will be estimating is:

$$\log P_i = \alpha_r + \sum_k \mathbf{X}_{ki}\beta_{k-r} + \epsilon_i$$

where we are not only allowing the constant term to vary by region (α_r), but also every other parameter (β_{k-r}).

To illustrate this approach, we will use the "spatial differentiator" of whether a house is in a coastal neighborhood or not (`coastal_neig`) to define the regimes. The rationale behind this choice is that renting a house close to the ocean might be a strong enough pull that people might be willing to pay at different *rates* for each of the house's characteristics.

To implement this in Python, we use the `OLS_Regimes` class in `spreg`, which does most of the heavy lifting for us:

```
# Pysal spatial regimes implementation
m5 = spreg.OLS_Regimes(
    # Dependent variable
    db[["log_price"]].values,
    # Independent variables
    db[variable_names].values,
    # Variable specifying neighborhood membership
    db["coastal"].tolist(),
    # Allow the constant term to vary by group/regime
    constant_regi="many",
    # Allow separate sigma coefficients to be estimated
    # by regime (False so a single sigma)
    regime_err_sep=False,
    # Dependent variable name
    name_y="log_price",
    # Independent variables names
    name_x=variable_names,
)
```

The result can be explored and interpreted similarly to the previous ones. If you inspect the `summary` attribute, you will find the parameters for each variable mostly conform to what you would expect, across both regimes. To compare them, we can plot them side-by-side on a bespoke table:

```
# Results table
res = pandas.DataFrame(
    {
        # Pull out regression coefficients and
        # flatten as they are returned as Nx1 array
        "Coeff.": m5.betas.flatten(),
        # Pull out and flatten standard errors
```

(continued on next page)

(continued from previous page)

```
        "Std. Error": m5.std_err.flatten(),
        # Pull out P-values from t-stat object
        "P-Value": [i[1] for i in m5.t_stat],
    },
    index=m5.name_x,
)
# Coastal regime
## Extract variables for the coastal regime
coastal = [i for i in res.index if "1_" in i]
## Subset results to coastal and remove the 1_ underscore
coastal = res.loc[coastal, :].rename(lambda i: i.replace("1_",
↪ ""))
## Build multi-index column names
coastal.columns = pandas.MultiIndex.from_product(
    [["Coastal"], coastal.columns]
)
# Non-coastal model
## Extract variables for the non-coastal regime
ncoastal = [i for i in res.index if "0_" in i]
## Subset results to non-coastal and remove the 0_ underscore
ncoastal = res.loc[ncoastal, :].rename(lambda i: i.replace("0_
↪", ""))
## Build multi-index column names
ncoastal.columns = pandas.MultiIndex.from_product(
    [["Non-coastal"], ncoastal.columns]
)
# Concat both models
pandas.concat([coastal, ncoastal], axis=1)
```

```
                Coastal                          Non-coastal            \
                 Coeff. Std. Error       P-Value      Coeff. Std. Error
CONSTANT        4.479904   0.025094  0.000000e+00    4.407242   0.021516
accommodates    0.048464   0.007881  8.253761e-10    0.090186   0.006474
bathrooms       0.247478   0.016566  1.381278e-49    0.143376   0.014268
bedrooms        0.189740   0.017923  5.783965e-26    0.112963   0.013827
beds           -0.050608   0.010743  2.522348e-06   -0.026272   0.008838
rt_Private_room -0.558628   0.028312  4.723759e-84   -0.529334   0.018918
rt_Shared_room  -1.052854   0.084174  1.836512e-35   -1.224459   0.042597
pg_Condominium   0.204447   0.033943  1.810152e-09    0.105307   0.028131
pg_House         0.075353   0.023378  1.274269e-03   -0.045447   0.017957
pg_Other         0.295485   0.038645  2.394157e-14    0.060753   0.027637
pg_Townhouse    -0.073508   0.049367  1.365396e-01   -0.010397   0.045673
                      P-Value
CONSTANT         0.000000e+00
accommodates     1.893020e-43
bathrooms        1.418804e-23
bedrooms         3.731742e-16
beds             2.964354e-03
rt_Private_room  3.546091e-162
rt_Shared_room   1.657163e-170
pg_Condominium    1.831822e-04
pg_House          1.140318e-02
pg_Other          2.796727e-02
pg_Townhouse      8.199294e-01
```

An interesting question arises around the relevance of the regimes. *Are estimates for each variable across regimes statistically different?* For this, the model object also

calculates for us what is called a Chow test. This is a statistic that tests the null hypothesis that estimates from different regimes are undistinguishable. If we reject the null, we have evidence suggesting the regimes actually make a difference.

Results from the Chow test are available on the `summary` attribute, or we can extract them directly from the model object, which we will do here. There are two types of Chow test. First is a global one that jointly tests for differences between the two regimes:

```
m5.chow.joint
```

```
(328.86902143020575, 7.113548767626822e-64)
```

The first value represents the statistic, while the second one captures the p-value. In this case, the two regimes are statistically different from each other. The next step then is to check whether each of the coefficients in our model differs across regimes. For this, we can pull them out into a table:

```
pandas.DataFrame(
    # Chow results by variable
    m5.chow.regi,
    # Name of variables
    index=m5.name_x_r,
    # Column names
    columns=["Statistic", "P-value"],
)
```

	Statistic	P-value
CONSTANT	4.832180	2.793329e-02
accommodates	16.735685	4.296522e-05
bathrooms	22.671471	1.922004e-06
bedrooms	11.503786	6.945459e-04
beds	3.060313	8.022620e-02
rt_Private_room	0.740097	3.896298e-01
rt_Shared_room	3.308838	6.890820e-02
pg_Condominium	5.057283	2.452265e-02
pg_House	16.792503	4.169771e-05
pg_Other	24.409876	7.786847e-07
pg_Townhouse	0.880564	3.480471e-01

As we can see in the table, most variables do indeed differ across regimes, statistically speaking. This points to systematic differences in the data generating processes across spatial regimes.

11.4.3 Spatial dependence

As we have just discussed, SH is about effects of phenomena that are *explicitly linked* to geography and that hence cause spatial variation and clustering. This encompasses many of the kinds of spatial effects we may be interested in when we fit linear regressions. However, in other cases, our focus is on the effect of the *spatial configuration* of the observations, and the extent to which that has an effect on the outcome we are considering. For example, we might think that the price of a house not only depends on whether it is a townhouse or an apartment, but also on whether it is surrounded by many more townhouses than skyscrapers with more apartments. This, we could hypothesize, might be related to the different "look and feel" a neighborhood with low-height, historic buildings has as compared to one with modern high-rises. To the extent these two different spatial configurations enter differently the house price determination process, we will be interested in capturing not only the characteristics of a house, but also of its surrounding ones. This kind of spatial effect is fundamentally different from SH in that is it not related to inherent characteristics of the geography but relates to the characteristics of the observations in our dataset and, specially, to their spatial arrangement. We call this phenomenon by which the values of observations are related to each other through distance *spatial dependence* [Ans88].

There are several ways to introduce spatial dependence in an econometric framework, with varying degrees of econometric sophistication (see [Ans02] for a good overview). Common to all of them however is the way space is formally encapsulated: through *spatial weights matrices (***W***)*, which we discussed in *Chapter 4*. In this section, we consider three ways in which spatial dependence, through spatial weights matrices, can be incorporated in a regression framework. We begin with the "least invasive" one, where we only modify the set of independent variables, and the move into more substantial modifications of the baseline linear model.

11.4.3.1 Exogenous effects: The SLX model

Let us come back to the house price example we have been working with. So far, we have hypothesized that the price of a house rented in San Diego through Airbnb can be explained using information about its own characteristics as well as some relating to its location such as the neighborhood or the distance to the main park in the city. However, it is also reasonable to think that prospective renters care about the set of neighbors a house has, not only about the house itself, and would be willing to pay more for a house that was surrounded by certain types of houses, and less if it was located in the middle of other types. How could we test this idea?

When it comes to regression, the most straightforward way to introduce spatial dependence between the observations in the data is by considering not only a given explanatory variable, but also its spatial lag. Conceptually, this approach falls more within the area of spatial feature engineering, which embeds space in a model through the explanatory variables it uses rather than the functional form of the model, and which we

delve into with more detail in *Chapter 12*. But we think it is interesting to discuss it in this context for two reasons. First, it provides "intellectual scaffolding" to learn the intuition of building spatial dependence into regression. And second, because it also illustrates how many of the techniques we cover in *Chapter 12* can be embedded in a regression model (and, by extension, in other predictive approaches).

In our example case, in addition to including a dummy for the type of house (`pg_XXX`), we can also include the spatial lag of each type of house. This addition implies we are also including as explanatory factor of the price of a given house the proportion neighboring houses in each type. Mathematically, this implies estimating the following model:

$$\log(P_i) = \alpha + \sum_{k=1}^{p} X_{ij}\beta_j + \sum_{k=1}^{p} \left(\sum_{j=1}^{N} w_{ij}x_{jk} \right) \gamma_k + \epsilon_i$$

where $\sum_{j=1}^{N} w_{ij}x_{jk}$ represents the spatial lag of the kth explanatory variable. This can be stated in *matrix* form using the spatial weights matrix, $\mathbf{W}$, as:

$$\log(P_i) = \alpha + \mathbf{X}\beta + \mathbf{WX}\gamma + \epsilon$$

This splits the model to focus on two main effects: β and γ. The β effect describes the change in y_i when X_{ik} changes by one.[1] The subscript for site i is important here: since we're dealing with a $\mathbf{W}$ matrix, it's useful to be clear about where the change occurs.

Indeed, this matters for the γ effect, which represents the *indirect* association of a change in X_i with the house price. This can be conceptualized in two ways. First, one could think of γ as simply *the association between the price in a given house and a unit change in its average surroundings.* This is useful and simple. But this interpretation blurs *where* this change might occur. In truth, a change in a variable at site i will result in a *spillover* to its surroundings: when x_i changes, so too does the *spatial lag* of any site near i. The precise size of the change in the lag will depend on the structure of $\mathbf{W}$, and it can be different for every site it is connected with. For example, think of a very highly connected "focal" site in a row-standardized weight matrix. This focal site will not be strongly affected if a neighbor changes by a single unit, since each site only contributes a small amount to the lag at the focal site. Alternatively, consider a site with only one neighbor: its lag will change by *exactly* the amount its sole neighbor changes. Thus, to discover the exact indirect effect of a change y caused by the change at a specific site x_i you would need to compute the *change in the spatial lag*, and then use that as your *change* in X. We will discuss this in the following section.

In Python, we can calculate the spatial lag of each variable whose name starts by `pg_` by first creating a list of all of those names, and then applying `pysal`'s `lag_spatial` to each of them:

[1] Since we use the log price as the dependent variable, our β coefficients can be interpreted as the percentage change in the price associated with a unit change in the explanatory variable.

```
# Select only columns in `db` containing the keyword `pg_`
wx = (
    db.filter(
        like="pg_"
        # Compute the spatial lag of each of those variables
    )
    .apply(
        lambda y: weights.spatial_lag.lag_spatial(knn, y)
        # Rename the spatial lag, adding w_ to the original␣
↪name
    )
    .rename(
        columns=lambda c: "w_"
        + c
        # Remove the lag of the binary variable for apartments
    )
    .drop("w_pg_Apartment", axis=1)
)
```

Once computed, we can run the model using OLS estimation because, in this context, the spatial lags included do not violate any of the assumptions OLS relies on (they are essentially additional exogenous variables):

```
# Merge original variables with the spatial lags in `wx`
slx_exog = db[variable_names].join(wx)
# Fit linear model with `spreg`
m6 = spreg.OLS(
    # Dependent variable
    db[["log_price"]].values,
    # Independent variables
    slx_exog.values,
    # Dependent variable name
    name_y="l_price",
    # Independent variables names
    name_x=slx_exog.columns.tolist(),
)
```

As in the previous cases, printing the `summary` attribute of the model object would show a full report table. The variables we included in the original regression display similar behavior, albeit with small changes in size, and can be interpreted also in a similar way. To focus on the aspects that differ from the previous models here, we will only pull out results for the variables for which we also included their spatial lags:

```
# Collect names of variables of interest
vars_of_interest = (
    db[variable_names].filter(like="pg_").join(wx).columns
)
```

(continued on next page)

(continued from previous page)

```
# Build full table of regression coefficients
pandas.DataFrame(
    {
        # Pull out regression coefficients and
        # flatten as they are returned as Nx1 array
        "Coeff.": m6.betas.flatten(),
        # Pull out and flatten standard errors
        "Std. Error": m6.std_err.flatten(),
        # Pull out P-values from t-stat object
        "P-Value": [i[1] for i in m6.t_stat],
    },
    index=m6.name_x
    # Subset for variables of interest only and round to
    # four decimals
).reindex(vars_of_interest).round(4)
```

	Coeff.	Std. Error	P-Value
pg_Condominium	0.1063	0.0222	0.0000
pg_House	0.0328	0.0157	0.0368
pg_Other	0.0862	0.0240	0.0003
pg_Townhouse	-0.0277	0.0338	0.4130
w_pg_Condominium	0.5928	0.0690	0.0000
w_pg_House	-0.0774	0.0319	0.0152
w_pg_Other	0.4851	0.0551	0.0000
w_pg_Townhouse	-0.2724	0.1223	0.0260

The spatial lag of each type of property (`w_pg_XXX`) is the new addition. We observe that, except for the case of townhouses (same as with the binary variable, `pg_Townhouse`), they are all significant, suggesting our initial hypothesis on the role of the surrounding houses might indeed be at work here.

As an illustration, let's look at some of the direct/indirect effects. The direct effect of the `pg_Condominium` variable means that condominiums are typically 11% more expensive ($\beta_{pg_Condominium} = 0.1063$) than the benchmark property type, apartments. More relevant to this section, any given house surrounded by condominiums *also* receives a price premium. But, since $pg_{Condominium}$ is a dummy variable, the spatial lag at site i represents the *percentage* of properties near i that are condominiums, which is between 0 and 1.[2] So, a *unit* change in this variable means that you would increase the condominium percentage by 100%. Thus, a .1 increase in `w_pg_Condominium` (a change of ten percentage points) would result in a 5.92% increase in the property house price ($\beta_{w_pg_Condominium} = 0.6$). Similar interpretations can be derived for all other spatially lagged variables to derive the *indirect* effect of a change in the spatial lag.

To compute the indirect change for a given site i, you may need to examine the predicted

[2] Discover this for yourself: what is the average of `numpy.array([True, True, True, False, False, True)]`?

values for its price. In this example, since we are using a row-standardized weights matrix with twenty nearest neighbors, the impact of changing x_i is the same for all of its neighbors and for any site i. Thus, the effect is always $\frac{\gamma}{20}$, or about 0.0296. However, it is interesting to consider this would not be the case for many other kinds of weights (like `Kernel`, `Queen`, `Rook`, `DistanceBand`, or `Voronoi`), where each observation has potentially a different number of neighbors. To illustrate this, we will construct the indirect effect for a specific i in the condominium group.

First, predicted values for y_i are stored in the `predy` attribute of any `spreg` model:

```
# Print first three predicted values
m5.predy[:3]
```

```
array([[5.27285901],
       [5.39966259],
       [4.28834686]])
```

To build new predictions, we need to follow the underlying equation of our model.

To illustrate the effect of a change in one of the values in a given location in other locations, we will switch one of the properties into the condominium category. Consider the third observation, which is the first apartment in the data:

```
# Print values for third observation for columns spanning
# from `pg_Apartment` to `pg_Townhouse`
db.loc[2, "pg_Apartment":"pg_Townhouse"]
```

```
pg_Apartment      1
pg_Condominium    0
pg_House          0
pg_Other          0
pg_Townhouse      0
Name: 2, dtype: object
```

Let's now make a copy of our data and change the value to magically turn the apartment into a condominium:

```
# Make copy of the dataset
db_scenario = db.copy()
# Make Apartment 0 and condo 1 for third observation
db_scenario.loc[2, ["pg_Apartment", "pg_Condominium"]] = [0,␣
↪1]
```

We've successfully made the change:

```
db_scenario.loc[2, "pg_Apartment":"pg_Townhouse"]
```

```
pg_Apartment       0
pg_Condominium     1
pg_House           0
pg_Other           0
pg_Townhouse       0
Name: 2, dtype: object
```

Now, we need to *also* update the spatial lag variates:

```
# Select only columns in `db_scenario` containing the keyword `pg_`
wx_scenario = (
    db_scenario.filter(
        like="pg"
        # Compute the spatial lag of each of those variables
    )
    .apply(
        lambda y: weights.spatial_lag.lag_spatial(knn, y)
        # Rename the spatial lag, adding w_ to the original name
    )
    .rename(
        columns=lambda c: "w_"
        + c
        # Remove the lag of the binary variable for apartments
    )
    .drop("w_pg_Apartment", axis=1)
)
```

And build a new exogenous $\mathbf{X}$ matrix, including the a constant 1 as the leading column

```
slx_exog_scenario = db_scenario[variable_names].join(wx_scenario)
```

Now, our new prediction (in the scenario where we have changed site `2` from an apartment into a condominium) can be computed by translating the model equation into Python code and plugging into it the simulated values we have just created:

```
# Compute new set of predicted values
y_pred_scenario = m6.betas[0] + slx_exog_scenario @ m6.betas[1:]
```

Note the only difference between this set of predictions and the one in the original `m6` model is that we have switched site `2` from apartment into condominium. Hence, every property which is *not* connected to site `2` (or is not site `2` itself) will be unaffected. The *neighbors* of site `2` however will have different predictions. To explore these, let's first identify who is in this group:

```
print(knn.neighbors[2])
```

```
[772, 2212, 139, 4653, 2786, 1218, 138, 808, 1480, 4241, 1631,
↪ 3617, 2612, 1162, 135, 23, 5528, 3591, 407, 6088]
```

Now, the effect of changing site 2 from an apartment into a condominium is associated with the following changes to the predicted (log) price, which we calculate by substracting the new predicted values from the original ones and subsetting only to site 2 and its neighbors:

```
# Difference between original and new predicted values
(
    y_pred_scenario
    - m6.predy
    # Subset to site `2` and its neighbors
).loc[[2] + knn.neighbors[2]]
```

```
              0
2      0.106349
772    0.029642
2212   0.029642
139    0.029642
4653   0.029642
2786   0.029642
1218   0.029642
138    0.029642
808    0.029642
1480   0.029642
4241   0.029642
1631   0.029642
3617   0.029642
2612   0.029642
1162   0.029642
135    0.029642
23     0.029642
5528   0.029642
3591   0.029642
407    0.029642
6088   0.029642
```

We see the first row, representing the direct effect, is equal exactly to the estimate for pg_Condominium. For the other effects, though, we have only changed w_pg_Condominium by .03 which roughly equates the marginal effect (w_pg_Condominium) divided by the weight of the spatial relationship between site 2 and every neighbor (in this case, $\frac{1}{20}$ for every neighbor, but note if different neighbors had different cardinalities, this would differ).

Introducing a spatial lag of an explanatory variable, as we have just seen, is the most straightforward way of incorporating the notion of spatial dependence in a linear regression framework. It does not require additional changes, it can be estimated with OLS, and the interpretation is rather similar to interpreting non-spatial variables, so long as aggregate changes are required.

The field of spatial econometrics however is a much broader one and has produced over the last decades many techniques to deal with spatial effects and spatial dependence in different ways. Although this might be an over-simplification, one can say that most of such efforts for the case of a single cross-section are focused on two main variations: the spatial lag and the spatial error model. Both are similar to the case we have seen in that they are based on the introduction of a spatial lag, but they differ in the component of the model they modify and affect.

11.4.3.2 Spatial error

The spatial error model includes a spatial lag in the *error* term of the equation:

$$\log P_i = \alpha + \sum_k \beta_k X_{ki} + u_i$$

$$u_i = \lambda u_{lag-i} + \epsilon_i$$

where $u_{lag-i} = \sum_j w_{i,j} u_j$. Although it appears similar, this specification violates the assumptions about the error term in a classical OLS model. Hence, alternative estimation methods are required. Pysal incorporates functionality to estimate several of the most advanced techniques developed by the literature on spatial econometrics. For example, we can use a general method of moments that account for heteroskedasticity [ADKP10]:

```
# Fit spatial error model with `spreg`
# (GMM estimation allowing for heteroskedasticity)
m7 = spreg.GM_Error_Het(
    # Dependent variable
    db[["log_price"]].values,
    # Independent variables
    db[variable_names].values,
    # Spatial weights matrix
    w=knn,
    # Dependent variable name
    name_y="log_price",
    # Independent variables names
    name_x=variable_names,
)
```

Similarly as before, the `summary` attribute will return a full-featured table of results.

For the most part, it may be interpreted in similar ways to those above. The main difference is that, in this case, we can also recover an estimate and inference for the λ parameter in the error term:

```
# Build full table of regression coefficients
pandas.DataFrame(
    {
        # Pull out regression coefficients and
        # flatten as they are returned as Nx1 array
        "Coeff.": m7.betas.flatten(),
        # Pull out and flatten standard errors
        "Std. Error": m7.std_err.flatten(),
        # Pull out P-values from t-stat object
        "P-Value": [i[1] for i in m7.z_stat],
    },
    index=m7.name_x
    # Subset for lambda parameter and round to
    # four decimals
).reindex(["lambda"]).round(4)
```

```
        Coeff.  Std. Error  P-Value
lambda  0.6449      0.0187      0.0
```

11.4.3.3 Spatial lag

The spatial lag model introduces a spatial lag of the *dependent* variable. In the example we have covered, this would translate into:

$$\log P_i = \alpha + \rho \log P_{lag-i} + \sum_k \beta_k X_{ki} + \epsilon_i$$

Although it might not seem very different from the previous equation, this model violates the exogeneity assumption, crucial for OLS to work. Put simply, this occurs when P_i exists on both "sides" of the equals sign. In theory, since P_i is included in computing P_{lag-i}, exogeneity is violated. Similarly to the case of the spatial error, several techniques have been proposed to overcome this limitation, and Pysal implements several of them. In the example below, we use a two-stage least squares estimation [Ans88], where the spatial lag of all the explanatory variables is used as instrument for the endogenous lag:

```
# Fit spatial lag model with `spreg`
# (GMM estimation)
m8 = spreg.GM_Lag(
    # Dependent variable
    db[["log_price"]].values,
    # Independent variables
```

(continued on next page)

(continued from previous page)

```
    db[variable_names].values,
    # Spatial weights matrix
    w=knn,
    # Dependent variable name
    name_y="log_price",
    # Independent variables names
    name_x=variable_names,
)
```

And let's summarize the coefficients in a table as before (usual disclaimer about the `summary` object applies):

```
# Build full table of regression coefficients
pandas.DataFrame(
    {
        # Pull out regression coefficients and
        # flatten as they are returned as Nx1 array
        "Coeff.": m8.betas.flatten(),
        # Pull out and flatten standard errors
        "Std. Error": m8.std_err.flatten(),
        # Pull out P-values from t-stat object
        "P-Value": [i[1] for i in m8.z_stat],
    },
    index=m8.name_z
    # Round to four decimals
).round(4)
```

	Coeff.	Std. Error	P-Value
CONSTANT	2.7440	0.0727	0.0000
accommodates	0.0698	0.0048	0.0000
bathrooms	0.1627	0.0104	0.0000
bedrooms	0.1604	0.0105	0.0000
beds	-0.0365	0.0065	0.0000
rt_Private_room	-0.4981	0.0151	0.0000
rt_Shared_room	-1.1157	0.0366	0.0000
pg_Condominium	0.1073	0.0209	0.0000
pg_House	-0.0004	0.0137	0.9766
pg_Other	0.1208	0.0215	0.0000
pg_Townhouse	-0.0186	0.0323	0.5653
W_log_price	0.3416	0.0148	0.0000

Similarly to the effects in the SLX regression, changes in the spatial lag regression need to be interpreted with care. Here, `W_log_price` applies consistently over all observations and actually changes the effective strength of each of the β coefficients. Thus, it is useful to use predictions and scenario-building to predict y when changing X, which allows you to analyze the *direct* and *indirect* components.

11.4.3.4 Other ways of bringing space into regression

We have covered here only a few ways to formally introduce space in a regression framework. There are however many other advanced spatial regression methods routinely used in statistics, data science, and applied analysis. For example, Generalized Additive Models [GOP15][Woo06] have been used to apply spatial kernel smoothing directly within a regression function. Other similar smoothing methods, such as spatial Gaussian Process Models [BFC10] or Kriging, conceptualize the dependence between locations as smooth as well. Other methods in spatial regression that consider graph-based geographies (rather than distance/kernel effects) include variations on conditional autoregressive model, which examines spatial relationships at locations *conditional* on their surroundings, rather than as jointly co-emergent with them. Full coverage of these topics is beyond the scope of this book, however, though [BGFS08] provides a detailed and comprehensive discussion. We have not covered these (and other existing ones) not because we do not think are important or useful, far from it, but because we consider them a bit more advanced than the level at which we wanted to pitch the chapter.

Both sets of models we have introduced in here and others that exist but that we have not covered share the feature of embedding space as a "first class" citizen in the regression framework. In all these cases, we conceptualize space (e.g., through a spatial weights matrix) and *modify* the functional form of our original model in a way that recognizes the location of its observations when generating predictions or inference. This approach results in modifications of the modeling framework to accommodate geography. In the next chapter, we discuss a different perspective to embed space in our modeling efforts. Rather than injecting space through the functional form, we will do it through the variables we use to explain/predict the outcome of interest. In this case, we will be using geography to *enrich* our data *before* it is passed through a modeling framework.

11.5 Questions

1. One common kind of spatial econometric model is the "Spatial Durbin Model," which combines the SLX model with the spatial lag model. Alternatively, the "Spatial Durbin Error Model" combines the SLX model with the spatial error model. Fit a Spatial Durbin variant of the spatial models we have fit in this chapter.
    - Do these variants improve the model fit?
    - What happens to the spatial autocorrelation parameters (ρ, λ) when the SLX term is added? Why might this occur?
2. Fortunately for us, spatial error models recover the same estimates (asymptotically) as a typical OLS estimate, although their confidence intervals will

change. Statistically, this occurs because OLS miscalculates when there is spatial correlation and/or spatial heteroskedasticity. How much do the confidence intervals change when the spatial error model is fit?

3. One common justification for the SLX model (and the Spatial Durbin variants) is about *omitted, spatially patterned variables*. That is, if an omitted variable is associated with the included variables *and* is spatially patterned, then we can use the spatial structure of our existing variables to mimic the omitted variable. In our spatial lag model,

    - what variables might we be missing that are important to predict the price of an Airbnb?
    - would these omitted variables have a similar spatial pattern to our included variables? Why or why not?

4. Where *spatial* regression models generally focus on how nearby observations are similar to one another, *platial* models focus on how observations in the same spatial group are similar to one another. These are often dealt with using multi-level or spatial mixed-effect models. When do these two ideas work together well? And, when might these disagree?

11.5.1 Challenge questions

The following discussions are a bit challenging but reflect extra enhancements to the discussions in the chapter that may solidify or enhance an advanced understanding of the material.

11.5.1.1 The random coast

In the section analyzing our naive model residuals, we ran a classic two-sample t-test to identify whether or not our coastal and non-coastal residential districts tended to have the same prediction errors. Often, though, it's better to use straightforward, data-driven testing and simulation methods than assuming that the mathematical assumptions of the t-statistic are met.

To do this, we can shuffle our assignments to coast and not-coast, and check whether or not there are differences in the distributions of the *observed* residual distributions and random distributions. In this way, we shuffle the observations that are on the coast, and plot the resulting cumulative distributions.

Below, we run 100 simulated re-assignments of districts to either "coast" or "not coast," and compare the distributions of randomly assigned residuals to the observed distributions of residuals. Further, we do this plotting by the *empirical cumulative density function*, not the histogram directly. This is because the *empirical cumulative density function* is usually easier to examine visually, especially for subtle differences.

The black lines in Figure 11.8 represent our simulations, and the colored patches below them represent the observed distribution of residuals. If the black lines tend to be on the left of the colored patch, then, the simulations (where prediction error is totally random with respect to our categories of "coastal" and "not coastal") tend to have more negative residuals than our actual model. If the black lines tend to be on the right, then they tend to have more positive residuals. As a refresher, positive residuals mean that our model is under-predicting, and negative residuals mean that our model is over-predicting. Below, our simulations provide direct evidence for the claim that our model may be systematically under-predicting coastal price and over-predicting non-coastal prices.

```
n_simulations = 100
f, ax = plt.subplots(1, 2, figsize=(12, 3), sharex=True,␣
↪sharey=True)
ax[0].hist(
    coastal,
    color=["r"] * 3,
    alpha=0.5,
    density=True,
    bins=30,
    label="Coastal",
    cumulative=True,
)
ax[1].hist(
    not_coastal,
    color="b",
    alpha=0.5,
    density=True,
    bins=30,
    label="Not Coastal",
    cumulative=True,
)
for simulation in range(n_simulations):
    shuffled_residuals = m1.u[numpy.random.permutation(m1.n)]
    random_coast, random_notcoast = (
        shuffled_residuals[is_coastal],
        shuffled_residuals[~is_coastal],
    )
    if simulation == 0:
        label = "Simulations"
    else:
        label = None
    ax[0].hist(
        random_coast,
        density=True,
        histtype="step",
        color="k",
```

(continued on next page)

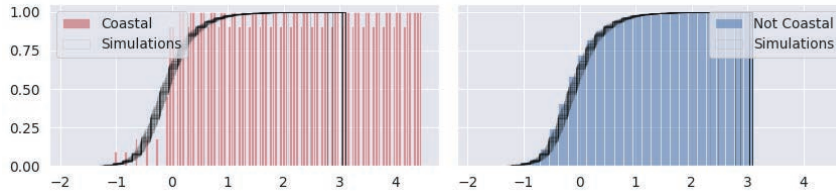


Fig. 11.8: Distributions showing the differences between coastal and non-coastal prediction errors. Some 'random' simulations are shown in black in each plot, where observations are randomly assigned to either 'Coastal' or 'Not Coastal' groups.

(continued from previous page)

```
        alpha=0.05,
        bins=30,
        label=label,
        cumulative=True,
    )
    ax[1].hist(
        random_coast,
        density=True,
        histtype="step",
        color="k",
        alpha=0.05,
        bins=30,
        label=label,
        cumulative=True,
    )
ax[0].legend()
ax[1].legend()
plt.tight_layout()
plt.show()
```

11.5.1.2 The K-neighbor correlogram

Further, it might be the case that spatial dependence in our mispredictions only matters for sites that are extremely close to one another, and it decays quickly with distance. To investigate this, we can examine the correlation between each site's residual and the *average* of the kth nearest neighbors' residuals, increasing k until the estimate stabilizes. This main idea is central to the geostatistical concept, the *correlogram*, which gives the correlation between sites of an attribute being studied as distance increases.

One quick way to check whether or not what we've seen is *unique* or *significant* is to compare it to what happens when we just assign neighbors randomly. If what we observe is substantially different from what emerges when neighbors are random, then the structure of the neighbors embeds a structure in the residuals. We won't spend too

much time on this theory specifically, but we can quickly and efficiently compute the correlation between our observed residuals and the spatial lag of an increasing k-nearest neighbor set:

```
correlations = []
nulls = []
for order in range(1, 51, 5):
    knn.reweight(
        k=order, inplace=True
    )  # operates in place, quickly and efficiently avoiding␣
↪copies
    knn.transform = "r"
    lag_residual = weights.spatial_lag.lag_spatial(knn, m1.u)
    random_residual = m1.u[numpy.random.permutation(len(m1.
↪u))]
    random_lag_residual = weights.spatial_lag.lag_spatial(
        knn, random_residual
    )  # identical to random neighbors in KNN
    correlations.append(
        numpy.corrcoef(m1.u.flatten(), lag_residual.
↪flatten())[0, 1]
    )
    nulls.append(
        numpy.corrcoef(m1.u.flatten(), random_lag_residual.
↪flatten())[
            0, 1
        ]
    )
```

And use the code below to generate Figure 11.9:

```
plt.plot(range(1, 51, 5), correlations)
plt.plot(range(1, 51, 5), nulls, color="orangered")
plt.hlines(
    numpy.mean(correlations[-3:]),
    *plt.xlim(),
    linestyle=":",
    color="k"
)
plt.hlines(
    numpy.mean(nulls[-3:]), *plt.xlim(), linestyle=":", color=
↪"k"
)
plt.text(
    s="Long-Run Correlation: ${:.2f}$".format(
        numpy.mean(correlations[-3:])
    ),
    x=25,
```

(continued on next page)

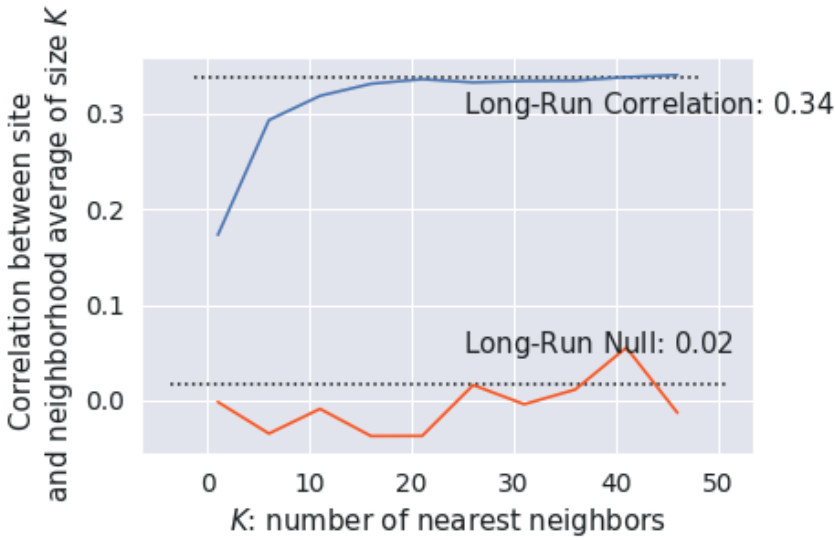


Fig. 11.9: Correlogram showing the change in correlation between prediction error at an Airbnb and its surroundings as the number of nearest neighbors increases. The null hypothesis, where residuals are shuffled around the map, shows no significant correlation at any distance.

(continued from previous page)

```
    y=0.3,
)
plt.text(
    s="Long-Run Null: ${:.2f}$".format(numpy.mean(nulls[-
↪3:])),
    x=25,
    y=0.05,
)
plt.xlabel("$K$: number of nearest neighbors")
plt.ylabel(
    "Correlation between site \n and neighborhood average of␣
↪size $K$"
)
plt.show()
```

Clearly, the two curves are different. The observed correlation reaches a peak around $r = .34$ when around 20 nearest listings are used. This means that adding more than 20 nearest neighbors does not significantly change the correlation in the residuals. Further, the lowest correlation is for the single nearest neighbor, and correlation rapidly increases as more neighbors are added close to the listing. Thus, this means that there does appear to be an unmeasured spatial structure in the residuals, since they are more similar to one

another when they are near than when they are far apart. Further, while it's not shown here (since computationally, it becomes intractable), as the number of nearest neighbors gets very large (approaching the number of observations in the dataset), the average of the kth nearest neighbors' residuals goes to zero, the global average of residuals. This means that the correlation of the residuals and a vector that is nearly constant begins to approach zero.

The null correlations, however, use randomly chosen neighbors (without reassignment). Thus, since sampling is truly random in this case, each average of k randomly chosen neighbors is usually zero (global mean). So, the correlation between the observed residual and the average of k randomly chosen residuals is also usually zero. Thus, increasing the number of randomly chosen neighbors does not significantly adjust the long-run average of zero. Taken together, we can conclude that there is distinct positive spatial dependence in the error. This means that our over- and under-predictions are likely to cluster.

11.6 Next steps

For additional reading on the topics covered in this chapter, please consult the following resources:

For a more in-depth discussion of the fundamentals of spatial econometrics and applications in both GUI and command-line software, consult:

Anselin, Luc and Sergio Rey. 2014. *Modern Spatial Econometrics in Practice: A Guide to GeoDa, GeoDaSpace, and Pysal.* GeoDa Press.

For additional mathematical detail and more extensive treatment of space-time models, consult:

Cressie, Noel and Christopher N. Wikle. 2011. *Statistics for Spatio-Temporal Data*. Singapore: Wiley Press.

For an alternative perspective on regression and critique of the spatial econometric perspective, consider:

Gibbons, Stephen and Henry G. Overman. 2012. "Mostly Pointless Spatial Econometrics." *Journal of Regional Science* 52: 172-191.

And for a useful overview of the discussions around multi-level modeling, with references therein to further resources, consider:

Owen, Gwilym, Richard Harris, and Kelvyn Jones. "Under Examination: Multilevel models, geography and health research." *Progress in Human Geography* 40(3): 394-412.

12

Spatial Feature Engineering

In machine learning and data science, we are often equipped with *tons* of data. Indeed, given the constellation of packages to query data services, free and open source datasets, and the rapid and persistent collection of geographical data, there is simply too much data to even represent coherently in a single, tidy fashion. However, we often need to be able to construct useful *features* from this rich and deep sea of data.

Where data is available, but not yet directly *usable*, *feature engineering* helps to construct representations or transformations of the original data that are more useful for modeling a given phenomenon of interest. In traditional machine learning, feature engineering involves applying additional *domain knowledge* to raw information in order to structure it in a manner that is meaningful for a model. Often, this involves some sort of *transformation* of the original dataset, which is a well-studied concept in both classical statistics and remains so in machine learning methods. While *feature engineering* always relies on this implicit domain knowledge, it is an extremely important part of adapting general-purpose algorithms to unique or distinctive problems facing the every-day data scientist.

Geography is one of the most high-quality, ubiquitous ways to introduce *domain knowledge* into a problem: everything has a position in both *space* and *time*. And, while things that happen near one another *in time* do not necessarily have a fundamental relationship, things that are *near* one another are often related. Thus, space is the ultimate *linkage key*, allowing us to connect different datasets together in order to improve our models and our predictions. This means that, even for *aspatial*, "non-geographic" data, you can use *spatial feature engineering* to create useful, highly relevant features for your analysis.

At its core, *spatial feature engineering* is the process of developing additional information from raw data using *geographic knowledge*. This distilling of information can occur *between* datasets, where geography is used to link information in separate datasets

together; or *within* datasets, where geography can be used to augment the information available for one sample by borrowing from nearby ones. This chapter is structured following that distinction: for cases where geography connects *different* datasets, we adopt the term "Map Matching", often used in industry; while we use the mirroring concept of "Map Synthesis" describing the use of geographical structure to derive new features from a given dataset. Technically speaking, some of the methods we review are similar across these two cases, or even the same; however, they can be applied in the context of "matching" or "synthesis", and we consider those conceptually different, hence their inclusion in both sections. Throughout the chapter, we use the Airbnb nightly rental prices in San Diego, as well as auxiliary datasets such as elevation or census demographics.

```
import geopandas, pandas, libpysal.weights as weights,␣
↪contextily
import matplotlib.pyplot as plt
import numpy
import cenpy
import osmnx
import seaborn
import rasterio
from rasterio.plot import show as rioshow
```

Throughout this chapter, we will use the same dataset to which we want to append more information or augment, and we will do so through leveraging geography. For the illustration, we will use the set of *Airbnb properties* we have used in *Chapter 11*. Let's read it:

```
airbnbs = geopandas.read_file("../data/airbnb/regression_db.
↪geojson")
```

12.1 What is spatial feature engineering?

At its core, *spatial feature engineering* is the process of developing additional information from raw data using *geographic knowledge*. This synthesis could occur *between* datasets, where geography is used to link samples in separate datasets together; or *within* datasets, where geography can be used to borrow information from nearby samples. Building linkages *between* datasets is often called "Map Matching", while we use the term "Map Synthesis" to describe the use of geographical structure to derive new features from existing data. We will cover both kinds of feature engineering.

To help us discuss this, a vocabulary is helpful. We will cover a few different kinds of features in this chapter. All of them can be constructed using either Map Synthesis or Map Matching operations:

- A *spatial summary feature* measures the attributes of observations that have some pre-specified spatial relationship with our target observations. This includes:

  - Taking the average or median value of features within a neighborhood of each target observation.
  - The *spatial lag*, used in previous chapters of this book, e.g., Chapters *3*, *6*, and *11*, is a kind of spatial summary feature, since it reflects the average value of the data in the neighborhood around each point.
  - Other kinds of spatial summary features might include the count of observations within a given distance or the standard deviation of ten nearest observations.
  - Summary features generally include *interpolated features* which involve a transfer of information from one spatial support to another, such as when the target locations are not the same as the locations in our ancillary data. Interpolated features become significantly more complex as well when the data is *areal*, as will be discussed later in this chapter.

- A *proximity feature* measures the distance from a target observation to some other observation or position in the map. This might be done in a map matching context, as we did before with the distance to Balboa Park in *Chapter 11*, or it might be done in a map synthesis context by measuring the distance to the nearest other observation.

12.2 Feature engineering using map matching

Space is the ultimate linkage key; map matching is all about exploiting this asset. Geographic information is collected at different scales, aggregated using bespoke geographic delineations, and ultimately stored in different datasets. Modeling and machine learning usually require the use of data aligned and coherently structured; to bring disparate geo-datasets together, the data scientist needs to be able to transfer information expressed for one geography into another. This is where map matching comes to the rescue. By using geographical location, we can connect datasets that have no common key or that are otherwise completely unrelated. This is a "magic trick" that we can pull off because of location that would be much harder, or impossible, with data lacking spatial information.

The details, and difficulty, of this transfer of information from one geography to another one depend on the nature of the "source" and "target", and on the precision with which we want to perform such transfer. In this context, there is always an inevitable trade-off between precision and sophistication: more precise transfer is usually possible at the expense of more advanced and involved techniques. Their additional cost in applying them, both in terms of cognitive load on the data scientist's part or in terms of computation, must be weighted in on an individual basis: sometimes we need the best possible estimates, at all costs; sometimes "good enough" is, well, good enough.

In this section, we cover a few cases that we consider represent the most common and widely used approaches. We begin with situations where we have two point datasets and all we need to calculate are descriptive statistics of one dataset to the observations

in the other. We switch the type of data and consider how to attach information from a continuous grid, stored in a raster file, to a set of points. Then we show the equivalent case for "moving" information for a polygon geography to a point dataset. These are all cases that, in their simplest form, involve mostly traditional GIS operations (e.g., buffer construction, spatial joins) and little in the way of statistical modeling. As an example of a case that is more involved, we cover the transfer of information from a set of points to another set of points in different locations, and of a polygon geography to another different polygon geography. For these final cases, but also more generally, we try to include examples that capture the essence of the method but keep the technique as simple as possible. For almost any of these cases we cover, the reader can find more sophisticated techniques that usually yield more accurate estimates. Where possible, we try to signpost these.

12.2.1 Counting *nearby* features

A first, conceptually straightforward, approach is to augment our dataset by counting how many points of a different dataset are in the vicinity of each observation. For example, we might want to know how many bars and restaurants each Airbnb has within a given radius. This count can then become an additional feature of our dataset, stored in a new column of `airbnbs`.

To obtain information on the location of restaurants and bars, we can download it from OpenStreetMap directly using `osmnx`. We first query all the points of interest (POIs) within the area our points cover, and then filter out everything except restaurants and bars. For that, we require a polygon that covers all our `airbnbs` points. From *Chapter 8*, we can recall that there are a few different hulls that can be used. We'll use the Convex Hull here (Figure 12.1), which is the smallest convex polygon that covers all the points in the set.

```
airbnbs_ch = airbnbs.unary_union.convex_hull
geopandas.GeoSeries([airbnbs_ch]).plot()
```

```
<AxesSubplot:>
```

Using this polygon, we can use the `osmnx` package to fetch POIs from OpenStreetMap. We can make our request more manageable by only requesting POIs that fall within specific categories. Below, we'll request POIs within San Diego that are "restaurants" or "bars," according to their metadata stored in OpenStreetMap. Once returned, we only use a few columns to keep the table small and tidy:

```
%%time
pois = osmnx.geometries_from_polygon(
    airbnbs_ch, tags={"amenity": ["restaurant", "bar"]}
).reset_index()[
```

(continued on next page)

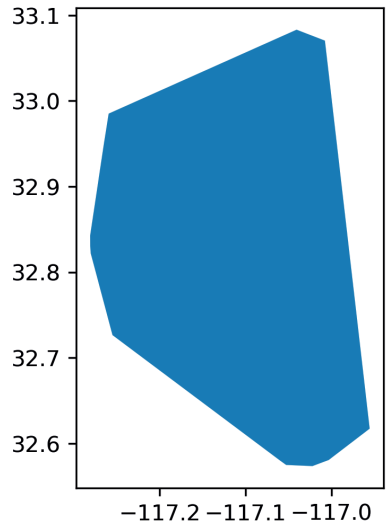


Fig. 12.1: Convex hull of the Airbnbs in San Diego.

(continued from previous page)

```
    [
        "element_type",
        "osmid",
        "amenity",
        "cuisine",
        "name",
        "geometry",
    ]
]
```

```
CPU times: user 1.75 s, sys: 47.5 ms, total: 1.8 s
Wall time: 1.81 s
```

The code snippet above sends a query to the OpenStreetMap server to fetch the data on amenities. Note that it *requires* internet connectivity to work. If you are working on the book *without* connectivity, a cached version of the dataset is available on the data folder and can be read as:

```
pois = geopandas.read_file("../data/cache/sd_pois.gpkg")
```

This provides us with every location within our convex hull that is tagged in the metadata stored in OpenStreetMap as a “restaurant” or “bar”. Overall, this provides us with about 1300 POIs:

```
pois.groupby("amenity").amenity.count()
```

```
amenity
bar             308
restaurant     1060
Name: amenity, dtype: int64
```

Once loaded into `pois` as a `GeoDataFrame`, we use the code below to generate Figure 12.2, which takes a peek at their location, as compared with Airbnb spots:

```
# Set up figure and axis
f, ax = plt.subplots(1, figsize=(12, 12))
# Plot Airbnb properties
airbnbs.plot(ax=ax, marker=".")
# Plot POIs in red
pois.plot(ax=ax, color="r")
# Add Stamen's Toner basemap
contextily.add_basemap(
    ax,
    crs=airbnbs.crs.to_string(),
    source=contextily.providers.Stamen.Toner,
)
# Remove axes
ax.set_axis_off()
# Display
plt.show()
```

Now for some feature engineering. It may be extremely useful to know whether an Airbnb is located in a "hot" location, with a lot of restaurants and bars to choose from. Alternatively, if Airbnbs are very remote, they might not be as lucrative for short, expensive "city-break" reservations. That is, Airbnb users may decide to reserve stays where there are a lot of dining and drinking opportunities, and thus may be *willing to pay more* for the same accommodation. We might be able to predict prices better if we know about the drinking and dining scene near the Airbnb.

Thus, we can *engineer features* in the Airbnb data using the nearby POIs. To do this, we can create a new feature for the Airbnb dataset –that is, a new column in `airbnbs`– which incorporates information about how many POIs are *nearby* each property. This kind of "feature counting" is useful in applications where the mere presence of nearby features can affect the quantity we are modeling.

To do this kind of feature engineering, let us first walk through what we need to do at a conceptual level:

1. Decide what is *nearby*. This will dictate how far we go from each Airbnb when counting the number of "nearby" bars and restaurants. For this

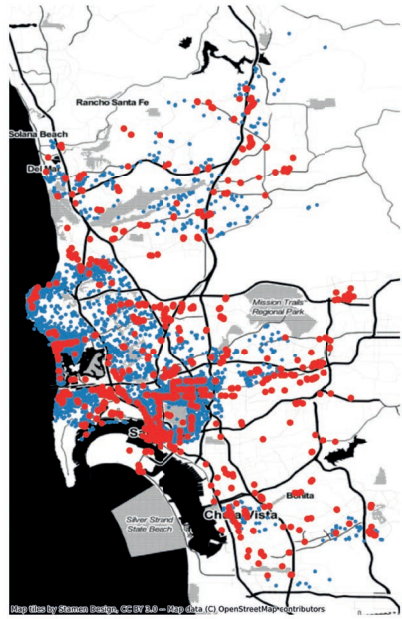


Fig. 12.2: Points of interest (POIs) and Airbnbs in San Diego.

example, we will use a 500-meter buffer, which is approximately the distance of a leisurely ten-minute walk.

2. For each Airbnb, determine whether POIs are *within* a leisurely ten-minute walk.
3. Count how many POIs are within the specified radius of each Airbnb.

At the end of this procedure, we have the number of bars and restaurants that are within a leisurely walk of the Airbnb, which might be useful in predicting the price of each Airbnb.

Let us now translate the list above into code. For part 1., we need to be able to measure distances in meters. However, `airbnbs` is originally expressed in degrees, since it is provided in terms of locations in latitude and longitude:

```
airbnbs.crs
```

```
<Geographic 2D CRS: EPSG:4326>
Name: WGS 84
Axis Info [ellipsoidal]:
- Lat[north]: Geodetic latitude (degree)
- Lon[east]: Geodetic longitude (degree)
```

(continued on next page)

(continued from previous page)

```
Area of Use:
- name: World.
- bounds: (-180.0, -90.0, 180.0, 90.0)
Datum: World Geodetic System 1984 ensemble
- Ellipsoid: WGS 84
- Prime Meridian: Greenwich
```

In addition, the `pois` are also provided in terms of their latitude and longitude:

```
pois.crs
```

```
<Geographic 2D CRS: EPSG:4326>
Name: WGS 84
Axis Info [ellipsoidal]:
- Lat[north]: Geodetic latitude (degree)
- Lon[east]: Geodetic longitude (degree)
Area of Use:
- name: World.
- bounds: (-180.0, -90.0, 180.0, 90.0)
Datum: World Geodetic System 1984 ensemble
- Ellipsoid: WGS 84
- Prime Meridian: Greenwich
```

Therefore, we need to convert this into a coordinate system that is easier to work with. Here, we will use a projection common for mapping in California, the California Albers projection:

```
airbnbs_albers = airbnbs.to_crs(epsg=3311)
pois_albers = pois.to_crs(epsg=3311)
```

```
pois_albers.crs
```

```
<Projected CRS: EPSG:3311>
Name: NAD83(HARN) / California Albers
Axis Info [cartesian]:
- X[east]: Easting (metre)
- Y[north]: Northing (metre)
Area of Use:
- name: United States (USA) - California.
- bounds: (-124.45, 32.53, -114.12, 42.01)
Coordinate Operation:
- name: California Albers
- method: Albers Equal Area
Datum: NAD83 (High Accuracy Reference Network)
- Ellipsoid: GRS 1980
- Prime Meridian: Greenwich
```

With this, we can create the radius of 500 meters around each Airbnb. This is often called *buffering*, where a shape is dilated by a given radius.

```
airbnbs_albers["buffer_500m"] = airbnbs_albers.buffer(500)
```

Now, `abb_buffer` contains a 500-meter circle around each Airbnb.

Using these, we can count the number of POIs that are within these areas using a *spatial join*. Spatial joins link geometries based on spatial relationships (or predicates). Here, we need to know the relationship: `pois within airbnb_buffers`, where `within` is the predicate relating `pois` to `airbnb_buffers`. Predicates are not always *reversible*: no `airbnb_buffer` can be `within` a `poi`. In `geopandas`, we can compute all pairs of relations between the `pois` and `airbnb_buffers` efficiently using the `sjoin` function, which takes a `predicate` argument defining the requested relationship between the first and second argument.

```
# Spatial join, appending attributes from right table to left␣
↪one
joined = geopandas.sjoin(
    # Right table - POIs
    pois_albers,
    # Left table - Airbnb with the geometry reset from the␣
↪original
    # points to the 500-meter buffer and selecting only `id`␣
↪and
    # `buffer_500m` column
    airbnbs_albers.set_geometry("buffer_500m")[["id", "buffer_
↪500m"]],
    # Operation (spatial predicate) to use for the spatial␣
↪join (`within`)
    predicate="within",
)
```

The resulting `joined` object contains a row for every pair of POI and Airbnb that are linked. From there, we can apply a group-by operation, using the Airbnb ID, and count how many POIs each Airbnb has within 500 meters of distance:

```
# Group POIs by Airbnb ID (`id`)
poi_count = (
    joined.groupby(
        "id"
        # Keep only POI id column (`osmid`)
    )[
        "osmid"
        # Count POIs by Airbnb + convert Series into DataFrame
    ]
    .count()
    .to_frame("poi_count")
```

(continued on next page)

(continued from previous page)

```
)
# Print top of the table
poi_count.head()
```

```
       poi_count
id
6             12
5570           7
9553          12
38245          1
69385          8
```

The resulting `Series` is indexed on the Airbnb IDs, so we can assign it to the original `airbnbs` table. In this case, we know by construction that missing Airbnbs in `poi_count` do not have any POI within 500m, so we can fill missing values in the column with zeros.

```
airbnbs_w_counts = airbnbs_albers.merge(
    poi_count, left_on="id", right_index=True
).fillna({"poi_count": 0})
```

We can visualize now (Figure 12.3) the distribution of counts to get a sense of how "well-served" Airbnb properties are arranged over space:

```
# Set up figure and axis
f, ax = plt.subplots(1, figsize=(9, 9))
# Plot quantile map of No. of POIs for every Airbnb
airbnbs_w_counts.plot(
    column="poi_count",
    scheme="quantiles",
    alpha=0.5,
    legend=True,
    ax=ax,
)
# Add basemap
contextily.add_basemap(
    ax,
    crs=airbnbs_albers.crs.to_string(),
    source=contextily.providers.Stamen.Toner,
)
# Remove axes
ax.set_axis_off();
```

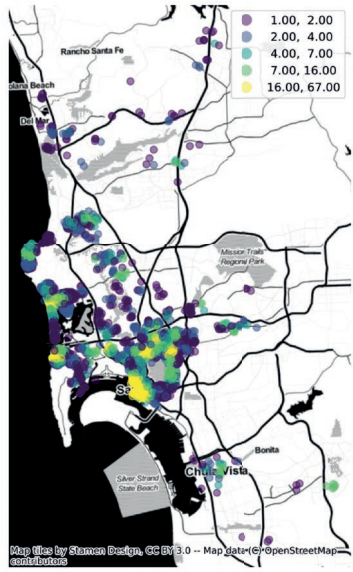


Fig. 12.3: Number of POIs within 500 meters of each Airbnb.

12.2.2 Assigning point values from surfaces: elevation of Airbnbs

We have just seen how to count points around each observation in a point dataset. In other cases, we might be confronted with a related but different challenge: transferring the value of a particular point in a surface to a point in a different dataset.

To make this more accessible, let us illustrate the context with an example question: *what is the elevation of each Airbnb property?* To answer this question, we require, at least, the following:

1. A sample of Airbnb property locations.
2. A dataset of elevation. We will use here the NASA DEM surface for the San Diego area.

We use the code below, which opens and plot the file with elevation data, to generate Figure 12.4:

```
# Open file
dem = rasterio.open("../data/nasadem/nasadem_sd.tif")
# Set up figure and axis
f, ax = plt.subplots()
# Display elevation data on created axis
```

(continued on next page)

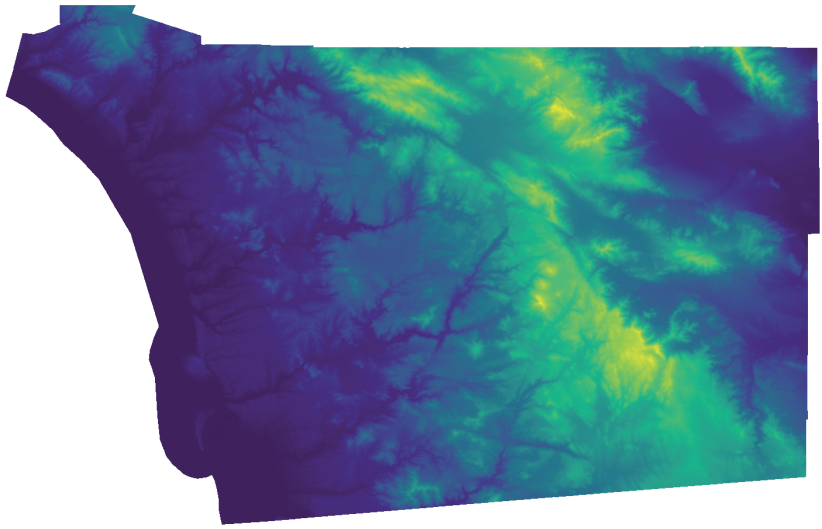


Fig. 12.4: Digital elevation model of the San Diego area.

(continued from previous page)

```
rioshow(dem, ax=ax)
# Remove axes
ax.set_axis_off();
```

Let's first check the CRS is aligned with our sample of point locations:

```
dem.crs
```

```
CRS.from_epsg(4326)
```

We have opened the file with `rasterio`, which has not read the entire dataset just yet. This feature allows us to use this approach with files that are potentially very large, as only requested data is read into memory. To extract a discrete set of values from the elevation surface in `dem`, we can use `sample`. For a single location, this is how it works:

```
list(dem.sample([(-117.24592208862305, 32.761619109301606)]))
```

```
[array([7], dtype=int16)]
```

Now, we can apply this logic to a sequence of coordinates. For that, we need to extract them from the `geometry` object:

```
# Create table with XY coordinates of Airbnb locations
abb_xys = pandas.DataFrame(
    {"X": airbnbs.geometry.x, "Y": airbnbs.geometry.y}
    # Convert from DataFrame to array of XY pairs
).to_records(index=False)
```

And then we can apply the same logic as above to sample elevation for the list of Airbnb locations:

```
# Save results as a DataFrame
elevation = pandas.DataFrame(
    # Sequence of elevation measurements sampled at the␣
↪Airbnb locations
    dem.sample(abb_xys),
    # Name of the column to be created to store elevation
    columns=["Elevation"],
    # Row index, mirroring that of Airbnb locations
    index=airbnbs.index,
)
# Print top of the table
elevation.head()
```

```
   Elevation
0        112
1          3
2         99
3        114
4         30
```

Now that we have a table with the elevation of each Airbnb property, we can plot the site elevations on a map (Figure 12.5) for visual inspection:

```
# Set up figure and axis
f, ax = plt.subplots(1, figsize=(9, 9))
# Join elevation data to original Airbnb table
airbnbs.join(
    elevation
    # Plot elevation at each Airbnb location as a quantile␣
↪choropleth
).plot(
    column="Elevation",
    scheme="quantiles",
    legend=True,
    alpha=0.5,
    ax=ax,
)
# Add Stamen's terrain basemap
```

(continued on next page)

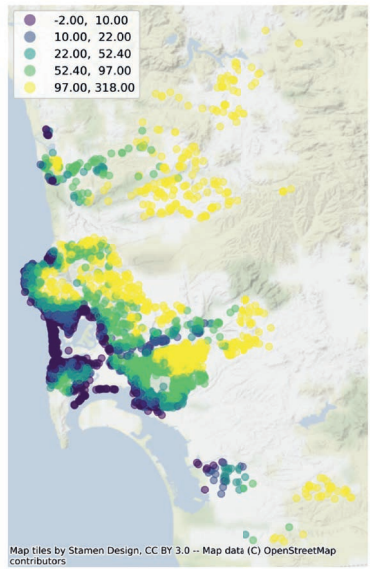


Fig. 12.5: Elevation above sea level at each Airbnb.

(continued from previous page)

```
contextily.add_basemap(
    ax,
    crs=airbnbs.crs.to_string(),
    source=contextily.providers.Stamen.TerrainBackground,
    alpha=0.5,
)
# Remove axes
ax.set_axis_off();
```

12.2.3 Point interpolation using scikit-learn

In the previous example, we have transferred information from a surface (stored in a raster layer) to a set of points; in other words, we have gone from surface to points. Sometimes, however, we do not have the luxury of a ready-made surface. Instead, all we have available is a set of points with measurements for the variable of interest that do not match the points we want the information for. In this situation, a solution we can rely on is *spatial interpolation*. For a continuous geographical field measured at a set of points, *spatial interpolation* methods provide us a way to guess at the value a field would take at sites we do not measure.

There are many sophisticated methods with which this can be done. *Kriging*, common in the sub-field of spatial statistics called "geostatistics", is one such practice based on the theory of *Gaussian Process Regression.* Another common approach, *geographically weighted regression*, provides unique model estimates at every control point, as well as predictions for places where there is no data. Here, though, we'll use a fairly basic k-nearest neighbor prediction algorithm from `scikit-learn` to demonstrate the process.

```
from sklearn.neighbors import KNeighborsRegressor
```

This algorithm will select the nearest ten listings, then compute the prediction using a weighted average of these nearest observations. To keep predictions relatively consistent, we will build an interpolation only for listings that are entire homes/apartments with two bedrooms:

```
two_bed_homes = airbnbs[
    airbnbs["bedrooms"] == 2 & airbnbs["rt_Entire_home/apt"]
]
```

Once subset, we can extract the XY coordinates for each of them into a two-dimensional array:

```
two_bed_home_locations = numpy.column_stack(
    (two_bed_homes.geometry.x, two_bed_homes.geometry.y)
)
```

To plot the interpolated surface, we must also construct a grid of locations for which we will make predictions. This can be done using `numpy.meshgrid`, which constructs all the combinations of the input dimensions as a grid of outputs.

```
# Extract bounding box of Airbnb locations
xmin, ymin, xmax, ymax = airbnbs.total_bounds
# Generate X and Y meshes for the space within the bounding␣
↪box
x, y = numpy.meshgrid(
    numpy.linspace(xmin, xmax), numpy.linspace(ymin, ymax)
)
```

To build an intuition on what they are we create Figure 12.6, which visualizes both meshes side-by-side:

```
# Set up figure
f, ax = plt.subplots(1, 2)
# Plot X mesh
ax[0].imshow(x)
# Plot Y mesh
ax[1].imshow(y)
```

(continued on next page)

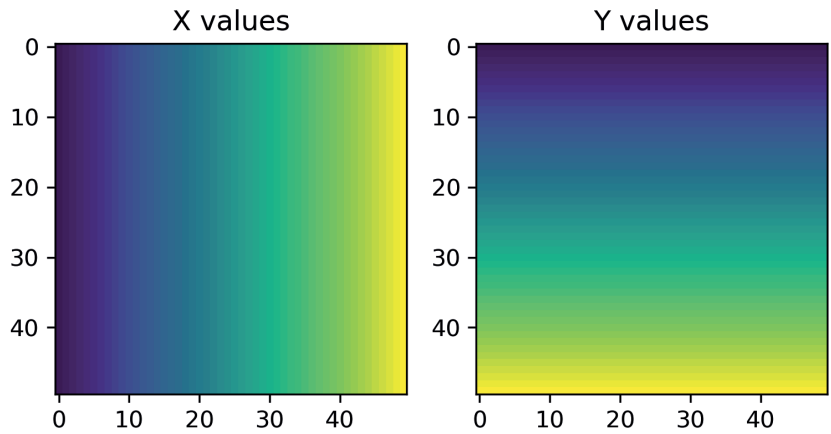


Fig. 12.6: Example grid showing the coordiantes used for interpolation.

(continued from previous page)

```
# Label X mesh
ax[0].set_title("X values")
# Label Y mesh
ax[1].set_title("Y values")
# Display
plt.show()
```

With these coordinates, we can make a GeoDataFrame containing the grid cells at which we would like to predict:

```
# Bind a 1D (flattened) of X and Y meshes into a 2D array
grid = numpy.column_stack((x.flatten(), y.flatten()))
# Convert a 1D (flattened) version of X and Y meshes into a␣
↪geo-table
grid_df = geopandas.GeoDataFrame(
    geometry=geopandas.points_from_xy(x=x.flatten(), y=y.
↪flatten())
)
```

We can visualize this grid together with the original Airbnb locations (Figure 12.7) to get a better sense of what we have just built:

```
# Plot grid points with size 1
ax = grid_df.plot(markersize=1)
# Plot on top Airbnb locations in red
two_bed_homes.plot(ax=ax, color="red")
# Add basemap
```

(continued on next page)

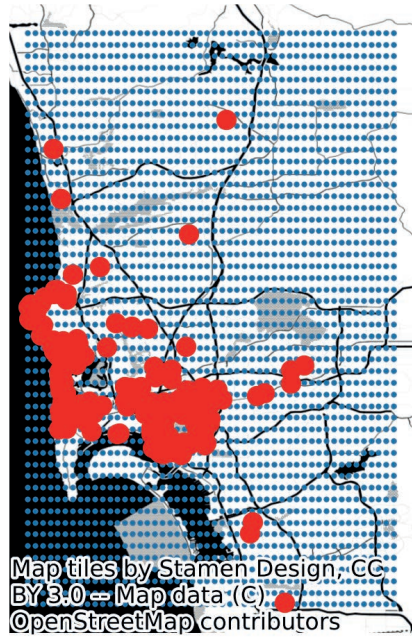


Fig. 12.7: Grid underlaid Airbnb locations used for interpolation.

(continued from previous page)

```
contextily.add_basemap(
    ax,
    crs=two_bed_homes.crs.to_string(),
    source=contextily.providers.Stamen.TonerBackground,
)
# Remove axes
ax.set_axis_off();
```

With this done, we can now construct the predictions. First we train the model:

```
model = KNeighborsRegressor(n_neighbors=10, weights="distance
↪").fit(
    two_bed_home_locations, two_bed_homes.price
)
```

And then we predict at the grid cell locations:

```
predictions = model.predict(grid)
```

The result can be displayed as a continuous choropleth (Figure 12.8), for example:

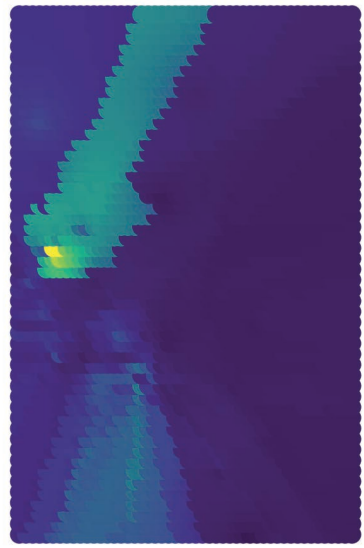


Fig. 12.8: Predicted Airbnb price using ten nearest neighbor interpolation.

```
ax = grid_df.plot(predictions)
ax.set_axis_off();
```

The map is a result not only of the underlying data but also the algorithm we have used. For example, you can see in Figure 12.9 (generated with the code below) that the surface gets smoother as you increase the number of nearest neighbors to consider.

```
# Set up figure and 8 axes
f, ax = plt.subplots(1, 8, figsize=(16, 4))
# Loop over eight values equally spaced between 2 and 100
for i, k_neighbors in enumerate(
    numpy.geomspace(2, 100, 8).astype(int)
):
    # Select axis to plot estimates for that value of k
    facet = ax[i]
    # Set up a KNN regressor instance with k_neighbors␣
↪neighbors
    predictions = (
        KNeighborsRegressor(
            n_neighbors=k_neighbors,
            weights="distance"
            # Fit instance to the 2-bed subset
        )
```

(continued on next page)

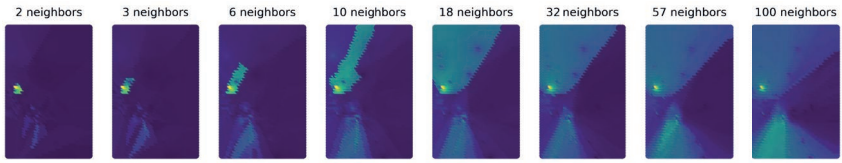


Fig. 12.9: Predicted nightly price using a varying number of nearest neighbors. Note the plot smooths considerably as more neighbors are added.

(continued from previous page)

```
        .fit(
            two_bed_home_locations,
            two_bed_homes.price
            # Get predictions for locations on the grid
        )
        .predict(grid)
    )
    # Plot predictions
    grid_df.plot(predictions, ax=facet)
    # Remove axis
    facet.axis("off")
    # Set axis title
    facet.set_title(f"{k_neighbors} neighbors")
```

Focusing in on central San Diego tells the story a bit more clearly, since there are interesting points there to show. The increasing number of nearest neighbors increases the smoothness of the interpolated surface. To visualize a zoom into the region, we begin by extracting its bounding box:

```
central_sd_bounds = [-117.179832, 32.655563, -117.020874, 32.
↪769909]
(
    central_xmin,
    central_ymin,
    central_xmax,
    central_ymax,
) = central_sd_bounds
```

And repeating the process above for this area, including building the meshes of coordinates:

```
# Build X and Y meshes
central_x, central_y = numpy.meshgrid(
    numpy.linspace(central_xmin, central_xmax),
    numpy.linspace(central_ymin, central_ymax),
)
```

(continued on next page)

(continued from previous page)

```
# Bind X and Y meshes in a 2D array
central_grid = numpy.column_stack(
    (central_x.flatten(), central_y.flatten())
)
# Bind X and Y meshes in a geo-table
central_grid_df = geopandas.GeoDataFrame(
    geometry=geopandas.points_from_xy(
        x=central_x.flatten(), y=central_y.flatten()
    )
)
```

Finally, we can reproduce the sequence of figures with different values of K only for the central part of the San Diego area (Figure 12.10):

```
# Set up figure and subplot
f, ax = plt.subplots(1, 5, figsize=(16, 4), sharex=True,␣
↪sharey=True)
# Loop over five values equally spaced between 2 and 100
for i, k_neighbors in enumerate(
    numpy.geomspace(2, 100, 5).astype(int)
):
    # Select axis to plot estimates for that value of k
    facet = ax[i]
    # Set up a KNN regressor instance with k_neighbors␣
↪neighbors
    predictions = (
        KNeighborsRegressor(
            n_neighbors=k_neighbors,
            weights="distance"
            # Fit instance to the 2-bed subset
        )
        .fit(
            two_bed_home_locations,
            two_bed_homes.price
            # Get predictions for locations on the grid
        )
        .predict(central_grid)
    )
    # Plot predictions
    central_grid_df.plot(predictions, ax=facet)
    # Remove axis
    facet.axis("off")
    # Set axis title
    facet.set_title(f"{k_neighbors} neighbors")
```

Fig. 12.10: Focus on downtown San Diego predictions for nearest neighbor interpolation.

Plenty more of these methods are implemented in `scikit-learn.neighbors`, as well as kriging methods in `GSTools` [MS21] and geographically weighted regression in `mgwr` [OLK+19].

12.2.4 Polygon to point

We now move on to a case where the information we are interested in matching to our set of points is stored in a polygon geography. For example, we would like to know the population density of the neighborhood in which each Airbnb is located. This is conceptually similar to the sampling example above, but in the case the information we want to sample is not a surface but a geo-table of polygons. To determine density, we will download population estimates at the census tract level, and "transfer" those estimates over to each Airbnb point. Geographically, the only challenge here is finding the containing polygon for each point, and then performing what is in spatial databases parlance known as a "spatial join", by which we link the two layers through their spatial connection.

Let us pull down the number of inhabitants from the American Community Survey for tracts in San Diego:

```
try:
    acs = cenpy.products.ACS()
    sd_pop = acs.from_msa(
        "San Diego, CA", level="tract", variables=["B02001_
↪001E"]
    )
    sd_pop.to_file("../data/cache/sd_census.gpkg", driver=
↪'GPKG')
except:
    print("Remote loading failed...")
```

```
Remote loading failed...
```

The code snippet above sends a query to the Census Bureau server to fetch the data for San Diego. Note that it *requires* internet connectivity to work. If you are working on the book *without* connectivity, a cached version of the dataset is available on the data folder and can be read as:

```
sd_pop = geopandas.read_file("../data/cache/sd_census.gpkg")
```

And calculate population density:

```
sd_pop["density"] = (
    sd_pop["B02001_001E"] / sd_pop.to_crs(epsg=3311).area
)
```

Now, to "transfer" density estimates to each Airbnb, we can rely on the spatial join in `geopandas`:

```
j = geopandas.sjoin(airbnbs, sd_pop.to_crs(airbnbs.crs))
```

The result is a table with one row per Airbnb and one column for each attribute we originally had for properties, *as well as* those of the tract where the area is located:

```
j.info()
```

```
<class 'geopandas.geodataframe.GeoDataFrame'>
Int64Index: 6110 entries, 0 to 6068
Data columns (total 28 columns):
 #   Column               Non-Null Count  Dtype
---  ------               --------------  -----
 0   accommodates         6110 non-null   int64
 1   bathrooms            6110 non-null   float64
 2   bedrooms             6110 non-null   float64
 3   beds                 6110 non-null   float64
 4   neighborhood         6110 non-null   object
 5   pool                 6110 non-null   int64
 6   d2balboa             6110 non-null   float64
 7   coastal              6110 non-null   int64
 8   price                6110 non-null   float64
 9   log_price            6110 non-null   float64
 10  id                   6110 non-null   int64
 11  pg_Apartment         6110 non-null   int64
 12  pg_Condominium       6110 non-null   int64
 13  pg_House             6110 non-null   int64
 14  pg_Other             6110 non-null   int64
 15  pg_Townhouse         6110 non-null   int64
 16  rt_Entire_home/apt   6110 non-null   int64
 17  rt_Private_room      6110 non-null   int64
 18  rt_Shared_room       6110 non-null   int64
 19  geometry             6110 non-null   geometry
 20  index_right          6110 non-null   int64
 21  GEOID                6110 non-null   object
 22  B02001_001E          6110 non-null   float64
 23  NAME                 6110 non-null   object
```

(continued on next page)

(continued from previous page)

```
 24  state                6110 non-null   object
 25  county               6110 non-null   object
 26  tract                6110 non-null   object
 27  density              6110 non-null   float64
dtypes: float64(8), geometry(1), int64(13), object(6)
memory usage: 1.4+ MB
```

12.2.5 Area to area interpolation

The final case of map matching we consider is transfer of information from one polygon/areal geography to a different one. This is a common use-case when an analysis requires data that is provided at different levels of aggregation and different boundary delineations.

There is a large literature around this problem under the umbrella of dasymetric mapping [EB01]. The conceptual idea is relatively straightforward: we want to apportion values from one set of polygons to another based on how much "geography" is shared. In its simplest case, we can do this based on area. We will assign values from the former geography to the latter one based on how much they share. Let us illustrate this with an example. We will call the geography for which we have data the "source", and that to which we want to transfer data the "target". Let's say we have a population count measure for each of three source polygons, 1, 2, 3, and we have a target polygon A for which we require an estimated population. If the intersection of polygon A with each of the source polygons amounts to 50%, 30%, and 20% of the total area of polygon A, respectively, then the population estimate for A will be a weighted average between the rates in 1, 2, and 3, where the weights are 0.5, 0.3, and 0.2, respectively.

We could also go in the other direction. If we had total population for polygon A, but population was not reported for polygons 1, 2, 3, we could use areal interpolation to allocate the population from A (now the source polygon) to each of the three target polygons. Polygon 1 would get 0.5 of the population of A, polygon 2 would get 0.3 of A's population, and C the remainder. Here we are assuming the area of A is completely exhausted by the areas of 1, 2, and 3, and that the latter polygons do not overlap with each other. Of course, underlying this exercise is the implicit assumption that the values we are interested in are uniformly distributed within each polygon in the source and target. In some cases, this is a valid assumption or, at least, it does not introduce critical errors; in others, this is not acceptable. Dasymetric mapping has proposed a large amount of sophistications that try to come up with more realistic estimates and that can incorporate additional information.

To implement dasymetric mapping in Python, the best option is `tobler`, a package from the Pysal federation designed exactly for this goal. We will show here the simplest case, that of areal interpolation where apportioning is estimated based on area, but the package also provides more sophisticated approaches.

```
from tobler.area_weighted import area_interpolate
```

```
/opt/conda/lib/python3.9/site-packages/rasterstats/io.py:24:␣
↪DeprecationWarning: Using or importing the ABCs from
↪'collections' instead of from 'collections.abc' is␣
↪deprecated since Python 3.3, and in 3.10 it will stop␣
↪working
  from collections import Iterable, Mapping
```

For the example, we need two polygon layers. We will stick with San Diego and use the set of Census Tracts read above, and a grid layer built using the H3 hexagonal spatial index [Bro18]. Our goal will be to create population estimates for each hexagonal cell.

First, let us load the H3 grid:

```
h3 = geopandas.read_file("../data/h3_grid/sd_h3_grid.gpkg")
```

We are ready to interpolate:

```
# Area interpolation from polygon geotable to polygon geo-
↪table
interpolated = area_interpolate(
    # Source geo-table (converted to EPSG:3311 CRS)
    source_df=sd_pop.to_crs(epsg=3311),
    # Target geo-table (converted to EPSG:3311 CRS)
    target_df=h3.to_crs(epsg=3311),
    # Extensive variables in `source_df` to be interpolated␣
↪(e.g. population)
    extensive_variables=["B02001_001E"],
    # Intensive variables in `source_df` to be interpolated␣
↪(e.g. density)
    intensive_variables=["density"],
)
```

There is quite a bit going on in the cell above, let us unpack it:

- Remember this method apportions data values based on area, so it makes sense to have an accurate estimate for the extent of each polygon. To do that, we convert each geography to Albers Equal (`EPSG:3311`), which is expressed in meters, using `to_crs`.
- The method `area_interpolate` then takes the source and the target `GeoDataFrame` objects using the same naming convention we have in our explanation.
- In addition, we need to specify which variables we would like to interpolate. here, Tobler makes a distinction:
  - *Extensive* variables, or absolute values such as counts, aggregates, etc. (which we use for population, `B02001_001E`)

- *Intensive* variables, such as rates, ratios, etc. (which we select for density, as it is the ratio of population over area)

The `interpolated` output object is a geo-table that contains the target polygons and estimates of the variable(s) we originally had for the source geography (population and density in this case). Figure 12.11 (generated with the code below) illustrates the transfer of information from one geography to the other with for the case of total population estimates.

```
# Set up figure and axes
f, axs = plt.subplots(1, 3, figsize=(18, 6))

# Extract bounding box from output for easier visual␣
↪comparison
minX, minY, maxX, maxY = interpolated.total_bounds
# Reproject tracts to EPSG:331
sd_pop.to_crs(
    epsg=3311
    # Clip to the bounding box
).cx[
    minX:maxX,
    minY:maxY
    # Quantile choropleth for tract population
].plot(
    column="B02001_001E", scheme="quantiles", k=10, ax=axs[0]
)
# Remove axes
axs[0].set_axis_off()

# Reproject H3 hexagons to ESPG:3311
h3.to_crs(
    epsg=3311
    # Plot hexagons
).plot(ax=axs[1], markersize=0.5)
# Remove axes
axs[1].set_axis_off()

# Plot hexagons with interpolated population
interpolated.plot(
    column="B02001_001E", scheme="quantiles", k=10, ax=axs[2]
)
# Remove axes
axs[2].set_axis_off()

# Adjust limits of tract map to hexagons bounding box
# for easier visual comparison
axs[0].set_xlim(minX, maxX)
axs[0].set_ylim(minY, maxY)
```

(continued on next page)

Fig. 12.11: Interpolation of areal information to a different geometry. The Uber H3 hexagon grid is shown in the middle, and the interpolated values for population are shown on the right.

(continued from previous page)

```
# Display
plt.show()
```

And Figure 12.12, generated with the code below, shows the equivalent for population density.

```
# Set up figure and axes
f, axs = plt.subplots(1, 3, figsize=(18, 6))

# Extract bounding box from output for easier visual␣
↪comparison
minX, minY, maxX, maxY = interpolated.total_bounds
# Reproject tracts to EPSG:331
sd_pop.to_crs(
    epsg=3311
    # Clip to the bounding box
).cx[
    minX:maxX,
    minY:maxY
    # Quantile choropleth for tract population density
].plot(
    column="density", scheme="quantiles", k=10, ax=axs[0]
)
# Remove axes
axs[0].set_axis_off()

# Reproject H3 hexagons to ESPG:3311
h3.to_crs(
    epsg=3311
    # Plot hexagons
```

(continued on next page)

Fig. 12.12: Interpolation of population density from Census Tracts to Uber H3 Hexagons.

(continued from previous page)

```
).plot(ax=axs[1], markersize=0.5)
# Remove axes
axs[1].set_axis_off()

# Plot hexagons with interpolated population density
interpolated.plot(
    column="density", scheme="quantiles", k=10, ax=axs[2]
)
# Remove axes
axs[2].set_axis_off()

# Adjust limits of tract map to hexagons bounding box
# for easier visual comparison
axs[0].set_xlim(minX, maxX)
axs[0].set_ylim(minY, maxY)
# Display
plt.show()
```

12.3 Feature engineering using map synthesis

Feature engineering with map matching is most helpful when you have additional information to use in the analysis. And, with the wealth of freely available data from censuses, satellites, and open volunteered geographic information vendors such as OpenStreetMap, map matching can be a very powerful tool for enriching and improving your analyses. However, it is sometimes *also* useful to only examine the data you've got, and use the spatial structure within to build better features or better models. While this might be done using spatially explicit models (like those covered in *Chapter 11*), it is also possible to use map synthesis to build spatial features and improve model performance.

There is an extensive amount of map synthesis features. In fact, many of them are usually derived for specific use cases and following the intuition of domain experts that can take technical guidance along the lines presented here. We discuss two main ones as an illustration, but reiterate that the sky really is the limit in this category. First, we will return to spatial *summary* features. Second, we will examine *regionalization features*, which detect and leverage geographical clusters in the data to improve prediction.

12.3.1 Spatial summary features in map synthesis

Just like in map matching, you can use spatial summary features in map synthesis to build better models. One approach involves constructing spatial summary measures of your training data. This is done in the same manner as in map matching, except we now refer only to the data on hand. Thus, we may want to determine whether nearby Airbnbs are "competing" with each Airbnb. We might do this by finding the distance to the nearest Airbnb with the same number of bedrooms, since two nearby listings that *also* sleep the same number of people likely will compete with one another for tenants.

12.3.1.1 Counting neighbors

A useful metric to characterize an observation in your data through geography is whether it is surrounded by other observations or not. This question (and the following one we cover) is another area of geographic data science where spatial weights matrices are useful.

If we define to be "surrounded by other observations" as in "having many neighbors", this translates into the cardinalities of a spatial weights matrix (i.e., the number of neighbors recorded in the matrix for each observation).

For example, we can "engineer a feature" that contains the number of Airbnb properties each property has within 500 meters by constructing a distance band object:

```
# Build distance band spatial weights matrix
d500_w = weights.DistanceBand.from_dataframe(
    airbnbs_albers, threshold=500, silence_warnings=True
)
```

And extracting its cardinalities:

```
card = pandas.Series(d500_w.cardinalities)
```

The `card` feature we have built will pick up areas of higher concentration of Airbnb properties with higher values, as we can see in Figure 12.13.

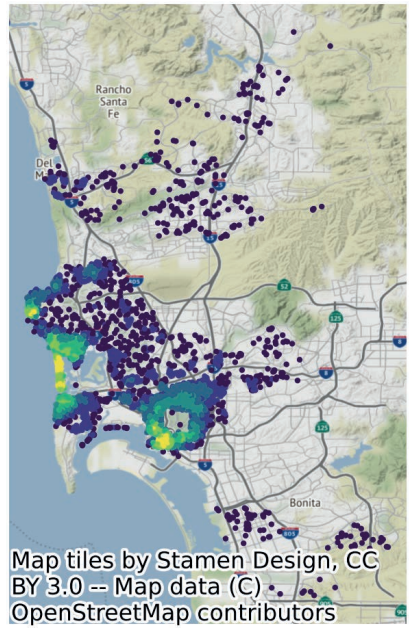


Fig. 12.13: Number of Airbnbs within 500 meters of each listing.

```
# Set up figure and axis
f, ax = plt.subplots(1)
# Append cardinalities to main Airbnb geo-table
airbnbs.assign(
    card=card
    # Plot cardinality quantile choropleth
).plot("card", scheme="quantiles", k=7, markersize=1, ax=ax)
# Add basemap
contextily.add_basemap(ax, crs=airbnbs.crs)
# Remove axes
ax.set_axis_off();
```

12.3.1.2 Distance buffers within a single table

If what we are interested in is finding the average number of bedrooms around each house, we might do this using a row-standardized version of the `DistanceBand` weight object we have just built, which considers Airbnb as "neighbors" if they are within the distance threshold, and computing the spatial lag of the number of bedrooms.

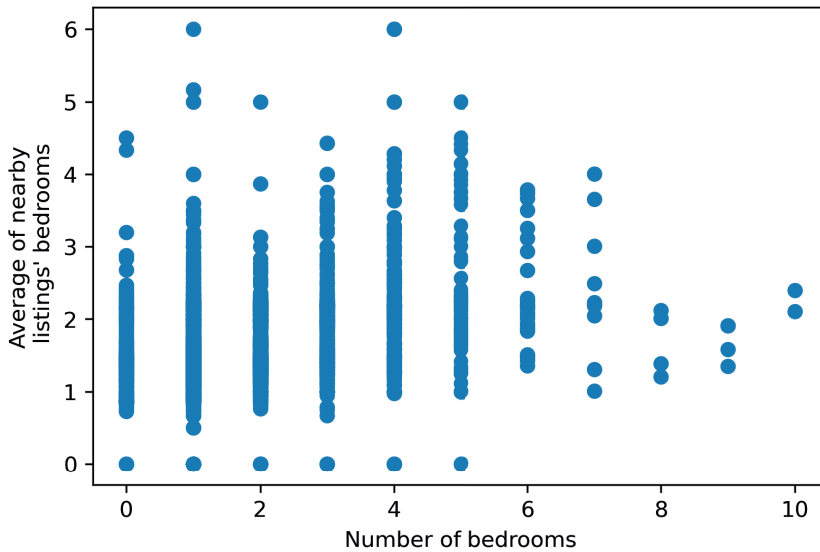


Fig. 12.14: Relationship between the number of bedrooms at an Airbnb and the typical number of bedrooms among nearby Airbnbs.

```
# Row standardize
d500_w.transform = "r"
# Compute spatial lag of No. of bedrooms
local_average_bedrooms = weights.lag_spatial(
    d500_w, airbnbs_albers[["bedrooms"]].values
)
```

While related, these features contain quite distinct pieces of information, and both may prove useful in modeling. This is shown in Figure 12.14, which compares them directly in a scatterplot.

```
plt.scatter(
    airbnbs_albers[["bedrooms"]].values, local_average_
↪bedrooms
)
plt.xlabel("Number of bedrooms")
plt.ylabel("Average of nearby\n listings' bedrooms");
```

If we were instead interested in the most common number of bedrooms, rather than the average, we could use the `lag_categorical` function, which will consider each value as a category and return the most common value in each neighbor set:

```
local_mode = weights.lag_categorical(
    d500_w, airbnbs_albers[["bedrooms"]].values
)
```

```
/opt/conda/lib/python3.9/site-packages/libpysal/weights/spatial_lag.py:167: DeprecationWarning: `np.int` is a␣
↪deprecated alias for the builtin `int`. To silence this warning, use `int` by itself. Doing this will not␣
↪modify any behavior and is safe. When replacing `np.int`, you may wish to use e.g. `np.int64` or `np.
↪int32` to specify the precision. If you wish to review your current use, check the release note link for␣
↪additional information.
Deprecated in NumPy 1.20; for more details and guidance: https://numpy.org/devdocs/release/1.20.0-notes.html
↪#deprecations
  normalized_labels = np.zeros(y.shape, dtype=np.int)
/opt/conda/lib/python3.9/site-packages/numpy/core/fromnumeric.py:3440: RuntimeWarning: Mean of empty slice.
  return _methods._mean(a, axis=axis, dtype=dtype,
/opt/conda/lib/python3.9/site-packages/numpy/core/_methods.py:189: RuntimeWarning: invalid value encountered␣
↪in double_scalars
  ret = ret.dtype.type(ret / rcount)
```

Since we are now treating the number of bedrooms as a discrete feature, we can use a `crosstab` from `pandas` to examine the relationship between a listing and the typical size of listings nearby:

```
crosstab = pandas.crosstab(
    airbnbs_albers.bedrooms, local_mode.flatten()
)
crosstab.columns.name = "nearby"
crosstab
```

nearby	0.0	1.0	2.0	3.0	4.0	5.0	6.0	7.0
bedrooms								
0.0	4	391	40	7	3	0	0	0
1.0	3	3064	215	33	17	5	0	1
2.0	1	978	269	11	3	1	0	0
3.0	2	429	164	34	12	3	0	0
4.0	0	164	60	11	30	5	3	0
5.0	0	51	17	3	8	18	0	1
6.0	0	12	10	1	4	2	0	0
7.0	0	6	1	2	1	0	0	0
8.0	0	3	1	0	0	0	0	0
9.0	0	3	1	0	0	0	0	0
10.0	0	2	0	0	0	0	0	0

From the table we can see the most common case (N=) is properties with one bedroom surrounded mostly by other properties with also only one bedroom. Similarly we also find out, for example, that the two properties with ten bedrooms in the dataset are surrounded by properties with mostly two bedrooms. The remaining cells in the table can be interpreted in a similar fashion.

If more complicated statistics are required, it can help to reexpress the construction of summary statistics as a *reduction* of the *adjacency list* representation of our weights, as done in *Chapter 4*. To recap, the *adjacency list* is a `pandas.DataFrame` where each

row contains a single link in our graph. It contains the identifier for some `focal` observation, the identifier for some `neighbor` observation, and a value for the `weight` of the link that connects the `focal` and `neighbor`:

```
adjlist = d500_w.to_adjlist()
```

```
adjlist.head()
```

```
   focal  neighbor    weight
0      0       136  0.023256
1      0       279  0.023256
2      0       317  0.023256
3      0       616  0.023256
4      0       761  0.023256
```

If we had the values for each for the neighbors in this adjacency list table, then we could use a `groupby()` to summarize the values of observations connected to a given focal observation. This merge can be done directly with the original data, linking the `neighbor` key in the adjacency list back to that observation in our source table:

```
adjlist = adjlist.merge(
    airbnbs_albers[["bedrooms"]],
    left_on="neighbor",
    right_index=True,
    how="left",
)
adjlist.head()
```

```
   focal  neighbor    weight  bedrooms
0      0       136  0.023256       1.0
1      0       279  0.023256       3.0
2      0       317  0.023256       2.0
3      0       616  0.023256       1.0
4      0       761  0.023256       1.0
```

Now, we need only to group the adjacency list by the focal observation and summarize the `bedrooms` column to obtain the median number of bedrooms for each `focal` observation.

```
adjlist.groupby("focal").bedrooms.median()
```

```
focal
0       1.0
1       2.0
2       1.0
4       1.0
```

(continued on next page)

(continued from previous page)

```
5       1.0
       ...
6105    1.0
6106    1.0
6107    1.0
6108    1.0
6109    1.0
Name: bedrooms, Length: 6035, dtype: float64
```

Since the mean and/or mode are the most commonly used measures of central tendency, the `lag_spatial` and `lag_categorical` functions cover many of the required uses in practice.

12.3.1.3 "Ring" buffer features

Sometimes, analysts might want to use multiple "bands" of buffer features. This requires that we build summaries of the observations that fall *only within* a given range of distances, such as the typical size of houses that are further than 500 meters, but still within one kilometer. This kind of "ring buffer", or annulus, is a common request in spatial analysis, and can be done in substantially the same way as before by increasing the `threshold` in a `DistanceBand` weight.

We can use our 500 meters weights from before to build the average again:

```
average_within_500 = weights.lag_spatial(
    d500_w, airbnbs_albers[["bedrooms"]].values
)
```

Then, we need to build the graph of Airbnbs that fall *between* 500 meters and 1 kilometer from one another. To start, we build the `DistanceBand` graph of all listings closer than 1 kilometer:

```
d1k_w = weights.DistanceBand.from_dataframe(
    airbnbs_albers, threshold=1000, silence_warnings=True
)
```

Then, using the `weights.set_operations` module, we can express set-theoretic relationships between graphs. Here, we need to *remove* the links in our 1 kilometer graph that are *also* links in the 500 meters graph. To do this, we need `w_difference(d1k_w, d500_w)`, the difference between the one kilometer graph and the 500 meters graph:

```
d1k_exclusive = weights.set_operations.w_difference(
    d1k_w, d500_w, constrained=False
)
```

```
/opt/conda/lib/python3.9/site-packages/libpysal/weights/
↪weights.py:172: UserWarning: The weights matrix is not␣
↪fully connected:
 There are 60 disconnected components.
 There are 32 islands with ids: 762, 907, 976, 1003, 1776,␣
↪1867, 2789, 2841, 3130, 3138, 3184, 3604, 3748, 3962, 3989,␣
↪4366, 4443, 4627, 4764, 4885, 4989, 5014, 5536, 5733, 5756,␣
↪5790, 5796, 5808, 6005, 6026, 6049, 6068.
  warnings.warn(message)
```

Then, we can compute the average size of listings between 500 meters and 1 kilometer in the same manner as before using our `d1k_exclusive` graph, which now omits all edges shorter than 500 meters.

```
d1k_exclusive.transform = "r"
average_500m_to_1k = weights.lag_spatial(
    d1k_exclusive, airbnbs_albers[["bedrooms"]].values
)
```

```
('WARNING: ', 762, ' is an island (no neighbors)')
('WARNING: ', 907, ' is an island (no neighbors)')
('WARNING: ', 976, ' is an island (no neighbors)')
('WARNING: ', 1003, ' is an island (no neighbors)')
('WARNING: ', 1776, ' is an island (no neighbors)')
('WARNING: ', 1867, ' is an island (no neighbors)')
('WARNING: ', 2789, ' is an island (no neighbors)')
('WARNING: ', 2841, ' is an island (no neighbors)')
('WARNING: ', 3130, ' is an island (no neighbors)')
('WARNING: ', 3138, ' is an island (no neighbors)')
('WARNING: ', 3184, ' is an island (no neighbors)')
('WARNING: ', 3604, ' is an island (no neighbors)')
('WARNING: ', 3748, ' is an island (no neighbors)')
('WARNING: ', 3962, ' is an island (no neighbors)')
('WARNING: ', 3989, ' is an island (no neighbors)')
('WARNING: ', 4366, ' is an island (no neighbors)')
('WARNING: ', 4443, ' is an island (no neighbors)')
('WARNING: ', 4627, ' is an island (no neighbors)')
('WARNING: ', 4764, ' is an island (no neighbors)')
('WARNING: ', 4885, ' is an island (no neighbors)')
('WARNING: ', 4989, ' is an island (no neighbors)')
('WARNING: ', 5014, ' is an island (no neighbors)')
('WARNING: ', 5536, ' is an island (no neighbors)')
('WARNING: ', 5733, ' is an island (no neighbors)')
('WARNING: ', 5756, ' is an island (no neighbors)')
('WARNING: ', 5790, ' is an island (no neighbors)')
('WARNING: ', 5796, ' is an island (no neighbors)')
('WARNING: ', 5808, ' is an island (no neighbors)')
```

(continued on next page)

(continued from previous page)

```
('WARNING: ', 6005, ' is an island (no neighbors)')
('WARNING: ', 6026, ' is an island (no neighbors)')
('WARNING: ', 6049, ' is an island (no neighbors)')
('WARNING: ', 6068, ' is an island (no neighbors)')
```

Thus, as we can see in Figure 12.15 (generated by the code below), the two features contain distinct, but related, information, and both may be valuable in their own right when attempting to predict outcomes of interest.

```
# Plot scatter
plt.scatter(
    average_within_500, average_500m_to_1k, color="k", marker=
↪"."
)
# Rename horizontal axis
plt.xlabel("Average size within 500 meters")
# Rename vertical axis
plt.ylabel("Average size\n beyond 500m but within 1km")
# Plot line of 45 degrees
plt.plot(
    [0, 5],
    [0, 5],
    color="orangered",
    linestyle=":",
    linewidth=2,
    label="1 to 1",
)
# Add legend
plt.legend();
```

As the plot shows, although the relationship between both is positive, there is quite a bit of variation and departure from the (red) line of equality. These represent cases with either larger (above the line) or smaller (below) average size in the 500 meters to 1 kilometer ring than in the 500 meters buffer.

12.3.2 Clustering as feature engineering

One unique way to use spatial or features information *within* your data as a feature in your existing models is to use *clustering*, as we saw in Chapters *8* and *10*, respectively. This can provide an indication of whether an observation exists in a given "place" geographically, or if an observation is a specific "kind" of observation. Data reduction of many variates into a derived categorical feature can be useful in training models. This is even more useful when the *spatial location* of a given observation indicates a useful characteristic about the kind of observation that may be found at that location.

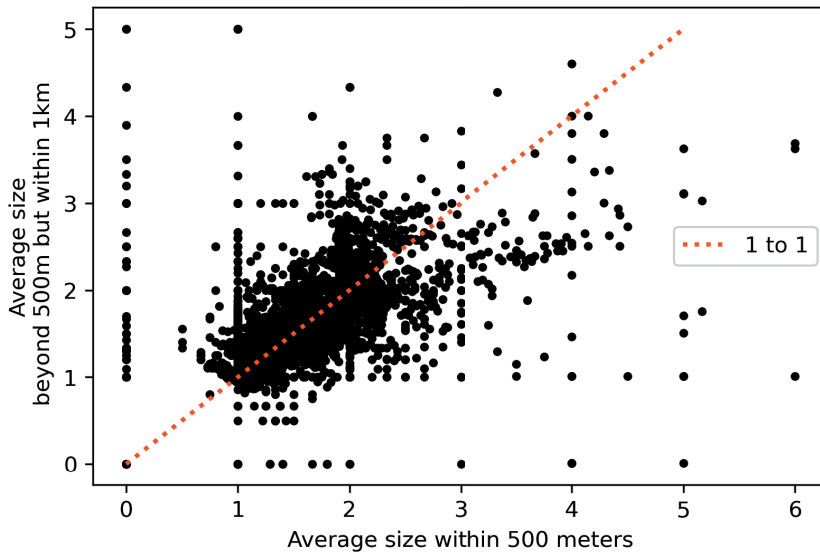


Fig. 12.15: Relationship between the size of Airbnbs between successive distance buffers around an Airbnb.

While sometimes we might prefer to introduce space and (co-)location through our model (as we did in *Chapter 11*), others we can instead use (spatial) cluster labels themselves as features. For example, to cluster the listings based on their location, we can use hierarchical DBSCAN [CMS13], an improved variant of the traditional DBSCAN algorithm used in *Chapter 8*. We import it through its own package [MHA17]:

```
from hdbscan import HDBSCAN
```

To apply to our Airbnb data, we need the coordinates as a two-dimensional array of XY pairs:

```
coordinates = numpy.column_stack(
    (airbnbs_albers.geometry.x, airbnbs_albers.geometry.y)
)
```

With a little tuning, we can decide on an effective parameterization. The main advantage of HDBSCAN over the traditional approach is that it reduces the number of tuning parameters from two to one: we only need to set the minimum number of observations we want to define a cluster. Here, we look for relatively large clusters of Airbnbs, those with about 25 listings or more, fit them to our array of coordinates, and store the cluster labels identified:

```
labels = HDBSCAN(min_cluster_size=25).fit(coordinates).labels_
```

The spatial distribution of these clusters will give us a sense of areas of San Diego with relatively high density of the observations. But the `labels` object does not contain clusters, only observation memberships. To derive those clusters geographically, we construct the convex hull of the observations in each detected cluster, creating thus a polygon that delimits every observation that is part of the cluster:

```
hulls = airbnbs_albers[["geometry"]].dissolve(by=labels).
↪convex_hull
```

The polygons in `hulls` (and displayed in Figure 12.16) provide an intermediate layer between the granularity of each individual location and the global scale of San Diego as a geographical unit. Since people tend to make locational decisions hierarchically (e.g., first they select *San Diego* as a destination, then they pick a particular *part* of San Diego, then choose a house within the area), this approach might give us reasonable insight into enclaves of Airbnb properties:

```
# Set up figure and axis
f, ax = plt.subplots(1, figsize=(9, 9))
# Plot individual Airbnb locations
airbnbs_albers.plot(
    # Color by cluster label
    column=labels,
    # Consider label as categorical
    categorical=True,
    # Add 50% of transparency
    alpha=0.5,
    # Include legend
    legend=True,
    # Draw on axis `ax`
    ax=ax,
    # Use circle as marker
    marker=".",
    # Position legend outside the map
    legend_kwds={"bbox_to_anchor": (1, 1)},
)
# Plot convex hull polygons for each cluster label
# except that for -1 (observations classified as noise)
hulls[hulls.index != -1].boundary.plot(color="k", ax=ax)
# Add basemap
contextily.add_basemap(
    ax,
    crs=airbnbs_albers.crs.to_string(),
    source=contextily.providers.CartoDB.VoyagerNoLabels,
)
```

(continued on next page)

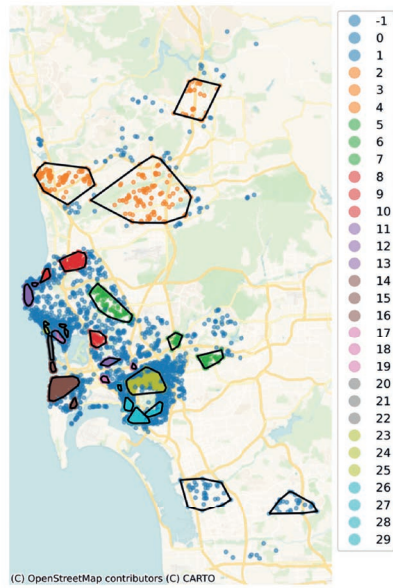


Fig. 12.16: Clusters in the locations of Airbnbs within San Diego.

(continued from previous page)

```
# Remove axes
ax.set_axis_off();
```

Note how the hierarchical nature of HDBSCAN, which picks density thresholds *locally* is at work in the map, where some of the clusters (e.g., orange ones in the north) display lower density than parts of the center which are *not* part of any cluster. These results also capture some information about the price of a listing. Figure 12.17 (generated with the code below) shows this by plotting the distribution of prices across the detected clusters.

```
# Set up figure
f = plt.figure(figsize=(8, 3))
# Add boxplots of price by HDBSCAN cluster
ax = airbnbs_albers.assign(labels=labels).boxplot(
    # Plot distribution of 'price'
    "price",
    # Group by cluster label, generating one boxplot/cluster
    by='labels',
    # Do not display individual outlier observations
    flierprops=dict(marker=None),
    # Draw visualization on the current axis (inside `f`)
    ax=plt.gca(),
)
```

(continued on next page)

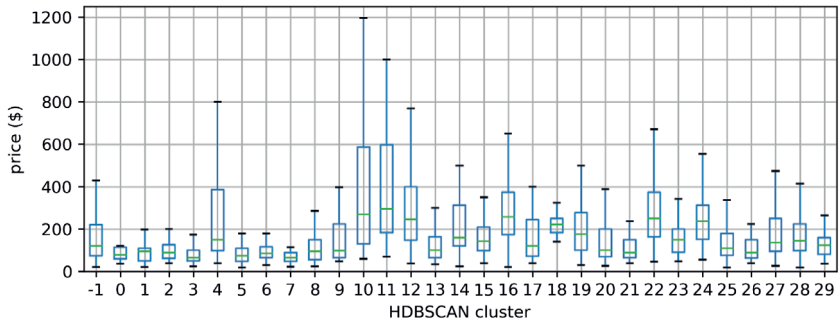


Fig. 12.17: Boxplot of price by detected 'competition cluster.' The clusters vary significantly in prices and could be used to train a model.

(continued from previous page)

```
# Set label for horizontal axis
ax.set_xlabel("HDBSCAN cluster")
# Set labels for vertical axis
ax.set_ylabel("price ($)")

# Remove default figure title
plt.gcf().suptitle(None)
# Remove default axis title
ax.set_title(None)
# Re-adjust vertical value range for easier legibility
ax.set_ylim(0, 1250);
```

If we want to capture the variation picked up by membership to these clusters in a regression model, we could use these labels as the basis for spatial fixed effects, for example, as we saw in *Chapter 11*.

12.4 Conclusion

Feature engineering is a powerful way to enrich your data and make the most out of them. It is often within reach whether you are able to mix in auxilliary data or not. At a minimum, it only requires that new variables are constructed from your existing data; however, if you have access to more data, spatial feature engineering provides the *ultimate linkage key*, a flexible and powerful tool with which you can unlock the value held in many *other* datasets that you may have access to even if they are not obviously connected to your own.

We have structured all the approaches presented in this chapter along two main groups: those which allow you to augment a given dataset, which we have termed *map synthesis*; and those that enable connecting disparate datasets through space. In both cases, each

operation involved in feature engineering, such as determining what the average value is in the area near each observation or identifying whether observations exist as part of a "cluster", are relatively simple operations. As building blocks that may be combined with each other, they build large, rich, and useful datasets that can be used directly in your existing methods of analysis.

Beyond feature engineering, statistical techniques we discuss in this book (particularly in Chapters *10* and *11*) can leverage spatial structure *directly* during learning/inference. These techniques allow you to embed space in non-spatial models to make them "geography aware". Furthermore, the techniques in this chapter (and the methods that extend upon them) are immediately useful for most practicing data scientists, and they can be integrated into nearly any analytical method or approach.

12.5 Questions

1. Thinking of your own research, provide an example where map matching would be a useful spatial feature engineering approach? How about an example where map synthesis would be useful for your research?
2. In the *Counting* nearby *features* example early in the chapter, there is a potential issue with under-counting the number of nearby bars and restaurants for certain Airbnbs in the dataset. Which Airbnbs are subject to neighbor under-counts and why?
3. How would you correct the under-count in the previous question?
4. Dasymetric mapping may introduce spurious spatial autocorrelation (see Chapters *6* and *7*) through the interpolation process. How does this occur and why is it important to acknowledge?
5. From the previous spatial regression chapter, is the SLX model a map matching or a map synthesis technique?
6. Feature engineering can be used in many different applied contexts. When might feature engineering actually *not* improve a model?

12.6 Next steps

For more information on machine learning methods, it is difficult to beat *Introduction to Statistical Learning* by:

James, Gareth, Daniela Witten, Trevor Hastie, and Robert Tibshirani. 2021. *Introduction to Statistical Learning* (2nd Edition). Wiley: New York.

In addition, there are many geospatial relationships that can be leveraged to merge datasets together. These *spatial join* techniques often require quite a bit of background

understanding in *geocomputation*, computation that focuses on the basic/fundamental geometric operations and relationships between geographic objects. However, simple introductions are offered by our partner book:

Tenkanen, Henrikki, Vuokko Heikinheimo, and David Whipp. 2023. *Python for Geographic Data Analysis*. CRC Press: Boca Raton, FL. https://pythongis.org

Finally, references on geographic interpolation methods are sometimes hard to come by, but a good and readable introduction to the domain is offered by:

Comber, Alexis and Wen Zeng. 2019. "Spatial Interpolation using areal features: A review of methods and opportunities using new forms of data with coded illustrations." *Geography Compass* 13. https://doi.org/10.1111/gec3.12465

Taylor & Francis
Taylor & Francis Group
http://taylorandfrancis.com

References

[Ans88] Luc Anselin. *Spatial Econometrics: Methods and Models*. Springer Netherlands, 1988. ISBN 9789401577991. URL: http://dx.doi.org/10.1007/978-94-015-7799-1, doi:10.1007/978-94-015-7799-1.

[Ans89] Luc Anselin. What is special about spatial data? alternative perspectives on spatial data analysis (89-4). National Center for Geographic Information and Analysis, University of California Santa Barbara, 1989.

[Ans95] Luc Anselin. Local indicators of spatial association-LISA. *Geographical Analysis*, 27:93–115, 1995.

[Ans02] Luc Anselin. Under the hood issues in the specification and interpretation of spatial regression models. *Agricultural Economics*, 27(3):247–267, Nov 2002. URL: http://dx.doi.org/10.1111/j.1574-0862.2002.tb00120.x, doi:10.1111/j.1574-0862.2002.tb00120.x.

[Ans03] Luc Anselin. Spatial externalities, spatial multipliers, and spatial econometrics. *International Regional Science Review*, 26(2):153–166, Apr 2003. URL: http://dx.doi.org/10.1177/0160017602250972, doi:10.1177/0160017602250972.

[AR14] Luc Anselin and Sergio J. Rey. *Modern spatial econometrics in practice: A guide to GeoDa, GeoDaSpace, and PySAL*. GeoDa Press, 2014.

[Arb06] Giuseppe Arbia. *Spatial econometrics: statistical foundations and applications to regional convergence*. Springer Science & Business Media, 2006.

[ADKP10] Irani Arraiz, David M. Drukker, Harry H. Kelejian, and Ingmar R. Prucha. A spatial Cliff-Ord-type model with heteroskedastic innovations:

Small and large sample results. *Journal of Regional Science*, 50(2):592–614, 2010.

[BRT15] Adrian Baddeley, Ege Rubak, and Rolf Turner. *Spatial point patterns: methodology and applications with R*. CRC Press, 2015.

[BGFS08] Sudipto Banerjee, Alan E. Gelfand, Andrew O. Finley, and Huiyan Sang. Gaussian predictive process models for large spatial data sets. *Journal of the Royal Statistical Society: Series B (Statistical Methodology)*, 70(4):825–848, Sep 2008. URL: http://dx.doi.org/10.1111/j.1467-9868.2008.00663.x, doi:10.1111/j.1467-9868.2008.00663.x.

[Ber64] Brian J L Berry. Approaches to regional analysis: a synthesis. *Annals of the Association of American Geographers*, 54:2–11, 1964.

[BAB20] Geoff Boeing and Dani Arribas-Bel. Gis and computational notebooks. In John P. Wilson, editor, *The Geographic Information Science & Technology Body of Knowledge (1st Quarter 2021 Edition)*. University Consortium for Geographic Information Science, 2020. doi:10.22224/gistbok/2021.1.2.

[BCK+15] K Bollen, J Cacioppo, R Kaplan, Jon Krosnick, and James Olds. Social, Behaviorial, and Economic Sciences: Perspectives on Robust and Reliable Science. Technical Report, National Science Foundation, Arlington, VA, 2015.

[Box76] George E P Box. Science and statistics. *Journal of the American Statistical Association*, 71(356):791–799, 1976. doi:10/108001621459.1976.10480949.

[BMPH97] Cynthia A Brewer, Alan M MacEachren, Linda W Pickle, and Douglas Herrmann. Mapping mortality: evaluating color schemes for choropleth maps. *Annals of the Association of American Geographers*, 87(3):411–438, 1997.

[Bro18] Isaac Brodsky. H3: uber's hexagonal hierarchical spatial index. *Available from Uber Engineering website: https://eng. uber. com/h3/[22 June 2019]*, 2018.

[BFC10] Chris Brunsdon, A. Stewart Fotheringham, and Martin E. Charlton. Geographically weighted regression: a method for exploring spatial nonstationarity. *Geographical Analysis*, 28(4):281–298, Sep 2010. URL: http://dx.doi.org/10.1111/j.1538-4632.1996.tb00936.x, doi:10.1111/j.1538-4632.1996.tb00936.x.

[CMS13] Ricardo JGB Campello, Davoud Moulavi, and Jörg Sander. Density-based clustering based on hierarchical density estimates. In *Pacific-Asia conference on knowledge discovery and data mining*, 160–172. Springer, 2013.

[CO81] Andrew David Cliff and J Keith Ord. *Spatial processes: models & applications*. Taylor & Francis, 1981.

[Coo17] Joshua Cook. *Docker for Data Science*. Apress, 2017. ISBN 9781484230121. URL: http://dx.doi.org/10.1007/978-1-4842-3012-1, doi:10.1007/978-1-4842-3012-1.

[CruwellvDE+19] Sophia Crüwell, Johnny van Doorn, Alexander Etz, Matthew C Makel, Hannah Moshontz, Jesse C Niebaum, Amy Orben, Sam Parsons, and Michael Schulte-Mecklenbeck. Seven easy steps to open science. *Zeitschrift für Psychologie*, 2019.

[CorversM21] Frank Cörvers and Ken Mayhew. Regional inequalities: causes and cures. *Oxford Review of Economic Policy*, 37(1):1–16, 2021.

[DSSC18] Teddy Dawson, J.S. Sandoval, Vasit Sagan, and Thomas Crawford. A spatial analysis of the relationship between vegetation and poverty. *ISPRS International Journal of Geo-Information*, 7:83, 03 2018. doi:10.3390/ijgi7030083.

[DDPP19] Nema Dean, Guanpeng Dong, Aneta Piekut, and Gwilym Pryce. Frontiers in residential segregation: understanding neighbourhood boundaries and their impacts. *Tijdschrift voor economische en sociale geografie*, 110(3):271–288, 2019.

[Don17] David Donoho. 50 Years of Data Science. *Journal of Computational and Graphical Statistics*, 26(4):745–766, October 2017. doi:10.1080/10618600.2017.1384734.

[DAR11] Juan C. Duque, Luc Anselin, and Sergio J. Rey. The max-p-regions problem. *Journal of Regional Science*, 52(3):397–419, Dec 2011. URL: http://dx.doi.org/10.1111/j.1467-9787.2011.00743.x, doi:10.1111/j.1467-9787.2011.00743.x.

[DRSurinach07] Juan Carlos Duque, Raúl Ramos, and Jordi Suriñach. Supervised regionalization methods: a survey. *International Regional Science Review*, 30(3):195–220, 2007.

[Eco] The Economist. Python has brought computer programming to a vast new audience. https://www.economist.com/science-and-technology/2018/07/19/python-has-brought-computer-programming-to-a-vast-new-audience. Accessed: 2021-06-04.

[EH16] Bradley Efron and Trevor Hastie. *Computer Age Statistical Inference*. Volume 5. Cambridge University Press, 2016.

[EB01] Cory L Eicher and Cynthia A Brewer. Dasymetric mapping and areal interpolation: implementation and evaluation. *Cartography and Geographic Information Science*, 28(2):125–138, 2001.

[EKS+96] Martin Ester, Hans-Peter Kriegel, Jörg Sander, Xiaowei Xu, et al. A density-based algorithm for discovering clusters in large spatial databases with noise. In *Kdd*, volume 96, 226–231, 1996.

[Ezc19] Roberto Ezcurra. Interregional Inequality and Civil Conflict: Are Spatial Disparities a Threat to Stability and Peace? *Defence and Peace Economics*, 30(7):759–782, November 2019. doi:10.1080/10242694.2018.1446621.

[FPP+10] Matthew C Fitzpatrick, Evan L Preisser, Adam Porter, Joseph Elkinton, Lance A Waller, Bradley P Carlin, and Aaron M Ellison. Ecological boundary detection using bayesian areal wombling. *Ecology*, 91(12):3448–3455, 2010.

[FD81] David Freedman and Persi Diaconis. On the histogram as a density estimator: l 2 theory. *Zeitschrift für Wahrscheinlichkeitstheorie und verwandte Gebiete*, 57(4):453–476, 1981.

[FHT+01] Jerome Friedman, Trevor Hastie, Robert Tibshirani, et al. *The elements of statistical learning*. Volume 1. Springer series in statistics New York, 2001.

[Gah99] Mark Gahegan. Guest editorial: what is geocomputation? *Transactions in GIS*, 3:203–206, 1999.

[Gea54] Robert C Geary. The contiguity ratio and statistical mapping. *The incorporated statistician*, 5(3):115–146, 1954.

[GH06] Andrew Gelman and Jennifer Hill. *Data Analysis Using Regression and Multilevel Hierarchical Models*. Cambridge University Press, 2006. ISBN 9780511790942. URL: http://dx.doi.org/10.1017/cbo9780511790942, doi:10.1017/cbo9780511790942.

[Get07] Arthur Getis. Reflections on spatial autocorrelation. *Regional Science and Urban Economics*, 37(4):491–496, 2007.

[GO92] Arthur Getis and JK Ord. The analysis of spatial association by use of distance statistics. *Geographical Analysis*, 24(3):189–206, 1992.

[GOP15] Steve Gibbons, Henry G. Overman, and Eleonora Patacchini. Spatial methods. *Handbook of Regional and Urban Economics*, pages 115–168, 2015. URL: http://dx.doi.org/10.1016/b978-0-444-59517-1.00003-9, doi:10.1016/b978-0-444-59517-1.00003-9.

[Git] Github. Basic writing and formatting syntax. https://help.github.com/en/github/writing-on-github/basic-writing-and-formatting-syntax. Accessed: 2021-06-04.

[Glu18] Konstantin Gluschenko. Measuring regional inequality: to weight or not to weight? *Spatial Economic Analysis*, 13(1):36–59, Jul 2018. URL: http://dx.doi.org/10.1080/17421772.2017.1343491, doi:10.1080/17421772.2017.1343491.

[Goo07] Michael F Goodchild. Citizens as sensors: the world of volunteered geography. *GeoJournal*, 69(4):211–221, 2007.

[GYC07] Michael F Goodchild, May Yuan, and Thomas Cova. Towards a general theory of geographic representation in GIS. *International Journal of Geographical Information Science*, 21:239–260, 2007.

[HHV93] David L Horn, Charles R Hampton, and Anthony J Vandenberg. Practical application of district compactness. *Political Geography*, 12:103–120, 1993.

[Ioa07] John P. A Ioannidis. Why most published research findings are false: Author's Reply to Goodman and Greenland. *PLoS Medicine*, 4(6):e215, Jun 2007. URL: http://dx.doi.org/10.1371/journal.pmed.0040215, doi:10.1371/journal.pmed.0040215.

[JC71] George F. Jenks and Fred C. Caspall. Error on choroplethic maps: definition, measurement, reduction. *Annals of the Association of American Geographers*, 61(2):217–244, Jun 1971. URL: http://dx.doi.org/10.1111/j.1467-8306.1971.tb00779.x, doi:10.1111/j.1467-8306.1971.tb00779.x.

[Jia13] Bin Jiang. Head/tail breaks: a new classification scheme for data with a heavy-tailed distribution. *The Professional Geographer*, 65(3):482–494, Aug 2013. URL: http://dx.doi.org/10.1080/00330124.2012.700499, doi:10.1080/00330124.2012.700499.

[JPF10] Ron Johnston, Michael Poulsen, and James Forrest. The intensity of ethnic residential clustering: exploring scale effects using local indicators of spatial association. *Environment and Planning A*, 42(4):874–894, 2010. doi:10.1068/a42181.

[KRKPerez+16] Thomas Kluyver, Benjamin Ragan-Kelley, Fernando Pérez, Brian Granger, Matthias Bussonnier, Jonathan Frederic, Kyle Kelley, Jessica Hamrick, Jason Grout, Sylvain Corlay, Paul Ivanov, Damián Avila, Safia Abdalla, Carol Willing, and Jupyter Development Team. Jupyter notebooks - a publishing format for reproducible computational workflows. In Fernando Loizides and Birgit Schmidt, editors, *Positioning and power in academic publishing: Players, agents and agendas*, 97–90. IOS Press, 2016.

[LC05] Haolan Lu and Bradley P Carlin. Bayesian areal wombling for geographical boundary analysis. *Geographical Analysis*, 37(3):265–285, 2005.

[MM21] Kirsi M. Manz and Ulrich Mansmann. Inequality indices to monitor geographic differences in incidence, mortality and fatality rates over time during the COVID-19 pandemic. *PLOS ONE*, 16(5):e0251366, May 2021. URL: https://dx.plos.org/10.1371/journal.pone.0251366 (visited on 2021-10-04), doi:10.1371/journal.pone.0251366.

[MHA17] Leland McInnes, John Healy, and Steve Astels. Hdbscan: hierarchical density based clustering. *Journal of Open Source Software*, 2(11):205, 2017.

[Mor48] Patrick AP Moran. The interpretation of statistical maps. *Journal of the Royal Statistical Society. Series B (Methodological)*, 10(2):243–251, 1948.

[MS21] Sebastian Müller and Lennart Schüler. Geostat-framework/gstools: v1.3.3 'pure pink'. August 2021. URL: https://doi.org/10.5281/zenodo.5169658, doi:10.5281/zenodo.5169658.

[Nie20] Michael Nielsen. *Reinventing Discovery*. Princeton University Press, Apr 2020. ISBN 9780691202846. URL: http://dx.doi.org/10.2307/j.ctvqsdst2, doi:10.2307/j.ctvqsdst2.

[Ord10] Keith Ord. Spatial autocorrelation: a statistician's reflections. In Luc Anselin and Sergio Rey, editors, *Perspectives on Spatial Data Analysis*, Advances in Spatial Science. Springer-Verlag, Berlin-Heidelberg, 2010.

[OLK+19] Taylor M Oshan, Ziqi Li, Wei Kang, Levi J Wolf, and A Stewart Fotheringham. Mgwr: a python implementation of multiscale geographically weighted regression for investigating process spatial heterogeneity and scale. *ISPRS International Journal of Geo-Information*, 8(6):269, 2019.

[PS03] T. Piketty and E. Saez. Income inequality in the United States, 1913-1998. *The Quarterly Journal of Economics*, 118(1):1–41, Feb 2003. URL: http://dx.doi.org/10.1162/00335530360535135, doi:10.1162/00335530360535135.

[RLG09] Sergio J Rey and Julie Le Gallo. Spatial analysis of economic convergence. In Terry C. Mills and Kerry Patterson, editors, *Palgrave handbook of econometrics*, pages 1251–1290. Palgrave, 2009.

[Rey09] Sergio J. Rey. Show me the code: spatial analysis and open source. *Journal of Geographical Systems*, 11(2):191–207, Apr 2009. URL: http://dx.doi.org/10.1007/s10109-009-0086-8, doi:10.1007/s10109-009-0086-8.

[Rey14] Sergio J. Rey. Spatial dynamics and space-time data analysis. In Manfred M. Fischer and Peter Nijkamp, editors, *Handbook of Regional Science*, pages 1365–1383. Springer Berlin Heidelberg, Berlin, Heidelberg, 2014. URL: https://doi.org/10.1007/978-3-642-23430-9_78, doi:10.1007/978-3-642-23430-9_78.

[RSastreGutierrez10] Sergio J. Rey and Myrna L. Sastré-Gutiérrez. Interregional inequality dynamics in mexico. *Spatial Economic Analysis*, 5(3):277–298, Sep 2010. URL: http://dx.doi.org/10.1080/17421772.2010.493955, doi:10.1080/17421772.2010.493955.

[RS12] Sergio J. Rey and Richard J. Smith. A spatial decomposition of the Gini coefficient. *Letters in Spatial and Resource Sciences*, 6(2):55–70, Sep 2012. URL: http://dx.doi.org/10.1007/s12076-012-0086-z, doi:10.1007/s12076-012-0086-z.

[Rip91] Brian D Ripley. *Statistical inference for spatial processes*. Cambridge University Press, 1991.

[RP18] Andrés Rodríguez-Pose. The revenge of the places that don't matter (and what to do about it). *Cambridge Journal of Regions, Economy and Society*, 11(1):189–209, Jan 2018. URL: http://dx.doi.org/10.1093/cjres/rsx024, doi:10.1093/cjres/rsx024.

[RBZ+19] Adam Rule, Amanda Birmingham, Cristal Zuniga, Ilkay Altintas, Shih-Cheng Huang, Rob Knight, Niema Moshiri, Mai H. Nguyen, Sara Brin Rosenthal, Fernando Pérez, and et al. Ten simple rules for writing and sharing computational analyses in jupyter notebooks. *PLOS Computational Biology*, 15(7):e1007007, Jul 2019. URL: http://dx.doi.org/10.1371/journal.pcbi.1007007, doi:10.1371/journal.pcbi.1007007.

[SORW21] Jeffery Sauer, Taylor Oshan, Sergio Rey, and Levi John Wolf. The importance of null hypotheses: understanding differences in local Moran's I_i under heteroskedasticity. *Geographical Analysis*, 2021. URL: https://onlinelibrary.wiley.com/doi/abs/10.1111/gean.12304, arXiv:https://onlinelibrary.wiley.com/doi/pdf/10.1111/gean.12304, doi:https://doi.org/10.1111/gean.12304.

[She08] Michael Shermer. Patternicity: Finding meaningful patterns in meaningless noise. *Scientific American*, 299(5):48, 2008.

[SW05] Anthony Shorrocks and Guanghua Wan. Spatial decomposition of inequality. *Journal of Economic Geography*, 5(1):59–81, 2005. URL: http://www.jstor.org/stable/26160606.

[Sil86] Bernard W Silverman. *Density estimation for statistics and data analysis*. Volume 26. CRC Press, 1986.

[SMKH14] Terry A Slocum, Robert B McMaster, Fritz C Kessler, and Hugh H Howard. *Thematic Cartography and Geovisualization*. Pearson, 2014.

[Tat17] Andrew J. Tatem. WorldPop, open data for spatial demography. *Scientific Data*, 4(1):170004, January 2017. doi:10.1038/sdata.2017.4.

[Tob70] Waldo R Tobler. A computer movie simulating urban growth in the Detroit region. *Economic geography*, 46(sup1):234–240, 1970.

[TPPGTZ14] Víctor Torres-Preciado, Mayren Polanco-Gaytán, and Miguel Tinoco-Zermeno. Technological innovation and regional economic growth in Mexico: a spatial perspective. *The Annals of Regional Science*, 52(1):183–200, January 2014. URL: https://ideas.repec.org/a/spr/anresc/v52y2014i1p183-200.html, doi:10.1007/s00168-013-0581-1.

[Tuk77] John W Tukey. *Exploratory data analysis*. Addison-Wesley, 1977.

[UvanMeeteren21] Justus Uitermark and Michiel van Meeteren. Geographical Network Analysis. *Tijdschrift voor Economische en Sociale Geografie*, 112(4):337–350, 2021. doi:10.1111/tesg.12480.

[WB18] Richard Webber and Roger Burrows. *The predictive postcode: The geodemographic classification of British society*. Sage, 2018.

[W+14] Hadley Wickham et al. Tidy data. *Journal of statistical software*, 59(10):1–23, 2014.

[Won04] David W. S. Wong. The Modifiable Areal Unit Problem (MAUP). In Donald G. Janelle, Barney Warf, and Kathy Hansen, editors, *WorldMinds: Geographical Perspectives on 100 Problems: Commemorating the 100th Anniversary of the Association of American Geographers 1904–2004*, pages 571–575. Springer Netherlands, Dordrecht, 2004. doi:10.1007/978-1-4020-2352-1_93.

[Woo06] Simon N. Wood. *Generalized Additive Models*. Chapman and Hall/CRC, Feb 2006. ISBN 9781420010404. URL: http://dx.doi.org/10.1201/9781420010404, doi:10.1201/9781420010404.

[WKW19] AM Wu, KK Kemp, and JP Wilson. Global measures of spatial association. *The geographic information science & technology body of knowledge*, 1st ed., UCGIS, New York, USA., 2019.

[ZRW+20] Xi Zhang, Huaxiang Rao, Yuwan Wu, Yubei Huang, and Hongji Dai. Comparison of spatiotemporal characteristics of the COVID-19 and SARS outbreaks in mainland China. *BMC infectious diseases*, 20(1):1–7, 2020.

INDEX

20:20 ratio, 228, 230, 234

Absolute Deviation around Class Means (ADCM), 117
adjacency, 8, 9, 67-69, 72, 76, 92, 95, 245, 246, 281, 356
agglomerative hierarchical clustering (AHC), 268, 273, 274
aggregation, 61, 124, 227, 268, 347
algorithm, 10, 106, 113-116, 119, 126, 213, 214, 217, 258, 260, 268, 273, 339, 342, 360, 369
alpha shape, 198, 203, 208, 219
aspatial, 246
attributes, 36, 74, 76, 105, 106, 108, 121, 153, 165, 185, 243, 252, 256, 259, 264, 267, 273, 326

big data, 51, 59
binary values, 80, 143
block weights, 86, 87, 101
border, 68, 72, 77, 92
bounding box, 205, 343
buffer, 84, 328, 331, 357, 359

cell, 7, 19, 39, 44, 45, 49, 51, 61, 118, 140, 147, 151, 189, 206-208, 217, 247, 256, 257, 341, 348
center of mass, 194, 196
centrography, 193, 196, 198
centroid, 38, 39, 79, 80
choropleth, 3, 105, 106, 108, 109, 117, 120, 121, 125, 126, 130, 131, 134, 137, 138, 153, 154, 159, 166, 189, 190, 227, 240, 254, 261, 341, 368
classification scheme, 106, 108, 121, 371
clustering, 115, 116, 138, 157, 174, 186, 203, 205, 209, 210, 213, 218, 251-254, 256-261, 268, 270, 272, 273, 275-277, 279-281, 288, 308, 368, 371
color scheme, 120-122
completely spatially random, 210-212
computational data, 12
computational notebooks, xvii, xix, 13, 16, 33, 368
conceptual data, 12
connected component, 91, 99, 281

connections, 5, 9, 12, 48, 88, 101
container, 18, 28-31
contiguity, 48, 68-70, 72-74, 77, 79, 80, 87, 88, 90, 96, 99-102, 151, 252, 254, 274-276, 280, 370
contiguous observations, 74, 99
convex hull, 198, 219, 329, 361
coordinate reference system, 137
coordinates, 42, 43, 45, 46, 77, 85, 185-187, 336, 339, 340, 343, 360
cores, 214
covariate, 295, 296, 299

dasymetric mapping, 347
data models, 4-7, 9, 11, 12, 35, 51, 64
data science, xiii-xvii, xix, 3, 10-13, 16, 18, 23-25, 28, 29, 32, 36, 50, 51, 67, 68, 71, 72, 91, 99, 105, 130, 157, 164, 182, 218, 252, 281, 318, 325, 352
data structures, xv, 4, 6, 7, 9, 11, 12, 24, 35-37, 48, 50, 51, 62, 64, 101
DBSCAN, 213-216, 219, 360
decomposition, 243, 245-247, 372, 373
dispersion, 157, 186, 193, 196, 205, 217, 226, 249
diverging, 121, 122
Docker, 18, 28-30, 32

econometric, 227, 250, 286, 308, 318, 324
edges, 49, 63, 64, 70, 75, 77, 91, 176, 358
environment, xiii, xvii, 17-20, 28, 32
equal intervals, 109, 120
extent, 7, 9, 93, 113, 131, 143, 152, 154, 198, 203, 208, 209, 212, 217, 219, 223, 238, 254, 259, 261, 286, 294, 308, 348

feature, xix, 16, 26, 35, 36, 39, 42, 79, 80, 122, 162, 179, 226, 278, 279, 283, 284, 308, 318, 326-328, 330, 336, 352, 355, 359
feature engineering, 294, 298, 325, 326, 330, 363, 364
fields, xiv, 7-10, 44, 51, 164, 218
fixed effects, 298-300, 302, 363

Gaussian, xvi, 192, 318, 368
Geary's C, 151, 152, 154, 155, 370
geodemographic, 252, 260, 282, 374
geographic tables, 6-8, 35-37, 42, 45, 47, 48, 51, 55, 75
geographical constraints, 252
GeoTIF, 45
Getis-Ord, 154, 155, 173, 174, 182, 184, 370
Gini Index, 94, 230, 231, 233-235, 239, 245, 246, 249, 250, 257, 258, 372
graph, xvi, 9, 36, 48-51, 63, 64, 72, 88, 90-92, 99-102, 154, 275, 280, 281, 318, 356-358

head tail algorithm, 114
hexagons, 192, 193
hexbin, 190, 192, 193, 219

interpolation, 157, 338, 339, 347, 364, 365, 369
intersection, xiv, 63, 185, 347

Jupyter, xviii, 13, 18, 19, 22, 28-30, 33, 371

K-nearest-neighbors (KNN), 85, 88, 91, 102, 154, 291
kernel density, 116, 149, 165, 192, 227
kernel weights, 80, 83
knn, 80, 277

line of perfect equality, 230, 231
linkage key, 327, 363

LISA, 163, 165, 166, 168, 170-179, 183, 367
local spatial autocorrelation, 98, 158, 160, 173, 176

map classification, 116, 117, 130
map matching, 327, 347, 351, 352, 364
map synthesis, 327, 351, 352, 364
markdown, xviii, 19
maximum breaks, 112, 113
mean, 36, 55, 92, 108, 112, 117, 119, 147, 149-151, 155, 162, 193-196, 198, 203, 217, 219, 230, 241, 245, 248, 249, 256, 258-260, 288, 320, 324, 357
median center, 195
minimum bounding rectangle, 200, 201, 203
minimum rotated rectangle, 201
Moran's I, 140, 147, 149-155, 157, 160, 162-165, 168, 172, 173, 175, 176, 179, 180, 182, 183, 238-240, 244, 254, 292, 372
multivariate process, 251, 252

neighbor, 68-75, 79-81, 84, 86-89, 93, 101, 145, 152, 153, 155, 210, 211, 274, 280, 288, 309, 314, 321-323, 339, 354, 364
neighborhood fixed effects, 302
network, 5, 6, 9, 49, 62, 63, 99, 100
nodes, 5, 49, 50, 63, 64, 70, 72, 101
noise, 157, 166, 215, 283, 289, 369, 373
notebook, xviii, 14, 16-20, 23, 26, 28, 35
null correlations, 324

objects, 4, 5, 7-11, 36, 37, 48, 51, 52, 61-63, 67, 68, 75, 78, 79, 85, 89, 99, 101, 177, 185, 187, 213, 228, 251, 287, 348, 365
open science, 13, 32, 35, 369
open source packages, 17, 18

point pattern, 185, 187, 193, 194, 200, 203-205, 207, 210, 212-214, 217-219
Points of Interest (POIs), 328-331, 333
Poisson, 211
polygon, 40, 55, 56, 59, 69-72, 76, 79, 80, 84, 93, 105, 152, 208, 253, 263, 328, 345, 347, 348, 361
PySAL, 367
Python, xiii, xv, xvii, xix, 7, 8, 13, 19, 23-26, 35-37, 45, 48-51, 59, 61, 64, 73, 101, 106, 121, 130, 147, 165, 190, 192, 194, 213, 214, 231, 260, 268, 285, 286, 299, 300, 305, 309, 313, 347, 369

Quadrat Statistic, 206
qualitative, 121, 124, 125, 240
quantiles, 97, 110, 111
Queen, 74, 76, 77, 84, 87, 90, 91, 99-102, 274, 276, 277, 281

raster, 10, 11, 55, 328, 338
regionalization, 8, 250, 252, 253, 273, 274, 276, 277, 279-281, 369
regression, 48, 74, 283-287, 294, 296, 299, 300, 304, 308-310, 315, 317-319, 324, 339, 345, 363, 364, 367, 368, 372
reproducibility, 14, 16, 17, 33, 260
reproducible platforms, 18, 28
ring, 357, 359
Rook, 69, 71, 72, 76, 77, 89, 96, 99, 100, 281

sequential, 121, 122, 125
simple features, 40
spatial configuration, 67, 130, 177, 179, 238, 254, 285
spatial constraint, 273-275
spatial dependence, 134, 246, 248, 250, 281, 294, 308, 309, 315, 321, 324
spatial error, 315, 316, 318, 319
spatial features, 131, 351
spatial graph, 6, 8, 9, 35, 48, 51, 92
spatial heterogeneity, 284, 294, 298, 304, 372
spatial join, 345, 346
spatial lag, 138, 140-143, 147, 148, 153, 154, 160, 162, 291, 308, 309, 311, 313, 315-319, 322, 327, 353
spatial network, 8, 36, 49
spatial randomness, 133, 145, 146, 150, 152, 153, 163, 214
spatial regimes, 304, 307
spatial regression, 74, 283, 285, 318, 364, 367
spatial structure, 91, 93, 158, 160, 176, 186, 253, 254, 256, 272, 280, 283, 288, 294, 297, 304, 319, 323, 351, 364
spatial weights, 8, 9, 48, 67, 68, 71, 72, 74-76, 78, 84-88, 91, 99, 102, 138, 140, 145, 147, 152-154, 160, 165, 177, 178, 238, 245, 246, 254, 273, 274, 281, 291, 308, 309, 318, 352
spillovers, xvi, 290
standard deviation, 112, 113, 117, 119, 183, 184, 217, 258, 327
surfaces, 4, 6-9, 11, 35, 36, 45, 47, 48, 51-53, 55, 56, 59, 61, 78, 158, 176-178, 180, 189

T (Theil), 230, 235, 239, 242, 244-246, 249, 250
time series, 228, 239, 244

variance, 164, 249, 259, 278, 289, 298
vector, 10, 11, 140, 178, 179, 246, 261, 286, 324

weights, 8, 9, 48, 67, 68, 70-81, 83-91, 93, 94, 99, 101, 102, 138, 140, 145, 147, 152-155, 160, 165, 177, 178, 238, 245, 246, 254, 273-275, 277, 281, 291, 308, 309, 312, 318, 347, 352, 355, 357